

ShiftSwap: A Shift Schedule and Substitute Management System for Small Part-Time Workplaces

소규모 매장 아르바이트 근무표·대타 관리 시스템
Team 7

Name	Student ID	Affiliation	E-mail
Hyewon Kim (김혜원)	24102028	Dept. of Industrial Engineering, ITM Program, SeoulTech	gokhw123@g.seoultech.ac.kr
Hyoungdo Kim (김형도)	18102075	Dept. of Industrial Engineering, ITM Program, SeoulTech	domo5776@seoultech.ac.kr
Minkyung Kim (김민경)	23102004	Dept. of Industrial Engineering, ITM Program, SeoulTech	min23@g.seoultech.ac.kr

Submission date	2026-12-04 (Week 14)
Deployed application	https://keemlogan.github.io/shiftswap/app/
Source repository	https://github.com/keemlogan/shiftswap
Submitted to	jihjim@gmail.com

Table of Contents

Roles of Team Members

Hyewon Kim — requirements lead

Hyoungdo Kim — project manager, design and implementation lead

Minkyung Kim — UI and quality lead

Abstract

1. Introduction

1.1 Motivation

1.2 Objectives

1.3 Scope

1.4 Development Approach

1.5 Organisation of the Report

2. Problem Statement, Background and Related Work

2.1 Current Process in the Reference Store

2.2 Problems of the Current Process

2.3 Legal Background

2.4 Related Work

2.5 The Gap ShiftSwap Fills

3. Requirements Specification

3.1 Requirements Elicitation

3.2 Stakeholders and Actors

3.3 Use-Case Model

3.4 Functional Requirements

3.5 Use-Case Descriptions

3.6 Business Rules

3.7 Non-Functional Requirements

3.8 Scenarios

3.9 Analysis Models

3.10 Requirements Validation

3.11 Requirements Management

4. Design Specification

4.1 Architecture

4.2 Deployment

4.3 Design Decisions

4.4 Class Design

4.5 Database Design

4.6 Dynamic Design: Sequence Diagrams

4.7 State Design

4.8 Algorithm Design

4.9 User-Interface Design

5. Implementation by Vibe Coding

5.1 Development Environment and Tools

5.2 The Vibe-Coding Process

5.3 Structure of the Final Prompt

5.4 Iterations of the Prompt

5.5 From Prompt to Code

5.6 Reproducibility Check

6. Testing and Validation

6.1 Test Strategy

6.2 Test Environment

6.3 Test Cases

6.4 Test Results

6.5 Boundary-Value Analysis

- 6.6 Concurrency Test
- 6.7 Verification of Non-Functional Requirements
- 6.8 Traceability Matrix
- 6.9 Defects and Specification Gaps Resolved During the Iterations

7. Deployment and User Guide

- 7.1 Deployment
- 7.2 User Guide
- 7.3 Demonstration Script
- 7.4 Troubleshooting

8. Conclusion

- 8.1 Achievement of the Objectives
- 8.2 Size and Complexity of the System
- 8.3 Limitations
- 8.4 Future Work
- 8.5 Lessons Learned

References

- Appendix A. Final Vibe-Coding Prompt (v8)
- Appendix B. Prompt Log (index)

Roles of Team Members

Team 7 has three members. The topic was proposed by Hyewon Kim, who works part-time in a small café and has seen at first hand how substitute shifts are arranged there. In Week 4 the team compared seven candidate topics and on 2026-09-27 selected ShiftSwap because it is a conventional transaction-processing application with a clear owner–worker workflow, more than five core use cases and a relational data model of twelve tables, which satisfies the minimum scope set by the term project description (five use cases, four tables).

Work was divided along the work breakdown structure (WBS) of the proposal, which has eight top-level packages that follow the course deliverables and the life-cycle phases: 1 Project management, 2 Proposal, 3 Requirements (interim report), 4 Design, 5 Implementation by vibe coding, 6 Testing, 7 Final report, and 8 Presentation. Each work package has one owner who is accountable for its deliverables. Every deliverable was also reviewed by a member who did not write it: requirements by Hyoungdo Kim and Minkyung Kim, design models by Hyewon Kim and Minkyung Kim, and test cases by Hyewon Kim and Hyoungdo Kim. The team met once a week, and the Gantt chart was updated after each meeting. The table below lists the responsibilities; the paragraphs after it describe each member's work in more detail.

Table 1. Responsibilities of team members mapped to WBS work packages and deliverables

Member	Role	WBS work packages owned	Main deliverables
Hyewon Kim	Requirements lead	2.1 Topic selection; 2.2.1 Problem statement and scope; 3.1 Elicitation; 3.2.1 FR and NFR tables, scenarios; 7.1.1 Requirements and design chapters	Comparison of seven candidate topics; problem statement P1–P3 and as-is process (Chapter 2); interview plan and notes; actors and use-case list; FR-01...FR-21; NFR-01...NFR-13; scenarios S-1...S-7; use-case descriptions UC-01...UC-13; requirements and design chapters of this report
Hyoungdo Kim	Project manager; design and implementation lead	1 Project management (1.1, 1.2); 2.2.2 Reviewed proposal PDF; 3.2.2 Business rules and data model draft; 4.1 UML models; 5 Implementation by vibe coding (5.1, 5.2); 7.2.1 Traceability matrix; 8.2.2 Live demo	WBS, Gantt chart, weekly milestone tracking with public-URL check, and meeting notes; canonical specification file <code>spec/spec.md</code> ; business rules BR-01...BR-12 and data model; use-case, class, ER, sequence, state and activity diagrams; final vibe-coding prompt and prompt log; increments 1 and 2; public URL on GitHub Pages; shared database on Supabase with a scheduled keep-alive job (5.2.3); traceability matrix; live demonstration
Minkyung Kim	UI and quality lead	4.2 UI design; 6 Testing (6.1, 6.2); 7.1.2 Testing and results chapters; 7.2.2 Final report PDF; 8.1 Slide deck; 8.2.1 Rehearsed talk	Screen wireframes, prototype screens and visual design rules; test cases TC-01x...TC-12x and passing test run; scenario walkthrough record and screenshots; cross-browser and 360/1440 px checks; testing chapters; integrated final report PDF (50–150 pages); PowerPoint deck

Hyewon Kim — requirements lead

Hyewon Kim owned the problem domain. The topic came from her own part-time job, and her workplace is the reference store of the project; she prepared the comparison of seven candidate topics that led to its selection on 2026-09-27 (WBS 2.1) and wrote the problem statement and scope of the proposal (2.2.1). She described the current substitute procedure step by step and supplied the figure of three to five one-to-one contacts per substitute request that appears as problem P1. She collected the legal material on the 2026 minimum wage (KRW 10,320), the weekly holiday allowance (주휴수당) under the Labor Standards Act, the five-employee threshold for premium pay and the probation reduction of the Minimum Wage Act. For elicitation (3.1) she prepared the interview plan for the owner and two co-workers (Section 3.1) and the list of actors and use cases; for the specification (3.2.1) she wrote the functional and non-functional requirements, the scenarios of Section 3.8 and the use-case descriptions of Section 3.5. She wrote the requirements and design chapters of this report (7.1.1). In the presentation she covers the problem and the requirements.

Hyoungdo Kim — project manager, design and implementation lead

Hyoungdo Kim was responsible for project management (WBS 1): the WBS and the Gantt chart, weekly milestone tracking with a check of the public URL, the weekly meeting notes and the task list with owners. He reviewed and submitted the pro-

posal (2.2.2). He turned the legal material into the business rules and drafted the data model (3.2.2), and he created the specification file `spec/spec.md`, which fixes every identifier (UC, FR, NFR, BR, TC), every state name and the seed data, so that the application, the UML models and the reports use the same names and numbers. He drew the UML models (4.1): use-case, class and ER diagrams, and sequence, state and activity diagrams. For implementation by vibe coding (5) he wrote the final prompt and kept the prompt log of iterations against the specification (5.1), produced increments 1 and 2 with the AI coding assistant, deployed the application on GitHub Pages, and set up the shared database on Supabase with a scheduled keep-alive job (5.2). He built the traceability matrix of the final report (7.2.1). In the presentation he covers the design and the implementation by vibe coding and gives the live demonstration of the deployed application (8.2.2).

Minkyung Kim — UI and quality lead

Minkyung Kim owned the user-interface design (WBS 4.2): the screen wireframes and prototype screens (4.2.1) for sign-in, weekly board, inbox, approvals, attendance, weekly summary, payroll and the owner and worker forms, and the visual design rules (4.2.2) that were later written into the final prompt. She owned testing (6): she wrote the unit test cases TC-01x to TC-12x from the business rules and ran them (6.1), and she carried out the scenario walkthroughs with screenshots and the cross-browser checks at 360 px and 1440 px screen width required by NFR-02 (6.2). She wrote the testing and results chapters (7.1.2), integrated the chapters of the three members into the final report PDF and checked the course format rules — A4, 12 pt, single spacing, 1-inch margins, 50–150 pages (7.2.2). She prepared the slide outline and the PowerPoint deck (8.1) and rehearsed the 15-minute talk (8.2.1). In the presentation she covers testing, deployment and the conclusion.

Abstract

In small stores such as cafés and convenience stores, part-time workers have fixed weekly shifts, and when one of them cannot come the owner must find a substitute by contacting other workers one at a time, edit a spreadsheet by hand, and later recheck each worker's weekly hours to decide whether the weekly holiday allowance (주휴수당) is still owed. In the reference store of this project, one substitute request typically costs the owner three to five separate contacts, and legal pay rules — the 2026 minimum wage of KRW 10,320, the 15-hour threshold for the holiday allowance, the five-employee threshold for premium pay, the limit on and premium for work of part-time workers beyond their contractual hours, and the probation reduction — are applied from memory. This report describes ShiftSwap, a web application built to remove these three problems. A worker requests a substitute for one of their own shifts; the system asks every co-worker who is not already working at that time, notifies all of them at once, and accepts only the first acceptance; if every one of them answers that they cannot, the request ends at once and the requester is told to contact the owner. The owner then approves the swap on a screen that shows the weekly contractual hours, scheduled hours and holiday-allowance eligibility of both workers before and after the change. In a workplace with five or more regular employees, the same screen warns when the swap would leave a part-time acceptor with more than 12 hours beyond their weekly contractual hours, the legal weekly limit for such work; the warning does not block the decision.

The system was developed with an incremental life-cycle model combined with UI prototyping, and every increment was implemented by vibe coding, that is, by generating code from natural-language prompts with an AI coding assistant. The report covers the problem statement and related work, a requirements specification of 13 use cases, 23 functional requirements, 16 non-functional requirements and 15 business rules, a design specification with UML 2.5 use-case, class, sequence, state and activity diagrams and an eleven-table relational schema, the final prompt that reproduces the system, the testing approach based on test cases derived from the business rules, and a user guide. The application is a static web application (HTML, CSS and JavaScript) deployed at <https://keemlogan.github.io/shiftswap/app/>. By default it keeps its data in a shared Supabase PostgreSQL database, so that the owner and every worker use one store from their own devices: the database server applies each change in one transaction, guarantees that only the first acceptance succeeds, checks deadlines against its own time and limits the rate of changes. Sign-in is a choice among demo accounts without a password, and all shared data are fictional. A local demonstration mode, which keeps the same tables in the browser with SQLite compiled to WebAssembly, serves as a fallback.

1. Introduction

This chapter explains why the team built ShiftSwap, what the project set out to achieve, which features are inside and outside the scope of the delivered system, and how the development process was organised. The last section outlines the rest of the report.

1.1 Motivation

Small stores in Korea — cafés, convenience stores, bakeries and small restaurants — depend on part-time workers (아르바이트). A typical store of this kind employs fewer than ten such workers, and each of them has a fixed weekly pattern of shifts, for example every Monday and Wednesday from 18:00 to 23:00. The pattern is agreed in the employment contract and changes rarely. What changes often is attendance on a particular day: a worker has an examination, falls ill or has a family event, and asks for someone else to cover the shift. In the café where one team member works, such requests are a routine part of running the store rather than an exception.

The owner, who in a store of this size is also the manager, handles each such case personally. The owner phones or messages the other workers one by one until someone agrees, updates the schedule spreadsheet, and tells the worker who asked. The work does not end there. Korean labour law ties part of a part-time worker's pay to the number of hours worked each week: a worker whose weekly contractual hours are 15 or more and who attended every scheduled working day is entitled to a paid weekly holiday, the weekly holiday allowance (주휴수당) [3], [5]. A swap therefore changes the hours and possibly the entitlement of two people at once, and the owner has to recheck both at the end of the month. In the reference store these checks are made from memory with a spreadsheet; the legal thresholds are not written down anywhere in the store.

The team chose this problem for three reasons. First, one member knows the process from the inside, so requirements could be drawn from a real workplace rather than from assumptions. Second, the problem is a transaction-processing problem of the kind the course recommends: it has several actors, a clear life cycle for each substitute request, and a relational data model. Third, the core difficulty is not the user interface but the rules — who is eligible, who wins when two people accept at the same moment, and how weekly and monthly pay change after a swap — which makes the requirements and design phases substantial and gives the testing phase concrete targets.

1.2 Objectives

The overall goal of the project is to deliver a working web application that replaces the telephone-and-spreadsheet process of a small store for three tasks: finding a substitute, keeping the schedule correct, and computing the weekly holiday allowance and monthly pay. The proposal broke this goal down into six measurable objectives, G1 to G6, which are kept unchanged in this report so that they can be checked at the end of the project (Chapter 8). Four of them are tied to the problems P1–P3 defined in Chapter 2; the other two follow from the course requirements.

Table 2. Project objectives, linked problems and measures of achievement

ID	Objective	Measure of achievement	Problem
G1	Remove the owner's one-to-one contacts when a substitute is needed.	The owner makes 0 contacts per request (reference store before the system: 3–5). All eligible workers are notified in the same transaction that creates the request (FR-07). The owner's only action is one approve or reject decision.	P1
G2	Let a candidate answer quickly.	A candidate accepts a request in at most 2 clicks from the inbox (NFR-01). Only the first acceptance can succeed (BR-02, NFR-04).	P1
G3	Keep the schedule and weekly hours of both workers correct after every swap.	On approval the shift is reassigned and the weekly summaries of both workers are recomputed with no manual step (FR-12). The approval screen shows both workers' hours and eligibility before and after the swap (FR-11); for the seed example it shows Choi Minh'o's scheduled hours changing from 16 to 21 with eligibility unchanged.	P2
G4	Apply the legal pay rules automatically and flag violations before payday.	Business rules BR-06 to BR-11 are implemented as tested functions, and every business rule has at least one unit test (spec §12), including the 15-hour boundary (14.99 h and 15 h) and the 40-hour cap. The payroll screen warns when a wage is below the applicable minimum and blocks confirmation until the warning is acknowledged (FR-16).	P3

ID	Objective	Measure of achievement	Problem
G5	Deliver a system of the size the course requires.	At least 5 core use cases (delivered: 13 active use cases, of which 8 are core) and at least 4 database tables (delivered: 11). The proposal planned 13 use cases and 12 tables; UC-02 and its table were retired in iteration 6, and UC-14 was added in iteration 7.	—
G6	Deliver on time and reproducibly.	Working system deployed at a public URL by Week 14 (2026-12-04, NFR-07). Re-running the final prompt in an empty folder produces a system that passes the same unit tests.	—

G6 also reflects a practical need of the target users: a small store has no server and no IT staff, so the system must run in a browser without installation and without an application server of its own (NFR-06), and it must be possible to rebuild it from its documentation alone.

1.3 Scope

The team classified features with the MoSCoW scheme (Must, Should, Could, Won't). Use-case priorities are the ones recorded in spec §3. The proposal defined thirteen use cases. Nine of them correspond one to one to the nine functions it announced (UC-01 to UC-09); UC-10 to UC-13 are supporting use cases that make settings, expiry, cancellation and role selection explicit, because the core use cases are incomplete without them. One of the nine, UC-02 Register availability, was retired in iteration 6 after the product owner's review of the running system (Section 3.11.4), which left twelve active use cases; UC-14, the automatic preparation of last month's payroll, was added in iteration 7 (Section 3.11.5), so the final system has thirteen. Iteration 8, which followed the revision of the proposal, added no use case (Section 3.11.6).

Table 3. Scope of the delivered system

Priority	Included features
Must	UC-01 Register worker and fixed schedule; UC-02 Register availability (planned in the proposal, retired in iteration 6); UC-03 Generate weekly schedule; UC-04 Request substitute (with automatic candidate selection, BR-01); UC-05 Respond to substitute request; UC-06 Approve or reject substitution; UC-08 Compute weekly hours and holiday allowance; UC-11 Expire overdue substitute request; UC-13 Sign in by role (demo account selection without a password); in-app notification inbox (FR-21); UC-14 Prepare last month's payroll automatically (added in iteration 7).
Should	UC-07 Record and confirm attendance; UC-09 Generate and confirm monthly payroll (base pay, holiday allowance, premium pay, minimum-wage warning, probation reduction); UC-10 Manage workplace settings and minimum wage table; UC-12 Cancel own substitute request; part-time overtime warning of the approval panel (F6, FR-24, added in iteration 8); Korean/English language switch; adjustable demo clock (local demonstration mode).
Won't (this version)	Password or external-account authentication (users select a demo account by role, UC-13); push notifications, KakaoTalk or SMS messaging (notifications are in-app only); bank transfer of wages; GPS or QR-code clock-in (excluded on purpose by NFR-12); multiple stores per owner; automatic optimisation of the weekly schedule; payslip export, tax and social-insurance deductions; annual leave, severance pay and the four-week averaging of the 15-hour rule beyond the weekly contractual value (see BR-06).

Two scope decisions need comment. The first concerns data and sign-in. Following the revised proposal, the system keeps the data of the demonstration store in a shared relational database on Supabase, a hosted PostgreSQL service with a free plan [16], so that the owner and every worker use one store from their own phones and a substitute request appears at once in the inbox of every eligible worker, as G1 requires. The system has no application server of its own: the static application reaches the database directly, and the database re-checks every change it receives (NFR-14). Sign-in, however, remains a choice among demo accounts without a password (UC-13); password or external-account authentication is out of scope. The database therefore guarantees the integrity of the data, the first acceptance, the server time and the rate limits, but not the identity of the user, and all shared data are fictional. A local demonstration mode, which keeps the same tables in one browser with sql.js [9], serves as a fallback when the shared database cannot be reached (NFR-16). The delivered system is therefore a multi-user demonstration system, not a production service. The second decision concerns payroll. The payroll function stops at gross pay: base pay, holiday allowance and premium pay. Deductions such as income tax and national insurance depend on rules that change every year and are outside the three problems the project addresses.

1.4 Development Approach

1.4.1 Choice of life-cycle model

Chapter 1 of the lecture presents the linear sequential (waterfall) model, the prototyping model, the incremental model, Boehm's spiral model, the Rational Unified Process and agile methods [6]. The team compared them against four facts about the project: the course fixes three document deadlines (proposal in Week 5, interim report in Week 7, final report in Week 14); the requirements for the substitute workflow were known well from the reference store, but the screens were not; code is produced by an AI coding assistant from prompts, so a working version can be generated in hours rather than weeks; and the team has three members with no dedicated customer on site.

Table 4. Life-cycle models considered

Model	Fit with this project	Decision
Waterfall	Matches the fixed course documents, but a phase must be completed before the next begins, and feedback on screens would arrive only after implementation.	Rejected as the main model; its document sequence is kept for the course deliverables.
Prototyping	Suited to the unknown part of the requirements (screens, approval comparison panel). The lecture warns that prototypes omit error checking and non-functional requirements.	Adopted for the user interface only.
Incremental	Delivers a minimal useful subset first and adds increments; reduces rework when requirements change; allows demonstration at each increment.	Adopted as the main model.
Spiral	Explicit risk analysis per loop; heavy for a three-person, fourteen-week project.	Not adopted; the main risks are tracked in the WBS instead.
RUP	Iterative phases with UML; its good practices (manage requirements, visual modelling) were adopted, but the four-phase structure adds little for a small system.	Practices adopted, process not.
Agile / XP	Test-first development and small releases fit vibe coding well; the course, however, requires full requirement and design documents.	Test-first practice adopted within increments.

The resulting approach is an **incremental model with UI prototyping**. The incremental model addresses the main weakness the lecture identifies in the waterfall model, the difficulty of accommodating change after the process is under way. The prototype addresses the part of the requirements that only becomes clear when users can see a screen. The known weaknesses of the incremental model are that the process is less visible and that structure degrades as increments are added [6]. The team countered the first with the fixed course documents and the prompt log, and the second with a layered architecture that was fixed before the first increment (Chapter 4) and with the rule that domain rules live only in one module of pure functions.

1.4.2 Increments

Following the proposal, the requirements and analysis models were written once for the whole system (WBS 3 Requirements and 4.1 UML models), and implementation was then delivered in two increments (WBS 5.2.1), followed by deployment (5.2.2), testing (6) and the freezing of the final prompt kept under prompt engineering (5.1). The revised proposal adds the set-up of the shared database with its scheduled keep-alive job in Week 9, before the first increment (5.2.3). The table below shows the content of each increment.

Table 5. Increments of the ShiftSwap development

Increment	Content	Use cases	Planned week and exit criterion
-----------	---------	-----------	---------------------------------

Increment	Content	Use cases	Planned week and exit criterion
Increment 1 (Must)	Database schema and seed data (shared database and local demonstration mode), demo clock, sign-in, worker and schedule registration, availability (retired in iteration 6), weekly board, the complete substitute workflow (request, candidate notification, first acceptance, approval with before/after comparison, expiry), weekly holiday allowance, inbox.	UC-01, UC-02, UC-03, UC-04, UC-05, UC-06, UC-08, UC-11, UC-13	End of Week 10, 2026-11-06 (WBS 5.2.1). Seed request with two candidates reproduced (three since iteration 6); worked example (Choi Minho, 3.2 h, KRW 33,024) shown on the approval screen; test groups TC-01x, TC-02x, TC-05x, TC-06x, TC-08x, TC-12x pass.
Increment 2 (Should)	Attendance recording and confirmation, monthly payroll with premium pay, minimum-wage warning and probation reduction, workplace settings and minimum-wage table, cancellation of an own request, part-time overtime warning of the approval panel (FR-24).	UC-07, UC-09, UC-10, UC-12	End of Week 11, 2026-11-13 (WBS 5.2.1). Test groups TC-09x, TC-10x, TC-11x, TC-15x pass together with all groups of Increment 1 (regression).
Deployment and testing	Deployment to GitHub Pages, test runs, scenario walkthroughs, cross-browser and responsive checks, reproduction run of the final prompt in an empty folder.	All	Weeks 12–14 (WBS 5.2.2, 6.1.2, 6.2). Deployed URL and all test groups run in Week 12; NFR-02 and NFR-06 checks and final prompt frozen in Week 13; final report, slides and live demonstration on 2026-12-04 (Week 14).

Increment 1 is the "minimal subset of the whole system that is still useful" in the sense of the lecture [6]: with it, a store can already find substitutes, approve swaps and see weekly holiday-allowance eligibility. If Increment 2 had slipped, the core of the project would still have been delivered. Placing all Must use cases in the first increment was the main mitigation of the scope-creep risk identified in the proposal. Three later changes of the requirements, in iterations 6, 7 and 8, went through the change procedure of Section 3.11.3 and are described in Sections 3.11.4 to 3.11.6.

1.4.3 UI prototyping

The prototyping model of the lecture runs requirement analysis, prototype development, prototype evaluation, and then implementation [6]. In this project the prototype was not a separate throwaway artefact. The first version of each screen was generated by the AI coding assistant from a short description, evaluated by the team against the scenarios of Section 3.8, and then either kept or regenerated. Three screens went through this loop more than once: the weekly board (how to show a handed-over shift), the inbox (how to reach the accept button in two clicks), and the approval comparison panel (which figures to show for the two workers). The decisions from these loops were written back into the specification as visual design rules, which is why the final prompt contains exact colours, spacing and a single decorative element (Chapter 5). The lecture notes that prototypes tend to skip error handling and non-functional requirements; to avoid this, the error messages and the non-functional requirements NFR-01 to NFR-06 were added to the prompt from the first increment onward rather than left for the end.

1.4.4 How vibe coding fits each increment

Vibe coding means that source code is produced by an AI coding assistant from natural-language prompts, and that the developer's work moves from typing code to specifying, reviewing and testing it. The team used the same cycle inside each increment:

1. **Specify.** The identifiers, rules, states and seed data needed by the increment are added to `spec/spec.md` first. Nothing is allowed into a prompt that is not in the specification.
2. **Prompt.** A prompt for the increment is written that refers to the specification by ID (for example, "implement BR-02 inside one transaction that re-checks the request state").
3. **Generate.** The AI coding assistant generates or modifies the code.
4. **Test.** The unit tests named after the spec test IDs are run with `node --test`, and the scenarios of Section 3.8 are walked through in the browser.
5. **Review and refine.** Failures and unwanted behaviour are traced either to an ambiguous prompt or to a gap in the specification. The prompt or the specification is corrected, never the generated code by hand alone, and the change is

recorded in the prompt log.

This cycle keeps the course deliverables and the code consistent: the final prompt is not written after the fact but is the accumulated result of the refinements made in all four increments. Test-first practice from extreme programming fits this cycle naturally, because a test named TC-081 holiday allowance at exactly 15 h is itself a precise prompt. Chapter 5 presents the final prompt and the prompt log in detail.

1.5 Organisation of the Report

Chapter 2 describes the current substitute process in the reference store, the three problems it causes, the legal background, and existing shift-scheduling products. Chapter 3 is the requirements specification: actors, use cases with full descriptions, functional and non-functional requirements, business rules, scenarios, analysis models and the validation of the requirements. Chapter 4 is the design specification with the architecture, class diagram, sequence diagrams, state diagrams and database design. Chapter 5 covers implementation by vibe coding, including the final prompt and the prompt log. Chapter 6 presents testing and validation and the traceability matrix. Chapter 7 explains deployment and gives a user guide for owners and workers. Chapter 8 concludes the report and lists limitations and future work. The references follow Chapter 8; the source code is submitted separately as an attached file.

2. Problem Statement, Background and Related Work

This chapter describes the problem that ShiftSwap addresses. Section 2.1 describes how substitutes are arranged today in the reference store and models the process as a data flow diagram. Section 2.2 states the three problems P1–P3 that the process causes. Section 2.3 summarises the legal rules that govern part-time pay in Korea and that the system must apply. Section 2.4 compares existing shift-scheduling products, and Section 2.5 identifies the gap that ShiftSwap fills.

2.1 Current Process in the Reference Store

2.1.1 The reference store

The requirements were drawn from one real workplace, a small café where Hyewon Kim works part-time. To protect the store and its staff, the report does not name the store; the demonstration data of the system uses a fictitious café, Dalbit Café (달빛카페), with one owner and four part-time workers, which has the same structure. The real store has the following characteristics, which are typical of the stores the system targets:

- fewer than ten part-time workers; whether the store reaches five regular employees, the threshold for premium pay (Sections 2.3.3 and 2.3.5), depends on how many workers it employs at a given time, so the system treats the number as a setting;
- the owner acts as the manager, sets the schedule and pays the wages; there is no separate HR or payroll staff;
- each worker has a fixed weekly pattern of shifts written in the employment contract, and the pattern changes rarely;
- the schedule is kept in a spreadsheet maintained by the owner, and messages about substitutes are sent by phone call or personal message;
- wages are paid monthly, and the owner calculates them at the end of the month from the spreadsheet.

2.1.2 The as-is substitute process, step by step

The following description is the current procedure for one substitute request, as reported by the team member who works in the reference store. It is written as the normal sequence; the variations are listed afterwards.

1. **The worker reports the absence.** A worker who cannot attend a coming shift sends the owner a personal message with the date and the reason, for example "I cannot come on Wednesday evening because of a midterm exam."
2. **The owner looks up possible substitutes.** The owner opens the schedule spreadsheet, checks who is not working at that time, and recalls from memory who is usually free on that weekday. Availability is not recorded anywhere.
3. **The owner contacts workers one by one.** The owner calls or messages the first person on the mental list and waits for an answer. If the person refuses or does not answer within a reasonable time, the owner contacts the next person. In the reference store this typically takes three to five contacts per request.
4. **A worker agrees.** When one worker agrees, the owner confirms to that worker and to the worker who asked. The other workers who were contacted may not be told that the shift has been filled.
5. **The owner updates the spreadsheet.** The owner changes the name in the cell for that shift and tells the workers concerned.
6. **The owner rechecks weekly hours at month end.** At the end of the month the owner adds up each worker's hours per week from the spreadsheet, including the swapped shifts.
7. **The owner decides the holiday allowance.** For each worker and week, the owner decides from memory whether the worker had at least 15 contractual hours and attended every working day, and whether a swapped shift counts as an absence.
8. **The owner calculates the monthly pay.** The owner multiplies hours by the hourly wage, adds the holiday allowance, and transfers the pay. Whether the hourly wage is still above the current minimum wage is not checked systematically.

Three variations occur regularly. Workers sometimes arrange a swap between themselves and only inform the owner afterwards, or not at all, in which case the spreadsheet does not match reality. Two workers may both agree when the owner has contacted them in parallel, and one of them must then be told that the shift is no longer free. Finally, when nobody agrees, the owner covers the shift personally, and this is often not recorded.

2.1.3 Data flow diagram of the as-is process

The figure below models the procedure as a data flow diagram (DFD) following the rules of Chapter 2 of the lecture [6]: boxes are processes named with verb phrases, arcs are data named with noun phrases, control is not shown, and actors are not included. Arcs that enter from outside the diagram carry data supplied by a person (the worker or the owner) whose identity is omitted by the DFD rules. Two outputs from one box mean that one or the other is produced: process 3 produces either a refusal, which feeds process 2 again, or a substitute agreement.

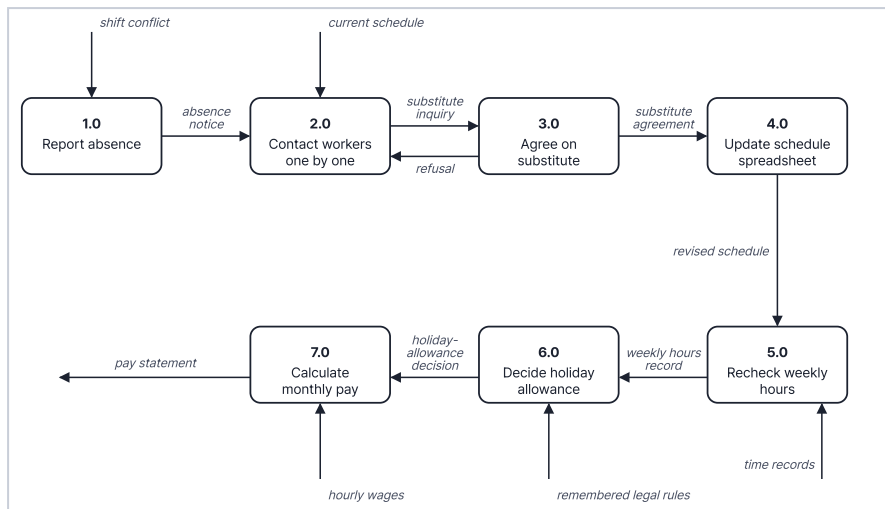


Figure 1. Data flow diagram of the as-is substitute and pay process in the reference store (lecture Chapter 2 notation: processes as verb phrases, data as noun phrases, actors omitted)

The diagram makes two properties of the current process visible. First, the loop between processes 2.0 and 3.0 carries one inquiry and one refusal at a time; the number of iterations of this loop is the cost described as problem P1. Second, processes 5.0 to 7.0 depend on data that is not recorded in the store: "time records" are notes kept by the owner, and "remembered legal rules" is literally the owner's memory. These two inputs are the source of problems P2 and P3.

2.2 Problems of the Current Process

The as-is process causes three problems. Each problem statement below gives the observable symptom, the cause in the data flow diagram of Section 2.1.3, and the consequence.

P1. Finding a substitute costs repeated one-to-one contacts

For every substitute request the owner contacts workers one at a time, typically three to five contacts per request in the reference store. The cause is that the information needed to choose a candidate — who is free at that time and who is willing to work extra — exists only in the owner's memory and in the workers' heads, so the owner has to ask. Because each inquiry waits for an answer, the elapsed time grows with the number of contacts, and a request made late in the evening may not be settled until the next day. When the owner contacts several people in parallel to save time, two of them may agree, and one has to be disappointed. The workers who refused or were not asked do not see whether the shift has been filled.

P2. Every swap changes the hours of two workers, and all recalculation is manual

A swap removes a shift from one worker's week and adds it to another's. Both workers' weekly scheduled hours change, and the holiday-allowance eligibility of either of them may change depending on how the missed shift is counted. In the current process the spreadsheet cell is edited (process 4.0), but the effect on hours is not calculated until the end of the month (process 5.0), when the owner must reconstruct from the edited spreadsheet which shifts were swapped and when. The owner approves a swap without seeing its effect on pay, and if the spreadsheet was not updated for a swap arranged directly between workers, the monthly calculation is wrong.

P3. Legal pay rules are applied from memory, so errors are not caught before payday

The weekly holiday allowance, the minimum wage, the five-employee threshold for premium pay, the limit on work of part-time workers beyond their contractual hours and the probation reduction are all conditional rules with numeric thresholds (Section 2.3). In the reference store they are applied from memory at month end. Errors that this practice allows include paying a holiday allowance to a worker below 15 contractual hours, not paying it to a worker at or above the threshold, continuing to pay the previous year's minimum wage after 1 January, applying the 90 % probation rate to a worker whose contract does not qualify for it, and, in a store with five or more regular employees, approving a swap that takes a part-time worker more than 12 hours beyond the contractual hours of the week. Because no check is made before the transfer, these errors are discovered, if at all, only when a worker complains.

Table 6. Summary of problems, causes in the as-is process and consequences

ID	Problem	Cause (DFD process)	Consequence
P1	Repeated one-to-one contacts to find a substitute	Availability not recorded; inquiries sent one at a time (2.0 ↔ 3.0)	3–5 contacts per request; delays; double acceptances when contacts are made in parallel

ID	Problem	Cause (DFD process)	Consequence
P2	Manual recalculation for two workers per swap	Spreadsheet edited without calculation (4.0); hours rebuilt at month end (5.0)	Swaps approved without knowing their pay effect; mismatches between schedule and reality
P3	Legal rules applied from memory	Rules not recorded (input "remembered legal rules" of 6.0 and 7.0)	Pay errors discovered only after payday

2.3 Legal Background

Part-time workers in Korea are covered by the same labour statutes as full-time workers and, in addition, by the Act on the Protection, etc. of Fixed-term and Part-time Employees (in this report, the Fixed-term and Part-time Employees Act). The provisions below determine the pay rules the system implements as business rules BR-06 to BR-10 and BR-15 (Chapter 3). The team read the statutes on the National Law Information Center [3], [4], [15] and the explanations of the government's legal information service for part-time workers [5]. The system is a tool for applying these rules consistently; it is not legal advice, and the rules were simplified in the ways stated below. Where the law leaves the method open, the system's rule is stated as a design decision of this project (Section 2.3.5).

2.3.1 Minimum wage

Under the Minimum Wage Act, the Minister of Employment and Labor announces the minimum hourly wage for each year. For 2026 the minimum hourly wage is KRW 10,320, up from KRW 10,030 in 2025, an increase of about 2.9 % [1], [2]. An employer who pays less than the minimum wage breaks the law regardless of what the contract says. For the system this means that the applicable minimum depends on the year of the pay period, so the minimum wage is stored as a table of yearly values (table MinimumWage, UC-10) rather than as a constant, and each payroll row is checked against the value for its year (BR-09).

2.3.2 Weekly paid holiday and the 15-hour threshold

Article 55 ① of the Labor Standards Act requires an employer to give a worker at least one paid holiday per week on average. Article 30 ① of the Enforcement Decree specifies that this paid holiday is given to a worker who has attended all the working days agreed for that week. Article 18 ③ excludes from this provision workers whose contractual working hours, averaged over four weeks, are less than 15 hours per week [3]. For a part-time worker the paid holiday is proportional to the worker's hours; the government's guidance computes the paid holiday hours as weekly contractual hours $\div 40 \times 8$, with the weekly hours capped at 40 [5]. The money paid for this holiday is what Korean workers call 주휴수당, rendered in this report as the weekly holiday allowance.

Two points required interpretation. The first is the four-week average. In the target stores a worker's contractual hours come from a fixed weekly schedule, which is the same every week; the four-week average therefore equals the weekly value, and the system uses the weekly contractual hours directly (BR-06). The second is a swapped shift. When a worker gives a shift to a colleague with the owner's approval, the working day has been changed by agreement, and the team's reading of [5] is that this should not count as a failure to attend. Because store practice differs, the system makes this a workplace setting (subAttendancePolicy = EXCUSED, the default, or ABSENT) rather than hard-coding one interpretation.

2.3.3 Premium pay and the five-employee threshold

Article 56 of the Labor Standards Act requires an additional 50 % of the ordinary wage for extended work (beyond 8 hours a day or 40 hours a week) and for night work between 22:00 and 06:00. Article 11 limits the application of the Act: a workplace that ordinarily employs fewer than five workers is subject only to the provisions listed in the Enforcement Decree, and Article 56 is not among them [3]. Many small cafés fall below this threshold; the demonstration store Dalbit Café is configured with four regular employees. The system therefore stores the number of regular employees as a workplace setting and computes premium pay only when it is five or more (BR-08). Holiday-work premium under Article 56 ② is outside the scope of the system. For part-time workers, the same threshold also governs the premium for hours beyond the contractual hours (Section 2.3.5).

2.3.4 Probation reduction

Article 5 ② of the Minimum Wage Act allows an employer to pay 90 % of the minimum wage during the first three months of employment for a worker under probation, but only if the employment contract is for one year or more, and not for workers in simple labour occupations designated by the Ministry of Employment and Labor (Enforcement Decree Art. 3) [4]. Many part-time jobs in cafés and convenience stores, such as serving and cashier work, fall into the designated simple labour occupations. The system requires all conditions to hold before it lowers the minimum-wage threshold for a worker (BR-10).

2.3.5 Work of part-time workers beyond contractual hours

The Fixed-term and Part-time Employees Act adds a rule that concerns substitutes directly. Like the premiums of Section 2.3.3, it applies only in workplaces with five or more regular employees (Art. 3 ②; Enforcement Decree Art. 2 and Annex 1). In such workplaces a part-time worker may work beyond their contractual hours only with their consent and for at most 12 hours a week, and these hours earn a premium of at least 50 % (Art. 6) [15]. Accepting a substitute shift usually creates such hours, so the rule bears on the owner's decision to approve a swap.

For counting these hours the system applies the following rules. They are design decisions of this project, taken where the law does not fix the method, and are not statements of law (BR-08 (iv) with its rules (a)–(e), and BR-15, in Chapter 3):

- a worker whose weekly contractual hours C , the sum of the fixed schedule, satisfy $0 < C < 40$ is treated as part-time; a worker without a fixed schedule ($C = 0$) or with $C \geq 40$ is not;
- the test is weekly against C , not daily: the hours of an ISO week are taken in chronological order, and every hour after the first C hours is beyond the contract;
- an hour that is both daily or weekly overtime (Section 2.3.3) and beyond the contract is counted once, and the night premium is added separately;
- the worker's consent is outside the system, and the 12-hour weekly limit is checked before approval: the approval panel warns when a swap would leave a part-time acceptor with more than 12 hours beyond the contractual hours in that week, without blocking the decision (BR-15).

Two other readings were considered and not adopted: a daily test against daily contractual hours, and the definition of part-time by comparison with a full-time worker doing the same kind of work in the same workplace. Both need data the system does not keep.

Table 7. Legal provisions and the business rules that implement them

Provision	Content used by the system	Business rule
Minimum Wage Act; MOEL notice for 2026 [1], [2]	Minimum hourly wage KRW 10,320 (2026), KRW 10,030 (2025)	BR-09
Labor Standards Act Art. 55 ①; Enforcement Decree Art. 30 ① [3]	One paid holiday per week for a worker who attended all agreed working days	BR-06, BR-07
Labor Standards Act Art. 18 ③ [3], [5]	No weekly paid holiday below 15 contractual hours per week (four-week average)	BR-06
Labor Standards Act Art. 11, Art. 56 [3]	+50 % for daily hours over 8, weekly hours over 40 and night hours 22:00–06:00; only with five or more regular employees	BR-08
Minimum Wage Act Art. 5 ②; Enforcement Decree Art. 3 [4]	90 % of the minimum wage for the first three months of probation, contract of one year or more, not simple labour	BR-10
Fixed-term and Part-time Employees Act Art. 6, Art. 3 ②; Enforcement Decree Art. 2, Annex 1 [15]	Work of a part-time worker beyond the contractual hours only with consent and for at most 12 hours a week; premium of at least 50 %; only with five or more regular employees (counting method: design decision, Section 2.3.5)	BR-08, BR-15

2.4 Related Work

Shift scheduling is an established product category. The team compared seven products on the basis of their own help pages and public descriptions, consulted on 29 September 2026: three international products, Deputy, When I Work and Homebase, and four Korean products, Shiftee, Shopl, Albami and Gig. The comparison records only what these pages state.

Scheduling products abroad already support a substitute workflow. In Deputy a worker can offer a shift to co-workers who are qualified and available; the manager is notified when the offer is accepted, and swaps can be set to require the manager's approval [10]. When I Work lets a worker offer a shift to all or selected eligible co-workers, subject by default to a manager's approval, and its Find Replacement function lets a manager offer a shift to eligible workers, and the first one to pick it up takes it [18], [19]. In Homebase a worker asks all or selected co-workers to cover a shift, and a manager gives final approval after one of them accepts [20]. Adopting these products in a small Korean store is not straightforward: Deputy's employee app is available in Korean, but managers who build schedules on its web interface must work in English [21], and the team found no mention of the Korean weekly holiday allowance in the help centres of the three products.

Korean products follow a different workflow, in which the manager edits the schedule or an employee sends a change request to the manager. Shiftee lets an employee request a new or changed shift, tags the request when it involves overtime,

night or holiday work, and computes the weekly holiday allowance automatically [22], [23]. Its help centre states that the allowance is recognised only when every contractual working day is worked, so that a worker contracted for Monday to Friday who works Tuesday to Saturday is not eligible [23]. Shopl lets employees request schedule changes that the manager approves or rejects [24]. Albami lets the owner manage recurring and daily schedules and set the holiday allowance for each worker [25], and Gig shares the owner's schedule with the workers and manages allowances including the holiday allowance [26]. The public feature descriptions of these four products do not include a substitute request between co-workers.

The table below summarises these statements. A dash means that the comparison does not cover the point for that product. The products change frequently, and the table reflects the pages as consulted on the date stated above.

Table 8. Comparison of shift-scheduling products (statements of the products' own help pages and public descriptions, consulted on 29 September 2026)

Product	Schedule changes	Substitute request between co-workers	Korean weekly holiday allowance	Sources
Deputy	A worker offers a shift to qualified and available co-workers; the manager is notified on acceptance; approval of swaps can be required	yes	no mention found in the help centre	[10]
When I Work	A worker offers a shift to all or selected eligible co-workers, by default subject to approval; with Find Replacement, the first eligible worker to pick up a shift offered by a manager takes it	yes	no mention found in the help centre	[18], [19]
Homebase	A worker asks all or selected co-workers to cover a shift; a manager gives final approval after one of them accepts	yes	no mention found in the help centre	[20]
Shiftee	An employee requests a new or changed shift; the request is tagged when it involves overtime, night or holiday work	not in the public feature description	computed automatically; recognised only when every contractual working day is worked	[22], [23]
Shopl	Employees request schedule changes that the manager approves or rejects	not in the public feature description	—	[24]
Albami	The owner manages recurring and daily schedules	not in the public feature description	set by the owner for each worker	[25]
Gig	The owner's schedule is shared with the workers	not in the public feature description	managed together with other allowances	[26]
ShiftSwap	A request reaches every free co-worker at once; the first acceptance wins; the owner approves or rejects (FR-07, FR-09, FR-12)	yes (BR-01, BR-02)	computed by BR-06 and BR-07; the change for both workers is shown at approval (FR-11); a handed-over shift is judged by a workplace setting	—

The team found no product that combines the co-worker substitute request with the Korean holiday-allowance rule, that is, one that shows the owner at the moment of approval how a swap changes the holiday-allowance eligibility of both workers. ShiftSwap adopts the substitute workflow that Deputy, When I Work and Homebase already offer and applies the Korean pay rules to it. Because a strict every-contract-day rule would also remove the allowance of a worker who hands a shift to a colleague, ShiftSwap makes the treatment of such a shift a workplace setting (Section 2.3.2, BR-06).

Besides products, the course material provides the methods used in this project. The life-cycle models and the requirements classification come from the course textbook [6] and from Sommerville's classification of non-functional requirements used in the lecture on requirements [7]. The UML diagrams follow the OMG UML 2.5.1 specification [8]. The shared data are kept in Supabase, a hosted PostgreSQL service with a free plan [16], and the working copy in the browser and the local demonstration mode use SQLite compiled to WebAssembly (sql.js) [9].

2.5 The Gap ShiftSwap Fills

ShiftSwap targets the part of the problem that the products compared in Section 2.4 do not address for small Korean stores. Its contribution is the combination of four elements in one workflow:

1. candidate selection from the existing shifts, so that a request reaches at once every co-worker who is not already working at that time, and ends with a clear message to the owner when none of them can take it (addresses P1). The proposal planned selection by recorded availability; this was replaced in iteration 6 (Section 3.11.4);
2. a first-acceptance-wins rule enforced in a single database transaction, so that two acceptances can never both succeed; in the shared database the server applies each change under a lock on the store and refuses a change computed on outdated data (addresses the double-acceptance variation of P1);
3. an approval step that shows, for both workers, contractual hours, scheduled hours and holiday-allowance eligibility before and after the swap, and recomputes the weekly summaries on approval (addresses P2);
4. pay rules taken from Korean law, including the premium for hours of part-time workers beyond their contractual hours, and stored as data where they change (minimum wage by year, number of regular employees, substitution-attendance policy), with warnings before payroll confirmation and, when a swap would leave a part-time acceptor with more than 12 hours beyond the contractual hours of the week, before approval (addresses P3).

The system is deliberately small. It serves one demonstration store, whose data are kept in a hosted shared database that the owner and every worker reach from their own phones; it has no application server of its own, keeps a local demonstration mode in the browser as a fallback, and does not replace a full payroll or HR system. Its value lies in making the rules explicit and applying them at the moment a decision is made, rather than at the end of the month. Chapter 3 turns these four elements into use cases, functional requirements and business rules.

3. Requirements Specification

This chapter specifies what ShiftSwap must do. It follows the requirements engineering activities of Chapter 8 of the lecture — elicitation, analysis, validation and management [6], [7] — and the contents recommended there for a software requirements specification: problem description, functional and non-functional requirements, scenarios, use-case diagram and supplementary diagrams. The problem description is Chapter 2. Section 3.1 explains how the requirements were elicited, and Section 3.2 identifies stakeholders and actors. Sections 3.3 to 3.5 present the use-case model, the functional requirements and a full description of each use case. Section 3.6 states the business rules, the domain requirements that carry the legal content of the system. Section 3.7 gives the non-functional requirements, Section 3.8 the scenarios, and Section 3.9 the analysis models. Sections 3.10 and 3.11 describe how the requirements were validated and how changes to them were managed.

All identifiers used in this chapter (A1–A3, UC-01–UC-14 with UC-02 retired, FR-01–FR-24 with FR-03 retired, NFR-01–NFR-16, BR-01–BR-15, TC groups) are defined in the specification file `spec/spec.md` of the repository, which the team treated as the single source of truth (Section 3.11). The wording of the requirements in this chapter is the wording of that file.

3.1 Requirements Elicitation

The lecture divides elicitation and analysis into requirements discovery, classification and organisation, prioritisation and negotiation, and specification [7]. For discovery the team used three sources.

3.1.1 Practice in the reference store

The first and main source was the practice of the reference store, described by Hyewon Kim from her own experience as a part-time worker there. She wrote down the as-is procedure of Section 2.1 step by step, including the variations (swaps arranged between workers, double agreement, the owner covering the shift). From this description the team extracted the entities the owner deals with — worker, fixed weekly shift, shift on a date, substitute request, the workers asked, the reply, attendance, weekly hours and monthly pay — and the decisions the owner makes, which became candidate use cases. This source is also the origin of the only quantitative statement about the current process used in the report, the figure of three to five contacts per substitute request (P1). The team did not collect further measurements in the store and does not report any.

3.1.2 Legal documents

The second source was the legislation and official guidance listed in Section 2.3: the Ministry of Employment and Labor notice on the 2026 minimum wage [1], [2], the Labor Standards Act and its Enforcement Decree [3], the Minimum Wage Act and its Enforcement Decree [4], the government's legal information service for part-time workers [5], and, from iteration 8, the Fixed-term and Part-time Employees Act and its Enforcement Decree [15]. These documents are the source of the domain requirements, which the lecture defines as constraints on the system from its domain of operation [7]. Each legal provision was converted into a business rule with explicit conditions and numbers (Section 3.6), and each business rule was reviewed against the text of the provision in the requirements review (Section 3.10). Where the law needed interpretation, the interpretation was written into the rule with a note (the four-week average of BR-06), made a workplace setting (the treatment of a handed-over shift, `subAttendancePolicy`), or stated as a design decision of the system that is not a statement of law (the counting of the hours of part-time workers beyond their contractual hours, BR-08 (iv) with its rules (a)–(e); Section 2.3.5).

3.1.3 Interview plan

The third source is a set of semi-structured interviews with the owner of the reference store and two co-workers, planned under WBS 3.1.1 (interview plan and notes) and scheduled to begin in Week 3. The interviews serve to confirm or correct what the team derived from the first two sources, not to produce statistics. The table below gives the plan. This report does not present interview results as numbers; where the requirements depend on a store-specific choice, the system makes it a setting instead of relying on the answer of one interviewee.

Table 9. Interview plan for requirements discovery (WBS 3.1.1)

Interviewee	Topic	Main questions	Requirements affected
Owner	Substitute procedure	How do you choose whom to contact first? What information do you need before agreeing to a swap? Would you accept a swap without seeing its effect on hours?	FR-06–FR-12, BR-01, BR-04
Owner	Pay calculation	How do you decide the weekly holiday allowance today? How do you treat a shift that a worker gave to a colleague? When do you update the hourly wage after a minimum-wage change?	FR-14–FR-16, BR-06–BR-11

Interviewee	Topic	Main questions	Requirements affected
Owner	Deadlines	How long before a shift must a substitute be confirmed? What happens when nobody answers?	FR-18, BR-03, BR-12
Co-workers (2)	Receiving requests	How do you prefer to be asked? How quickly do you usually answer? Have you ever agreed to a shift that someone else had already taken?	FR-07, FR-09, FR-10, NFR-01
Co-workers (2)	Availability and privacy	Would you record the times you are available for extra shifts? Which of your data should other workers not see?	FR-03 (retired in iteration 6), NFR-11
All	Devices	Which device do you use to check the schedule? Do you use a computer at the store?	NFR-02, NFR-06

3.1.4 Domain analysis and organisation

In addition, the team examined the public descriptions of existing shift-scheduling products (Section 2.4) to find what is common to the domain. Shift trades with manager approval appear in several products, which confirmed that owner approval (UC-06) is an expected step and not a peculiarity of the reference store. The candidate requirements from all sources were then classified into the groups used below (worker and schedule management, substitute workflow, attendance and pay, administration and access), prioritised with the MoSCoW scheme (Section 1.3), and specified in the specification file. One prioritisation decision needs explanation. Perfect attendance is a condition of the holiday allowance, which might suggest that attendance recording (UC-07) is a Must. It is a Should because the holiday-allowance rule treats a shift that has not been marked ABSENT as attended (Section 3.9.2); Increment 1 therefore computes a correct holiday allowance for every week without absences even before attendance recording exists.

3.2 Stakeholders and Actors

The lecture defines stakeholders as everyone with an interest in the system, including end users, managers, maintenance engineers and domain experts [7]. Actors are the subset of stakeholders, and of external systems, that interact with the system directly. The two tables below list the stakeholders and the three actors of the use-case model.

Table 10. Stakeholders of ShiftSwap

Stakeholder	Interest in the system	How represented in the requirements
Store owner	Fewer contacts per substitute; correct schedule; correct and lawful pay; no disputes on payday.	Actor A2; goals G1, G3, G4; FR-11, FR-12, FR-15, FR-16.
Part-time worker (requester)	Find a substitute quickly without personally asking colleagues; keep the holiday allowance where possible.	Actor A1; FR-06, FR-19; before/after comparison (FR-11).
Part-time worker (candidate)	Be asked only when not already working at that time; answer with one tap, including "can't"; know at once whether a shift is still free; privacy of wage and phone number.	Actor A1; BR-01, BR-02, BR-13; FR-22; NFR-01, NFR-11, NFR-15.
Legislator and labour authorities	Minimum wage, weekly paid holiday, premium pay and the limit on work of part-time workers beyond their contractual hours applied as the law requires.	Business rules BR-06 to BR-10 and BR-15; NFR-10.
Course instructor	A system that demonstrates the full life cycle and can be reproduced from the final prompt.	Organisational requirements NFR-07 to NFR-09; goals G5, G6.
Development team	A scope that can be delivered in one semester; a system that three students can test.	MoSCoW scope; static web application and hosted shared database without an application server of its own (NFR-06, NFR-08); local demonstration mode with the demo clock (A3, NFR-16).

Table 11. Actors of the use-case model

ID	Actor	Type	Description
A1	Worker	Primary, human	Part-time employee. Views own schedule, requests substitutes, accepts or declines ("can't") requests from co-workers, records attendance.
A2	Owner	Primary, human	Store owner (also acts as manager). Registers workers and fixed schedules, approves/rejects substitutions, confirms attendance, reviews weekly summaries, confirms monthly payroll, edits settings.
A3	System Clock	Secondary, system	Time-triggered actor. Expires unanswered/unapproved requests at their deadline, closes weeks for weekly summary, and on or after the first day of a month prepares the previous month's payroll draft (UC-14, from iteration 7). In the shared mode it uses the time of the shared database server (Korea Standard Time, read-only in the app); in the local demonstration mode it is driven by the adjustable "demo clock" (iteration 8).

A worker who receives a substitute request is still actor A1; the use-case descriptions call this worker the *candidate* and the worker who made the request the *requester*. A separate candidate actor is not used, because every worker can be a requester in one request and a candidate in another, and the difference is a role in a particular request, not a kind of user. The System Clock is modelled as an actor because expiry (UC-11) happens without any user action: it is triggered by the passage of time. In the shared mode, time is the time of the database server, which also enforces the deadlines itself (BR-12, NFR-14), so that no user can move it. In the local demonstration mode, time is the adjustable demo clock, so that expiry can be shown in a few seconds instead of waiting for a real deadline. Because the system has no application server of its own, the duties of the System Clock are carried out by whichever copy of the application is open, and their results are saved like any other change.

The owner could in principle be a specialisation of worker, since the owner is also stored in the table Worker (with role OWNER). In the use-case model the two are kept as separate actors because they have no use cases in common except sign-in (UC-13) and attendance (UC-07), and in attendance their steps differ: the worker records times, the owner confirms them.

3.3 Use-Case Model

The figure below shows the use-case diagram. It contains thirteen active use cases. UC-01 to UC-09 correspond one to one to the nine main functions announced in the proposal; UC-02 (register availability) was retired in iteration 6 (Section 3.11.4), and its identifier is kept but not reused. UC-10 to UC-13 are supporting use cases, already listed in the proposal, which make settings, expiry, cancellation and role selection explicit so that the core use cases are complete. UC-14, the automatic preparation of last month's payroll, was added in iteration 7 at the request of the product owner (Section 3.11.5). Iteration 8 added no use case but changed the descriptions of UC-06 (part-time overtime warning), UC-11 (server time) and UC-13 (no passwords, notice of shared data) (Section 3.11.6). The table after the diagram lists the use cases with their primary actor, priority and the database tables they mainly use.

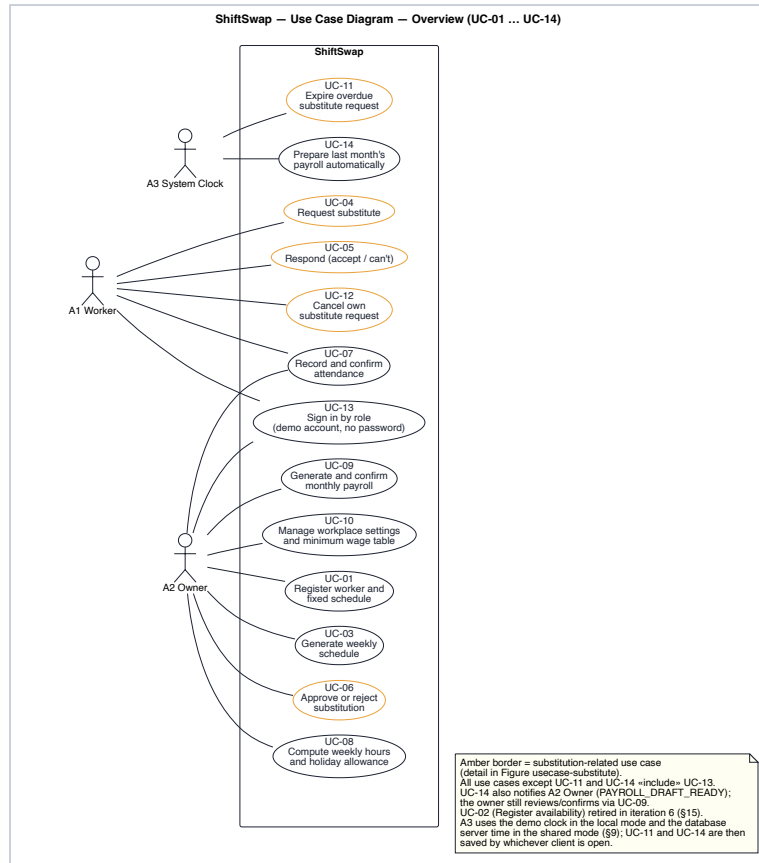


Figure 2. Use-case diagram of ShiftSwap (UML 2.5)

Table 12. Use cases of ShiftSwap

ID	Name	Primary actor	Priority	Main tables
UC-01	Register worker and fixed schedule	Owner	Must	Worker, FixedSchedule
UC-02	Register availability — retired in iteration 6	—	—	— (table Availability removed)
UC-03	Generate weekly schedule	Owner	Must	Shift
UC-04	Request substitute	Worker	Must	SubRequest, SubRequestTarget, Notification
UC-05	Respond to substitute request (accept / can't)	Worker (candidate)	Must	SubRequestTarget, SubRequest, Notification
UC-06	Approve or reject substitution	Owner	Must	SubRequest, Shift, WeeklySummary
UC-07	Record and confirm attendance	Worker, Owner	Should	Attendance, Shift
UC-08	Compute weekly hours and holiday allowance	Owner (System)	Must	WeeklySummary, Workplace
UC-09	Generate and confirm monthly payroll	Owner	Should	Payroll, MinimumWage, Worker
UC-10	Manage workplace settings and minimum wage table	Owner	Should	Workplace, MinimumWage
UC-11	Expire overdue substitute request	System Clock	Must	SubRequest, SubRequestTarget, Notification
UC-12	Cancel own substitute request	Worker	Should	SubRequest, SubRequestTarget
UC-13	Sign in by role (demo account selection, no password)	Worker, Owner	Must	Worker

ID	Name	Primary actor	Priority	Main tables
UC-14	Prepare last month's payroll automatically	System Clock	Must	Payroll, Notification

The lecture describes «include» as a common function that a use case always uses and «extend» as an exceptional condition that adds behaviour to a base use case [6]. The diagram uses both relationships as follows.

- **UC-04 «include» Find eligible candidates.** Candidate selection (BR-01) is always performed when a request is created. It is drawn as a separate included use case because it is the most important rule of the substitute workflow and because it has its own activity diagram (activity diagram AD-2 in Section 3.9).
- **UC-06 «include» UC-08.** Approving a substitution always recomputes the weekly summaries of the requester and the acceptor. The before/after comparison shown before approval uses the same computation.
- **UC-09 «include» UC-08.** The monthly payroll takes its holiday allowance from the weekly summaries, so they are recomputed when a payroll draft is generated.
- **UC-11 «extend» UC-04.** The extension point is "deadline reached with no approved acceptance". Expiry is the exceptional ending of a request, triggered by the System Clock.
- **UC-12 «extend» UC-04.** Cancellation is the other exceptional ending, triggered by the requester while the request is still open.
- **All use cases except UC-11 and UC-14 «include» UC-13.** Every user action requires that the user has signed in, and the role chosen at sign-in decides which use cases are available. UC-11 and UC-14 are the only use cases not started by a signed-in user; both are started by the System Clock.

In the diagram the Worker is also associated with UC-01, although the Owner is its primary actor. The association represents the worker as the subject of the registration, who afterwards sees the registered fixed schedule on the weekly board; the worker cannot create or change a fixed schedule.

3.4 Functional Requirements

The lecture defines functional requirements as statements of the services the system should provide, how it should react to particular inputs and how it should behave in particular situations [7]. Following the SRS contents of the lecture, each requirement below has an identifier, a group, a statement and the use case it belongs to; the priority is inherited from the use case. The statements use "shall" for mandatory behaviour. The table below is the complete list.

Table 13. Functional requirements

ID	Group	Requirement	UC	Priority
FR-01	Workers and schedules	The owner shall register a worker with name, phone, hourly wage (KRW), contract start date, contract end date, probation end date (optional), simple-labor flag, and active flag.	UC-01	Must
FR-02	Workers and schedules	The owner shall register one or more fixed weekly shifts per worker (weekday, start time, end time); fixed shifts of the same worker must not overlap. Deleting a fixed-schedule row keeps the shifts it already generated (their fixedScheduleId becomes NULL).	UC-01	Must
FR-03	—	Retired in iteration 6: availability slots are no longer collected (Section 3.11.4).	—	—
FR-04	Workers and schedules	The system shall generate the shifts of a given week (Mon–Sun) from all active fixed schedules; generating the same week twice shall not create duplicates.	UC-03	Must
FR-05	Workers and schedules	The owner shall be able to add, move or delete a single shift of a generated week. Deleting a shift that has a substitute request or an attendance record is refused with an explanatory message.	UC-03	Must
FR-06	Substitute workflow	A worker shall create a substitute request for one of their own future SCHEDULED shifts, with an optional reason and a response deadline.	UC-04	Must
FR-07	Substitute workflow	On creation the system shall compute the eligible candidates (BR-01), create one SubRequestTarget per candidate, and put a notification in each candidate's inbox at once.	UC-04	Must

ID	Group	Requirement	UC	Priority
FR-08	Substitute workflow	If no worker is eligible, the system shall still create the request and end it at once as FAILED (BR-13); the requester is told that no co-worker can take the shift and to contact the owner, and the owner receives a REQUEST_FAILED notification.	UC-04	Must
FR-09	Substitute workflow	A candidate shall accept a request, or answer can't (decline), from their inbox or Home. The first acceptance makes that candidate the request's acceptor (BR-02); later acceptances shall be refused with an explanatory message.	UC-05	Must
FR-10	Substitute workflow	When a request is accepted, the system shall notify the requester and the owner, and close all other pending targets.	UC-05	Must
FR-11	Substitute workflow	The approval screen shall show, for both the requester and the acceptor, the weekly contractual hours, weekly scheduled hours and holiday-allowance eligibility before and after the swap, and highlight any eligibility change.	UC-06	Must
FR-12	Substitute workflow	On approval the system shall reassign the shift to the acceptor (keeping the original worker), recompute the weekly summaries of both workers, and notify both. On rejection the shift stays unchanged and both are notified.	UC-06	Must
FR-13	Attendance and pay	A worker shall record actual start and end time of a shift they worked; the owner shall confirm it or mark the shift ABSENT.	UC-07	Should
FR-14	Attendance and pay	The system shall compute, per worker and week, contractual hours, scheduled hours, actual hours, perfect-attendance flag, holiday-allowance eligibility and holiday-allowance hours (BR-06, BR-07).	UC-08	Must
FR-15	Attendance and pay	The owner shall generate a monthly payroll draft per worker (base pay, holiday allowance, premium pay, total) and confirm it; a confirmed payroll is read-only.	UC-09	Should
FR-16	Attendance and pay	The payroll screen shall warn when a worker's hourly wage is below the applicable minimum wage (BR-09), and apply the probation reduction only when BR-10 holds.	UC-09	Should
FR-17	Administration	The owner shall edit workplace settings: store name, number of regular employees, substitution-attendance policy (BR-06), and yearly minimum wage rows.	UC-10	Should
FR-18	Substitute workflow	When a request's deadline passes, or its shift starts, while it is still REQUESTED or ACCEPTED, the system shall mark it EXPIRED, close its targets and notify requester and owner.	UC-11	Must
FR-19	Substitute workflow	A requester shall cancel their own request while it is REQUESTED or ACCEPTED; all PENDING targets become CLOSED, and every target that was PENDING or ACCEPTED receives a REQUEST_CANCELLED notification.	UC-12	Should
FR-20	Access	Users shall sign in by choosing a demo account without a password (password authentication is out of scope, Section 1.3), either from a guided three-step demo tour or from the list of all accounts; the UI shows only the menus of that role. In the shared mode the sign-in screen states that the data are shared with everyone who opens the app and must be fictional; the tour's progress follows the shared data, and restarting the tour there resets the demo data for everyone (the rate-limited reset for everyone, NFR-05).	UC-13	Must
FR-21	Access	Every user shall have an in-app notification inbox listing notifications newest first with unread count.	UC-04-06, 11, 12	Must

ID	Group	Requirement	UC	Priority
FR-22	Substitute workflow	When every target of a REQUESTED request has answered can't (DECLINED), the request ends as FAILED (BR-13): the requester receives "모든 동료가 대타가 불가능하다고 해요. 사장님께 연락드려 보세요." ("Every co-worker said they can't cover this shift. Please contact the owner.") and the owner receives a REQUEST_FAILED notification whose only action is 확인했어요 (acknowledge, which marks it read).	UC-05	Must
FR-23	Attendance and pay	On or after the first day of a month, the system shall prepare the previous month's payroll as a DRAFT (the same computation as FR-15) without any user action, notify the owner with PAYROLL_DRAFT_READY, and never overwrite rows that already exist; the owner still reviews, acknowledges warnings and confirms (FR-15, FR-16).	UC-14	Must
FR-24	Substitute workflow	In a workplace with five or more regular employees, the approval screen (FR-11) shall warn when the swap would leave a part-time acceptor with more than 12 hours beyond their weekly contractual hours in that week (BR-15), showing the hours beyond contract; the warning does not block the decision.	UC-06	Should

Of the 24 identifiers, 23 functional requirements are active; FR-03 was retired together with UC-02, FR-23 was added in iteration 7, and FR-24 in iteration 8. The priority of a requirement is that of its use case with one exception: FR-24, the part-time overtime warning, has Should priority within the Must use case UC-06, as the revised proposal specifies for function F6. Every active functional requirement maps to at least one use case, as the traceability rule of the specification requires (spec §12). The reverse also holds: every use case is realised by at least one functional requirement. UC-08 is realised by FR-14 alone, but the computation it describes is detailed by business rules BR-05 to BR-07. The notification inbox (FR-21) is not a use case of its own; it is the shared output of the five use cases that create notifications. The kinds of notification are fixed by the specification: REQUEST_RECEIVED, REQUEST_ACCEPTED, REQUEST_APPROVED, REQUEST_REJECTED, REQUEST_EXPIRED, REQUEST_CANCELLED, REQUEST_FAILED, TARGET_CLOSED and PAYROLL_DRAFT_READY. REQUEST_FAILED replaced the earlier NO_CANDIDATE in iteration 6, and PAYROLL_DRAFT_READY, which UC-14 sends to the owner, was added in iteration 7.

The following conventions apply to all requirements. Dates are ISO YYYY-MM-DD, times are HH:MM in 24-hour notation, and datetimes are YYYY-MM-DDTHH:MM. A time interval [s, e) includes its start and excludes its end, so a shift ending at 18:00 does not overlap one starting at 18:00. An end time that is not later than the start time means that the shift ends on the following day. Money amounts are integers in Korean won.

3.5 Use-Case Descriptions

The lecture notes that a use case is represented both in a diagram, as an overview, and in a more detailed textual form [6]. This section gives the textual form for every active use case, UC-01 and UC-03 to UC-14, in one fixed structure: identifier and name, actors, preconditions, trigger, main flow as numbered steps, alternative flows, exception flows, postconditions, the business rules applied, the related functional and non-functional requirements, and the screen of the application on which the use case is performed. The fields correspond to the scenario contents required by the SRS outline of the lecture (related actors, initial condition, final condition, exceptional case and step-by-step flow of action). Alternative flows describe other successful paths; exception flows describe paths on which the goal of the use case is not reached. The screens are described in Chapter 4 and shown in the user guide of Chapter 7.

Two conventions keep the descriptions short. "The system validates" always means that invalid input leaves the data unchanged and produces a message that states the cause and the correction, as required by the copy rules of the design. Every use case except UC-11 and UC-14 includes UC-13, which is therefore stated only as a precondition.

UC-01 Register worker and fixed schedule

Table 14. Use-case description UC-01

ID	UC-01
Name	Register worker and fixed schedule
Actors	Primary: Owner (A2). Secondary: Worker (A1), as the subject of the registration.
Preconditions	- The owner is signed in (UC-13). - The workplace exists (seed: Dalbit Café).
Trigger	A new worker is hired, or the contract or fixed weekly shifts of an existing worker change.

Main flow	1. The owner opens the <i>Staff</i> tab. 2. The system lists the workers of the workplace with their contract period, hourly wage and weekly contractual hours. 3. The owner selects <i>Add worker</i>. 4. The owner enters name, phone, hourly wage (KRW), contract start date, contract end date (empty for an open-ended contract), probation end date (optional), simple-labor flag and active flag. 5. The system validates the input: name present, hourly wage a positive whole number of won, contract start date present, contract end not before contract start, probation end not before contract start. 6. The system stores the worker with role WORKER and shows the worker in the list. 7. The owner adds a fixed weekly shift by choosing weekday, start time and end time. 8. The system checks that the new fixed shift does not overlap another fixed shift of the same worker (FR-02) and stores it. 9. Steps 7-8 are repeated for each fixed shift. The system shows the resulting weekly contractual hours (sum of fixed-shift hours, BR-06).
Alternative flows	• A1 Edit worker. At step 3 the owner selects an existing worker and changes its data; the flow continues at step 5. • A2 Deactivate worker. The owner clears the active flag. The worker is no longer a candidate for substitute requests (BR-01) and no new shifts are generated for them (FR-04); existing shifts and records remain. • A3 Remove a fixed shift. The owner deletes a fixed shift. Shifts already generated from it keep existing, and their link to the fixed schedule is cleared (FR-02); later weeks are generated without it.
Exception flows	• E1 Overlapping fixed shift. At step 8 the new fixed shift overlaps an existing one of the same worker. The system does not store it and names the conflicting weekday and times. • E2 Invalid data. At step 5 a required field is missing or a date is inconsistent. The system keeps the form open and states which field is wrong and how to correct it.
Postconditions	• The worker and the fixed schedule are stored. • The worker's weekly contractual hours are defined by the fixed schedule.
Business rules	BR-06 (contractual hours), BR-09 and BR-10 (wage, contract and probation fields used later)
Related FR / NFR	FR-01, FR-02; NFR-11 (phone and wage visible only to owner and the worker)
UI screen	Staff (직원): worker cards and worker detail with the fixed-schedule editor (owner)

UC-02 Register availability (retired)

Up to iteration 5, workers registered weekly availability slots in the *Me* tab, and BR-01 asked only workers whose slot covered the whole shift. UC-02 and its requirement FR-03 were retired in iteration 6, together with the table Availability, after the product owner found that the availability check led to requests that nobody could answer and that keeping the slots up to date was extra work for the workers (Section 3.11.4). The identifier UC-02 is kept retired and is not reused, following the change procedure of Section 3.11.3.

UC-03 Generate weekly schedule

Table 15. Use-case description UC-03

ID	UC-03
Name	Generate weekly schedule
Actors	Primary: Owner (A2).
Preconditions	• The owner is signed in (UC-13). • At least one active worker has a fixed schedule.
Trigger	The owner opens a week (Monday to Sunday, BR-05) that has not yet been generated, or wants to adjust a generated week.

Main flow	1. The owner opens the <i>Schedule</i> tab. 2. The owner navigates with <i>previous</i> / <i>next</i> to the week to be generated. 3. The owner selects <i>Create this week's schedule</i>, the primary action shown when the displayed week has no shifts. 4. For each active worker and each of the worker's fixed shifts, the system creates a shift on the corresponding date of the week with status SCHEDULED, unless a shift for that fixed schedule and date already exists. 5. The system shows the week as seven columns, Monday to Sunday, with shifts ordered by start time.
Alternative flows	• A1 Week already generated. At step 4 all shifts already exist; the system creates no duplicates (FR-04) and shows the week unchanged. • A2 Add a single shift. The owner adds a shift for a worker on a date and time outside the fixed schedule (FR-05). • A3 Move or delete a shift. The owner changes the date or time of a shift, or deletes it (FR-05).
Exception flows	• E1 Overlap after edit. An added or moved shift would overlap another shift of the same worker. The system refuses the change and names the conflicting shift. • E2 Shift in use. The owner tries to delete a shift that has a substitute request or an attendance record. The system refuses and suggests changing its time or worker instead (FR-05).
Postconditions	• All shifts of the week exist exactly once per fixed schedule and date. • New shifts have status SCHEDULED and no original worker.
Business rules	BR-05 (week definition)
Related FR / NFR	FR-04, FR-05; NFR-02, NFR-03
UI screen	Schedule (근무표), owner view, with the shift detail panel

UC-04 Request substitute

Table 16. Use-case description UC-04

ID	UC-04
Name	Request substitute
Actors	Primary: Worker (A1) as requester. Secondary: other Workers (A1) as candidates, Owner (A2) as receiver of notifications.
Preconditions	• The worker is signed in (UC-13). • The worker is the current worker of a shift that is SCHEDULED and starts in the future. • The shift has no open (REQUESTED or ACCEPTED) request.
Trigger	The worker learns that they cannot work one of their coming shifts.

Main flow	1. The worker starts the request flow with <i>Find a substitute</i> from the <i>Swaps</i> tab, from the next-shift card on Home, or from the detail panel of one of their own shifts (the shift is then preselected). 2. Step 1 of 3: the system lists the worker's own future SCHEDULED shifts; a shift that already has an open request is shown disabled with the reason. The worker chooses a shift. 3. Step 2 of 3: the worker chooses a reason (optional; chips or free text) and a deadline, preset to the earlier of now + 24 hours and the shift start; the chosen deadline is repeated as a sentence. 4. Step 3 of 3: the system finds the eligible candidates at this moment — every active co-worker who is not already working at that time (included use case <i>Find eligible candidates</i>, BR-01) — and shows a summary of shift, reason and deadline with the names of the recipients. 5. The worker selects <i>Send substitute request</i>. 6. The system validates the request (BR-03), creates it with status REQUESTED, determines the eligible candidates again inside the transaction, and creates one SubRequestTarget with response PENDING for each. 7. The system puts a REQUEST_RECEIVED notification in the inbox of every candidate at once. 8. The system shows a result screen with the progress tracker at its first step (Requested → Accepted → Approved) and the recipients, whose answers are shown as they arrive (waiting / can't / accepted). On the schedule the shift is marked with a dashed amber outline.
Alternative flows	• A1 No eligible candidate. At step 4 the review already warns that no co-worker is free at that time; at step 6 no worker is eligible. The system still creates the request, with no targets, and ends it at once as FAILED (BR-13). It sends REQUEST_FAILED to the requester and the owner and tells the requester to contact the owner (FR-08). • Extension point <i>nobody can take it</i>. When every candidate answers can't, the request ends as FAILED in UC-05 (FR-22). • Extension point <i>deadline reached with no approved acceptance</i>. UC-11 extends this use case when the deadline or the shift start passes while the request is open. • Extension point <i>requester withdraws</i>. UC-12 extends this use case when the requester cancels the open request.
Exception flows	• E1 Shift not eligible for a request. The shift belongs to another worker, is not SCHEDULED or has already started. The system refuses the request and states the reason. • E2 Open request exists. The shift already has a REQUESTED or ACCEPTED request. The system refuses and points to the existing request. • E3 Invalid deadline. The deadline is not after now or is later than the shift start. The system keeps the form open and states the allowed range.
Postconditions	• A request in state REQUESTED exists with one target per eligible candidate, or, without candidates, a request in state FAILED. • Each candidate has an unread REQUEST_RECEIVED notification, or the requester and the owner have a REQUEST_FAILED notification. • The shift itself is unchanged.
Business rules	BR-01, BR-03, BR-13
Related FR / NFR	FR-06, FR-07, FR-08, FR-21; NFR-11
UI screen	Swaps (대타) tab and the three-step request flow; Home next-shift card; Schedule shift detail (worker)

UC-05 Respond to substitute request (accept / can't)

Table 17. Use-case description UC-05

ID	UC-05
Name	Respond to substitute request (accept / can't)
Actors	Primary: Worker (A1) as candidate. Secondary: requester (A1) and Owner (A2), who are notified.

Preconditions	- The candidate is signed in (UC-13). - The candidate has a target with response PENDING for a request.
Trigger	A REQUEST_RECEIVED notification arrives (unread count on the bell icon; card on Home).
Main flow	- The candidate opens Home (or follows the notification from the bell icon). - Home shows, first, a card for each pending request: who asks, the shift, the reason, a deadline countdown and the effect on the candidate's own week (scheduled hours before → after, holiday allowance unchanged or changed), with <i>Accept</i> and <i>Can't</i>. - The candidate selects <i>Accept</i> (one tap). - Within one database transaction the system reads the request again and confirms that it is still REQUESTED (BR-02). - The system sets the request to ACCEPTED with the candidate as acceptor, sets the candidate's target to ACCEPTED and all other PENDING targets to CLOSED. - The system sends REQUEST_ACCEPTED to the requester and the owner, and TARGET_CLOSED to the other candidates whose targets were closed (FR-10). - The system confirms the result in a message that also says what happens next (the owner's approval).
Alternative flows	A1 Can't. At step 3 the candidate selects <i>Can't</i> (one tap). The target becomes DECLINED, and the card closes with a short confirmation. If other targets are still PENDING, the request stays REQUESTED and remains open for them, and the answer notifies no one. A2 Nobody can take it. If the answer of A1 leaves every target DECLINED, the request ends as FAILED (BR-13, FR-22). The requester receives REQUEST_FAILED with the message that all co-workers said they can't and that they should contact the owner; the owner receives REQUEST_FAILED as a card on Home whose only action is <i>Got it</i>.
Exception flows	E1 Already taken. At step 4 the request is no longer REQUESTED because another candidate accepted first. The transaction makes no change, and the card says that the other candidate was faster, naming them, and closes (FR-09). In the shared mode the two acceptances may be computed at the same time in two browsers; the server applies the first and refuses the second as computed on an outdated revision, and the second browser reloads the data and runs the acceptance again, which then ends in the same way (NFR-04). E2 Request closed. At step 4 the request has been cancelled, has expired or has failed. The system makes no change and shows the state of the request (REQUEST_CLOSED). E3 Candidate now busy. Since the request was created, the candidate has been given another shift that overlaps the requested one. The acceptance is refused (ACCEPTOR_BUSY) and nothing changes (BR-02).
Postconditions	- On success: request ACCEPTED, exactly one target ACCEPTED, no target PENDING; requester and owner notified. - On can't: the candidate's target is DECLINED; the request is unchanged, or FAILED if no target is left PENDING and none accepted.
Business rules	BR-02, BR-13
Related FR / NFR	FR-09, FR-10, FR-21, FR-22; NFR-01 (at most 2 clicks), NFR-04 (transaction), NFR-14 (server checks), NFR-15 (answers visible to others)
UI screen	Home (홈) request card; Swaps tab, received requests (worker); Home FAILED card (requester and owner)

UC-06 Approve or reject substitution

Table 18. Use-case description UC-06

ID	UC-06
Name	Approve or reject substitution
Actors	Primary: Owner (A2). Secondary: requester and acceptor (A1), who are notified.

Preconditions	- The owner is signed in (UC-13). - A request is in state ACCEPTED.
Trigger	The owner receives a REQUEST_ACCEPTED notification or opens Home, where accepted requests are listed first.
Main flow	- The owner opens Home. - Home lists a decision card for each ACCEPTED request; open REQUESTED requests are listed separately, read-only, with their deadline countdown and each candidate's response. - The decision card states the swap in one sentence ("Seoyeon's Wednesday shift is covered by Minh"). - For the requester and the acceptor, the system computes the weekly contractual hours, weekly scheduled hours, holiday-allowance eligibility and holiday-allowance amount for the week of the shift, before and after the swap (uses UC-08). - The card shows one impact line per person; a change of eligibility is highlighted in amber with the words "holiday allowance changes" (FR-11). In a workplace with five or more regular employees, if the acceptor is part-time and would work more than 12 hours beyond their weekly contractual hours in that week after the swap, the card also shows a warning with the hours beyond contract (FR-24, BR-15). <i>Details</i> opens the full before/after table. - The owner selects <i>Approve</i>. - The system confirms that the request is still ACCEPTED, assigns the shift to the acceptor, records the requester as the shift's original worker if none is recorded yet, and sets the request to APPROVED with the decision time (BR-04). - The system recomputes the weekly summaries of both workers (includes UC-08). - The system sends REQUEST_APPROVED to the requester and the acceptor. - On the schedule the shift shows the handover marker: the acceptor's name in bold, the original worker struck through, an amber left edge.
Alternative flows	A1 Reject. At step 6 the owner selects <i>Reject</i>. The request becomes REJECTED, the shift is unchanged, and REQUEST_REJECTED is sent to both workers. A2 Part-time limit warning shown. The warning of step 5 does not block the decision: the worker's consent and the owner's decision are outside the system. The owner may approve (step 6) or reject (A1).
Exception flows	E1 Request no longer ACCEPTED. Between steps 3 and 7 the request expired (UC-11) or was cancelled (UC-12). The system refuses the decision and shows the current state. E2 Acceptor now busy. Since accepting, the acceptor has been given another shift that overlaps the requested one. The approval is refused (ACCEPTOR_BUSY) and the request stays ACCEPTED, so the owner can reject it (BR-02).
Postconditions	- On approval: shift assigned to the acceptor, original worker kept; request APPROVED; weekly summaries of both workers up to date. - On rejection: request REJECTED; shift and summaries unchanged.
Business rules	BR-04, BR-06, BR-07, BR-15 (with BR-08 (a))
Related FR / NFR	FR-11, FR-12, FR-21, FR-24; NFR-03
UI screen	Home (홈) decision card with the <i>Details</i> table (owner); Schedule

UC-07 Record and confirm attendance

Table 19. Use-case description UC-07

ID	UC-07
Name	Record and confirm attendance
Actors	Primary: Worker (A1) and Owner (A2).

Preconditions	- The user is signed in (UC-13). - The shift has started ($\text{start} \leq \text{now}$). For recording, the shift is still SCHEDULED and the worker is its current worker.
Trigger	A shift has been worked, or the owner reviews the past shifts.
Main flow	- The worker opens the <i>Me</i> tab (or the "shifts to record" section of Home). - The system lists the worker's past shifts that have no attendance record. - The worker enters the actual clock-in and clock-out times and saves. - The system stores the attendance record as unconfirmed. - The owner opens Home, section "attendance records to confirm". - The system lists unconfirmed records with the scheduled and the recorded times. - The owner confirms a record. - The system marks the record confirmed and sets the shift to WORKED; the worker's weekly summary is recomputed.
Alternative flows	A1 Mark absent. At step 7 (or for a shift with no record) the owner marks the shift ABSENT. The shift status becomes ABSENT, which breaks perfect attendance for that week (BR-06). A2 Correct a record. Before confirmation the worker changes the recorded times.
Exception flows	E1 Incomplete times. A time is missing or clock-in equals clock-out. The system does not store the record and explains the correction. E2 Already closed. Once the owner has confirmed the record or marked the shift absent, the worker can no longer change it. E3 Nothing to confirm. The owner tries to confirm a shift the worker has not recorded. The system asks the owner to have it recorded or to mark the shift absent.
Postconditions	- The shift is WORKED with a confirmed record, or ABSENT. - Actual hours are available for the weekly summary and the payroll.
Business rules	BR-06 (perfect attendance), BR-11 (actual hours for base pay)
Related FR / NFR	FR-13; NFR-12 (no location data; self-reported and owner-confirmed)
UI screen	Me (내 정보) and Home (worker); Home (owner)

UC-08 Compute weekly hours and holiday allowance

Table 20. Use-case description UC-08

ID	UC-08
Name	Compute weekly hours and holiday allowance
Actors	Primary: Owner (A2); the computation is performed by the system and is also included by UC-06 and UC-09.
Preconditions	The owner is signed in (UC-13).
Trigger	The owner opens the weekly summary in the <i>Pay</i> tab, or UC-06 / UC-09 includes this use case.

Main flow	1. The owner opens the <i>Pay</i> tab, segment <i>Weekly summary</i>, and selects a week (identified by its Monday, BR-05). 2. For each worker who has a shift in the week, as current or as original worker, the system computes the contractual hours from the fixed schedule. 3. The system computes the scheduled hours from the shifts currently assigned to the worker in the week, and the actual hours from confirmed attendance. 4. The system determines perfect attendance from the shifts whose original worker is the worker, applying the workplace's substitution-attendance policy (BR-06). 5. The system determines holiday-allowance eligibility and holiday-allowance hours (BR-06, BR-07). 6. The system stores one weekly summary per worker and week, replacing an earlier one. 7. The system shows the table with the columns of FR-14 and, for each worker who is not eligible, a short reason such as "contract hours below 15" or "absent on Tue 09-29".
Alternative flows	• A1 Included by UC-06. Only the requester and the acceptor are computed, once with the current assignment (before) and once with the shift reassigned (after); after approval the result is stored. • A2 Included by UC-09. All weeks whose Sunday falls in the payroll month are computed before the payroll draft is built.
Exception flows	• E1 Week not generated. The selected week has no shifts, so no worker has a summary for it. The system says that there are no shifts in the week and offers to open the week on the schedule (UC-03).
Postconditions	• Weekly summaries of the week reflect the current shifts, attendance and settings.
Business rules	BR-05, BR-06, BR-07
Related FR / NFR	FR-14; NFR-03, NFR-10
UI screen	Pay (급여), weekly summary segment (owner); Home decision card

UC-09 Generate and confirm monthly payroll

Table 21. Use-case description UC-09

ID	UC-09
Name	Generate and confirm monthly payroll
Actors	Primary: Owner (A2).
Preconditions	• The owner is signed in (UC-13). • The minimum-wage table has a row for the year of the month or an earlier year.
Trigger	The end of a pay month, or the owner wants to check the pay of the current month.
Main flow	1. The owner opens the <i>Pay</i> tab, segment <i>Monthly payroll</i>, and selects a month. 2. The owner selects <i>Create September payroll draft</i> (the month named in the button). 3. The system recomputes the weekly summaries of the weeks whose Sunday falls in the month (includes UC-08). 4. For each worker with shifts or holiday pay in the month the system computes base hours and base pay (worked hours for WORKED shifts, scheduled hours for SCHEDULED shifts, nothing for ABSENT shifts), holiday pay, premium pay and total (BR-11, BR-08). 5. The system checks each worker's hourly wage against the applicable minimum wage, taking the probation reduction into account only when all its conditions hold (BR-09, BR-10). 6. The system stores one DRAFT row per worker, replacing earlier DRAFT rows of the month. 7. The system shows the table with the flags <i>estimated</i> and <i>below minimum wage</i>. 8. The owner acknowledges each minimum-wage warning; the acknowledgement is stored on the row (minWageAck = 1). 9. The owner selects <i>Confirm</i>. 10. The system sets the rows to CONFIRMED; they are read-only from now on.

Alternative flows	• A1 No warning. If no row has a minimum-wage warning, step 8 is skipped. • A2 Regenerate. The owner generates the month again before confirming; only DRAFT rows are replaced, so acknowledgements given on the old draft must be given again (BR-09). • A3 Draft already prepared. On or after the first day of the following month, the System Clock has already performed steps 3–6 (UC-14). The owner is told by the PAYROLL_DRAFT_READY notification and on Home, opens the prepared draft and continues at step 8.
Exception flows	• E1 Warning not acknowledged. At step 9 a warning is still unacknowledged. Confirmation is blocked and the row is highlighted. • E2 Month already confirmed. Confirmed rows are not replaced by regeneration and cannot be edited. • E3 Minimum wage not yet updated. The table has no row for the year of the month. The system uses the latest earlier year, which may be too low; the owner adds the new year in Settings (UC-10) and generates the draft again.
Postconditions	• Payroll rows for the month exist in DRAFT or CONFIRMED state. • No confirmed row has an unacknowledged minimum-wage warning.
Business rules	BR-07, BR-08, BR-09, BR-10, BR-11
Related FR / NFR	FR-15, FR-16; NFR-10
UI screen	Pay (급여), monthly payroll segment (owner); Home payroll-warning section

UC-10 Manage workplace settings and minimum wage table

Table 22. Use-case description UC-10

ID	UC-10
Name	Manage workplace settings and minimum wage table
Actors	Primary: Owner (A2).
Preconditions	• The owner is signed in (UC-13).
Trigger	The store changes (name, number of regular employees), the owner decides the substitution-attendance policy, or a new minimum wage is announced.
Main flow	1. The owner opens <i>Settings</i> from the header menu. 2. The system shows store name, number of regular employees, substitution-attendance policy (EXCUSED or ABSENT) and the minimum-wage table by year. 3. The owner changes a value or adds a minimum-wage row (year, hourly amount). 4. The system validates the input and stores it. 5. The system confirms the change; computations made afterwards use the new values.
Alternative flows	• A1 Update an existing year. The owner changes the amount of an existing year; the row is updated.
Exception flows	• E1 Invalid value. The number of regular employees is negative, or the hourly amount is not a positive integer. The system refuses the value and explains the valid range.
Postconditions	• Settings and minimum-wage table are stored. • Existing CONFIRMED payroll rows are not changed.
Business rules	BR-06 (policy), BR-08 (regular employees), BR-09 (minimum wage)
Related FR / NFR	FR-17; NFR-10
UI screen	Settings (설정), from the header menu (owner)

UC-11 Expire overdue substitute request

Table 23. Use-case description UC-11

ID	UC-11
Name	Expire overdue substitute request
Actors	Primary: System Clock (A3). Secondary: requester (A1), candidates (A1) and Owner (A2), who are notified.
Preconditions	At least one request is REQUESTED or ACCEPTED.
Trigger	The application starts, the user navigates to another screen, or, in the local demonstration mode, the demo clock is changed.
Main flow	- The system reads the current time from the clock: in the shared mode the server time (Korea Standard Time), in the local demonstration mode the demo clock. - The system selects all REQUESTED or ACCEPTED requests whose deadline $\leq$ now or whose shift start $\leq$ now (BR-12). - For each selected request the system sets the status to EXPIRED and closes its PENDING targets without notifying them. - The system sends REQUEST_EXPIRED to the requester and the owner. - In the shared mode the copy of the application that performed steps 1–4 saves the result to the shared database like any other change.
Alternative flows	A1 Nothing overdue. At step 2 no request qualifies; the use case ends without change.
Exception flows	E1 Accepted but not approved. A request that was ACCEPTED but not decided by the owner before the deadline also expires; the swap does not take place and the shift stays with the requester. E2 Expiry refused by the server. In the shared mode the server accepts EXPIRED only when the deadline or the shift start has passed by server time (BR-12). If it refuses the change, the application reloads the shared data and pauses the System Clock duties for a minute; the request stays open.
Postconditions	- No open request has a deadline or shift start in the past. - The shifts of expired requests are unchanged.
Business rules	BR-12
Related FR / NFR	FR-18, FR-21; NFR-14 (server time)
UI screen	None of its own (effects visible on Home, in the Swaps tab and in the notification list)

UC-12 Cancel own substitute request

Table 24. Use-case description UC-12

ID	UC-12
Name	Cancel own substitute request
Actors	Primary: Worker (A1) as requester. Secondary: candidates (A1), who are notified.
Preconditions	- The requester is signed in (UC-13). - The requester has a request in state REQUESTED or ACCEPTED.
Trigger	The requester can work the shift after all, or has arranged the matter otherwise.

Main flow	1. The requester opens the <i>Swaps</i> tab, section "sent requests" (or Home). 2. The system shows each open request as a progress tracker with the candidates' answers so far. 3. The requester selects <i>Cancel request</i> on an open request. 4. The system sets the request to CANCELLED and sets every PENDING target to CLOSED. 5. The system sends REQUEST_CANCELLED to every target that was PENDING or ACCEPTED; targets already DECLINED or CLOSED are not notified. 6. The list shows the request as CANCELLED; the shift stays with the requester.
Alternative flows	• A1 Accepted request. If the request was ACCEPTED, the acceptor's target keeps the response ACCEPTED, and the acceptor receives the REQUEST_CANCELLED notification, so the acceptor knows that the swap will not take place.
Exception flows	• E1 Request already decided. The request is APPROVED, REJECTED, FAILED or EXPIRED. The <i>Cancel</i> action is not offered.
Postconditions	• The request is CANCELLED and no target is PENDING.
Business rules	BR-03 (only one open request per shift; cancelling frees the shift for a new request)
Related FR / NFR	FR-19, FR-21
UI screen	Swaps (대타) tab, sent requests; Home (worker)

UC-13 Sign in by role (demo account selection, no password)

Table 25. Use-case description UC-13

ID	UC-13
Name	Sign in by role (demo account selection, no password)
Actors	Primary: Worker (A1) and Owner (A2).
Preconditions	• The application is loaded in the browser.
Trigger	A user opens the application or signs out.
Main flow	1. The system shows the sign-in screen as a demo guide: a title, a note that the demo has no passwords, a notice on the data mode (in the shared mode: everyone who opens the page sees and changes the same fictional data, and no personal information may be entered; in the local demonstration mode: the data are kept in this browser only), a three-step tour card (① view Lee Seoyeon's request, ② accept it as Choi Minho, ③ approve it as the owner Park Jiyoung), and the list of all five accounts with role and one-line summary. 2. The user selects a tour step, which signs in as that person and opens the screen where the step happens, or selects any account from the list. 3. The system keeps the chosen account for the browser session and shows only the navigation of that role (FR-20).
Alternative flows	• A1 Switch account or sign out. The user selects <i>View as someone else</i> in the header or <i>Sign out</i> in the menu; the account list opens without changing any data. • A2 Reset demo data. In the local demonstration mode the user selects <i>Reset demo data</i> in the <i>Demo tools</i> panel and confirms; the system restores the seed data and the demo clock (NFR-05). In the shared mode the action is <i>Reset the demo data for everyone</i>: after confirmation the server restores the seed data, moved to the current date, for every user; it allows this at most once a minute and 50 times a day (NFR-05, NFR-14). • A3 Switch the data mode. The user switches between the shared mode and the local demonstration mode in the <i>Demo tools</i> panel or on the sign-in screen; the choice is remembered in the browser.

Exception flows	• E1 Session ended. A new browser tab or session has no chosen account; the sign-in screen is shown. • E2 Shared database unavailable. The shared database cannot be reached when the application starts. The system says so and offers the local demonstration mode with the seed data (NFR-16).
Postconditions	• A user with a role is active, and the menus shown correspond to that role.
Business rules	—
Related FR / NFR	FR-20; NFR-05, NFR-11, NFR-14, NFR-16
UI screen	Sign-in demo guide; header account switch; Demo tools panel

UC-14 Prepare last month's payroll automatically

Table 26. Use-case description UC-14

ID	UC-14
Name	Prepare last month's payroll automatically
Actors	Primary: System Clock (A3). Secondary: Owner (A2), who is notified and later confirms the draft (UC-09).
Preconditions	• The previous month P is not earlier than the month of the earliest contract start. • No Payroll row exists yet for P.
Trigger	The application starts, the user navigates to another screen, or, in the local demonstration mode, the demo clock is changed, and the current time (in the shared mode the server time) is on or after the first day of a month (BR-14).
Main flow	1. The system reads the current time and determines the previous month P. 2. The system checks that P is not earlier than the month of the earliest contract start and that P has no Payroll row. 3. The system recomputes the weekly summaries of the weeks whose Sunday falls in P and builds one DRAFT row per worker, exactly as in UC-09, steps 3–6 (BR-06 to BR-11). 4. The system sends one PAYROLL_DRAFT_READY notification to the owner. 5. The owner's Home shows a card saying that last month's payroll draft is ready, with an action that opens it (the owner continues with UC-09, step 8).
Alternative flows	• A1 Already prepared. At step 2 P already has rows, DRAFT or CONFIRMED, because the draft was prepared earlier in the month or generated by the owner. The use case ends without change, so the clock running again later in the month creates no second draft and no second notification. • A2 Before the first contract. P lies before the earliest contract start; nothing is prepared.
Exception flows	• E1 Minimum-wage warning. A row of the draft carries a minimum-wage warning. The draft is still prepared; confirmation remains blocked until the owner acknowledges the warning (BR-09).
Postconditions	• P has a DRAFT payroll, or had rows already; nothing has been confirmed. • The owner has at most one PAYROLL_DRAFT_READY notification for P.
Business rules	BR-14; BR-06 to BR-11 (through the payroll computation)
Related FR / NFR	FR-23, FR-15, FR-16, FR-21
UI screen	None of its own (effects visible on the owner's Home and in Pay → Monthly)

3.6 Business Rules

The business rules are the domain requirements of ShiftSwap. They state precisely, with conditions and numbers, what the functional requirements refer to by name: which workers are eligible, when an acceptance succeeds, when the holiday allowance is owed, and how pay is computed. The lecture warns that a system that does not satisfy its domain requirements

may be unworkable [7]; for this system, a wrong rule means a wrong pay slip. The rules are therefore written so that each can be implemented as one pure function in the domain-rule module (Chapter 4) and tested on its own (Section 3.10).

The table below lists the fifteen rules. The column "Rule" repeats the wording of the specification. The column "Rationale" explains why the rule has this form, and the column "Source" names the law or the store practice it comes from.

Table 27. Business rules with rationale and source

ID	Rule	Rationale	Source
BR-01	Eligible candidates. A worker <i>W</i> is eligible for a request on shift <i>S</i> (date <i>d</i> , start <i>s</i> , end <i>e</i>) iff: <i>W</i> is an active WORKER; <i>W</i> ≠ requester; <i>W</i> has no shift on date <i>d</i> overlapping [<i>s</i> , <i>e</i>) (someone already working then cannot take it). Availability is not used (removed in iteration 6); every other co-worker is asked.	Every co-worker who could take the shift is asked at once, which removes the owner's search (P1); whether they can is their own answer (BR-13) rather than a slot the system guesses from. A worker already working at that time is excluded because a yes would be impossible. Half-open intervals let back-to-back shifts (e.g. 12:00–18:00 and 18:00–23:00) coexist.	Store practice (Section 2.1, steps 2–3); product-owner review in iteration 6
BR-02	First acceptance wins. Acceptance succeeds only if the request is REQUESTED at the moment of the transaction and the candidate still has no overlapping shift on that date (re-checked at acceptance and again at approval, error ACCEPTOR_BUSY); it sets request.status = ACCEPTED, request.acceptorId = <i>W</i> , target(<i>W</i>).response = ACCEPTED, and all other PENDING targets → CLOSED.	A broadcast request can be answered by several candidates at nearly the same time. The rule makes the outcome deterministic and avoids the double agreement observed in the current process. The state check must be inside the transaction; a check before it would allow a race.	Store practice (double agreement, Section 2.1.2); NFR-04
BR-03	Request validity. Only the shift's current worker can request; shift must be SCHEDULED and start in the future; at most one open (REQUESTED/ACCEPTED) request per shift; deadline must be after now and no later than the shift start.	Prevents requests on someone else's shift, on past or completed shifts, and competing requests for one shift. A deadline after the shift start would be meaningless because the shift would already have to be covered.	Workflow consistency
BR-04	Approval effect. Approve: shift.workerId = acceptor; shift.originalWorkerId keeps the first worker (set only if null); request → APPROVED. Reject: request → REJECTED, shift unchanged. Owner can approve only an ACCEPTED request.	The owner stays responsible for the schedule, as in the current process. Keeping the first worker as original worker lets the holiday-allowance rule count a given-away shift for the right person, even after a chain of swaps.	Store practice (owner decides); BR-06 needs the original worker
BR-05	Week. A week runs Monday 00:00 to Sunday 24:00 (ISO week). Weekly records are keyed by the Monday date.	The weekly holiday allowance is defined per week; a fixed week definition makes weekly summaries unambiguous and gives each one a single key.	ISO 8601 week [11]; Labor Standards Act Art. 55 [3]
BR-06	Holiday-allowance eligibility (주휴). Eligible iff (a) weekly contractual hours ≥ 15, and (b) perfect attendance: every shift of the week whose <i>original</i> worker is <i>W</i> and that was not given away was not ABSENT (a shift that is still SCHEDULED does not break attendance, so the weekly result is a projection until the week is over). Shifts <i>W</i> gave away by an APPROVED substitution count according to Workplace.subAttendancePolicy: EXCUSED (default — the working day was changed by agreement, so it does not break attendance) or ABSENT (it breaks attendance). Contractual hours = sum of <i>W</i> 's fixed-schedule hours per week. Substitute shifts <i>W</i> took do not change contractual hours.	Implements the two legal conditions. Contractual hours come from the contract, not from the shifts actually worked, so taking extra substitute shifts cannot create an entitlement and giving one away cannot remove it. The law averages 15 h over four weeks; the fixed schedule is constant, so the average equals the weekly value. The treatment of a handed-over shift is not stated in the law and is therefore a setting.	Labor Standards Act Art. 55 ①, Enforcement Decree Art. 30 ①, Art. 18 ③ [3]; [5]

ID	Rule	Rationale	Source
BR-07	Holiday-allowance hours. hours = $\min(\text{contractualHours}, 40) / 40 \times 8$, rounded to 2 decimals; pay = hours $\times$ Worker.hourlyWage, rounded down to the won per week (monthly holiday pay is the sum of the rounded weekly amounts).	Proportional paid holiday for part-time workers, capped at one full 8-hour day. Rounding down money avoids overstating an amount by fractions of a won; hours are rounded to 2 decimals so that the displayed and the computed value agree.	[5]; Labor Standards Act Art. 18 ① [3]
BR-08	Premium pay. Applies only if Workplace.regularEmployees ≥ 5 . Then +50 % of wage for (i) daily hours beyond 8, (ii) weekly hours beyond 40 not already counted in (i), (iii) night hours 22:00–06:00, and (iv) for a part-time worker, hours beyond the weekly contractual hours not already counted in (i) or (ii) (iteration 8). Night premium adds to overtime premium. If regularEmployees $< 5 \rightarrow$ premium = 0. In a monthly payroll, weekly overtime (ii) uses only the days of that ISO week that fall inside the month. Premium pay = premium hours $\times$ wage $\times 0.5$, rounded down. <i>System rules for (iv), design decisions of this system where the law does not fix the method, not statements of law:</i> (a) a worker whose weekly contractual hours C (the sum of the current fixed schedule, as in BR-06) satisfy $0 < C < 40$ is treated as part-time; a worker without a fixed schedule ($C = 0$) or with $C \geq 40$ is not; (b) the test is weekly against C, not daily; (c) the hours of an ISO week are taken in chronological order and every hour after the first C hours is beyond contract; the hours of a shift count on its work date; in a monthly payroll the days of that week outside the month only determine where the first C hours end, and only hours on dates inside the month are paid in that month; (d) an hour that meets more than one of (i), (ii) and (iv) is counted once, so the overtime hours of a part-time worker on a day are the larger of the daily hours beyond 8 and the hours beyond contract; (e) the night premium (iii) is added separately. The worker's consent to hours beyond contract is outside the system; the 12-hour weekly limit is checked before approval (BR-15).	Small stores below the threshold are exempt, but a store can grow; making the number of employees a setting keeps the rule correct in both cases. Counting weekly overtime only beyond what daily overtime already covers avoids paying the same hour twice; restricting a week to its days inside the month keeps each hour in one payroll. Item (iv) implements the premium for work of part-time workers beyond their contractual hours; the law does not state how these hours are counted, so rules (a)–(e) are the system's own and are marked as such. Considered and not adopted: a daily test against daily contractual hours, and the definition of part-time by comparison with a full-time worker doing the same kind of work in the same workplace; both need data the system does not keep.	Labor Standards Act Art. 11, Art. 56 [3]; Fixed-term and Part-time Employees Act Art. 3 ②, Art. 6, Enforcement Decree Art. 2, Annex 1 [15]; rules (a)–(e): design decision (iteration 8)
BR-09	Minimum-wage check. Warn if worker.hourlyWage $<$ MinimumWage(year).hourly $\times$ (0.9 if BR-10 applies else 1). The payroll still computes with the worker's wage; the warning is shown on the payroll row and blocks confirmation until the owner acknowledges it, which sets Payroll.minWageAck = 1. Re-generating a DRAFT month replaces its rows, so acknowledgements must be given again.	The system cannot change a contract, but it must stop an unlawful wage from being paid unnoticed (P3). Blocking confirmation until acknowledgement forces a conscious decision without hiding the actual amount. Storing the acknowledgement on the row keeps it across reloads; because a regenerated draft may contain different amounts, it must be acknowledged again.	Minimum Wage Act; 2026 notice [1], [2]

ID	Rule	Rationale	Source
BR-10	Probation reduction. The minimum may be reduced to 90 % only if all hold: contract length ≥ 1 year (contractEnd – contractStart ≥ 365 days, or contractEnd empty = open-ended), the date is within 3 months of contractStart and $\leq$ probationEnd, and simpleLabor = false. In a monthly payroll the conditions are evaluated on the first day of the month, or on contractStart if later. "Within 3 months" means from contractStart (inclusive) to the same calendar date three months later (exclusive); a worker without probationEnd is never on probation.	All legal conditions are required together; each is a separate field of the worker record so that the rule can be checked mechanically. An open-ended contract counts as one year or more. A monthly payroll needs one evaluation date; the first day of the month (or the contract start, if later) is the date on which the month's wage begins to apply.	Minimum Wage Act Art. 5 ②, Enforcement Decree Art. 3 [4]
BR-11	Monthly payroll. Month = calendar month. Base pay = Σ actual hours of WORKED shifts in the month $\times$ wage (recorded clock-in/out); SCHEDULED shifts in the month count with their scheduled hours and set estimated = 1 on the row; ABSENT shifts count 0. Holiday allowance = Σ holiday pay of weeks whose Sunday falls in the month. Total = base + holiday + premium. Amounts are integers in KRW, rounded down.	Wages are paid monthly while the holiday allowance is weekly; assigning each week to the month of its Sunday makes every week count exactly once. The "estimated" flag lets the owner check pay during the month without mistaking it for the final amount; an absent shift earns nothing.	Store practice (monthly pay)
BR-12	Expiry. A REQUESTED or ACCEPTED request whose deadline $\leq$ now, or whose shift start $\leq$ now, becomes EXPIRED. Accept and approve check this themselves inside their transaction (REQUEST_CLOSED), independent of the periodic expiry run. In the shared mode "now" is the server time: the server refuses an acceptance or approval at or after the deadline or the shift start, and an expiry before it (NFR-14).	An open request must not remain open after it can no longer be useful. Expiring ACCEPTED requests as well means that an owner who does not decide in time leaves the shift with the original worker, which is the safe default. With several devices, only the server's clock gives every user the same "now".	Workflow consistency
BR-13	No taker. A REQUESTED request ends as FAILED (final) when it has no eligible candidate at creation, or when every SubRequestTarget has response DECLINED. FAILED sends REQUEST_FAILED to the requester and the owner; the owner's only action is to acknowledge (mark the notification read). Accepting a FAILED request is refused (REQUEST_CLOSED).	Once everyone asked has said no, waiting for the deadline only delays the moment at which the owner has to act. Ending the request at once tells the requester to call the owner and gives the owner a clear task, the same one as in the current process but without the search. The owner's card has no other action because the system has nothing left to try.	Product-owner review in iteration 6 (Section 3.11.4)
BR-14	Automatic monthly payroll draft. Whenever the System Clock runs (app start, every route change, after the demo clock changes), let M be the month of now and $P = M - 1$. If P is not earlier than the month of the earliest contractStart and no Payroll row exists yet for P, generate the DRAFT for P exactly as generatePayroll does and insert one PAYROLL_DRAFT_READY notification for the owner. It runs at most once per month (existing rows — DRAFT or CONFIRMED — mean it has already run) and never confirms anything.	The owner asked that last month's pay be ready without a manual step. Using the existence of rows as the "already done" marker makes the rule idempotent without a separate flag, and never overwriting rows protects both confirmed pay and a draft the owner has already acknowledged. Confirmation stays manual because a minimum-wage warning needs a conscious decision (BR-09).	Product-owner request in iteration 7 (Section 3.11.5)

ID	Rule	Rationale	Source
BR-15	Part-time weekly limit warning (F6, iteration 8). When Workplace.regularEmployees ≥ 5 and the acceptor of an ACCEPTED request is part-time ($0 < C < 40$, BR-08 (a)), let S be the acceptor's scheduled hours in the ISO week of the requested shift after the swap (every shift of the week of which they would be the current worker, as WeeklySummary.scheduledHours). If $S - C > 12$, the approval preview (FR-11, FR-24) shows the warning with $S - C$ (for example "13 h beyond contract, limit 12 h"). Exactly 12 hours beyond contract gives no warning. The warning does not block approval: the worker's consent and the owner's decision are outside the system.	Accepting a substitute shift usually creates hours beyond contract, and approval is the last moment at which the owner can still decide otherwise. The system does not know whether the worker has consented, so it warns and does not block. Scheduled hours are used because the hours will be worked in the future.	Fixed-term and Part-time Employees Act Art. 6 [15]; weekly test against the fixed-schedule hours: system rule of BR-08 (a)–(c); revised proposal, F6 (Section 3.11.6)

A worked example shows how BR-06 and BR-07 combine. Choi Minho has fixed shifts on Tuesday and Thursday 18:00–23:00 and Saturday 12:00–18:00, which is $5 + 5 + 6 = 16$ contractual hours per week. Since $16 \geq 15$, condition (a) holds. If he attended all his shifts, his holiday-allowance hours are $\min(16, 40) / 40 \times 8 = 3.2$ hours, and at his wage of KRW 10,320 the holiday pay is $3.2 \times 10,320 = \text{KRW } 33,024$. If he also takes Lee Seoyeon's Wednesday shift, his scheduled hours for that week rise from 16 to 21, but his contractual hours and his holiday pay stay the same. Lee Seoyeon's contractual hours are 10, so she is not eligible either way. This example is used throughout the report and is reproduced by the application. A second worked example, for BR-08 (iv) and BR-15, is scenario S-9 in Section 3.8.

3.7 Non-Functional Requirements

Non-functional requirements are constraints on the services of the system, such as timing, reliability, process and standards, and they often apply to the system as a whole [7]. Following the classification used in the lecture, the table below groups them into product requirements (how the delivered product must behave), organisational requirements (consequences of the policies of the developing organisation, here the course) and external requirements (factors outside the system and its development, here the law). Each requirement is stated so that it can be verified; the last column names the verification method used in Chapter 6.

Table 28. Non-functional requirements (Product / Organisational / External)

ID	Class	Sub-class	Requirement	Verification
NFR-01	Product	Usability	A candidate can accept a request in at most 2 clicks from the inbox.	UI walkthrough
NFR-02	Product	Usability	All screens are usable at 360 px width (mobile) and 1440 px (desktop) without horizontal scrolling.	Browser check at both widths
NFR-03	Product	Efficiency / Performance	With 20 workers and 12 weeks of shifts, any screen renders in under 1 second on a mid-range laptop.	Timed load with seeded data
NFR-04	Product	Reliability	"First acceptance wins" is enforced inside one database transaction that re-checks the request state; two acceptances can never both succeed. In the shared mode the server applies each change set in one transaction under a row lock on the store and refuses a change set computed on an older revision, which the app then recomputes on the current data.	Unit tests TC-05x, TC-16x; REST test of the shared database
NFR-05	Product	Reliability	Data persists across page reloads — in the shared database (shared mode) or in browser storage (local demonstration mode) — and can be reset to the seed data (in the shared mode by the rate-limited reset for everyone).	Manual test; REST test
NFR-06	Product	Portability	Runs in current Chrome, Edge, Safari, Firefox without installation and without an application server of its own (the shared mode uses only the hosted database service).	Manual test

ID	Class	Sub-class	Requirement	Verification
NFR-07	Organisational	Delivery	Working system deployed by Week 14 (2026-12-04).	Deployed URL
NFR-08	Organisational	Implementation	Static web application (HTML/CSS/JavaScript ES modules) hosted on GitHub Pages; shared data in a Supabase PostgreSQL database reached with the publishable key through its REST and RPC interface; SQLite compiled to WebAssembly (sql.js) as the working copy in the browser (shared mode) and as the only store of the local demonstration mode; built by vibe coding from the final prompt.	Repository
NFR-09	Organisational	Standards	Models follow UML 2.5 notation; code identifiers in English.	Review
NFR-10	External	Legislative	Pay rules follow the Minimum Wage Act (2026 minimum hourly wage KRW 10,320), the Labor Standards Act (Art. 18 ☺, Art. 55, Enforcement Decree Art. 30; Art. 11 / 56 premium threshold) and the Fixed-term and Part-time Employees Act (Art. 6, work of part-time workers beyond contractual hours); where the law leaves the method open, the system's rule is stated as a design decision (BR-08, BR-15).	Unit tests TC-08x, TC-09x, TC-15x
NFR-11	External	Privacy	A worker's phone number and wage are visible only to the owner and that worker (Personal Information Protection Act principle of minimum disclosure).	Role test
NFR-12	External	Ethical	The system does not collect location data; attendance is self-reported and owner-confirmed.	Design review
NFR-13	Product	Usability	Task-first design: each role starts on a Home screen that lists the items waiting for that person with the action on the item; every screen has exactly one primary action; any action that changes hours or pay shows its effect (hours before → after, holiday-allowance amount) before the user commits; the default UI language is Korean with a complete English alternative.	Headless check counts one primary button per screen; walkthrough
NFR-14	Product	Security	The shared database accepts writes only through SECURITY DEFINER functions with an empty search_path (app_commit, app_reset); the API roles may only read the app_ tables (row-level-security read policies, explicit grants, no write grants). The server itself re-checks what it can decide without knowing the user: the state transitions of the request, acceptance and approval only before the deadline and the shift start, expiry only when overdue (BR-12, server time), at most one open request per shift, read-only confirmed payroll, creation times within 10 minutes of server time. It limits every store to 60 change sets per minute and 3,000 per day, a change set to 256 kB and 2,000 rows, a store to 20,000 rows, and resets to one per minute and 50 per day. Identity is not verified (demo accounts without passwords).	REST test of the shared database (iteration 8)
NFR-15	Product	Efficiency	In the shared mode a change saved by one user appears on the open screen of every other user within 10 seconds (revision polling, and at once when the window regains focus).	Two-browser check
NFR-16	Product	Dependability	The shared database is kept active by a scheduled read query every six hours (GitHub Actions, .github/workflows/supabase-keepalive.yml); when it cannot be reached, the app offers the local demonstration mode with the seed data (risk R6).	Workflow log; manual test

Several of these requirements shape the design more than any single functional requirement. NFR-08 fixes the technology: a static web application without an application server of its own, whose shared data are kept in a hosted PostgreSQL database and whose browser holds a working copy in SQLite. Because no application server stands between the browser and the data, the database must protect the data itself: NFR-14 allows the API only to read, lets changes pass only through two functions that re-check the rules the server can decide, and limits the rate and the size of changes. Sign-in remains a choice of demo account (FR-20) rather than a password check, as the revised proposal specifies, so NFR-14 guarantees integrity but not identity, and all shared data are fictional. NFR-05 requires persistence in the shared database or, in the local demonstration mode, in browser storage, with a reset to seed data; NFR-15 bounds the delay after which one user's change appears on another user's screen; and NFR-16 answers risk R6 of the proposal, the pausing of an inactive free-plan database [16], [17], with a scheduled read query and the local mode as fallback. NFR-04 turns the business rule BR-02 into a reliability requirement with a concrete mechanism, one transaction that re-checks the state; in the shared mode the row lock and the revision check of the server extend this mechanism across browsers. NFR-01 and NFR-02 follow from the users: candidates answer on their phones, often between classes, and the owner works both on a phone and on a computer. NFR-13 was added in iteration 4 (Chapter 5), after a usability review of the running application: it turns the design principles of the final user interface — a Home screen per role that lists what is waiting, one primary action per screen, the effect on hours and pay shown before the user commits, and Korean as the default language — into a checkable requirement. NFR-12 records a deliberate choice. Location-based clock-in would make attendance harder to falsify, but it would collect personal data that the problem does not require, and the reference store does not use it today.

The lecture observes that non-functional requirements may be more critical than functional ones, because a system that violates them may be useless [7]. For ShiftSwap the most critical one is NFR-10: a system that computes pay faster but not lawfully would make problem P3 worse, not better. For this reason NFR-10 is verified by unit tests at the rule level rather than by inspection of screens.

3.8 Scenarios

A scenario is a real-life example of how the system can be used, and according to the lecture it should include the starting situation, the normal flow of events, what can go wrong, other concurrent activities, and the state when the scenario finishes [7]. The nine scenarios below use the demonstration data of the specification: Dalbit Café (regular employees 4, substitution-attendance policy EXCUSED), the owner Park Jiyoung and the workers Lee Seoyeon, Choi Minho, Jung Hana and Kang Doyun. The first table below summarises the workers; the demo clock starts at Monday 2026-09-28 09:00, and the weeks beginning 2026-09-21 and 2026-09-28 have been generated, the first with confirmed attendance for all shifts. These dates are those of the local demonstration mode. In the shared mode the clock is the server time, and the same seed is moved by whole weeks so that the seed request is still open; weekdays, times and the course of each scenario stay the same, and only the dates differ (spec §10). Since iteration 7 the seed also contains each worker's work history: every fixed shift from the contract start up to 2026-09-20 is worked and confirmed, and the payroll of every month up to August 2026 is confirmed. Together the scenarios cover every use case at least once and include the concurrent acceptance and the expiry of a request.

Table 29. Workers of the demonstration data (seed)

Worker (id)	Wage (KRW)	Contract	Fixed shifts (contractual h/week)
Lee Seoyeon (2)	10,500	2026-03-02 ~ 2027-02-28	Mon, Wed 18:00–23:00 (10 h)
Choi Minho (3)	10,320	2026-06-01 ~ 2026-12-31	Tue, Thu 18:00–23:00; Sat 12:00–18:00 (16 h)
Jung Hana (4)	10,320	2026-09-01 ~ open-ended; probation end 2026-11-30; not simple labour	Fri 17:00–23:00; Sun 10:00–16:00 (12 h)
Kang Doyun (5)	10,000	2026-08-15 ~ 2026-11-15; simple labour	Sat 10:00–16:00; Sun 16:00–22:00 (12 h)

S-1 Requesting a substitute for an exam evening (UC-13, UC-04)

Starting situation. Lee Seoyeon has a midterm exam on Thursday morning and cannot work her fixed shift on Wednesday 2026-09-30, 18:00–23:00. No request exists for that shift.

Normal flow. Seoyeon opens ShiftSwap on her phone and signs in by choosing her demo account. In the *Swaps* tab she starts *Find a substitute*, chooses her Wednesday shift in step 1, gives the reason "Midterm exam" and sets the deadline to Tuesday 2026-09-29 21:00 in step 2, and in step 3 sees that the request will go to Choi Minho, Jung Hana and Kang Doyun. The system checks the request (BR-03) and then checks every worker against BR-01. None of the three co-

workers has a shift on Wednesday evening, so all three are eligible. The system creates the request with three targets and puts a notification in the inboxes of Minhø, Hana and Doyun. Seoyeon sees the result screen with the tracker at "Requested" and the three recipients, and the Wednesday shift on the schedule now has a dashed amber outline.

What can go wrong. If Seoyeon had tried to set the deadline after the shift start, the flow would have refused it and shown the allowed range. If she had already had an open request for the shift, step 1 would have shown the shift disabled with the reason.

Concurrent activities. None; the owner is not involved at this stage and receives no notification because candidates exist.

Final state. Request REQUESTED with targets for Minhø, Hana and Doyun, all PENDING. This is the state stored in the seed data.

S-2 Two candidates accept at nearly the same time (UC-05)

Starting situation. The request of S-1 is REQUESTED. Choi Minhø, Jung Hana and Kang Doyun see the request card on their Home screens.

Normal flow. Minhø taps *Accept* on the request card on his Home screen, which also tells him that his week will go from 16 to 21 scheduled hours with the holiday allowance unchanged (NFR-01, NFR-13). In one transaction the system reads the request, finds it REQUESTED, sets it to ACCEPTED with Minhø as acceptor, sets Minhø's target to ACCEPTED and the targets of Hana and Doyun to CLOSED. Seoyeon and the owner Park Jiyoung receive REQUEST_ACCEPTED; Hana and Doyun receive TARGET_CLOSED.

Concurrent activities. Doyun had opened his Home screen before Minhø's acceptance and still sees the *Accept* button. He taps it a moment after Minhø. His transaction reads the request again, finds it ACCEPTED instead of REQUESTED, and changes nothing. His card now says that Minhø was faster and closes.

What can go wrong. Without the re-check inside the transaction, both acceptances could succeed and two workers would come to the same shift — the double agreement of the current process. BR-02 and NFR-04 exclude this, and test group TC-05x tests it.

The same case in the shared mode. Minhø and Doyun use their own phones, and each application runs the acceptance on its working copy of the store at the same revision. Minhø's change set reaches the shared database first; the server applies it under the lock of the store row and increments the revision. Doyun's change set was computed on the previous revision, so the server refuses it as stale; his application reloads the current data and runs the acceptance again, which now finds the request ACCEPTED and ends with the message that Minhø was faster. Independently of the revision check, the server allows the state ACCEPTED only from REQUESTED (NFR-04, NFR-14; test TC-163).

Final state. Request ACCEPTED, acceptor Choi Minhø; Minhø's target ACCEPTED, those of Hana and Doyun CLOSED.

S-3 The owner approves the swap after checking its effect (UC-06, UC-08)

Starting situation. The request of S-2 is ACCEPTED. Park Jiyoung has an unread REQUEST_ACCEPTED notification.

Normal flow. Jiyoung signs in as owner. Her Home screen starts with a decision card for Seoyeon's request, with one impact line per person; *Details* opens the full comparison for the week of 2026-09-28:

Table 30. Before/after comparison shown to the owner in scenario S-3

Worker	Contractual h	Scheduled h (before → after)	Eligible (before → after)	Holiday h
Lee Seoyeon (requester)	10	10 → 5	no → no	0
Choi Minhø (acceptor)	16	16 → 21	yes → yes	3.2

No eligibility changes, so no line is highlighted. Jiyoung selects *Approve*. The system assigns the Wednesday shift to Minhø, records Seoyeon as its original worker, sets the request to APPROVED, recomputes both weekly summaries and notifies both workers. On the weekly board the shift now shows Minhø's name in bold, Seoyeon's name struck through, and the amber handover edge.

What can go wrong. If Jiyoung had waited until after Tuesday 21:00, the request would have expired (S-4 shows this case) and *Approve* would not be offered. The demonstration walkthrough S7 of the specification shows the case in

which the comparison changes the decision. With the policy set to ABSENT, Choi Minhø requests a substitute for his Saturday 2026-10-03 shift, 12:00–18:00; Lee Seoyeon and Jung Hana are eligible, and Kang Doyun is excluded because his own Saturday 10:00–16:00 shift overlaps. When Seoyeon accepts, the panel shows Minhø's eligibility changing from yes to no and highlights that line, because under the ABSENT policy a given-away shift breaks his perfect attendance for the week of 2026-09-28.

Concurrent activities. Seoyeon could cancel the request (UC-12) while Jiyoung has the card open; the approval would then be refused because the request is no longer ACCEPTED.

Final state. Request APPROVED; shift worker Choi Minhø, original worker Lee Seoyeon; Minhø's holiday allowance for the week remains 3.2 h (KRW 33,024).

S-4 A request expires without an answer (UC-04, UC-11)

Starting situation. The demo clock shows Monday 2026-09-28 09:00. Jung Hana cannot work her Friday shift on 2026-10-02, 17:00–23:00.

Normal flow. Hana requests a substitute without changing the proposed deadline, which is the earlier of now + 24 h and the shift start, i.e. Tuesday 2026-09-29 09:00. Under BR-01 Lee Seoyeon, Choi Minhø and Kang Doyun are eligible, since none of them has a shift on Friday. They receive the notification, but none of them responds. On Tuesday the owner uses +1 day in the *Demo tools* panel, which moves the clock to 2026-09-29 09:00. The System Clock actor runs the expiry check (UC-11); the deadline is ≤ now, so the request becomes EXPIRED, the three targets are closed, and Hana and the owner receive REQUEST_EXPIRED.

What can go wrong. If one of them had accepted but the owner had not approved before the deadline, the request would expire in state ACCEPTED in the same way (BR-12), and the shift would stay with Hana. The scenario uses the local demonstration mode, in which the clock can be moved. In the shared mode the clock is the server time and cannot be moved: the request expires at its real deadline, and the server refuses an acceptance made after it.

Concurrent activities. The request of S-1 to S-3 is independent; each request expires on its own deadline.

Final state. Request EXPIRED; the Friday shift is still Hana's. Hana may create a new request for the same shift, because no open request exists any more.

S-5 Nobody can take the shift (UC-04, UC-05, UC-12; demonstration walkthrough S8)

Starting situation. Jung Hana cannot work her shift on Sunday 2026-10-04, 10:00–16:00. No request exists for it.

Normal flow. Hana creates a request. Under BR-01 Lee Seoyeon, Choi Minhø and Kang Doyun are eligible: Seoyeon and Minhø have no Sunday shift, and Doyun's Sunday shift 16:00–22:00 begins when Hana's ends and does not overlap it. All three receive the request. Each of them taps *Can't* on the card on their Home screen; the first two answers notify no one. With the third answer every target is DECLINED, so the request ends as FAILED (BR-13, FR-22). Hana sees the card "Every co-worker said they can't cover this shift. Please contact the owner." The owner's Home shows the card "No one can cover" for Hana's Sunday shift, whose only action is *Got it*; acknowledging removes it.

What can go wrong. If Hana finds that she can work after all while answers are still missing, she cancels the request in the *Swaps* tab (UC-12), and it becomes CANCELLED. If nobody answers at all, the request expires at its deadline (BR-12). In no case does it stay open. If no co-worker had been free at the time of the shift, the request would have ended FAILED at once, when it was created (FR-08).

Concurrent activities. A candidate who has not yet answered could still accept; the request then continues as in S-2, and the earlier answers of can't have no effect.

Final state. Request FAILED, all three targets DECLINED; the shift is still Hana's, and the owner knows that the matter needs a personal call.

S-6 Reviewing the weekly holiday allowance (UC-07, UC-08)

Starting situation. All shifts of the week of 2026-09-21 have confirmed attendance (seed data). The owner wants to know who is owed a weekly holiday allowance for that week.

Normal flow. Jiyoung opens the weekly summary in the *Pay* tab and selects the week of 2026-09-21. The system computes, for each worker, contractual, scheduled and actual hours and applies BR-06 and BR-07. Lee Seoyeon (10 h), Jung

Hana (12 h) and Kang Doyun (12 h) are below 15 contractual hours; each row shows "contract hours below 15". Choi Minh (16 h) worked all his shifts and is eligible with 3.2 holiday hours.

What can go wrong. Suppose instead that Minh had not come on Tuesday 2026-09-22 and the owner had marked that shift ABSENT (UC-07). Perfect attendance would fail, Minh would not be eligible, and his row would show "absent on Tue 09-22".

Concurrent activities. A worker may still be recording attendance for the current week; this does not affect the closed week of 2026-09-21.

Final state. Weekly summaries of the week of 2026-09-21 stored for all four workers; only Minh is eligible.

S-7 Monthly payroll with a minimum-wage warning (UC-09, UC-10)

Starting situation. The owner prepares the September 2026 payroll on Monday 28 September. The months up to August are confirmed (seed history), and the minimum-wage table holds 10,030 for 2025 and 10,320 for 2026.

Normal flow. Jiyoung opens the monthly payroll in the *Pay* tab, selects September 2026 and creates the draft. Four weeks have their Sunday in September — the weeks ending 6, 13, 20 and 27 September — so Minh's holiday pay for September is $4 \times \text{KRW } 33,024 = \text{KRW } 132,096$ (BR-11). With base pay for 69 hours (KRW 712,080), his total is KRW 844,176. The other workers have no holiday allowance because their contractual hours are below 15. Lee Seoyeon's total is KRW 472,500 for 45 hours, Jung Hana's KRW 495,360 for 48 hours and Kang Doyun's KRW 480,000 for 48 hours; the month's total is KRW 2,292,036. The shifts of 28 to 30 September have not yet been worked, so they count with their scheduled hours and the affected rows are flagged "estimated". Premium pay is zero for everyone, because the workplace has 4 regular employees (BR-08). The system then checks each wage. Kang Doyun earns KRW 10,000, below the 2026 minimum of 10,320; the probation reduction does not apply to him, because his contract (2026-08-15 to 2026-11-15) is shorter than one year and his work is simple labour (BR-10). His row therefore carries a minimum-wage warning. Jung Hana's contract is open-ended, she is within three months of her contract start and before her probation end, and her work is not simple labour, so all BR-10 conditions hold and her threshold is 90 % of 10,320 = KRW 9,288; her wage of 10,320 is above it. Seoyeon (10,500) and Minh (10,320) are not below the minimum. Jiyoung acknowledges Doyun's warning, which is stored on his row (`minWageAck` = 1), decides to correct his wage with him, and confirms the payroll; the rows become read-only.

What can go wrong. If Jiyoung tries to confirm without acknowledging the warning, confirmation is blocked (BR-09). If she waits until 1 October, she does not need to create the draft at all: the System Clock prepares it (UC-14, scenario S-8).

Concurrent activities. If the café hired a fifth regular employee and the owner changed the setting in Settings (UC-10) before generating the draft, premium pay would apply; each seed evening shift from 18:00 to 23:00 contains one night hour (22:00–23:00), which would then earn an additional 50 % of the wage.

Final state. September 2026 payroll CONFIRMED for four workers; Doyun's minimum-wage warning acknowledged.

S-8 Last month's payroll is prepared automatically (UC-14, UC-09; demonstration walkthrough S9)

Starting situation. The demo clock shows a day in September 2026. In *Pay* → *Monthly* the owner can page back through every month since the first contract, March 2026, and sees March to August as confirmed and read-only; no payroll exists for September.

Normal flow. In the *Demo tools* panel the clock is moved to 2026-10-01. The System Clock runs (UC-14): the previous month, September, has no payroll rows, so the system prepares its DRAFT exactly as the owner's *Create draft* would and sends `PAYROLL_DRAFT_READY` to the owner. Jiyoung's Home shows the card "9월 급여 초안이 준비됐어요" ("September payroll draft is ready") with an action that opens *Pay* → *Monthly* for September. There she sees the draft, marked as prepared automatically on the first of the month, with Kang Doyun's below-minimum warning, which she must acknowledge before she can confirm (UC-09).

What can go wrong. If the owner had already created the September draft herself in September, the System Clock would find rows for September on 1 October and prepare nothing (BR-14). If the draft contained a mistake in the underlying data, the owner could correct it and regenerate the draft before confirming, as in UC-09.

Concurrent activities. The clock runs again on every screen change during October; because September now has rows, no second draft and no second notification are created. In the shared mode the draft is prepared by the first copy of the application that runs on or after the first of the month by server time, and every other user then finds the rows already present.

Final state. September 2026 payroll in DRAFT for all workers with shifts in September, one PAYROLL_DRAFT_READY notification for the owner, nothing confirmed.

S-9 A swap takes a part-time worker beyond the weekly limit (UC-10, UC-03, UC-05, UC-06; demonstration walkthrough S10)

Starting situation. The request of S-1 is REQUESTED. The owner Park Jiyoung sets the number of regular employees to 5 in Settings (UC-10) and adds a single shift for Choi Minhoo on Friday 2026-10-02, 10:00–18:00 (UC-03, alternative flow A2). Minhoo's week of 2026-09-28 then has 24 scheduled hours against 16 contractual hours; he is part-time under BR-08 (a), because $0 < 16 < 40$.

Normal flow. Minhoo accepts Lee Seoyeon's Wednesday request (UC-05). Jiyoung's decision card shows Minhoo's week going from 24 to 29 scheduled hours and, because the workplace has five regular employees, the warning of BR-15 that Minhoo would work 13 hours beyond contract this week while the limit for part-time workers is 12 hours a week ($29 - 16 = 13 > 12$; FR-24). The warning does not block the decision; whether Minhoo consents is outside the system, and Jiyoung decides whether to approve.

Worked example (BR-08 (iv)). If the 29 hours are worked as scheduled, the chronological order Tuesday 5, Wednesday 5 (the swapped shift), Thursday 5, Friday 8 and Saturday 6 reaches the 16 contractual hours one hour into Friday. The remaining 7 hours on Friday and the 6 hours on Saturday are beyond contract, 13 hours in total, all on dates in October. No day exceeds 8 hours, so no hour is also daily overtime. October's premium therefore includes $13 \times 10,320 \times 0.5 = \text{KRW } 67,080$ for these hours; the night hour 22:00–23:00 of each evening shift is paid separately under item (iii) (tests TC-09A, TC-09C).

What can go wrong. With the Friday shift 10:00–17:00 instead, the card shows $23 \rightarrow 28$ hours, exactly 12 hours beyond contract, and no warning. With 4 regular employees, the setting of the seed, there is no warning and no premium, because BR-08 and BR-15 apply only from five regular employees (tests TC-153 to TC-155).

Concurrent activities. Another candidate could accept first; the card would then evaluate BR-15 for that worker instead of Minhoo.

Final state. If Jiyoung approves, the request is APPROVED and Minhoo holds 29 scheduled hours in the week, 13 of them beyond contract; his contractual hours and his holiday allowance for the week (3.2 h) are unchanged. If she rejects, the shift stays with Seoyeon.

The table below shows which use cases each scenario exercises; the retired UC-02 has no column. Every active use case appears in at least one scenario, and the Must use cases appear in several.

Table 31. Coverage of use cases by scenarios

Scenario	UC-01	03	04	05	06	07	08	09	10	11	12	13	14
S-1	(seed)	(seed)	●									●	
S-2				●								●	
S-3					●		●					●	
S-4			●							●		●	
S-5			●	●							●	●	
S-6						●	●					●	
S-7							●	●	●			●	
S-8								●				●	●
S-9		●		●	●		●		●			●	

UC-01 and UC-03 are exercised through the seed data, which is the result of registering four workers with their fixed schedules and generating two weeks; S-9 also adds a single shift (UC-03, A2). The user guide in Chapter 7 walks through these two use cases on the screens.

3.9 Analysis Models

The use-case descriptions state the flow of each use case, but three parts of the system are easier to check in a diagram: the candidate selection of BR-01, the holiday-allowance decision of BR-06 and BR-07, and the life cycle of a substitute request.

The lecture lists activity diagrams and state diagrams among the supplementary diagrams of an SRS [7]. This section presents these three analysis models. The class, sequence and remaining state diagrams belong to the design and are presented in Chapter 4.

3.9.1 Activity diagram AD-2: candidate selection

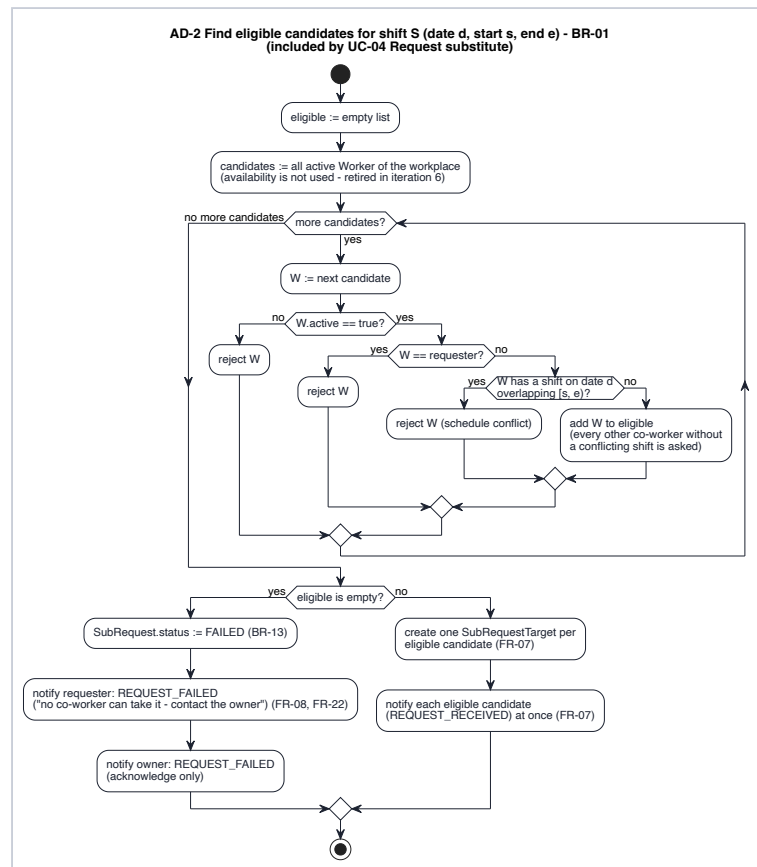


Figure 3. Activity diagram AD-2 — find eligible candidates for a shift (BR-01), included by UC-04

The figure above shows the activity included by UC-04. The system starts with an empty list and examines every active worker of the workplace in turn; the owner is not among them. A worker is rejected at the first failed condition, in the order of BR-01: inactive, the requester, or a conflicting shift on the date of the request. A worker who passes all three checks is added to the list; since iteration 6 there is no availability check. After the loop the diagram branches: an empty list ends the request as FAILED and sends REQUEST_FAILED to the requester and the owner (FR-08, BR-13); otherwise one target is created per candidate and all candidates are notified at once (FR-07). The notification step is outside the loop, which is what makes the notification simultaneous rather than sequential — the difference between the new process and the loop of processes 2.0 and 3.0 in the data flow diagram of Section 2.1.3.

The table below traces the activity for the seed request of scenario S-1 (Wednesday 2026-09-30, 18:00–23:00). The owner Park Jiyoung is also a row of the table Worker; her role is OWNER, so she is never a candidate.

Table 32. Trace of AD-2 for the seed request (Wednesday 2026-09-30, 18:00–23:00)

Worker	Active worker	Not requester	No conflicting shift	Result
Park Jiyoung (owner)	no (role OWNER)	—	—	rejected
Lee Seoyeon	yes	no	—	rejected
Choi Minho	yes	yes	yes (no Wednesday shift)	eligible
Jung Hana	yes	yes	yes (no Wednesday shift)	eligible
Kang Doyun	yes	yes	yes (no Wednesday shift)	eligible

3.9.2 Activity diagram AD-1: holiday-allowance determination

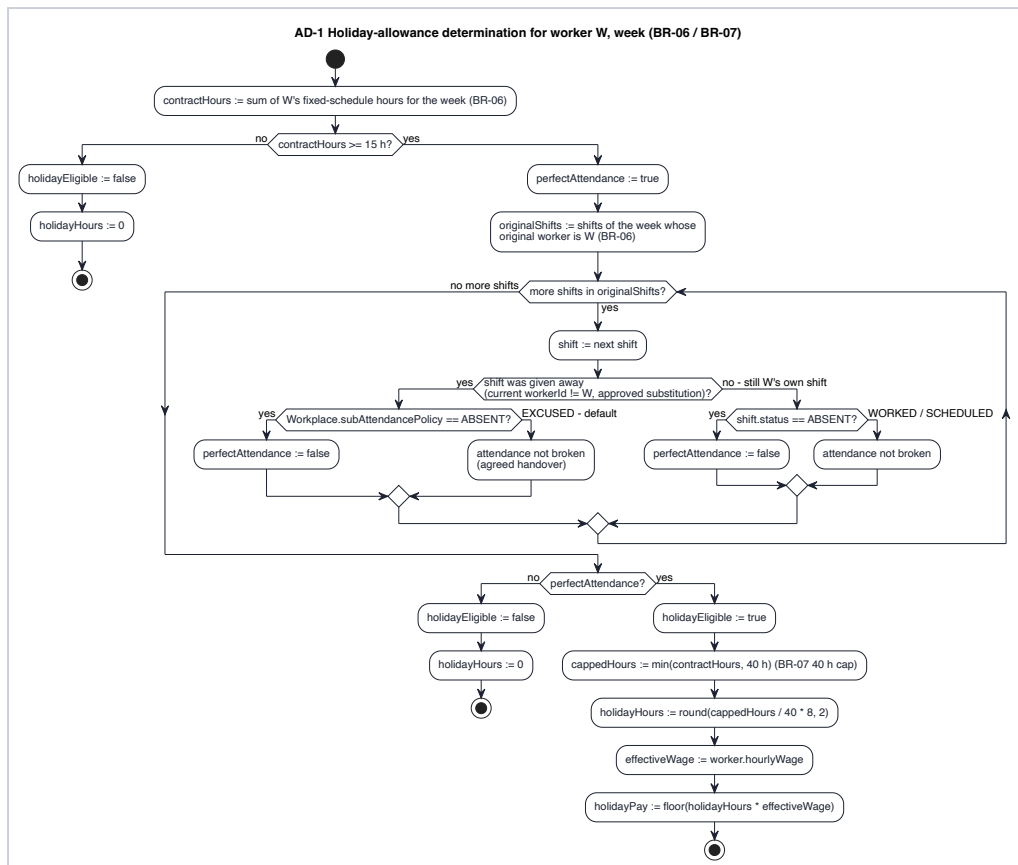


Figure 4. Activity diagram AD-1 — holiday-allowance determination for one worker and week (BR-06, BR-07)

The figure above shows the decision of BR-06 and BR-07 for one worker and one week. The first decision tests the contractual hours against 15; a worker below the threshold is not eligible, and no attendance is examined. For a worker at or above the threshold, the loop examines every shift of the week whose *original* worker is the worker. A shift that was given away by an approved substitution is judged by the workplace policy: under EXCUSED it does not break attendance, under ABSENT it does. A shift still held by the worker breaks attendance only if it is ABSENT. A shift that is WORKED or still SCHEDULED does not break it, so the weekly summary of the current week can be shown before the week is over and is updated if the owner later marks an absence. If attendance is perfect, the hours are computed with the 40-hour cap and rounded to two decimals, and the pay is rounded down to the won.

The diagram makes two properties of the rule easy to check with the users. First, substitute shifts that the worker *took* do not appear in the diagram at all: they affect scheduled hours but not the holiday allowance, which is the reason Choi Minh'o's allowance in scenario S-3 does not change. Second, the only place where the policy setting matters is one branch inside the loop, which shows that the setting cannot affect a worker below 15 hours.

3.9.3 State diagram: substitute request

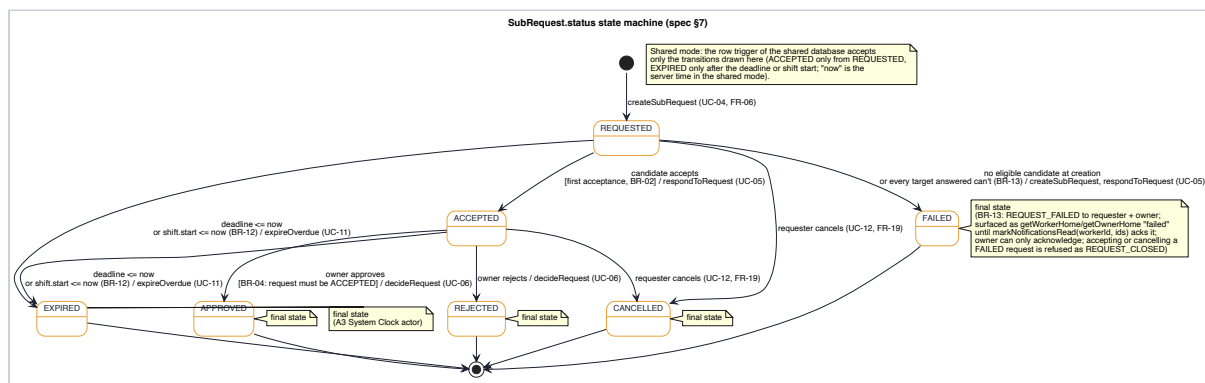


Figure 5. State machine diagram of a substitute request (SubRequest.status)

The figure above shows the life cycle of a substitute request. A request is created in REQUESTED (UC-04). The first acceptance moves it to ACCEPTED (UC-05, BR-02); the owner's decision moves it to APPROVED or REJECTED (UC-06, BR-04). A REQUESTED request moves to FAILED when it has no candidate at creation or when every candidate has answered can't (UC-04, UC-05, BR-13). From both open states, the System Clock can move it to EXPIRED (UC-11, BR-12) and the requester can move it to CANCELLED (UC-12). The five states APPROVED, REJECTED, FAILED, EXPIRED and CANCELLED are final. Each transition is labelled with the event, the guard in brackets where there is one, and the operation that performs it. In the shared mode the row triggers of the shared database enforce the same transitions a second time and refuse every other change of SubRequest.status (NFR-14).

The lecture gives three rules for state diagrams used in a requirements specification: all states must be domain significant, all sequences from the scenarios must be allowed, and all prohibited sequences must not be allowed [7]. The lecture also gives the general rules that there is one initial state, every state is reachable from it, every state has a path to a stop state, and every transition is labelled with an event [6]. The table below checks the diagram against these rules.

Table 33. Check of the SubRequest state diagram against the lecture's rules

Rule	Check
One initial state; every state reachable; path to a stop state from every state	Satisfied. The initial transition leads to REQUESTED; every other state is reachable from REQUESTED; the five final states lead to the stop state, and both open states have transitions to final states.
Every transition labelled with an event	Satisfied. The initial transition and each of the eight transitions between states are labelled (e.g. "candidate accepts [first acceptance, BR-02]").
All states domain significant	Satisfied. Each state corresponds to a situation the owner and workers talk about ("asked", "someone took it", "approved", "rejected", "nobody can", "too late", "withdrawn"). The internal state of targets (PENDING, ACCEPTED, DECLINED, CLOSED) is kept out of this diagram.
All scenario sequences allowed	Satisfied. S-1-S-3: REQUESTED → ACCEPTED → APPROVED. S-4: REQUESTED → EXPIRED (and ACCEPTED → EXPIRED in its variant). S-5: REQUESTED → FAILED (and REQUESTED → CANCELLED in its variant). UC-06 alternative flow A1: ACCEPTED → REJECTED.
Prohibited sequences not allowed	Satisfied. There is no transition REQUESTED → APPROVED (the owner cannot approve without an acceptor, BR-04), no transition ACCEPTED → FAILED (once someone has accepted, later answers of can't do not matter), no transition ACCEPTED → ACCEPTED (a second acceptance cannot replace the first, BR-02), and no transition leaving a final state (an approved swap cannot be cancelled or expire).

3.10 Requirements Validation

Requirements validation demonstrates that the requirements define the system the customer really wants; the lecture stresses that fixing a requirements error after delivery can cost up to 100 times as much as fixing an implementation error [7]. The lecture names five checks — validity, consistency, completeness, realism and verifiability — and three techniques — requirements reviews, prototyping and test-case generation. ShiftSwap used all three techniques; they are carried out within the requirements package (WBS 3), the UI design package (4.2) and the testing package (6).

3.10.1 Requirements reviews

Reviews were held while the requirements were being written, at the weekly team meeting. Following the review rule of the proposal, the requirements written by Hyewon Kim were reviewed by Hyungdo Kim and Minkyung Kim; the analysis models by Hyewon Kim and Minkyung Kim. The review of each requirement applied the five checks of the lecture, using the questions in the table below. Findings and their resolution were entered in the weekly meeting notes (WBS 1.2.1); a finding was closed only when the specification file had been changed or the reason for no change had been written down.

Table 34. Review checks applied to each requirement

Check	Question asked in the review	Example of how the specification answers it
Validity	Does the requirement support a problem P1-P3 or a goal G1-G6?	Each FR belongs to a use case, and each use case to a problem (objectives table of Section 1.2 and Section 2.2).
Consistency	Does the requirement conflict with another requirement or rule?	FR-18 (expire when the shift starts) and BR-03 (deadline no later than the shift start) use the same limit.

Check	Question asked in the review	Example of how the specification answers it
Completeness	Are all outcomes covered, including failures and time-outs?	Every open state of a request has an exit by time (UC-11) and by the requester (UC-12).
Realism	Can it be implemented in one semester without an application server of its own?	Notifications are in-app only (FR-21); push and SMS are in the Won't list; the shared data are kept in a hosted database service (NFR-08).
Verifiability	Can a test or an inspection decide whether it is met?	Adjectives such as "fast" or "easy" are replaced by numbers: 2 clicks (NFR-01), 360 px (NFR-02), 1 second with 20 workers (NFR-03).

Applying these checks to the domain produced several points where the first reading of the problem was ambiguous. The table below lists them together with the resolution that the specification contains. They are the points most likely to be questioned by a store owner, and they show why the business rules are written with explicit conditions.

Table 35. Ambiguities in the domain and their resolution in the specification

Question	Resolution	Where
Does a shift handed over by agreement break perfect attendance?	The law does not say. Made a workplace setting, EXCUSED by default; both variants tested.	BR-06, FR-17
The 15-hour threshold is a four-week average. Which weeks?	Contractual hours come from the fixed schedule, which is the same every week, so the average equals the weekly value.	BR-06
Do extra shifts taken as a substitute raise the holiday allowance?	No. Contractual hours are not changed by substitute shifts; only scheduled hours change.	BR-06
What happens when two candidates accept?	The first acceptance wins inside one transaction; the second is refused with the name of the acceptor.	BR-02, NFR-04
What if the owner never decides on an accepted request?	It expires at its deadline or at the shift start; the shift stays with the requester.	BR-12
Who is the "original" worker after two swaps of the same shift?	The first worker; the field is set only when empty.	BR-04
How is a shift that ends after midnight measured?	An end time not later than the start time means the next day; supported by the rule functions and tested, although the seed data has no such shift.	spec §8, TC-09x
In which month is the holiday pay of a week that spans two months paid?	In the month of the week's Sunday.	BR-11
Who is told when a request is cancelled or expires, and who when someone declines?	A recipients table per notification kind: cancel notifies every target that was PENDING or ACCEPTED; expiry notifies requester and owner and closes pending targets silently; a single can't notifies no one, and the last can't that leaves nobody triggers REQUEST_FAILED.	spec §8, FR-19, FR-22
What happens when nobody can take the shift? (raised by the product owner in iteration 6)	The request ends as FAILED as soon as every target has answered can't, or at once when nobody can be asked; the requester is told to contact the owner, and the owner gets an acknowledge-only card.	BR-13, FR-08, FR-22
Is a week that has not ended yet already "perfect"?	Only an ABSENT shift breaks attendance; a SCHEDULED shift does not, so the weekly result is a projection until the week is over.	BR-06
Who is a part-time worker, and how are hours beyond the contractual hours counted? (iteration 8)	The law does not fix the method. System rules, stated as design decisions: part-time means $0 < C < 40$ contractual hours; a weekly test against C in chronological order; an hour is counted once; the night premium is separate; consent is outside the system, and the 12-hour limit is warned about before approval.	BR-08 (iv), BR-15
Whose clock decides a deadline when every user has their own device? (iteration 8)	In the shared mode the server time; the server refuses an acceptance or approval after the deadline and an expiry before it.	A3, BR-12, NFR-14

Question	Resolution	Where
What does the shared database guarantee without passwords? (iteration 8)	Integrity, first acceptance, server time and rate limits, but not identity; all shared data are fictional, and the sign-in screen says so.	FR-20, NFR-14

3.10.2 Prototyping

Prototyping uses an executable model of the system to check requirements [7]. The UI prototype of the key screens — weekly board, inbox and approval panel — belongs to WBS 4.2.1 (screen wireframes and prototype screens, owner Minkyung Kim), with wireframes scheduled to start in Week 7. It was generated with the same tool chain as the final system, so that it could be evolved rather than thrown away. The proposal planned to show it to the owner of the reference store and two co-workers during validation. The prototype was used to check the requirements that are hard to judge on paper: whether the before/after table of FR-11 contains the figures an owner needs, whether the accept button can be reached in two clicks (NFR-01), and whether a handed-over shift is recognisable on the weekly board. Prototype feedback is recorded in the meeting notes together with the review findings; like the interviews, it is used to confirm or correct requirements and is not reported as statistics. Because the prototype was generated by the AI coding assistant from the same specification, a change requested on the prototype was made in the specification first, following the change procedure of Section 3.11.

3.10.3 Test-case generation

The third technique develops tests from the requirements to check that they are testable [7]. For ShiftSwap the business rules were the natural starting point, because each rule is a function with inputs and a result. The specification requires that every business rule maps to at least one test case (spec §12). Test cases are grouped by a two-digit prefix, and each test is named with its full identifier, for example TC-081 holiday allowance at exactly 15 h. The table below gives the mapping from business rules to test groups and the boundary cases each group must contain. The test results are reported in Chapter 6.

Table 36. Mapping of business rules to test-case groups

Business rule	Test group	Cases required by the specification and the final prompt	Related requirements
BR-01	TC-01x	Interval overlap including back-to-back shifts; each eligibility condition failing on its own; the seed request yielding exactly Choi Minho, Jung Hana and Kang Doyun.	FR-07, FR-08
BR-03	TC-02x	Request on someone else's shift, on a past or non-SCHEDULED shift, a second open request, a deadline before now or after the shift start.	FR-06
BR-02	TC-05x	Two acceptances of the same request, of which only the first succeeds; other targets closed.	FR-09, FR-10, NFR-04
BR-04	TC-06x	Approve sets the acceptor and keeps the first original worker; reject leaves the shift unchanged; approval of a non-ACCEPTED request refused.	FR-12
BR-05, BR-06, BR-07	TC-08x	Eligibility at 14.99 h (not eligible) and 15 h (eligible); the 40 h cap; both substitution-attendance policies; the worked example (Choi Minho, 3.2 h, KRW 33,024).	FR-14, NFR-10
BR-08	TC-09x	Premium zero at 4 regular employees and non-zero at 5; daily and weekly overtime without double counting; night hours, including a shift that ends after midnight. For part-time workers (iteration 8, TC-096 to TC-09C): hours beyond contract only with $0 < C < 40$, weekly test in chronological order across a month boundary, an hour counted once with daily or weekly overtime, night hours added separately, the worked example of S-9.	FR-15, NFR-10
BR-09, BR-10	TC-10x	Warning below the minimum; probation reduction only when all conditions hold, each condition failing on its own (contract under one year, after three months, after probation end, simple labour).	FR-16, NFR-10
BR-11	TC-11x	Monthly totals; holiday pay assigned by the week's Sunday; estimated flag when attendance is missing; rounding down to the won.	FR-15
BR-12	TC-12x	Expiry at the deadline and at the shift start, for REQUESTED and ACCEPTED requests; no expiry of final states.	FR-18
BR-13	TC-025, TC-13x	No candidate at creation ends the request FAILED; the last can't ends it FAILED with REQUEST_FAILED to requester and owner; an earlier can't changes nothing; accepting a FAILED request is refused.	FR-08, FR-09, FR-22

Business rule	Test group	Cases required by the specification and the final prompt	Related requirements
BR-14	TC-14x	The draft for the previous month appears on the first day of the month and not before; it is prepared only once and never overwrites DRAFT or CONFIRMED rows; one PAYROLL_DRAFT_READY for the owner; the seeded history equals what the rules compute.	FR-23
BR-15	TC-15x	12 h beyond contract gives no warning, 12.5 h and 13 h do; no warning with 4 regular employees, with $C \geq 40$ or without a fixed schedule; the walkthrough of S-9 with 24 → 29 h (warning) and 23 → 28 h (no warning).	FR-11, FR-24

Writing the test cases before implementation had a validation effect of its own: a rule for which no precise expected result could be written was, by the lecture's definition, not verifiable, and had to be made more precise. The rounding clauses of BR-07 and BR-11 ("rounded to 2 decimals", "rounded down to the won") are examples of precision that a test demands and a prose rule easily omits. The non-functional requirements that are not covered by unit tests have the verification methods listed in the table of Section 3.7 (walkthrough, browser checks, timed load, role test, design review). The shared mode added one test group that is not derived from a business rule: TC-16x checks the change sets, the re-run of a stale command, the mapping of server errors and the currency of the seed template of the shared database against an in-process fake server (NFR-04, NFR-14, NFR-15); the shared database itself is tested over its REST interface with the publishable key.

3.11 Requirements Management

Requirements management keeps track of individual requirements and of the links between dependent requirements, so that the impact of a change can be assessed, and it needs a formal process for change proposals [7]. In a vibe-coding project this activity has an additional purpose: the requirements are also the input of the code generator, so a requirement that changes in one document but not in the prompt produces a system that no longer matches its specification. The team therefore organised requirements management around two files in the repository.

3.11.1 The specification file as single source of truth

The file `spec/spec.md` contains every identifier and every fact that more than one artefact uses: actors, use cases, functional and non-functional requirements, business rules, states, the database schema, the seed data, the test groups and the reference list. Its first line states the rule that governs it: every artefact must use the identifiers, names, states, rules and numbers defined there, and anything not defined there must be added there first before it is used anywhere else. The final prompt (Chapter 5) instructs the AI coding assistant to treat the specification as the contract and not to add features that are not in it, and the prompt is always given together with the full specification. The UML sources, the reports and the test names all refer to the same identifiers. As a result, a requirement has one wording, and the report quotes it rather than paraphrasing it.

3.11.2 Traceability

Traceability links are part of the specification itself. Each functional requirement names its use case; each use case names its tables; each business rule is referenced by the requirements that use it; each test group names the rules it tests; and the source code cites the business rule it implements in the comment of each rule function. The specification states the traceability rule explicitly (spec §12): every FR maps to at least one UC, every UC to at least one screen and service function, and every BR to at least one test case. The full traceability matrix is given in Chapter 6. With these links, the impact of a change can be found by searching for an identifier across the repository.

3.11.3 Change procedure and the prompt log

A proposed change to the requirements followed five steps:

1. The change is raised at the weekly meeting and entered in the meeting notes (WBS 1.2.1), with its reason (a review finding, prototype feedback, a failing test or a legal source).
2. The requirements lead and the project manager assess its impact by following the traceability links: which use cases, rules, tables, screens and test cases are affected, and whether the change moves a feature across the MoSCoW boundary.
3. If accepted, the change is made in `spec/spec.md` first. Identifiers are never reused: a removed requirement keeps its number retired, and a new one receives the next free number.
4. The prompt is updated to refer to the changed specification, the code is regenerated or modified by the AI coding assistant, and the affected test groups are run together with all others as regression tests.
5. The change is recorded in the prompt log with the prompt version, the specification change, and the result of the test run.

The prompt log serves as the change history that the lecture requires. Each entry records a prompt version, what was asked, what the AI coding assistant produced, what was wrong with it, and whether the correction was made in the prompt or in the specification. Chapter 5 presents the log. The distinction matters: a correction in the prompt concerns how the system is

built, while a correction in the specification changes what the system is required to do and therefore passes through the steps above.

Several kinds of change were anticipated in the requirements and turned into data rather than code, so that they do not require a requirements change at all. A new minimum wage for 2027 is a new row in the minimum-wage table (UC-10), not a change to BR-09. A store that reaches five regular employees changes a setting, and BR-08 and BR-15 apply automatically. A store that reads the law differently on handed-over shifts changes the substitution-attendance policy. These are the changes that the domain makes most likely, and the design of Chapter 4 keeps them out of the code.

3.11.4 Change request in iteration 6

The largest change to the requirements after the interim report came from the product owner, who used the running application after iteration 5. It went through the five steps of Section 3.11.3 and is recorded in §15 of the specification. This section follows it from the observation to the updated traceability links.

Observed dead end. Under the old BR-01, a worker was asked only if one of their availability slots covered the whole shift. In use, this produced requests that no one could answer: when no slot matched, the owner received a NO_CANDIDATE notification and nothing else, and had nothing to do in the system. At the same time, the slots were extra data that every worker had to keep up to date, although whether someone can take a particular shift is known only to that person on that day.

Decision. Availability is removed. Every other active co-worker without an overlapping shift is asked (BR-01), and the first acceptance still wins (BR-02). A candidate can now answer *can't*, and a request that nobody can take ends early as FAILED instead of waiting for its deadline (BR-13, FR-22). The requester is then told to contact the owner, and the owner receives a notification whose only action is to acknowledge it. The same ending applies when nobody can be asked at all (FR-08). UC-02, FR-03 and the table Availability are retired; following step 3 of the procedure, their identifiers are kept and not reused, and the new rule and requirement receive the next free numbers.

Impact analysis. The traceability links of the specification were followed from each changed identifier. The table below lists what the change touched.

Table 37. Impact of the iteration 6 change request

Artefact	Change
Use cases	UC-02 retired (12 active use cases); UC-04 alternative flow A1 ends the request FAILED; UC-05 renamed "accept / can't", with the new alternative flow A2 "nobody can take it"
Functional requirements	FR-03 retired; FR-08 and FR-09 reworded; FR-22 new (21 active functional requirements: FR-01, FR-02, FR-04 to FR-22)
Business rules	BR-01 loses its availability condition and is restricted to active workers; BR-13 new (13 rules)
States and notifications	SubRequest.status gains the final state FAILED; the notification kind NO_CANDIDATE is replaced by REQUEST_FAILED, sent to the requester and the owner
Database	Table Availability removed (11 tables); the CHECK constraint of SubRequest.status admits FAILED
Seed data and scenarios	Availability slots removed from the seed; the seed request now has three candidates (Choi Minho, Jung Hana, Kang Doyun); scenarios S-1, S-2 and S-4 updated; S-5 replaced by the demonstration walkthrough S8 (nobody can take the shift)
Screens	Me: the availability editor removed, the worker's profile shown instead; Home (worker): <i>Can't</i> on the request card, and the FAILED card for the requester; Home (owner): the acknowledge-only card "No one can cover"; request flow step 3: recipients are all free co-workers
Services and prompt	The service that saved availability slots is removed; answering can't checks whether any target is still PENDING and ends the request FAILED if none is; prompt version 6 gains a done criterion for walkthrough S8
Tests	Tests that relied on availability rewritten (TC-014, TC-015, TC-025) and expected values updated for three candidates; seven new tests: TC-13A to TC-13F for BR-13, FR-08 and FR-22, and TC-003 for the replacement of a database saved before the change (84 tests in total, Section 6.9)

Traceability after the change. The retired identifiers leave gaps that the traceability rule of spec §12 must still satisfy: every active functional requirement maps to a use case, every active use case to a screen and a service function, and every business rule, including the new BR-13, to at least one test. The matrix of Section 6.8 was regenerated from the code after the change and shows FR-03 and UC-02 as retired and FR-22 and BR-13 with their tests. The change also shows why step 3 of the procedure forbids reusing identifiers: the prompt log, the earlier reports and the tests of iterations 1 to 5 still refer to FR-03 and UC-02 with their old meaning, and a new requirement under the same number would make those records ambiguous.

3.11.5 Change request in iteration 7

Iteration 7 added a requirement instead of removing one. It followed the same procedure as iteration 6 and is recorded in §16 of the specification.

Request. After using the application, the product owner asked for two things: payroll data for the past months, based on the employees' contracts, and last month's payroll filled in automatically on the first day of every month. Until then, the seed contained only two weeks of shifts, so *Pay* → *Monthly* showed nothing before September 2026, and every monthly draft had to be started by hand.

Decision. The seed now contains each worker's full work history from the contract start: every fixed shift up to 2026-09-20 is worked with a confirmed attendance, and the payroll of every month from the first contract month to August 2026 is confirmed, computed by the same rules as any other payroll. A new use case, UC-14, has the System Clock as its actor: on or after the first day of a month it prepares the previous month's DRAFT and notifies the owner (FR-23, BR-14). Confirmation stays with the owner, because a minimum-wage warning must be acknowledged by a person (BR-09). The new identifiers take the next free numbers: UC-14, FR-23 and BR-14.

Impact analysis. The table below lists what the change touched, found by following the traceability links from UC-09 and FR-15, whose computation the new use case reuses.

Table 38. Impact of the iteration 7 change request

Artefact	Change
Use cases	UC-14 new, with the System Clock as actor (13 active use cases); UC-09 gains the alternative flow "draft already prepared"
Functional requirements	FR-23 new (22 active functional requirements)
Business rules	BR-14 new (14 rules); BR-06 to BR-11 unchanged and reused for the history and the automatic draft
Notifications	New kind PAYROLL_DRAFT_READY, sent once per month to the owner (nine kinds)
Database	No schema change (11 tables); the existence of Payroll rows for a month marks the draft as prepared
Seed data and scenarios	Work history from each contract start and confirmed payroll to August 2026; new scenario S-8 (demonstration walkthrough S9); the September 2026 figures of the payroll scenario now include the weeks before 21 September
Screens	Owner Home: card "9월 급여 초안이 준비됐어요" with an action that opens the draft; Pay → Monthly: month switcher back to the first contract month, confirmed months read-only, a note on an automatically prepared draft
Services and prompt	The System Clock runs the new service prepareMonthlyPayroll after expireOverdue; the seed is built by calling the same services as the application; prompt version 7 gains a done criterion for walkthrough S9
Tests	New test group TC-14x for BR-14, FR-23 and the seeded history (Section 6.9)

Traceability after the change. FR-23 maps to UC-14, UC-14 to the service that prepares the draft and to the owner's Home, and BR-14 to the test group TC-14x, so the rule of spec §12 holds for the new identifiers as well (Section 6.8). The change also shows a pattern that the two iterations share: in both, the product owner judged the running system, not a document, and the change entered the specification first and the code only afterwards.

3.11.6 Change request in iteration 8

Iteration 8 differs from the two before it in its source: the change came from the revision of the proposal on 2026-09-29, not from the use of the running system. It followed the same procedure and is recorded in §17 of the specification.

Request. The revised proposal adopts a shared relational database on Supabase, so that the owner and every worker use one store from their own phones, and keeps the browser-only version as a local demonstration mode (fallback, risk R6). It adds the rule for part-time workers in workplaces with five or more regular employees — hours beyond the contractual hours need the worker's consent, are limited to 12 a week and earn a premium of at least 50 % [15] — with a warning in the approval panel (F6, Should) when a swap would leave the acceptor with more than 12 hours beyond contract in that week. It adds risk R6 (a free-plan project is paused after seven days of low activity [16]; scheduled workflows of a public repository are disabled after 60 days without repository activity [17]; a keep-alive read runs every six hours), and it states that password authentication is out of scope and that the System Clock uses the server time in the shared mode. The product owner decided to convert the application to the shared database.

Decision. The shared mode works with a working copy and change sets: every command runs the unchanged service function on an SQLite copy of the store in the browser, and only the changed rows are sent to the server, which applies them only to the revision they were computed on, under a row lock, and whose triggers re-check what the server can decide without knowing the user (NFR-14). The service layer, the rules and their tests therefore stay the same in both modes. Two alternatives were considered and not adopted: one server function per use-case step, with which every rule would exist twice, in SQL and in JavaScript, and the two could disagree; and direct table writes under row-level-security policies, which give no transaction across several tables and so no guarantee of first acceptance. Authentication stays out of scope (FR-20); the server guarantees integrity, first acceptance, server time and limits, not identity, and all shared data are fictional. A reset for everyone restores the seed, moved by whole weeks to the current date, at most once a minute. The part-time rule enters as BR-08 (iv) with the system rules (a)–(e), BR-15 and FR-24; the readings considered and not adopted are recorded in BR-08. The new identifiers take the next free numbers, and nothing was renumbered.

Impact analysis. The table below lists what the change touched, found by following the traceability links from BR-02, BR-08, BR-12, UC-13 and NFR-08.

Table 39. Impact of the iteration 8 change request

Artefact	Change
Actors and use cases	A3 System Clock uses the server time in the shared mode; no new use case (13 active); UC-06 gains the part-time warning (step 5, alternative flow A2); UC-11 reads the server time and gains the exception flow E2; UC-13 is renamed "demo account selection, no password" and gains the notice of shared data, the reset for everyone, the switch of the data mode and the exception flow for an unreachable database
Functional requirements	FR-24 new, priority Should (23 active functional requirements); FR-20 reworded
Non-functional requirements	NFR-04, NFR-05, NFR-06, NFR-08 and NFR-10 reworded; NFR-14 (security), NFR-15 (efficiency) and NFR-16 (dependability) new (16 requirements)
Business rules	BR-08 gains item (iv) with the system rules (a)–(e); BR-12 states the server time; BR-15 new (15 rules)
Database	The same 11 tables in both modes. The shared database holds them as 11 app_ tables with a store_id column, plus the service tables app_store and app_store_gate; three functions app_snapshot, app_commit and app_reset; 7 row triggers that re-check the rules of NFR-14
Seed data and scenarios	The seed is also kept as a template in the shared database (353 rows) and moved by whole weeks on reset; scenario S-2 describes first acceptance in the shared mode; new scenario S-9 (demonstration walkthrough S10)
Screens	Sign-in: notice of the data mode and link to the other mode; Demo tools in the shared mode: server time (read-only), reset for everyone with confirmation, switch to the local mode; owner decision card: the part-time warning; a busy state while a change is saved, and messages for a concurrent change, a refused change, an unreachable database and the rate limits
Services and prompt	Service functions unchanged; new shared-data gateway (app/core/shared.js, app/config.js) that loads the snapshot, sends change sets, re-runs stale commands and polls the revision; database script db/app-shared.sql; keep-alive workflow; prompt version 8 covers the shared mode and the part-time rules
Tests	New tests TC-096 to TC-09C (BR-08 (iv)), group TC-15x (BR-15, FR-24) and group TC-16x (shared-data gateway); 125 unit and integration tests in total; REST test of the shared database with 64 checks against a throwaway store (Chapter 6)

Traceability after the change. FR-24 maps to UC-06 and to the owner's decision card; BR-08 (iv) maps to TC-096 to TC-09C and BR-15 to TC-15x; NFR-14 is verified by the REST test of the shared database. The rule of spec §12 therefore holds for the new identifiers as well (Section 6.8). The change also shows the value of stating legal content and design decisions separately: the proposal and the specification give the statute as the source of the 12-hour limit and the 50 % premium, while the counting method is recorded as the system's own rule, so that a different reading of the law would change BR-08 (a)–(e) without touching the statement of the law.

4. Design Specification

This chapter describes how ShiftSwap provides the functions specified in Chapter 3. The lecture divides design into architectural design (the structure of the system), interface design (the interfaces between its parts) and detailed design (the algorithms of the individual parts) [6]. Section 4.1 presents the architecture with its two data modes and Section 4.2 the deployment. Section 4.3 records the main design decisions and the alternatives that were rejected. Sections 4.4 and 4.5 give the static design: the class model and the database. Sections 4.6 and 4.7 give the dynamic design: sequence diagrams for the four central use cases and state machines for the three entities with a life cycle. Section 4.8 specifies the algorithms of the business rules, and Section 4.9 the design of the user interface.

The design is described as it is implemented. Every module, function and table named in this chapter — including the objects of the shared database in `db/app-shared.sql` — exists under the same name in the repository, and the functions are the ones called by the tests of Chapter 6. Where a UML diagram shows an operation at a more abstract level than the code, the text gives the name of the implementing function. The parts added in iteration 8 — the shared-data gateway `app/core/shared.js`, the configuration `app/config.js` and the shared database — are described in the same way.

4.1 Architecture

4.1.1 Layers

ShiftSwap has a layered architecture with two data modes, introduced in iteration 8 (Section 3.11.6). In the *shared mode*, the default, the owner and the workers use one store in a shared database on Supabase, a hosted PostgreSQL service with a free plan [16]; everyone who opens the application sees and changes the same data. In the *local demonstration mode*, which keeps the behaviour of iterations 1 to 7, the same tables exist only in the user's browser. In both modes all application code runs in the browser, and a static web host delivers the files. In the shared mode the browser also calls three functions of the shared database through its REST interface, using only the publishable key of the project; the system has no application server of its own (NFR-06). The table below lists the seven parts of the architecture. Each layer may call only the layers below it, and the domain-rule layer calls nothing at all. The application services and the rules are the same code in both modes.

Table 40. Layers and modules of ShiftSwap

Layer	Module (file)	Responsibility
Presentation	<code>app/index.html</code> , <code>app/main.js</code> , <code>app/ui/*.js</code> , <code>app/styles.css</code>	Screens per role in fifteen modules — one per screen (for example <code>home.js</code> , <code>requests.js</code> , <code>me.js</code>) plus the shared <code>components.js</code> , <code>slots.js</code> (fixed-schedule slot editor) and <code>i18n.js</code> ; hash routing (<code>#/home</code> , <code>#/swaps</code> , ...); role menus; clock chip, <i>Demo tools</i> and language controls; choice of the data mode; translation of error codes into English or Korean text. Every command goes through <code>ctx.run()</code> , which in the shared mode waits until the change is saved and shows a busy state meanwhile.
Shared-data gateway	<code>app/core/shared.js</code> , <code>app/config.js</code>	Shared mode only: loads the snapshot of the store, sends the change set of each command with the revision it was computed on, runs a command again when that revision is stale, saves the work of the System Clock, polls the revision of the store, and maps the errors of the server to messages. <code>config.js</code> holds the project URL, the publishable key and the store slug.
Application services	<code>app/core/services.js</code>	One exported function per use-case step, each running inside one transaction; read-only query functions for the screens; creation of Notification rows. The same code in both modes.
Domain rules	<code>app/core/rules.js</code>	Pure functions for BR-01 to BR-11, BR-13 and BR-15: overlap, duration, week start, eligibility, no taker, request validity, contractual hours, weekly summary, holiday hours, premium hours including hours beyond contract, the part-time weekly limit check, minimum wage, probation, payroll row. No database, no DOM.

Layer	Module (file)	Responsibility
Data access	app/core/db.js	sql.js start-up, schema of spec §8, seed data of spec §10, tx(), all(), get(), run(). Local mode: persistence to localStorage and reset. Shared mode: the in-memory working copy built from the rows of the server (loadTables) and the table snapshots from which change sets are computed (dumpTables).
Clock	app/core/clock.js	now(): in the shared mode the server time in Korea Standard Time (useServerTime()), which cannot be set; in the local mode the demo clock with setNow() and advance().
Shared database	db/app-shared.sql (Supabase PostgreSQL)	Thirteen app_ tables, of which the API roles may read twelve and write none; the functions app_snapshot, app_commit and app_reset; row triggers that re-check the rules the server can decide itself (NFR-14; Section 4.5.2).

The most important boundary is the one around rules.js. Every decision that depends on a business rule — whether a worker is eligible, whether a request is valid, whether a holiday allowance is owed, how much premium is paid — is made by a function of this module that receives plain values and returns plain values. The service layer reads the data, calls the rule, and writes the result. This has three consequences. The rules can be unit-tested without a database (the 36 tests of tests/rules.test.js). The rules can be read by someone who knows the labour law but not SQL. And a change of a rule cannot accidentally change how data is stored, because the rule module has no access to storage.

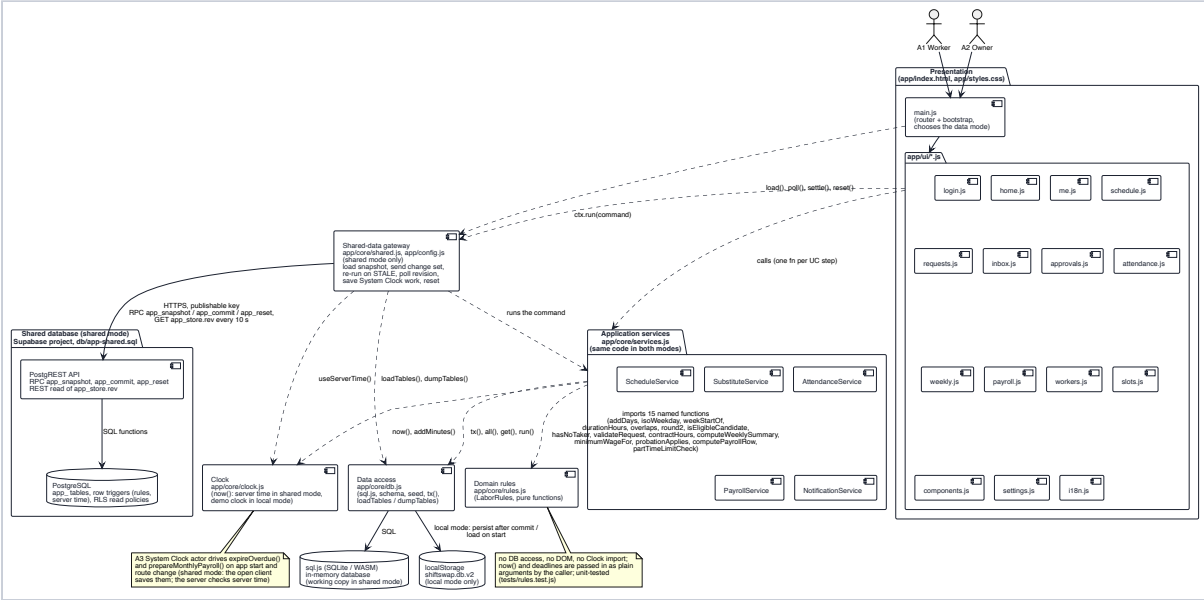


Figure 6. Component diagram of ShiftSwap (iteration 8): the UI, services and rules run on an in-memory sql.js working copy, which the gateway keeps in step with the shared database (shared mode) or which is saved in localStorage (local demonstration mode)

The component diagram above shows the same structure. The actors reach only the router main.js, which renders one screen module of app/ui/. Screens call the services, and the services depend on the rules, the data-access module and the clock. In the shared mode the router passes every command to the shared-data gateway, which uses the data-access module and the clock and reaches the shared database over HTTPS. The diagram lists the fifteen functions that services.js imports from rules.js, and it shows no dependency from the rules to the clock: the current time is passed to the rules as an argument (for example, validateRequest receives now from the service), and rules.js imports nothing. The rule functions are therefore deterministic, which is what allows tests such as TC-023 to fix the time in the test itself.

4.1.2 Transactions and persistence

The data-access module keeps one SQLite database in memory, created by sql.js [9]. Every service command is wrapped in tx(fn), which executes BEGIN, runs the function, and then executes COMMIT; if the function throws, it executes ROLLBACK and passes the error on. A nested call of tx() joins the outer transaction instead of opening a second one, so that decideRequest can call recomputeWeek and both changes commit or roll back together. In the local demonstration mode, the whole database is exported as bytes after a successful commit, encoded in base64 and saved to localStorage under the key

`shiftswap.db.v2` (v1 until iteration 7, when the seed gained the work history). On start, `openDatabase` loads the saved copy if one exists and has the current schema; otherwise it creates the schema, inserts the seed data and saves it (NFR-05).

In the shared mode the in-memory database is a working copy of one store of the shared database and is not saved in the browser. On start, the gateway calls `app_snapshot(store)`, which returns every row of the store, the revision number of the store and the server time, and `loadTables` builds the working copy from these rows with the schema of spec §8. A command runs the unchanged service function inside `tx()` on the working copy. The gateway then compares the tables with their state at the last synchronisation (`dumpTables`) and sends the changed rows — inserts, updates and deletes by primary key — together with the revision on which they were computed to `app_commit(store, base_rev, changes)`. The server applies this change set in one transaction: it locks the store row with `SELECT ... FOR UPDATE`, refuses the change set with `STALE` if its base revision is no longer the current one, checks the rate limits and sizes, applies the deletes, updates and inserts, lets the row triggers check the rules it can decide itself, checks the deferred foreign-key and uniqueness constraints (`SET CONSTRAINTS ALL IMMEDIATE`) and increments the revision. Any violation rolls back the whole change set (Section 4.5.2). On `STALE` the gateway reloads the snapshot and runs the same command again on the current data, at most three times; when a rule of the server refuses the change, it reloads and runs the command once more, so that the user sees the service's own message rather than an error of the server (spec §9).

Because a failing service throws a `ServiceError` inside `tx()`, a rejected operation never leaves a partial change behind. This is the mechanism behind NFR-04: the acceptance of a request re-reads the request inside the transaction and, if it is no longer `REQUESTED`, throws before anything is written. In the shared mode a command that fails before it changes anything sends nothing to the server, and the server's row lock and revision check extend the guarantee to acceptances made in different browsers (Section 4.6.2).

4.1.3 The clock and the System Clock actor

All time-dependent behaviour — whether a shift is in the future, whether a deadline has passed, the default deadline, the time stamps of notifications — uses `now()` from `clock.js`. In the shared mode `now()` returns the server time in Korea Standard Time: every snapshot and every commit returns the time of the server, and the clock keeps its difference from the clock of the device, so that deadlines are judged by the same time in every browser. This clock cannot be set. In the local demonstration mode `now()` returns the demo clock, which starts at 2026-09-28 09:00 (the Monday of the seed data), is stored under `shiftswap.clock.v1`, and can be set or advanced by one hour or one day in *Demo tools*. The System Clock actor A3 of the use-case model is realised by the router: before rendering any screen, `main.js` calls `expireOverdue()` and then, since iteration 7, `prepareMonthlyPayroll()`. Both therefore run on start, on every route change and, in the local mode, after every change of the demo clock: a request whose deadline has passed is expired before any user can see or act on it, and on or after the first day of a month the previous month's payroll draft exists before the owner opens Home (UC-14). In the shared mode these duties are performed by whichever browser has the application open. Their changes are committed in the background like those of a command, and the server accepts an expiry only when the request is overdue by its own time (BR-12). If the server refuses such a change, the working copy is reloaded and the duties pause for one minute.

4.2 Deployment

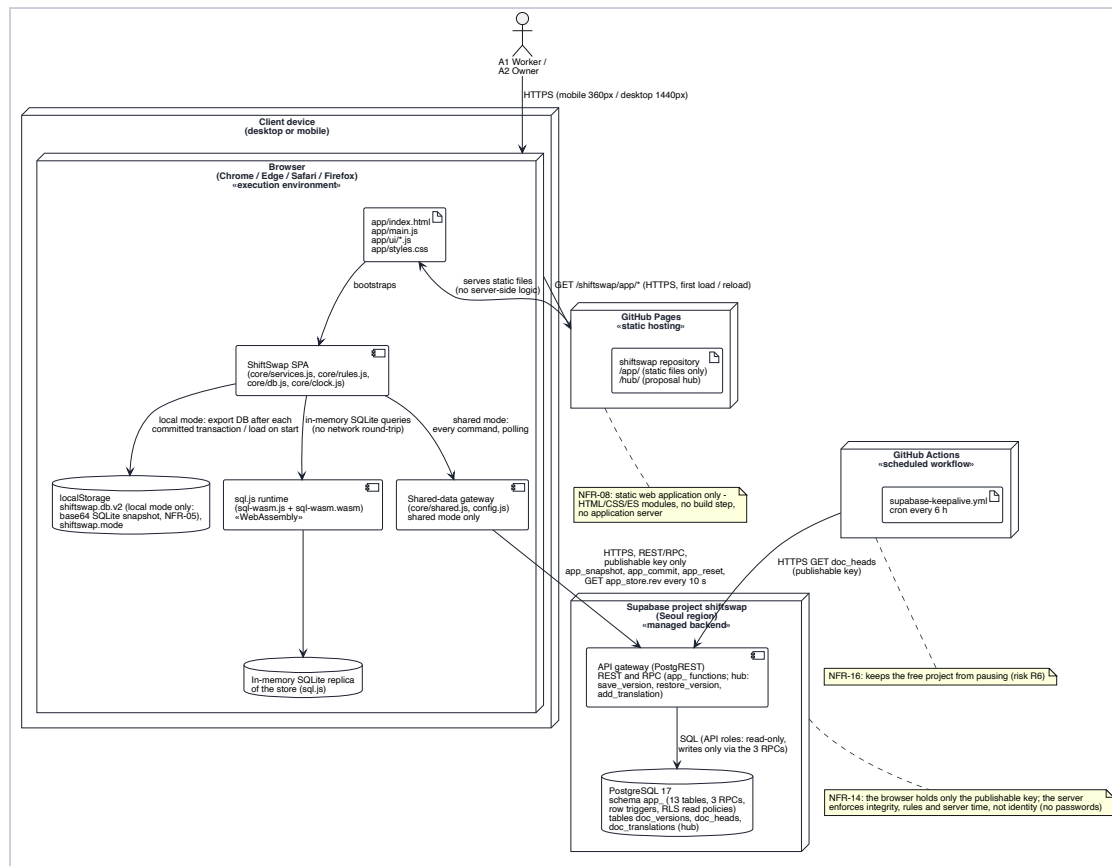


Figure 7. Deployment diagram of ShiftSwap: static files from GitHub Pages, the shared data in the Supabase project reached over HTTPS with the publishable key, and the six-hourly keep-alive workflow

The deployment diagram above shows the nodes of the deployed system. GitHub Pages serves the static files of the repository over HTTPS [13]; it runs no application code. The browser on the user's device downloads `index.html`, the JavaScript modules, the style sheet and the two `sql.js` files and runs all application code. In the local demonstration mode nothing else is contacted. In the shared mode the browser also connects over HTTPS to the Supabase project `shiftswap`, whose PostgreSQL database holds the shared store (Section 4.5.2); it reads the store and calls the three database functions through the REST interface of the project with the publishable key, which permits nothing else. The `sql.js` files (`sql-wasm.js` and `sql-wasm.wasm`, version 1.14.2) are copied into `app/vendor/`, so the application does not depend on a content-delivery network at run time. All URLs in the application are relative, so the same files work under the sub-path `/shiftswap/app/` of the deployed site and under any local static server. Chapter 7 describes the deployment procedure.

A third element of the deployment is the scheduled workflow `.github/workflows/supabase-keepalive.yml` of the repository (NFR-16). On the free plan of Supabase, a project that shows low activity over seven days is paused, and it stays paused until it is resumed manually from the Supabase dashboard [16]. The workflow therefore sends a read query to the project every six hours (cron `23 */6 * * *`, and on demand); the service documentation states that a few user requests to the database each day are typically enough to keep a project from being paused [16]. The query reads the version heads of the project hub (Section 7.1.5), which are stored in the same project, so it keeps the database of the application active as well. GitHub disables scheduled workflows in a public repository after 60 days without repository activity [17]; the team therefore also checks the public URL every week until grading ends and before every submission, as planned in the proposal.

The browser keeps six values between visits, listed in the table below. In the shared mode the database itself is not kept: the working copy is built again from the server on every start. If the browser refuses storage (for example in some private-browsing modes), `main.js` falls back to an in-memory store, and the application still works for the length of the visit.

Table 41. Browser storage used by ShiftSwap

Key	Storage	Content
<code>shiftswap.db.v2</code>	localStorage	Local mode only: the whole SQLite database, base64-encoded; rewritten after every committed transaction.
<code>shiftswap.clock.v1</code>	localStorage	Local mode only: the demo clock, YYYY-MM-DDTHH:MM.

Key	Storage	Content
shiftswap.mode	localStorage	The chosen data mode, shared or local, set by ?mode= or by the switch on the sign-in screen and in <i>Demo tools</i> (iteration 8).
shiftswap.tour.v1	localStorage	The first step of the guided tour on the sign-in screen, once completed; the other two steps are read from the data.
shiftswap.lang	localStorage	The chosen language, en or ko.
shiftswap.user	sessionStorage	The id of the signed-in demo account (UC-13); cleared when the tab is closed or the user signs out.

4.3 Design Decisions

The table below records the design decisions that shaped the system, the alternatives considered, and the reason for the choice. Several decisions follow directly from the organisational requirement NFR-08; the others were made during design and the build iterations recorded in the prompt log (Chapter 5). Decisions D13 to D18 were made in iteration 8, when the revised proposal introduced the shared database (Section 3.11.6), and the entries of D1, D3, D6 and D7 state how that iteration changed them.

Table 42. Design decisions, alternatives and rationale

#	Decision	Alternatives considered	Rationale
D1	Client-only application on static hosting	Server with a REST API and a hosted database; serverless functions	NFR-08 and NFR-06 require a system that runs without installation and without an application server of its own. A static site can be reproduced from the final prompt without running costs, and anyone can open it from the URL. Until iteration 7 the price was that data was local to one browser. Iteration 8 keeps the static hosting and the client-side code and adds a hosted database for the shared data (D13); the client-only form remains as the local demonstration mode (D18).
D2	SQLite in the browser (sql.js, WebAssembly)	Plain JSON objects in localStorage; IndexedDB	The course requires a relational database of at least four tables, and the business rules need joins, uniqueness constraints and transactions. sql.js provides real SQL with CHECK, UNIQUE and BEGIN/COMMIT/ROLLBACK, and the same schema runs unchanged in the Node tests. Since iteration 8 the same database also serves as the working copy of the shared mode, so the services need no second implementation (D13).
D3	Persist the whole database after each commit	Persist individual rows; persist on page unload	The database of a small store is a few hundred kilobytes at most, so exporting it after each commit is cheap. Saving on unload would lose data if the tab is killed. Since iteration 8 this applies to the local demonstration mode; in the shared mode only the changed rows are sent to the server (D13).
D4	Business rules as pure functions in one module	Rules inside SQL queries; rules inside the service functions	Pure functions can be tested at boundary values without a database (Section 6.4) and read against the legal source. Each function cites its BR in a JSDoc comment.
D5	One service function per use-case step, each in one transaction	Object-oriented service classes; direct SQL from the screens	The mapping from use case to code becomes one-to-one, which makes the traceability matrix derivable from the code. A transaction per step guarantees that a step either happens completely or not at all.

#	Decision	Alternatives considered	Rationale
D6	First acceptance enforced by a conditional update inside the transaction	Optimistic locking with a version column; a lock table	UPDATE SubRequest SET status = 'ACCEPTED' ... WHERE id = ? AND status = 'REQUESTED' changes exactly one row for the first acceptance and none for any later one; the service checks the number of changed rows (NFR-04). In the shared mode the update runs on the working copy of each browser, and the row lock and revision check of the server decide which acceptance is saved first (D13, Section 4.6.2).
D7	Adjustable demo clock instead of the real time	Real system time; time only in tests	Expiry, deadlines and weekly results must be demonstrable in a presentation of fifteen minutes. The seed data is fixed to the week of 2026-09-28, so the clock must be too. Since iteration 8 this applies to the local demonstration mode; the shared mode uses the server time (D14).
D8	Weekly summaries and payroll stored as derived tables	Compute on every screen and never store	A confirmed payroll must not change when shifts change later (FR-15), so payroll must be stored. Weekly summaries are stored so that payroll can sum the holiday hours of the month's weeks with one query; they are recomputed whenever their inputs change.
D9	Notifications as rows of a table, written by the services	Events published to the screens; browser notifications	An in-app inbox (FR-21) that survives reloads needs stored rows. Writing them in the same transaction as the state change guarantees that a notification exists exactly when the change happened.
D10	Errors as <code>ServiceError(code, message, params)</code>	Plain exceptions with English text	The UI translates the code into English or Korean and fills in the parameters (for example the name in "Already taken by ..."), while the tests assert on the code.
D11	No framework, no build step; hash routing	A component framework with a bundler	NFR-08 forbids a build step. Hash routing (<code>#/inbox</code>) works on static hosting without server rewrites. Eleven screens are few enough to render with template strings.
D12	An old saved database is replaced by the seed, not migrated	Migration with ALTER TABLE	When <code>Payroll.minWageAck</code> was added in iteration 2, replacing kept the schema exactly as spec §8. Only demonstration data is lost (TC-002).
D13	Shared mode by a working copy and change sets	One database function per use-case step; direct writes to the tables under row-level-security policies	Each command runs the unchanged services on an in-memory copy of the store, and only its row changes go to the server, which applies them under a row lock and only to the revision on which they were computed (Section 4.1.2). The services, the rules and their tests therefore remain one implementation for both modes. One function per step would implement every rule twice, in SQL and in JavaScript, and the two could disagree; direct table writes allow no transaction across several tables and therefore no guarantee of first acceptance.
D14	Integrity rules and time enforced by the server	Rely on the checks of the client and on the clock of the device	A browser can be modified, so the server re-checks what it can decide without knowing the user: the state transitions of spec §7, acceptance and approval only before the deadline and the shift start, expiry only when overdue, read-only confirmed attendance and payroll, and creation times within ten minutes of the server time (NFR-14). For the same reason the clock of the shared mode is the server time (BR-12).

#	Decision	Alternatives considered	Rationale
D15	No passwords: the server guarantees integrity, not identity	User accounts with passwords and data restricted to their owner	Password authentication is outside the scope of the project (FR-20). Anyone who opens the application can therefore act as any demo account. The server guarantees integrity, first acceptance, server time and the limits (D16); all shared data are fictional, and the sign-in screen says so. Phone numbers and wages are still shown only to the owner and the worker concerned (NFR-11).
D16	Rate limits and size caps per store	No limits; limits per visitor	Without accounts a visitor cannot be identified, so the store is the unit that the server limits: at most 60 change sets a minute and 3,000 a day, 256 kB and 2,000 rows per change set, and 20,000 rows per store (NFR-14). The counters are updated in the same transaction as the change set.
D17	Reset for everyone by a database function from a seed template	A reset by the client through ordinary change sets; no reset	A demonstration needs a known starting state. <code>app_reset</code> rebuilds the store from the template <code>app_seed_template()</code> , which is generated from the seed of spec §10 and moved by whole weeks to the current date (Section 4.5.2); it is limited to once a minute and 50 times a day. A reset through change sets would have to delete requests and notifications, which the server never accepts from a client.
D18	Local demonstration mode kept	Shared mode only	The local mode keeps the behaviour of iterations 1 to 7 with the adjustable demo clock. It is the fallback when the shared database cannot be reached (NFR-16), the mode of the automated checks and screenshots, which need a fixed clock, and a mode for a demonstration that other visitors cannot disturb.

[illegible]

The five service classes are a logical grouping of the exported functions of `services.js` by concern; in the code they are functions of one ES module, not JavaScript classes. The class `LaborRules` stands for the module `rules.js`, whose functions are static in the sense that they have no state. The tables below list the implementing functions with their parameters.

Table 43. Service classes and their implementing functions in services.js			
Logical class	Function (real signature)	UC	Responsibility

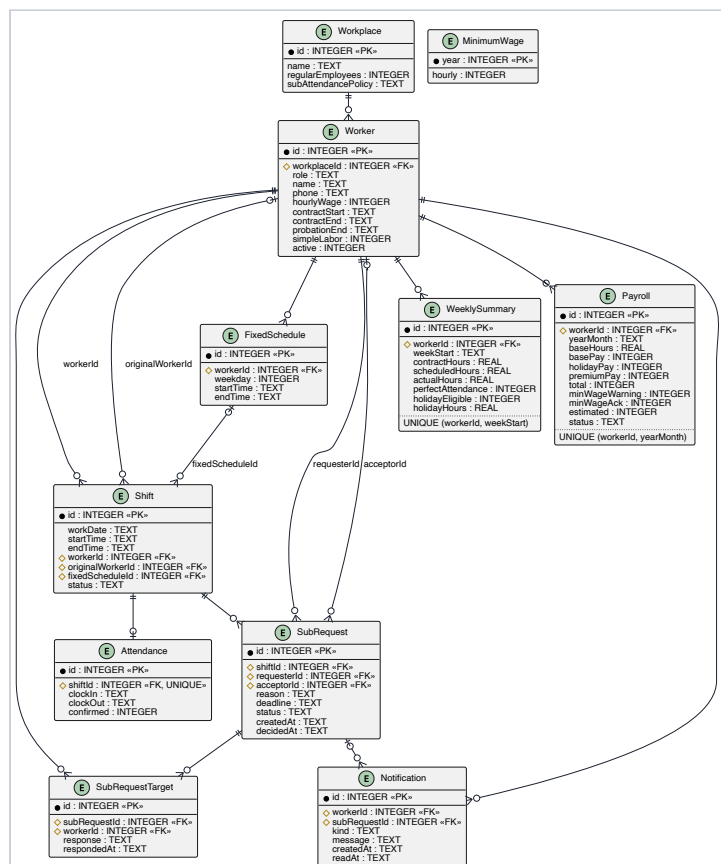
Logical class	Function (real signature)	UC	Responsibility
ScheduleService	registerWorker(data)	UC-01	Validate and insert or update a worker (FR-01).
	saveFixedSchedule(workerId, slots)	UC-01	Replace a worker's fixed shifts; refuse overlaps; unlink shifts of deleted rows (FR-02).
	generateWeek(weekStart)	UC-03	Create the week's shifts from active fixed schedules without duplicates (FR-04).
	editShift(shiftId, changes)	UC-03	Add, move or delete one shift; refuse overlaps and deletion of shifts in use (FR-05).
	updateSettings({name, regularEmployees, subAttendancePolicy, minimumWages})	UC-10	Workplace settings and minimum-wage table (FR-17).

Logical class	Function (real signature)	UC	Responsibility
SubstituteService	createSubRequest({shiftId, requesterId, reason, deadline})	UC-04	Validate (BR-03), select candidates (BR-01), create targets and notifications; without candidates end the request FAILED (FR-06–FR-08, BR-13).
	respondToRequest(requestId, workerId, response)	UC-05	Accept or answer can't; first acceptance wins; the last can't ends the request FAILED (BR-02, BR-13, FR-09, FR-10, FR-22).
	decideRequest(requestId, decision)	UC-06	Approve or reject; reassign the shift and recompute both weeks (BR-04, FR-12).
	cancelRequest(requestId, requesterId)	UC-12	Cancel an open request; close pending targets; notify (FR-19).
	expireOverdue()	UC-11	Expire open requests past deadline or shift start (BR-12, FR-18).
AttendanceService	recordAttendance(shiftId, workerId, clockIn, clockOut)	UC-07	Worker records times of a started shift (FR-13).
	confirmAttendance(shiftId)	UC-07	Owner confirms; shift becomes WORKED; week recomputed.
	markAbsent(shiftId)	UC-07	Owner marks a started shift ABSENT; week recomputed.
	recomputeWeek(weekStart, workerIds)	UC-08	Compute and store WeeklySummary rows (FR-14, BR-06, BR-07).
PayrollService	generatePayroll(yearMonth)	UC-09	Build DRAFT rows of a month (FR-15, FR-16, BR-08–BR-11).
	acknowledgeMinWage(yearMonth, workerId)	UC-09	Set minWageAck = 1 on a warned DRAFT row (BR-09).
	confirmPayroll(yearMonth)	UC-09	Confirm the month's drafts if every warning is acknowledged.
	prepareMonthlyPayroll(nowIso) → {yearMonth, created}	UC-14	Called by the System Clock: if the previous month has no rows, build its DRAFT as generatePayroll does and notify the owner with PAYROLL_DRAFT_READY; otherwise do nothing (FR-23, BR-14).
NotificationService	listNotifications(workerId)	FR-21	Inbox newest first, unread count, live state of each request.
	markNotificationsRead(workerId, ids)	FR-21, FR-22	Mark all of the user's notifications read, or only the given ones (acknowledging a FAILED card).

The operations in the diagram carry the real parameters of the implementation, including `acknowledgeMinWage(yearMonth, workerId)`, which was added to the payroll service in iteration 2 when the acknowledgement became a stored column, and `confirmPayroll(yearMonth)`, which confirms all DRAFT rows of a month at once. In addition to the commands, `services.js` exports read-only query functions used by the screens, which do not change data: `getWorkplace`, `getSettings`, `getWorker`, `listWorkers`, `getFixedSchedules`, `getWeekShifts`, `listRequestableShifts`, `listMyRequests`, `listOwnerRequests`, `getApprovalPreview`, `previewCandidates`, `getShiftDetail`, `getWorkerHome`, `getOwnerHome`, `listAttendance`, `getPayroll`, `unreadCount`, `menuFor`, `routesFor` and `defaultDeadline`. The two Home queries also return the unacknowledged `REQUEST_FAILED` notifications, which the screens show as FAILED cards. Iteration 7 added `listPayrollMonths()`, which returns every month from the first contract month to the month of the demo clock with its status (`CONFIRMED`, `DRAFT` or `NONE`) and whether the System Clock prepared its draft; it feeds the month switcher of *Pay* → *Monthly*. The class diagram groups these read-only queries in the class `HomeQueries`. Among them,

The class `LaborRules` corresponds to the twelve functions of `rules.js` named in the final prompt: `overlaps`, `durationHours`, `weekStartOf`, `isEligibleCandidate` (BR-01), `validateRequest` (BR-03), `contractHours`, `holidayHours` and `computeWeeklySummary` (BR-06, BR-07), `premiumHours` (BR-08), `minimumWageFor` (BR-09), `probationApplies` (BR-10) and `computePayrollRow` (BR-07–BR-11). Iteration 6 added a thirteenth function, `hasNoTaker` (BR-13), which decides from the responses of a request's targets whether anybody can still take it. Iteration 8 added `isPartTime`, which treats a worker with weekly contractual hours C of $0 < C < 40$ as part-time (BR-08 (a)), and `partTimeLimitCheck` (BR-15), which returns the hours beyond contract and whether the approval preview must warn; `premiumHours` and `computePayrollRow` received the optional parameters `contractHours`, `month` and `context` for the hours beyond contract (BR-08 (iv)). Each function receives plain values and returns plain values; their algorithms are given in Section 4.8.

4.5 Database Design



The database has eleven tables, nearly three times the minimum of four set by the course; the twelfth table of earlier versions, Availability, was removed in iteration 6 (Section 3.11.4). The schema is the SQL of spec §8, executed verbatim by `db.js`. All identifiers are English (NFR-09). SQLite has few column types; the schema uses `INTEGER` for keys, counts, flags (0/1) and amounts in won, `REAL` for hours, and `TEXT` for dates (YYYY-MM-DD), times (HH:MM), datetimes (YYYY-MM-DDTHH:MM) and enumerations, which are restricted by `CHECK` constraints. Storing dates and times as ISO text has a practical advantage: they sort and compare correctly as strings, so the services can compare a deadline with `now()` directly. Money is never stored as a floating-point number. Since iteration 8 the same eleven tables also exist in the shared database, where they are held in PostgreSQL with a store column and composite keys; Section 4.5.2 gives this physical mapping.

The tables fall into four groups: master data (Workplace, Worker, FixedSchedule, MinimumWage), transactions (Shift, SubRequest, SubRequestTarget, Attendance), derived results (WeeklySummary, Payroll) and communication (Notification). The table below summarises each table with its key columns and the requirements and rules that use it; every table has an `INTEGER PRIMARY KEY` column `id` except `MinimumWage`, whose key is the year. The complete schema with every column, type and constraint is the SQL of spec §8 in the specification file `spec/spec.md` in the repository (<https://github.com/keem-logan/shiftswap/blob/main/spec/spec.md>).

Table 44. Database tables: purpose, key columns and the requirements that use them

Table	Purpose	Key columns and constraints	Used by
Workplace	The store and its settings	<code>name</code> ; <code>regularEmployees</code> (default 4); <code>subAttendancePolicy</code> $\in$ {EXCUSED, ABSENT}	FR-17; BR-06, BR-08
Worker	Owner and workers with contract and wage data	<code>role</code> $\in$ {OWNER, WORKER}; <code>name</code> ; <code>phone</code> ; <code>hourlyWage</code> (KRW); <code>contractStart</code> , <code>contractEnd</code> (null = open-ended), <code>probationEnd</code> ; <code>simpleLabor</code> ; <code>active</code>	FR-01, FR-20; BR-01, BR-09, BR-10; NFR-11
FixedSchedule	Fixed weekly shifts of a worker	<code>workerId</code> $\rightarrow$ Worker; <code>weekday</code> 1–7 (CHECK); <code>startTime</code> , <code>endTime</code>	FR-02, FR-04; BR-06
Shift	One shift on a date	<code>workDate</code> , <code>startTime</code> , <code>endTime</code> ; <code>workerId</code> (current) and <code>originalWorkerId</code> (first owner after a swap) $\rightarrow$ Worker; <code>fixedScheduleId</code> (null for added shifts); <code>status</code> $\in$ {SCHEDULED, WORKED, ABSENT}	FR-04, FR-05, FR-12, FR-13; BR-01, BR-03, BR-04, BR-06, BR-11
SubRequest	A substitute request and its state	<code>shiftId</code> $\rightarrow$ Shift; <code>requesterId</code> , <code>acceptorId</code> $\rightarrow$ Worker; <code>reason</code> ; <code>deadline</code> ; <code>status</code> $\in$ seven states including FAILED (CHECK); <code>createdAt</code> , <code>decidedAt</code>	FR-06–FR-12, FR-18, FR-19, FR-22; BR-02–BR-04, BR-12, BR-13
SubRequestTarget	One eligible candidate of a request	<code>subRequestId</code> $\rightarrow$ SubRequest; <code>workerId</code> $\rightarrow$ Worker; <code>response</code> $\in$ {PENDING, ACCEPTED, DECLINED, CLOSED}; <code>respondedAt</code>	FR-07, FR-09, FR-10, FR-19, FR-22; BR-01, BR-02, BR-13
Attendance	Recorded clock-in and clock-out of a shift	<code>shiftId</code> $\rightarrow$ Shift, UNIQUE; <code>clockIn</code> , <code>clockOut</code> (HH:MM on the shift date); <code>confirmed</code> 0/1	FR-13; BR-06, BR-11
WeeklySummary	Derived weekly hours and holiday allowance	<code>workerId</code> , <code>weekStart</code> (Monday), UNIQUE together; <code>contractHours</code> , <code>scheduledHours</code> , <code>actualHours</code> ; <code>perfectAttendance</code> , <code>holidayEligible</code> ; <code>holidayHours</code>	FR-11, FR-14; BR-05–BR-07
Payroll	Monthly pay per worker	<code>workerId</code> , <code>yearMonth</code> , UNIQUE together; <code>baseHours</code> , <code>basePay</code> , <code>holidayPay</code> , <code>premiumPay</code> , <code>total</code> (KRW integers); <code>minWageWarning</code> , <code>minWageAck</code> , <code>estimated</code> ; <code>status</code> $\in$ {DRAFT, CONFIRMED}	FR-15, FR-16, FR-23; BR-07–BR-11, BR-14
MinimumWage	Minimum hourly wage per year	<code>year</code> (primary key); <code>hourly</code>	FR-16, FR-17; BR-09
Notification	In-app message to one user	<code>workerId</code> $\rightarrow$ Worker; <code>subRequestId</code> $\rightarrow$ SubRequest; <code>kind</code> (nine kinds); <code>message</code> ; <code>createdAt</code> ; <code>readAt</code> (null = unread)	FR-07, FR-08, FR-10, FR-12, FR-18, FR-19, FR-21, FR-22, FR-23

4.5.1 Normalisation and integrity

The master and transaction tables are in third normal form: every non-key column depends on the whole primary key and on nothing else. There are no repeating groups — a worker's several fixed shifts are rows of their own table rather than columns of Worker — and names of workers are stored only in Worker and joined where needed. Two tables deliberately hold derived data. WeeklySummary is a cache of values that can be recomputed from Shift, Attendance, FixedSchedule and Workplace; it is kept consistent by recomputing the affected rows inside the transaction that changes their inputs (approval, attendance confirmation, marking absent, payroll generation, and opening the weekly summary screen). Payroll is a snapshot: once CONFIRMED, it must keep the amounts that were paid even if shifts change afterwards (TC-115), which is why it cannot be a view.

Integrity rules are enforced at two levels. The schema enforces what SQL can express locally: primary keys, NOT NULL, the permitted values of every status column, one attendance record per shift (Attendance.shiftId UNIQUE), one summary per worker and week, and one payroll row per worker and month. The services enforce the rules that involve several rows or the clock: no overlapping fixed shifts or shifts of one worker (FR-02, FR-05), at most one open request per shift (BR-03), first acceptance wins (BR-02), and no deletion of a shift that has a request or attendance record (FR-05). Foreign keys are declared with REFERENCES for documentation and for the ER diagram; the services never delete a row that is still referenced. In the shared mode a third level is added: the shared database enforces its foreign keys, allows at most one open request per shift with an index, and re-checks the life cycles and time rules with row triggers (Section 4.5.2), so that a modified client cannot bypass the rules that the server can decide itself.

4.5.2 Physical design of the shared database

The shared database, defined in db/app-shared.sql and applied to the Supabase project shiftswap, maps the logical model of spec §8 to PostgreSQL. The file is idempotent, so it can be applied again after a change. The table below summarises the mapping; the text after it describes the functions, the triggers and the access rights.

Table 45. Physical mapping of the data model to the shared PostgreSQL database

Aspect	Design in db/app-shared.sql
Tables	The eleven tables of spec §8 as app_workplace, app_worker, app_fixed_schedule, app_shift, app_sub_request, app_sub_request_target, app_attendance, app_weekly_summary, app_payroll, app_minimum_wage and app_notification, with the same columns in snake case (workDate → work_date). Two service tables complete them: app_store (id, slug, revision, time of the last reset) and app_store_gate (the rate-limit counters), thirteen tables in all.
Stores and keys	Every mirrored table has a column store_id referencing app_store with ON DELETE CASCADE; the primary key is (store_id, id), for MinimumWage (store_id, year). The ids are those of the working copy, so a row keeps its id in both databases.
Foreign keys and uniqueness	Composite foreign keys such as (store_id, worker_id) → app_worker (store_id, id), and the uniqueness constraints of spec §8 extended by store_id, all DEFERRABLE INITIALLY DEFERRED, so that the rows of a change set may arrive in any order; app_commit checks them before it returns (SET CONSTRAINTS ALL IMMEDIATE).
One open request per shift	A partial unique index app_sub_request_open_idx on (store_id, shift_id) for the rows with status REQUESTED or ACCEPTED (BR-03).
Types and CHECKs	Dates as date; times and datetimes as text with the formats of spec §8, checked by regular expressions; flags as smallint restricted to 0 and 1; hours as double precision; amounts as integers; the status and enumeration columns restricted by the same CHECK lists as in SQLite, and text columns limited in length.
Row triggers	Seven triggers on SubRequest, SubRequestTarget, Shift, Attendance, Payroll, Notification and Workplace (described below).
Access	Row-level security with read policies on the twelve readable tables; SELECT granted to the API roles on these tables and nothing else; app_store_gate is not readable. EXECUTE is granted only on app_snapshot, app_commit and app_reset and revoked on every helper function.

The three public functions form the whole interface of the shared database. app_snapshot(store) is a read-only SQL function that returns the revision, the server time and every row of the store, grouped by the table names of the working copy. app_commit(store, base_rev, changes) and app_reset(store) are SECURITY DEFINER functions with an empty search_path, so that they write with the rights of their owner and cannot be redirected to objects of the caller. app_commit refuses a change set larger than 256 kB or 2,000 row operations, locks the row of the store, refuses a stale base revision (STALE), checks and updates the counters of app_store_gate (60 change sets a minute, 3,000 a day), and applies the

deletes, the updates and then the inserts. A delete is accepted only for FixedSchedule, Shift, WeeklySummary, Payroll and MinimumWage rows; requests, targets, attendance records and notifications are never deleted by a client. An update or delete that finds no row is refused as a conflict. After the inserts the function refuses a store of more than 20,000 rows (STORE_FULL), checks the deferred constraints and increments the revision. Because all of this runs in one transaction, a refused change set also leaves the rate-limit counters unchanged.

The row triggers re-check what the server can decide without knowing the user. The trigger on SubRequest permits only the transitions of the state machine of spec §7 (Section 4.7.1): a new request must be REQUESTED, or FAILED when it has no candidates, and must be made by the current worker of a SCHEDULED shift, with a deadline between the server time and the shift start; an acceptance must come from one of the request's targets and, like an approval, must happen before the deadline and the shift start by server time; an expiry is accepted only when the request is overdue by server time (BR-12); the shift, the requester, the deadline and the reason never change, and a request is never deleted. The other triggers keep a target's worker and request fixed and an answered target unchanged, permit only the shift transitions of Section 4.7.2 and keep the original worker of a shift (BR-04), keep a confirmed attendance record unchanged, make CONFIRMED payroll rows read-only and refuse a confirmation while a minimum-wage warning is unacknowledged (BR-09), allow only the read time of a notification to change, and allow the workplace of a store to be updated but not created or deleted. The creation time of a new request or notification must lie within ten minutes of the server time.

`app_reset(store)` is limited to one reset a minute and 50 a day per store. It replaces the rows of the store with the seed template `app_seed_template()`, whose 353 rows are generated from the seed code of spec §10 by `tools/seed-sql.mjs`; a unit test checks that the template is up to date (TC-16A). All dates of the template are moved by the smallest number of whole weeks that leaves the seed request of Lee Seoyeon open for at least six hours, so weekdays, times and the walk-throughs of spec §10 remain valid. Shifts that have not ended by the server time become SCHEDULED without attendance, and creation times later than the server time are set to it. The template contains no Payroll rows, because calendar months do not move with whole weeks; the first client that opens a store without Payroll rows completes the payroll history with the same code as the local seed (`completeSharedSeed`) and saves it like any other System Clock work.

4.6 Dynamic Design: Sequence Diagrams

The lecture describes a sequence diagram as the interactions that take place during a particular use case, with the objects along the top and function calls as horizontal arrows [6]. Five sequence diagrams — SD-1, SD-2, SD-3a, SD-3b and SD-4 — cover the use cases with the most interaction. For each sequence diagram, the numbered steps name the functions of the implementation.

4.6.1 SD-1 Request substitute (UC-04)

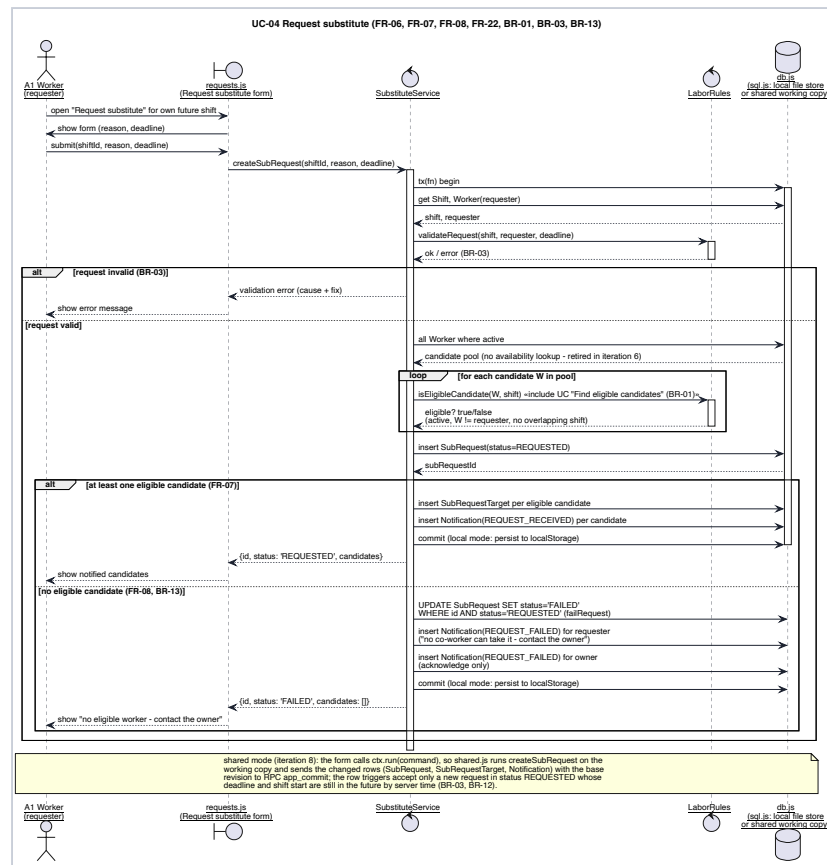


Figure 10. SD-1 — request substitute; in the shared mode the created rows are saved through app_commit (UC-04; FR-06–FR-08, BR-01, BR-03)

1. The worker starts the request flow. Its first step lists the worker's requestable shifts with `listRequestableShifts(workerId)`, and its second step fills the deadline with `defaultDeadline(shift)`, the earlier of `now + 24 h` and the shift start.
2. When the worker sends the request from the review step, the screen calls `createSubRequest({shiftId, requesterId, reason, deadline})`, which opens a transaction with `tx()`.
3. The service reads the shift and counts the open requests on it, then calls `validateRequest` with the current time from `now()`. If the result contains an error code, it throws a `ServiceError` with that code; the transaction rolls back and the screen shows the translated message (the "request invalid" branch).
4. Otherwise the service inserts the `SubRequest` with status `REQUESTED`.
5. For every worker, the service reads that worker's shifts on the date and calls `isEligibleCandidate` (the loop of the diagram; the included use case *Find eligible candidates*).
6. For each eligible candidate it inserts a `SubRequestTarget` with response `PENDING` and a `REQUEST_RECEIVED` notification. If there is none, `hasNoTaker` holds for the empty list of responses: the service sets the request to `FAILED` and sends `REQUEST_FAILED` to the requester and the owner (BR-13).
7. The transaction commits. In the local mode the database is saved to `localStorage`; in the shared mode the gateway sends the created rows as one change set to `app_commit`, whose trigger checks the status, the deadline and the creation time against the server time, and the screen waits for this save. The service returns the request id, its status (`REQUESTED` or `FAILED`) and the list of candidates, which the screen shows.

4.6.2 SD-2 Accept request with concurrent second acceptance (UC-05)

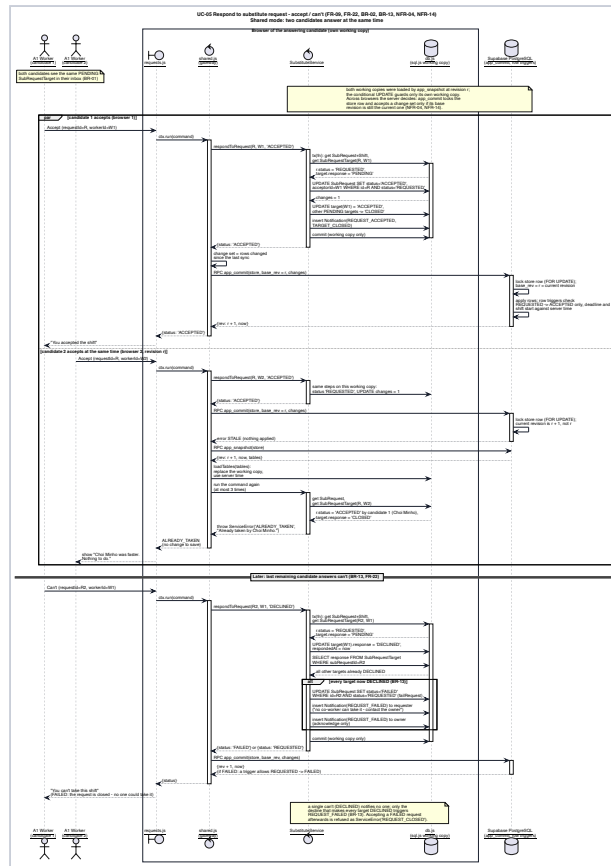


Figure 11. SD-2 — respond to a request; of two simultaneous acceptances the server applies the first, and the second receives STALE, reloads, re-runs the service and is told that Choi Minho was faster; the last answer of can't ends the request (UC-05; FR-09, FR-10, FR-22, BR-02, BR-13, NFR-04)

- Both candidates see the same pending request in their own browsers, built for their screens by the service queries (`listNotifications`, and on the Home screen `getWorkerHome`), with *Accept* and *Can't*. Both working copies are at revision r of the store.
- Candidate 1 selects *Accept*; the screen passes `respondToRequest(requestId, workerId, 'ACCEPTED')` to the gateway (`ctx.run`), which runs it on the working copy inside a transaction.
- Inside the transaction the service reads the request and the candidate's target. Both are `REQUESTED` and `PENDING`, so the service executes the conditional update `UPDATE SubRequest SET status = 'ACCEPTED', acceptorId = ? WHERE id = ? AND status = 'REQUESTED'`. The database reports one changed row.
- The service sets the candidate's target to `ACCEPTED`, sets every other `PENDING` target to `CLOSED` with a `TARGET_CLOSED` notification, and sends `REQUEST_ACCEPTED` to the requester and the owner. The transaction commits on the working copy, and the gateway sends the changed rows with base revision r to `app.commit`. The server locks the store row, finds revision r current, lets the trigger check the transition `REQUESTED` $\rightarrow$ `ACCEPTED`, the target and the deadline and shift start against the server time, applies the rows and returns revision $r + 1$.
- Candidate 2 selects *Accept* at nearly the same moment. On the working copy of the second browser the request is still `REQUESTED`, so the same steps succeed locally, and the gateway sends its change set with base revision r . If it arrives while the first change set is being applied, the server waits for the row lock; in either case it then finds revision $r + 1$ and refuses the change set with `STALE`, and nothing of it is written.
- The gateway of candidate 2 reloads the snapshot and runs `respondToRequest` again on the current data. The first read now finds the request `ACCEPTED` with another acceptor, and the service throws `ServiceError('ALREADY_TAKEN')` with the acceptor's name, which the screen shows as "already taken by Choi Minho". Because the re-run changed nothing, nothing more is sent. Within one working copy — and in the local mode, where both accounts use the same database — the conditional update alone decides: an update that finds the status no longer `REQUESTED` changes zero rows, and the service refuses with the same code.
- The last part of the diagram shows an answer of *can't* from the last candidate whose target is still `PENDING` (BR-13, FR-22). Inside the transaction the service sets the target to `DECLINED` and reads the responses of all targets of the request again. If every target is now `DECLINED`, the service sets the request to `FAILED` with a conditional update on `status = 'REQUESTED'` and sends `REQUEST_FAILED` to the requester and the owner; otherwise the request stays

REQUESTED and no one is notified. In the shared mode this change set also goes through `app_commit`, so two answers of can't given at the same time are serialised by the revision check, and the second is recomputed on data that already contain the first.

SQLite has no `SELECT ... FOR UPDATE`, so the service does not lock the row before reading it. Instead, the condition `status = 'REQUESTED'` is part of the update itself, and the check of the changed-row count happens in the same transaction; only one of two competing updates can affect the row. This is sufficient within one database. Across browsers, where each acceptance is first computed on a separate working copy, the guarantee of NFR-04 comes from the server: `app_commit` applies change sets one at a time under the row lock of the store and only to the revision on which they were computed, so of two change sets computed on the same revision only the first is applied (D6, D13). In addition, the trigger permits ACCEPTED only from REQUESTED, and at most one open request per shift is allowed by the partial unique index. The scenario was tested with the throwaway store of the REST test (Chapter 6) and with two headless browsers (`tools/shared-check.mjs`), in which one of two simultaneous acceptances succeeded and the other was told that the request had already been taken.

4.6.3 SD-3 Approve substitution with weekly recomputation (UC-06)

The approval use case is shown in two diagrams: the comparison that the owner sees before deciding (SD-3a), and the decision itself (SD-3b).

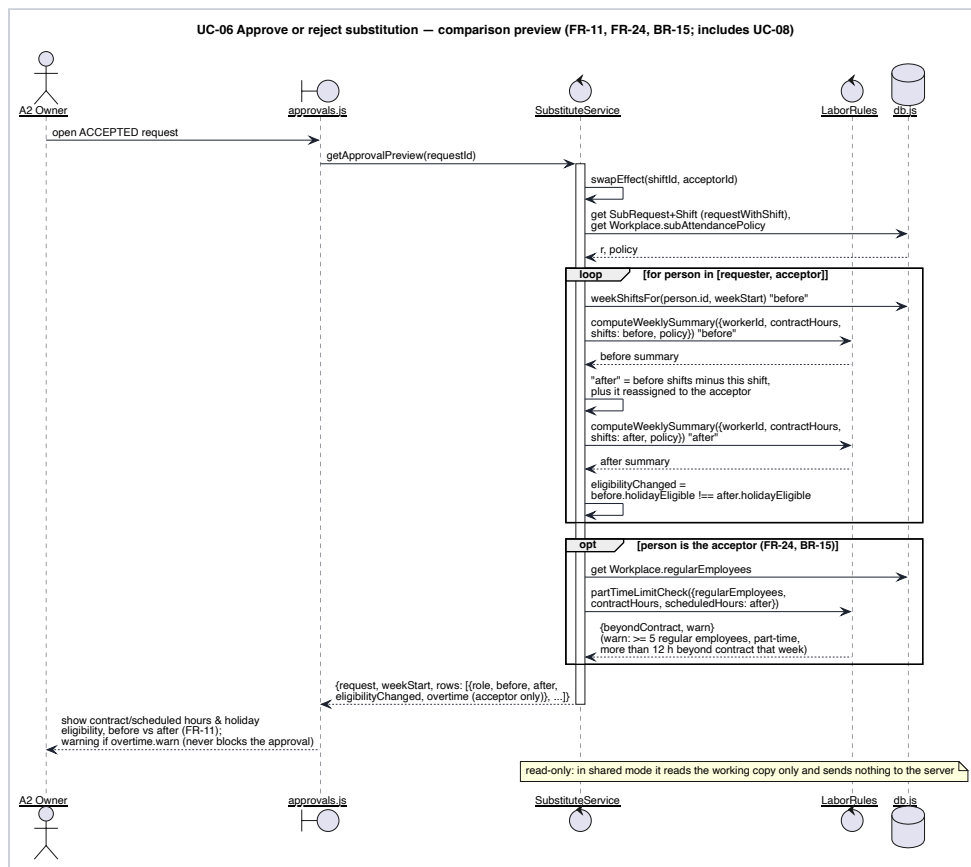


Figure 12. SD-3a — approval preview: before/after comparison for requester and acceptor, with a warning when the acceptor would exceed the part-time limit (UC-06; FR-11, FR-24, BR-15; uses UC-08)

1. The owner opens an accepted request; the screen calls `getApprovalPreview(requestId)`. The service reads the request with its shift (`requestWithShift`) and the workplace's substitution-attendance policy.
2. For the requester and then the acceptor, the service reads the shifts of the week in which the person is current or original worker (`weekShiftsFor`) and calls `computeWeeklySummary` for this "before" list.
3. It builds the "after" list by removing the requested shift and adding it back reassigned to the acceptor, and calls `computeWeeklySummary` again.
4. It sets `eligibilityChanged` when the eligibility before and after differ, and returns one row per person with both summaries. The screen shows the impact lines and highlights a changed eligibility (FR-11). The preview writes nothing to the database.
5. Since iteration 8 the acceptor's row also carries `overtime`, the result of `partTimeLimitCheck` with the workplace's number of regular employees, the acceptor's contractual hours and the scheduled hours of the "after" summary (BR-15, FR-24). If `overtime.warn` is set, the decision card and the details show that the acceptor would work the given number

of hours beyond contract, with the limit of 12 hours a week for part-time workers (Section 4.8.3). The warning does not disable *Approve*.

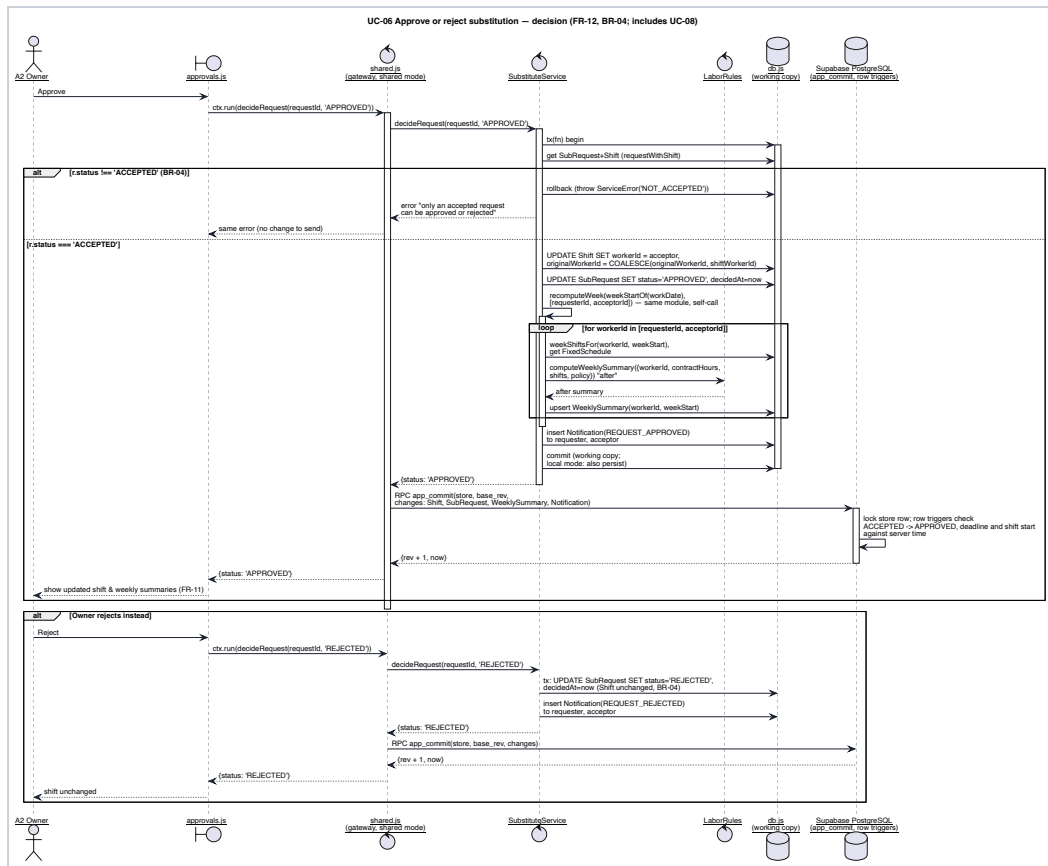


Figure 13. SD-3b — approve or reject: shift reassignment and weekly re-computation, computed on the working copy and committed to the shared database through the gateway (UC-06; FR-12, BR-04; includes UC-08)

1. The owner selects *Approve*; the screen calls `decideRequest(requestId, 'APPROVED')`, which opens a transaction and reads the request with its shift. If the request is not **ACCEPTED** (BR-04), the service throws **NOT_ACCEPTED** and the transaction rolls back.
2. Otherwise it executes `UPDATE Shift SET workerId = acceptor, originalWorkerId = COALESCE(originalWorkerId, shiftWorkerId)`, so the first owner of the shift is kept even after several swaps, and sets the request to **APPROVED** with the decision time.
3. It calls `recomputeWeek(weekStart, [requesterId, acceptorId])` in the same module. The nested call joins the same transaction; for each of the two workers it reads the week's shifts and fixed schedule, calls `computeWeeklySummary` and writes the result with an upsert (`INSERT ... ON CONFLICT (workerId, weekStart) DO UPDATE`).
4. It sends **REQUEST_APPROVED** to the requester and the acceptor and commits. In the shared mode the gateway sends the changed `Shift`, `SubRequest`, `WeeklySummary` and `Notification` rows as one change set to `app_commit`, whose trigger accepts **ACCEPTED** → **APPROVED** only before the deadline and the shift start by server time; if the request expired in the meantime, the command is re-run on reloaded data and the owner sees the service's own message. The schedule, read with `getWeekShifts(weekStart)`, now shows the handover marker because the shift has an original worker.
5. If the owner rejects instead, `decideRequest(requestId, 'REJECTED')` sets the request to **REJECTED**, leaves the shift unchanged and sends **REQUEST_REJECTED** to both workers; in the shared mode this change set is saved in the same way.

Figure 14. SD-4 — generate and confirm monthly payroll including the part-time premium, by the owner or automatically by the System Clock; in the shared mode the System Clock draft is saved by the open client (UC-09, UC-14; FR-15, FR-16, FR-23, BR-08–BR-11, BR-14, BR-15; includes UC-08)

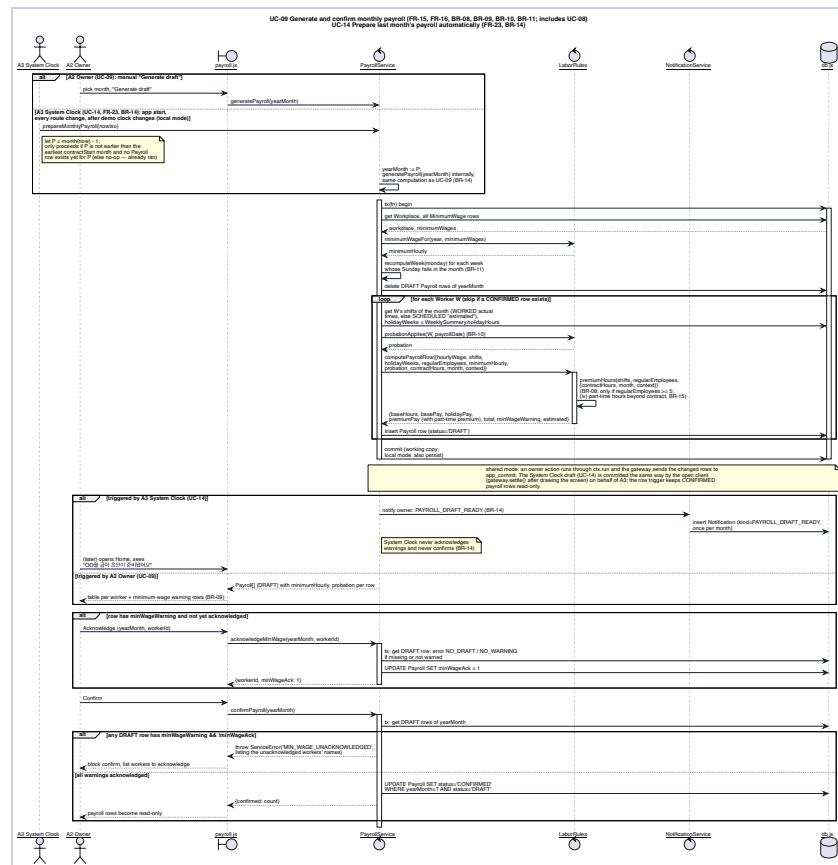


Figure 14. SD-4 — generate and confirm monthly payroll including the part-time premium, by the owner or automatically by the System Clock; in the shared mode the System Clock draft is saved by the open client (UC-09, UC-14; FR-15, FR-16, FR-23, BR-08–BR-11, BR-14, BR-15; includes UC-08)

1. The draft is started in one of two ways, shown as alternatives at the top of the diagram. The owner selects a month in the monthly payroll of the *Pay* tab and creates the draft; the screen calls `generatePayroll(yearMonth)`. Or the System Clock, which runs on application start, on every route change and, in the local mode, after a change of the demo clock, calls `prepareMonthlyPayroll(nowIso)`. This service takes the previous month *P* and continues only if *P* is not earlier than the first contract month and has no Payroll row yet; it then performs the same generation for *P* (BR-14). Otherwise it returns at once, which makes repeated runs within a month harmless.
2. The service looks up the minimum wage of the year with `minimumWageFor` and determines the weeks whose Sunday falls in the month. For each of these weeks it calls `recomputeWeek` (the included UC-08).
3. It deletes the DRAFT rows of the month; CONFIRMED rows are kept and their workers are skipped.
4. For each worker it reads the month's shifts that are not ABSENT. A WORKED shift with recorded times contributes its clock-in and clock-out; any other shift contributes its scheduled times and is marked estimated (BR-11). It reads the holiday hours of the month's weeks from `WeeklySummary`.
5. It evaluates `probationApplies(worker, date)` on the first day of the month or the contract start, if later (BR-10), and calls `computePayrollRow`, which computes base pay, holiday pay, premium pay through `premiumHours`, the total and the minimum-wage warning. Since iteration 8 the service also passes the worker's contractual hours, the payroll month and, as context, the worker's shifts of the first and last ISO weeks of the month on dates outside it, so that `premiumHours` can count the hours of a part-time worker beyond contract (BR-08 (iv), Section 4.8.3). The row is inserted as DRAFT.
6. After an automatic preparation the service inserts one PAYROLL_DRAFT_READY notification for the owner, whose Home then shows that last month's draft is ready; it never acknowledges a warning or confirms. After a manual generation the screen shows the rows. In both cases, a row with a warning has a button to acknowledge it, which calls `acknowledgeMinWage(yearMonth, workerId)` and sets `minWageAck = 1`.
7. Confirming the month calls `confirmPayroll(yearMonth)`. If any DRAFT row has a warning without acknowledgement, the service throws MIN_WAGE_UNACKNOWLEDGED; otherwise all DRAFT rows of the month become CONFIRMED.

In the shared mode the owner's commands of this diagram are saved through `app_commit` like any other command. A draft prepared by the System Clock is computed by whichever client has the application open while it draws a screen and is then saved by `gateway.settle()`; the trigger on the Payroll table keeps CONFIRMED rows read-only and refuses a confirmation while a warning is unacknowledged, so a draft computed by a second client at the same moment is refused as stale and not applied twice.

The division of work between the service and the rules is visible in the diagram: the service calls `minimumWageFor` and `probationApplies` and passes their results to `computePayrollRow`, which calls `premiumHours` itself. This keeps `computePayrollRow` independent of dates and of the database.

4.7 State Design

Three entities have a life cycle that the business rules constrain: the substitute request, the shift and the payroll row. The lecture's rules for state diagrams — one initial state, every state reachable, a path to a final state from every state, and every transition labelled with an event [6] — hold for all three. For each transition the text names the function that performs it, which is how the design was checked against the code.

4.7.1 SubRequest

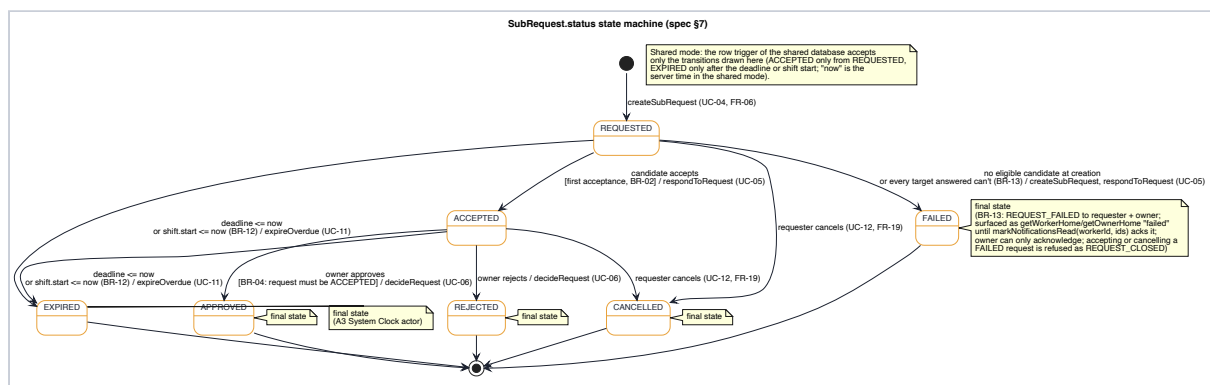


Figure 15. State machine of `SubRequest.status`; in the shared mode the row trigger of the server accepts only the drawn transitions and judges expiry by server time

Table 46. State transitions of `SubRequest.status`

From	Event [guard]	To	Function
(initial)	worker submits a valid request [BR-03]	REQUESTED	<code>createSubRequest</code>
REQUESTED	candidate accepts [status = REQUESTED at the moment of the transaction]	ACCEPTED	<code>respondToRequest(..., 'ACCEPTED')</code>
REQUESTED	no eligible candidate at creation [BR-13]	FAILED	<code>createSubRequest</code> , in the same transaction
REQUESTED	candidate answers can't [another target still PENDING]	REQUESTED	<code>respondToRequest(..., 'DECLINED')</code> (target only)
REQUESTED	candidate answers can't [every target DECLINED, BR-13]	FAILED	<code>respondToRequest(..., 'DECLINED')</code> , then <code>failRequest</code>
REQUESTED, ACCEPTED	deadline $\leq$ now or shift start $\leq$ now	EXPIRED	<code>expireOverdue</code>
REQUESTED, ACCEPTED	requester cancels [requester = caller]	CANCELLED	<code>cancelRequest</code>
ACCEPTED	owner approves [status = ACCEPTED]	APPROVED	<code>decideRequest(..., 'APPROVED')</code>
ACCEPTED	owner rejects [status = ACCEPTED]	REJECTED	<code>decideRequest(..., 'REJECTED')</code>

The targets of a request have a smaller life cycle of their own: PENDING → ACCEPTED (the accepting candidate), DECLINED (an answer of can't), or CLOSED (another candidate accepted, the request expired, or the requester cancelled). Only PENDING targets are ever closed; a target that has accepted keeps the response ACCEPTED even if its request is later cancelled or expires.

In the shared mode the table above is enforced twice: by the services on the working copy, and by the trigger on `app_sub_request`, which refuses any other transition, an acceptance or approval at or after the deadline or the shift start, and an expiry before it, all judged by the server time (Section 4.5.2). The trigger on `app_sub_request_target` enforces the smaller life cycle of the targets.

4.7.2 Shift

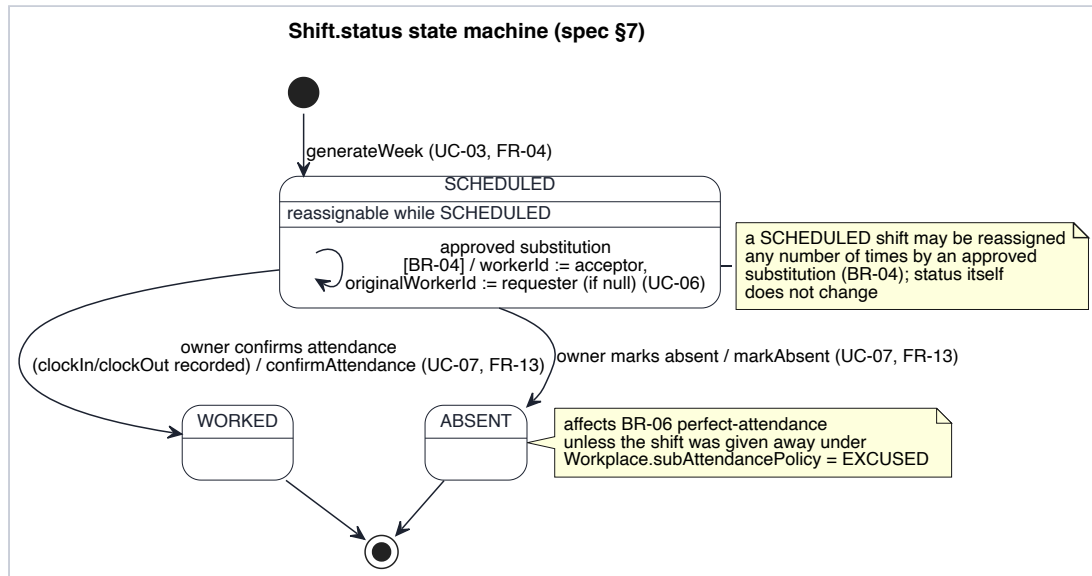


Figure 16. State machine of Shift.status

A shift is created as SCHEDULED by `generateWeek` or `editShift`. An approved substitution (`decideRequest`) changes its worker but not its status. The owner moves it to WORKED with `confirmAttendance`, which requires a recorded attendance, or to ABSENT with `markAbsent`, which requires that the shift has started. WORKED and ABSENT are final with respect to requests and attendance recording: `validateRequest` refuses a request on a shift that is not SCHEDULED, and `recordAttendance` refuses to change a shift that the owner has closed. The attendance record itself has two states, recorded (`confirmed = 0`) and confirmed (`confirmed = 1`).

4.7.3 Payroll

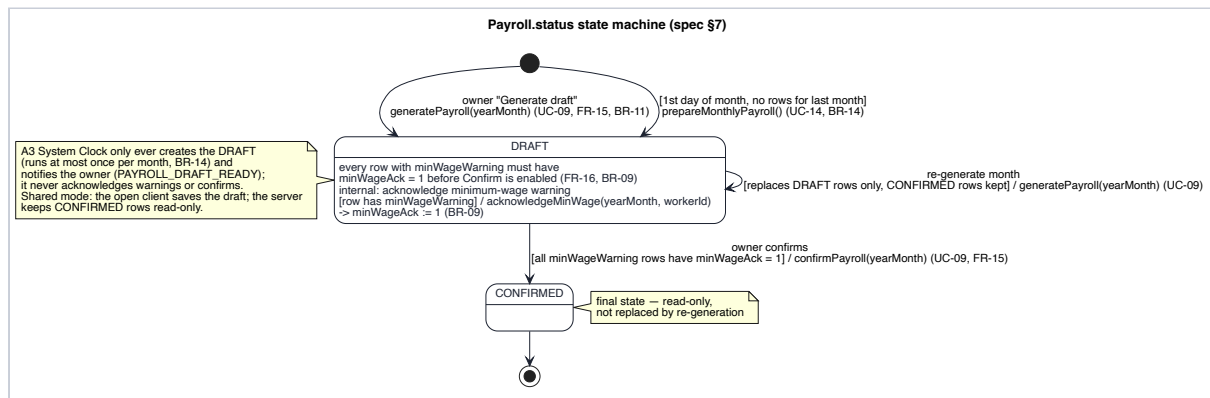


Figure 17. State machine of Payroll.status; in the shared mode the open client saves the System Clock draft, and the server keeps CONFIRMED rows read-only

A payroll row is created as DRAFT in one of two ways: by the owner through `generatePayroll`, or, since iteration 7, by the System Clock through `prepareMonthlyPayroll` on or after the first day of the following month, provided that the month has no rows yet (BR-14). The diagram shows these as two initial transitions. Regenerating the month replaces the DRAFT row with a new one whose `minWageAck` is 0. `acknowledgeMinWage` sets `minWageAck = 1` on a warned DRAFT row, and `confirmPayroll` moves all DRAFT rows of the month to CONFIRMED, but only if every warned row is acknowledged.

CONFIRMED rows are never replaced or changed; in the shared mode the trigger on app_payroll enforces this on the server.

4.8 Algorithm Design

This section specifies the algorithms of the business rules in pseudocode. The pseudocode was derived from rules.js and uses the same names; it omits type conversions and formatting. Times are handled on a minute axis of the shift's date: span(start, end) returns the start and end in minutes after midnight and adds 1,440 minutes to the end when the end is not later than the start, so that an overnight shift is one continuous interval.

4.8.1 Candidate selection (BR-01)

```
function isEligibleCandidate(worker, requesterId, shift, workerShifts):
  if not worker.active:           return false
  if worker.role ≠ WORKER:       return false  -- the owner is never a candidate
  if worker.id = requesterId:    return false
  for x in workerShifts:         -- the worker's shifts on shift.workDate
    if x.workDate = shift.workDate and overlaps(x, shift):
      return false
  return true                    -- no availability check since iteration 6

function hasNoTaker(responses):  -- BR-13; responses of the request's targets
  return every r in responses is DECLINED  -- true for an empty list: no candidate at all

function overlaps(a, b):         -- half-open intervals [start, end)
  (as, ae) := span(a); (bs, be) := span(b)
  return as < be and bs < ae
```

The service applies this function to every worker with role WORKER (createSubRequest). The owner is not in this set, and the order of the checks follows the activity diagram AD-2 in Section 3.9. With five workers and at most a few shifts per worker and day, the cost is negligible. The half-open comparison makes shifts that meet at a boundary, such as 10:00–16:00 and 16:00–22:00, non-overlapping (TC-011); this is why Kang Doyun, whose Sunday shift starts at 16:00, is asked in walk-through S8 for Jung Hana's shift that ends at 16:00. hasNoTaker is evaluated after each answer of can't and once at creation, where an empty list of candidates makes it true, so both endings of BR-13 share one function.

4.8.2 Weekly summary and holiday allowance (BR-06, BR-07)

```
function computeWeeklySummary(W, contract, shifts, policy):
  -- shifts: the week's shifts whose current worker or original worker is W
  scheduled := 0; actual := 0; reasons := []
  if contract < 15: reasons.add(CONTRACT_BELOW_15)
  for s in shifts ordered by date and start:
    original := s.originalWorkerId if set, else s.workerId
    if s.workerId = W:
      scheduled += duration(s)
      if s.status = WORKED and s has clockIn/clockOut:
        actual += duration(s.clockIn, s.clockOut)
      if original ≠ W: continue  -- a shift W took as substitute
      if s.workerId = W and s.status = ABSENT: reasons.add(ABSENT on s.workDate)
      if s.workerId ≠ W and policy = ABSENT: reasons.add(GAVE_AWAY on s.workDate)
  perfect := no reason other than CONTRACT_BELOW_15
  eligible := contract ≥ 15 and perfect
  hours := round2(min(contract, 40) / 40 × 8) if eligible else 0
  return (contract, scheduled, actual, perfect, eligible, hours, reasons)
```

Three properties of the rule are visible in the pseudocode. A shift the worker took as a substitute adds to scheduled hours but is skipped for attendance (the continue), so an absence on someone else's shift does not cost the substitute his own allowance (TC-087). A SCHEDULED shift is never a reason, so a week in progress is a projection (TC-085). And the policy is consulted in exactly one place. Holiday pay is computed later, per week: floor(hours × hourlyWage), and the month sums the rounded weekly amounts (BR-07). For Choi Minh, contract = 16, so hours = round2(16 / 40 × 8) = 3.2 and pay = floor(3.2 × 10,320) = 33,024.

4.8.3 Premium hours and the part-time limit (BR-08, BR-15)

```
function premiumHours(shifts, regularEmployees, options):
  -- options (iteration 8): contractHours C, payroll month, context shifts
```

```

if regularEmployees < 5: return (daily 0, weekly 0, night 0, partTime 0, total 0)
byDay := map date → hours; night := 0
for s in shifts: -- worked or scheduled intervals of the month
  (a, b) := span(s)
  byDay[s.workDate] += (b - a) / 60
  night += minutes of [a, b] inside 22:00–06:00 windows / 60
daily := 0; byWeek := map monday → regular hours
for (date, h) in byDay:
  extra := max(0, h - 8); daily += extra
  byWeek[weekStartOf(date)] += h - extra -- only hours not already counted as daily
overtime
weekly := Σ over weeks of max(0, regular - 40)
partTime := 0
if isPartTime(C): -- 0 < C < 40, rule (a)
  hoursOf := byDay + hours of context shifts on dates outside the month
  for date in hoursOf in chronological order: -- rule (c): the first C hours of the ISO
week are within contract
    before := hours of the week before date; upTo := before + hoursOf[date]
    if date is not in the month: continue -- context only places the first C hours;
never paid
    overContract := max(0, upTo - max(C, before))
    beyond[week] += max(0, overContract - max(0, byDay[date] - 8)) -- rule (d): not
counted in (i)
    partTime := Σ over weeks of max(0, beyond[week] - weekly overtime of that week) -- nor
in (ii)
  return (daily, weekly, night, partTime, daily + weekly + night + partTime)

premiumPay := floor(total × hourlyWage × 0.5)

function partTimeLimitCheck(regularEmployees, C, S): -- BR-15; S = scheduled hours of the week
after the swap
  beyondContract := round2(max(0, S - C))
  return (beyondContract, warn = regularEmployees ≥ 5 and isPartTime(C) and beyondContract >
12)

```

The night windows are checked against three intervals on the minute axis, from 22:00 on the previous day to 06:00, from 22:00 to 06:00 of the next day, and the following night, which covers any shift of less than 24 hours. Weekly overtime is computed from the regular hours only, so an hour that already earned daily overtime is not counted again (TC-094). Because the function receives only the shifts of the payroll month, a week that spans two months contributes only its days inside the month, as BR-08 requires.

Iteration 8 added part (iv) of BR-08, the hours of a part-time worker beyond the weekly contractual hours, which earn a premium of at least 50 % in a workplace with five or more regular employees under the Fixed-term and Part-time Employees Act [15]. The Act does not fix how these hours are to be counted, so the algorithm implements system rules that are design decisions of this project, not statements of law (spec BR-08 (a)–(e)): (a) a worker whose contractual hours C, the sum of the current fixed schedule as in BR-06, satisfy $0 < C < 40$ is part-time; (b) the test is weekly against C, not daily; (c) the hours of an ISO week are taken in chronological order, each shift on its work date, and every hour after the first C hours is beyond contract — in a monthly payroll the days of that week outside the month only determine where the first C hours end; (d) an hour that meets more than one of (i), (ii) and (iv) is counted once; and (e) night hours are added separately. Without the options argument the function returns exactly the result of iteration 7. For the walkthrough S10 of spec §10, Choi Minh (C = 16) works 29 hours in the week of 28 September in the order Tuesday 5, Wednesday 5, Thursday 5, Friday 8 and Saturday 6 hours; the contractual hours end one hour into Friday, so 13 hours are beyond contract and October's premium includes $13 \times 10,320 \times 0.5 = 67,080$ won for them.

partTimeLimitCheck implements the warning of BR-15, which the approval preview computes for the acceptor with the scheduled hours S of the "after" summary (Section 4.6.3). The limit of 12 hours a week beyond contract is taken from the Act [15]; the comparison of the scheduled hours of the swap week with the fixed-schedule hours is again the system rule of BR-08 (a)–(c). Exactly 12 hours beyond contract gives no warning and 13 hours give one (TC-15x). The warning never blocks the approval, because the worker's consent and the owner's decision lie outside the system.

4.8.4 Probation reduction and minimum-wage check (BR-09, BR-10)

```

function probationApplies(worker, date):
  if worker.contractStart is empty or worker.probationEnd is empty: return false
  if worker.simpleLabor: return false
  if worker.contractEnd is set and daysBetween(contractStart, contractEnd) < 365: return false
  if date < contractStart: return false
  if date ≥ addMonths(contractStart, 3): return false

```

```

return date ≤ probationEnd

minimumWageFor(year) := hourly of the latest MinimumWage row with row.year ≤ year
floor    := minimumWageFor(year) × (0.9 if probationApplies(worker, evaluationDate) else 1)
warning  := worker.hourlyWage < floor
evaluationDate := max(first day of the month, contractStart)

```

Every condition of BR-10 is a separate early return, so each can be tested on its own (TC-102): the simple-labour flag, a contract shorter than 365 days, a date three months or more after the contract start, a date after the probation end, and a missing probation end. `addMonths` clamps the day to the length of the target month, so a contract starting on 31 August ends its three months on 30 November. For the seed data in September 2026, Jung Hana's floor is $0.9 \times 10,320 = 9,288$ won and Kang Doyun's is 10,320 won; his wage of 10,000 won produces the warning.

4.8.5 Monthly payroll (BR-11)

```

function generatePayroll(yearMonth):                -- services.js, inside tx()
  weeks := Mondays of the weeks whose Sunday lies in yearMonth
  for monday in weeks: recomputeWeek(monday)
  delete DRAFT rows of yearMonth
  for W in workers with role WORKER:
    if W has a CONFIRMED row for yearMonth: continue
    shifts := W's shifts in yearMonth with status ≠ ABSENT, each as
      (clockIn, clockOut, estimated = false) if WORKED with a record
      else (startTime, endTime, estimated = true)
    holidayWeeks := W's WeeklySummary.holidayHours for the weeks above
    if shifts empty and no holiday hours: continue
    row := computePayrollRow(W.hourlyWage, shifts, holidayWeeks,
      regularEmployees, minimumWageFor(year),
      probationApplies(W, evaluationDate),
      contractHours(W), yearMonth, context) -- BR-08 (iv), iteration
8
    insert row as DRAFT

function computePayrollRow(...):                    -- rules.js
  baseHours := round2(Σ duration(s))
  basePay   := floor(baseHours × wage)
  holidayPay := Σ floor(h × wage) for h in holidayWeeks
  premiumPay := floor(premiumHours(shifts, regularEmployees,
    (contractHours, month, context)).total × wage × 0.5)
  return (baseHours, basePay, holidayPay, premiumPay,
    total = basePay + holidayPay + premiumPay,
    minWageWarning = wage < floor, estimated = any s.estimated)

```

Applied to Choi Minho in September 2026 with the seed data, which since iteration 7 contains his work history from the contract start, his fixed shifts in September amount to 69 hours: five Tuesdays and four Thursdays of 5 hours and four Saturdays of 6 hours. All of them up to 26 September were worked; his shift on Tuesday 29 September is still scheduled and counts with its 5 scheduled hours, which sets the estimated flag. Base pay is $69 \times 10,320 = 712,080$ won. Four weeks have their Sunday in September (6, 13, 20 and 27 September), and in each of them he was eligible with 3.2 holiday hours, so holiday pay is $4 \times 33,024 = 132,096$ won and the total is 844,176 won. These values are asserted by TC-114 and TC-088.

4.9 User-Interface Design

The user interface was redesigned in iteration 4 (Chapter 5) after a usability review of the running application. The earlier design followed the use cases: one screen per function (inbox, approvals, weekly summary, payroll and so on), each reached from a menu. The review found that this made the user find the right screen before acting, and that the most important information — what an action does to someone's hours and pay — was on a separate screen from the action. The final design, fixed in section 2 of the final prompt and in NFR-13, is organised around the user's tasks instead. This section describes its principles, its navigation, its main screens and its visual tokens; Chapter 7 shows the screens.

4.9.1 Design principles

Eight principles apply to every screen and take precedence over the description of an individual screen.

Table 47. User-interface principles of the final design

#	Principle	Meaning in ShiftSwap
---	-----------	----------------------

#	Principle	Meaning in ShiftSwap
1	One screen, one job	Every screen has exactly one primary action, drawn as the only filled blue button; on phones it is a full-width button fixed above the tab bar. Other actions are grey or text buttons.
2	What needs me comes first	Each role opens on a <i>Home</i> screen that lists what is waiting for this person, most urgent first, each item with its own action button.
3	Show the consequence before the commitment	Accepting, approving and confirming payroll show their effect in one sentence with numbers before the button, for example "Minho: this week 16 h → 21 h · holiday allowance unchanged (33,024 won)".
4	Ask one thing at a time	Requesting a substitute is a three-step flow (choose shift → reason and deadline → review and send) with a step indicator and a review that names the recipients.
5	Say what happened and what happens next	Every action ends with a result screen or message that states the outcome and the next step; no state changes silently.
6	Status as progress	A request is shown as a three-step tracker Requested → Accepted → Approved; rejected, expired and cancelled requests show a labelled end state with the reason.
7	The user's words	Korean is the default language, in the polite everyday style; English is available in the menu. Identifiers such as FR, UC, BR and status codes never appear. Dates read like "9월 30일(수) 18:00–23:00" or "Wed, Sep 30 · 18:00–23:00", with relative times where they help ("by 21:00 tomorrow", "D-2").
8	Nothing blocks without a reason	A disabled button always has a one-line explanation under it, for example that one minimum-wage warning must be acknowledged before the payroll can be confirmed.

Principles 2 and 3 carry the core idea of ShiftSwap into the interface. The system already knew, at the moment of each decision, what the decision would do to both workers' hours and holiday allowance (FR-11); the redesign moves that knowledge onto the decision itself, so a candidate sees the effect of accepting and the owner sees the effect of approving without opening another screen.

4.9.2 Navigation

Each role has four main destinations, shown as a left navigation column of 220 px on screens of 900 px and wider and as a bottom tab bar of 64 px on narrower screens. The tab bar is hidden while the request flow is open. The table below maps the destinations to the use cases.

Table 48. Navigation of the final design: destinations per role and the use cases they serve

Role	Destination	Content	Use cases
Worker	Home (홈)	Requests received with Accept / Can't; own open requests as trackers with each co-worker's answer; a FAILED card with <i>Got it</i> when nobody could take a request; next shift with <i>Find a substitute</i> ; this week's hours and holiday allowance; shifts to record	UC-04, UC-05, UC-07, UC-12, FR-21
	Schedule (근무표)	Week as time grid (desktop) or day list (phone); shift detail with the actions the viewer may take	UC-03 (view), UC-04
	Swaps (대타)	Received, sent and past requests; primary action <i>Find a substitute</i> starts the three-step flow	UC-04, UC-05, UC-12
	Me (내 정보)	Own profile (contract, weekly contract hours, wage, phone), attendance records	UC-07

Role	Destination	Content	Use cases
Owner	Home (홈)	"N things to handle today": decision cards for accepted swaps with impact lines and <i>Approve</i> ; requests waiting for replies; attendance to confirm; payroll warnings; today's and this week's staffing	UC-06, UC-07, UC-09, FR-21
	Schedule (근무표)	Time grid; <i>Create this week's schedule</i> for an empty week, otherwise <i>Add shift</i> ; shift detail with change time, change worker, delete	UC-03
	Staff (직원)	Worker cards; worker detail with the fixed-schedule editor; primary <i>Add worker</i>	UC-01
	Pay (급여)	Segments <i>Weekly summary</i> and <i>Monthly payroll</i> ; month and week chosen with < >	UC-08, UC-09
Both	Header	Store name; bell icon with unread count opening the notification list; compact clock chip — the demo clock in the local mode, the server time in the shared mode — opening <i>Demo tools</i> ; current person with <i>View as someone else</i> ; menu with Settings (owner), language and sign-out	UC-13, UC-10, UC-11, FR-21

The menus per role are no longer fixed in the router alone. `services.js` exports the pure function `menuFor(role)`, which returns the destinations a role may open; the router uses it both to draw the navigation and to redirect any other route (FR-20). This makes the role restriction testable without a browser. Two further service queries, `getWorkerHome(workerId)` and `getOwnerHome()`, return in one call everything a Home screen needs, including the before/after effect of accepting or approving, so the Home screens contain no business logic of their own.

The *Demo tools* panel (a sheet on phones, a pop-over on desktops) holds what is needed only for demonstration. In the local mode it contains the demo-clock input, *+1 hour*, *+1 day*, *Reset demo data* with its confirmation and, since iteration 8, *Switch to the shared demonstration mode*. In the shared mode it states that everyone who opens the page sees and changes the same fictional data, shows the server time without a way to change it, and offers *Reset the demo data for everyone*, whose confirmation says that all requests, approvals and attendance records made so far are erased for everyone, and *Switch to the local demonstration mode*. In the earlier design these controls occupied the header permanently.

4.9.3 Key screens

- **Sign-in as a demo guide (UC-13).** The title "달빛카페 데모" and the sentence that the demo needs no passwords are followed by a *3-minute tour* card with three numbered steps: view Lee Seoyeon's request, accept it as Choi Minho, approve it as the owner Park Jiyoung. Each step signs in as that person and opens the screen where the step happens, and completed steps are marked. Below, all five accounts are listed with their one-line summaries. Since iteration 8 a notice states the data mode: in the shared mode, that everyone who opens the page sees and changes the same fictional data and that no personal information may be entered; in the local mode, that the data are kept in this browser only. Each notice has a button that switches to the other mode. The tour is offered in both modes; in the shared mode its progress follows the shared data, so it may show steps completed by other visitors, and restarting it resets the demo data for everyone after a confirmation.
- **Worker Home.** In order, each section omitted when empty: a greeting with today's date; *Requests for you*, one card per pending request with the requester, shift, reason, deadline countdown, the effect on the worker's own week, and *Accept* (one tap, NFR-01) / *Can't*; the worker's own open requests as trackers with each co-worker's answer so far (waiting / can't / accepted); for a request that ended FAILED, the card "Every co-worker said they can't cover this shift. Please contact the owner." with *Got it*; the next shift with *Find a substitute*; this week's hours with the holiday allowance in words and money and a bar against the 15-hour line; past shifts still to record.
- **Request flow (UC-04).** Step 1 lists the worker's own future scheduled shifts, with a shift that already has an open request shown disabled and explained. Step 2 offers reason chips (exam, hospital, family event, personal, free text) and deadline chips ("24 hours from now", "2 hours before the shift", custom), with the default of BR-03. Step 3 summarises the request and names the recipients computed at that moment — every active co-worker who is not already working at that time (BR-01) — or warns that no co-worker is free and that the owner will be told at once (FR-08). The result screen shows the tracker at its first step.
- **Owner Home.** A heading with the number of things to handle; a decision card per accepted swap with a sentence ("Seoyeon's Wednesday 18:00–23:00 shift is covered by Minho"), one impact line per person (scheduled hours before → after, holiday allowance before → after, a change highlighted in amber) and *Approve / Reject*, with *Details* opening the full table of FR-11 — since iteration 8 with the warning of BR-15 when a part-time acceptor would work more than 12 hours beyond contract that week, which does not disable *Approve*; then requests waiting for replies with each co-

worker's answer; a card "No one can cover" for each request that ended FAILED, whose only action is *Got it* (FR-22); attendance to confirm, payroll warnings, and today's and this week's staffing.

- **Schedule (UC-03).** The time grid of the previous design on wide screens (hour axis from the earliest start to the latest end of the week, 44 px per hour, overlapping shifts side by side) and a day list with today expanded on phones. Selecting a shift opens a detail panel with its history ("Seoyeon → Minho · approved 9/28") and only the actions the viewer may take.
- **Pay (UC-08, UC-09).** The weekly summary as one card per worker, with eligibility stated in words and the reason ("contract 10 h per week · below 15 h"). The monthly payroll with *Create September payroll draft*, rows or cards with flags (estimated, probation minimum 90 %, below minimum wage), the below-minimum sentence with *I have checked* (acknowledgeMinWage), and *Confirm September payroll*, disabled with its reason until all warnings are acknowledged.
- **Saving and errors in the shared mode (iteration 8).** While a command is being saved, the screen is dimmed, does not accept input and is marked as busy for screen readers (aria-busy). A change set that is stale is recomputed without a message; the user sees a message only if it is still stale after three re-runs ("Other people are changing the same data right now"), if the server refuses the change (the latest data are then shown), if the database cannot be reached (the change is not saved), or if a limit of the store is reached (60 changes a minute, 3,000 a day, 20,000 rows, one reset a minute, 50 resets a day). If the shared database cannot be reached on start, the application shows this with a button that opens the local demonstration mode. Changes made by other users appear within about ten seconds, or at once when the window regains focus; the screen is redrawn only when no sheet is open, nothing has been typed into it and no change is being saved (NFR-15).
- **Staff, Me and Settings.** Worker cards and a fixed-schedule editor with weekday chips; the worker's own profile and attendance records on *Me*, which no longer has an availability editor; settings in which the number of regular employees and the substitution-attendance policy each carry a sentence explaining their effect.

4.9.4 Visual design

The visual design is specified in section 3 of the final prompt with exact values, so that a regenerated system looks the same. The table below lists the design tokens.

Table 49. Design tokens of the final user interface

Token	Value	Use
--ink / --ink-2 / --ink-3	#191F28 / #4E5968 / #8B95A1	Main text; secondary text; captions
--surface	#F2F4F6	Page background
--paper	#FFFFFF	Cards
--line	#E5E8EB	Dividers and grid lines
--shift / --shift-soft	#2B5BD7 / #EBF1FD	Primary buttons and the user's own shifts / their background
--handover / --handover-soft	#E9A23B / #FDF3E1	Substitution: handover edge and stripe, pending outline, changed eligibility
--alert / --alert-soft	#D93A30 / #FDECEB	Errors and the minimum-wage warning
--ok	#1F8A55	Eligible, confirmed, success
Type	Pretendard Variable	One font for all text, with tabular figures for times, hours and money. Body 16 px / 1.6; Home heading 26 px / 700; card title 17 px / 700; key number 28 px / 700; captions 13 px, never below 12 px.
Cards	16 px radius, 20–24 px padding	White on the grey page, no border, no shadow; 12 px between cards, 32 px between sections.
Buttons	52 px (phone) / 44 px (desktop) high, 12 px radius	Primary filled blue; secondary filled grey; text buttons; pressed state scales to 96 %; touch targets at least 44 × 44 px.
Icons and motion	24 px line icons; 180 ms	Inline SVG icons for tabs, bell, clock and chevrons; sheets and step changes slide and fade in 180 ms; no motion under prefers-reduced-motion.

As in the earlier design, one element carries meaning through decoration: a handed-over shift has a 6-pixel amber left edge and a faint 45° amber stripe — the only gradient in the style sheet — with the new worker's name in bold and the original worker struck through, and a shift with an open request has a dashed amber outline and the chip "대타 구하는 중" (looking for a substitute). Content is limited to 720 px width for Home, flows and lists, and uses up to 1,200 px for the schedule grid and the payroll table; at 360 px no screen scrolls horizontally (NFR-02). Accessibility rules remain: visible focus rings, real buttons, labelled inputs, status never shown by colour alone, and a step indicator that is announced to screen readers ("step 2 of 3").

5. Implementation by Vibe Coding

ShiftSwap was implemented by vibe coding: its source code was generated by an AI coding assistant from natural-language prompts, and the team's work consisted of writing the specification and the prompt, running and reviewing the result, and revising the prompt until it reliably produced the system specified in Chapters 3 and 4. This chapter describes how that was done. Section 5.1 lists the development environment. Section 5.2 describes the process that was repeated in every iteration. Section 5.3 explains the structure of the final prompt, whose full text is Appendix A. Section 5.4 presents the eight iterations recorded in the prompt log (Appendix B), with examples of prompt sentences before and after revision. Section 5.5 shows how the prompt maps to the code, and Section 5.6 describes the reproducibility check, in which the system was rebuilt from the final prompt in an empty folder.

5.1 Development Environment and Tools

Table 50. Development environment and tools

Tool	Version	Use in the project
AI coding assistant	agent mode with file and shell access	Generated and modified all application and test code from the prompt and the specification; ran the tests and the browser checks; recorded ambiguities in the prompt log.
Node.js / npm	v24.14.0 / 11.9.0	Running the test suite (<code>node:test</code>) and installing the one development dependency.
sql.js	1.14.2	SQLite compiled to WebAssembly; the npm package provides the files copied into <code>app/vendor/</code> and the database for the Node tests [9].
Google Chrome (headless)	current stable	Browser checks of every screen at a desktop and a phone width (1440 px and 360 px up to version 3; 1280 px and 390 px, later 360 px, with <code>tools/check.mjs</code> from version 4 on); printing the reports to PDF.
PlantUML	1.2025.2	Drawing the UML diagrams of Chapters 3 and 4 from text sources in <code>uml/src/</code> [14].
Git / GitHub	2.48.1	Version control of specification, prompts, code, models and reports in the repository <code>keemlogan/shiftswap</code> .
GitHub Pages	—	Static hosting of the application at https://keemlogan.github.io/shiftswap/app/ [13].
Supabase (PostgreSQL with PostgREST)	free plan, project shiftswap	From iteration 8: the hosted database of the shared mode. The tables, row triggers, grants and the three functions of <code>db/app-shared.sql</code> are called by the application over HTTPS through the PostgREST interface with the publishable key only; the same project stores the versions of the collaboratively edited documents of the project hub (<code>db/hub-collab.sql</code>).
node-postgres (pg)	8.23.0	Administration only: applying the SQL files to the database and creating and deleting the throwaway stores used by the shared-mode checks, through the throwaway connection of a small script kept outside the repository, so that the connection string is never committed. The application does not use it.
Project hub, collaborative editing	<code>hub/collab.js</code>	Review of the proposal, the interim report and the final report by the team: versioned editing in the browser with labels, highlighting of changes against the parent version and restoring of earlier versions (Section 7.1.5).

The application has no framework, no bundler and no build step (NFR-08), and no client library: the vendored `sql.js` files are its only third-party code. In the shared mode it also depends on the hosted database, which it calls with the browser's `fetch` function; the local demonstration mode needs nothing but the static files. The only entry in `package.json` besides the test script is `sql.js` as a development dependency for the tests.

5.2 The Vibe-Coding Process

The team used the same cycle for every version of the prompt. The figure below shows it; the numbered steps explain each stage and who performed it.

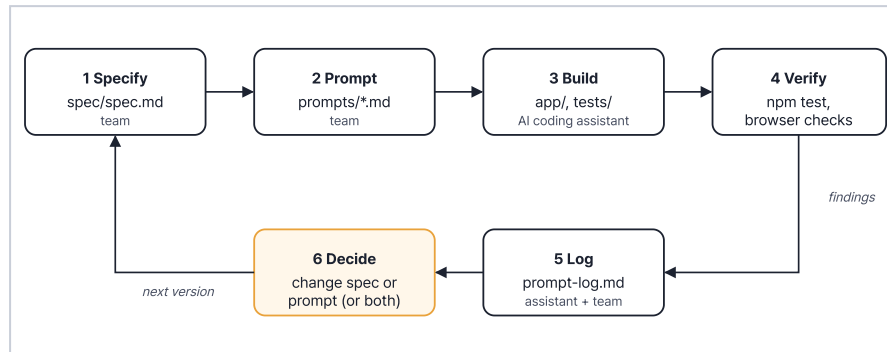


Figure 18. The vibe-coding cycle used for every prompt version

- Specify.** Everything the system must do is first written into `spec/spec.md`: actors, use cases, requirements, business rules, states, the SQL schema, the seed data and the test groups. The file is not reproduced in this report; it is `spec/spec.md` in the repository (<https://github.com/keemlogan/shiftswap/blob/main/spec/spec.md>). The specification is the contract. A behaviour that is not in it is not requested, and a behaviour that is in it may not be changed by the prompt.
- Prompt.** The prompt tells the AI coding assistant *how* to build what the specification defines: the technology, the file layout, the names of the functions, the screens, the visual design and the conditions under which the work is finished. It refers to the specification by identifier rather than repeating it.
- Build.** The prompt, followed by the complete specification, is given to the AI coding assistant in an empty folder. The assistant creates the files, installs `sql.js`, vendors its WebAssembly files, writes the tests and runs them.
- Verify.** The team runs `npm test` and walks through the scenarios of Section 3.8 in the browser, including the done criteria of the prompt. Browser checks at a desktop and a phone width are scripted with headless Chrome; from version 4 on, the script `tools/check.mjs` is part of the repository.
- Log.** Whenever the specification was silent or ambiguous, the assistant was instructed to make a reasonable choice, implement it, and record it in the prompt log in the form "what was unclear → what the implementation does → suggested wording for the next version". Review findings from the team were added to the same log.
- Decide.** For each logged item the team decided whether it belongs in the specification (it changes what the system does) or only in the prompt (it changes how the system is built), made the change, and started the next version. The rule of Section 3.11.3 applies: requirements change in the specification first.

The division of work in this cycle is deliberate. The AI coding assistant is fast at producing code but has no knowledge of the reference store or of the team's interpretation of the law, so every decision about behaviour is taken by the team and written down. Conversely, the working rule was not to fix behaviour by editing generated code by hand. When a result was wrong, the cause was traced to the prompt or the specification and corrected there, so that the next build — and any future rebuild from the final prompt — produces the corrected behaviour without depending on a manual patch.

5.3 Structure of the Final Prompt

The final prompt is version 8. It is 136 lines of Markdown and is used together with the 316-line specification, which is appended to it verbatim, and with three acceptance test files and two server files, which are placed in the empty folder before the build starts. The full text is reproduced in Appendix A. The table below describes its parts in order and the reason each exists.

Table 51. Parts of the final prompt (version 8) and their purpose

Part	Content	Why it is there
Header and usage note	Version history (v1 → ... → v8, with the reason for each version and pointers to the earlier versions and the log); how to use the prompt: place the three test files and the two supplied server files in an empty folder, then paste the prompt followed by the complete specification into a coding agent.	Makes the prompt self-contained and states the reproduction procedure, so that anyone can repeat the build (Section 5.6).

Part	Content	Why it is there
Role and contract paragraph	What is being built and for whom; the instruction to treat the specification as the contract, to use its identifiers, table and column names, state names and seed data exactly, and not to add features that are not in it.	Prevents the most common failure of generated code: plausible but unrequested features and renamed identifiers that break traceability.
§0 Acceptance tests	The supplied files <code>tests/rules.test.js</code> , <code>tests/services.test.js</code> and, since version 8, <code>tests/shared.test.js</code> are the executable form of the specification and fix the API: module paths, exported names, argument order, return shapes, error codes and the Node-side database factory. They must be read first and must not be modified; where a test and the prose disagree on an API detail the test wins, and a disagreement on behaviour must be reported, not guessed. The server side of the shared mode is supplied as well (<code>db/app-shared.sql</code> and <code>tools/seed-sql.mjs</code>); it must be read but neither changed nor run.	Added in version 5 after reproduction run 1 (Section 5.6.2): names alone did not fix the interfaces, so two correct rebuilds disagreed at the API level. Tests are an unambiguous contract for signatures that prose cannot state compactly. The server files are supplied because the shared database is deployed once and used by every copy of the application; a rebuild must work against it rather than create its own.
§1 Technology constraints	Static site with ES modules and no build step; <code>sql.js</code> vendored into <code>app/vendor/</code> ; since version 8 the two data modes — the shared mode with <code>app/config.js</code> , the replica, the change sets, the re-run rules and polling of the gateway <code>app/core/shared.js</code> , the server clock and the command wrapper <code>ctx.run()</code> , and the local demonstration mode with its persistence keys; the complete folder layout with the name of every module and every exported function; test files, test naming and the test script; when the System Clock duties run.	Fixes the architecture of Chapter 4 before any code exists. Naming every service and rule function makes the code predictable across rebuilds, lets the tests of one build run against another, and keeps the traceability matrix valid.
§2 Screens and behaviour	§2.0 eight UX principles that override any individual screen description (one primary action per screen, consequence before commitment, one question at a time, and others); §2.1 navigation per role, the Demo tools panel of both modes and the messages of the shared mode; §2.2–§2.8 the sign-in demo guide with the notice of the current mode, worker Home, request flow, owner Home, Schedule, Pay, and Staff / Me / Settings, each with its sections in order and the exact Korean wording of key labels, including the part-time warning of BR-15 on the owner's decision card.	The specification says what the system must do; this part says how the user meets each function. Stating the principles first gives the assistant a rule for every case the screen descriptions do not cover. It carries NFR-01, NFR-11 and NFR-13 into concrete screen behaviour.
§3 Visual design	Fonts, type scale, colour tokens, the single decorative element (the amber handover marker), layout widths and breakpoints, copy rules, accessibility rules; the heading says "do not deviate".	Without exact values, each generation produced a different look. Fixed tokens make the result reproducible and keep the interface consistent with the design of Section 4.9.

Part	Content	Why it is there
§4 Done criteria	Eight checkable conditions: all test groups pass, including named boundary cases; the seed state (with three candidates for the seed request) and walkthrough S7 are reproduced; the app starts without console errors; the sign-in tour works from Seoyeon's tracker through Minhø's one-tap acceptance to the owner's approval and the handover marker; walkthrough S8 works in the browser, from Hana's request through three answers of can't to the owner's acknowledgement; walkthrough S9 works in the browser, with March to August 2026 confirmed and the September draft prepared once when the demo clock reaches 1 October; every screen has exactly one primary button, every disabled button a visible reason, and all touch targets at least 44 × 44 px (criteria 3 to 7 are checked in the local demonstration mode); and the shared mode works with two browsers on a throwaway store, where of two simultaneous acceptances exactly one succeeds, the owner's approval appears in the other browser and the reset for everyone restores the open request.	Tells the assistant when it has finished and gives the team the acceptance tests of Section 6.1. The service parts of criteria 4 to 6 are automated as TC-063, TC-13E and the TC-14x group; criterion 7 is a headless count, and criterion 8 is checked by <code>tools/shared-check.mjs</code> .
Specification (appended)	The complete <code>spec/spec.md</code> : §0–§17.	The contract itself. Keeping it in a separate file lets the reports and the UML models refer to the same text.

Three principles run through the prompt. First, it is **specific where variation is harmful** — names, schema, keys, colours, sizes — and says nothing where variation does not matter, such as the internal structure of a screen module. Second, it **refers rather than repeats**: rules are cited by their BR identifier, so there is only one definition of each rule. Third, it **states its own acceptance test**. A prompt whose done criteria are "the walkthrough of criterion 4 works" can be checked; a prompt that asks for "a good substitute workflow" cannot.

5.4 Iterations of the Prompt

The prompt went through eight iterations, summarised in the table below and recorded in detail in the prompt log (Appendix B). Each iteration ended with the full test suite passing. The first five iterations changed how the system is built or corrected it; the sixth to the eighth changed what the system is required to do.

Table 52. Overview of the prompt iterations

Iteration	Prompt	Source of changes	Result
1	v1 → build	First build of the whole system from prompt v1; 21 ambiguities recorded by the AI coding assistant	Working system; 52 tests, all passing
2	v1 → v2	The 21 build ambiguities, plus gaps found in the document review of the proposal and interim report against the specification (20 numbered changes)	58 tests (6 new), 58 passing; headless Chrome 70 of 70 checks passed, 0 console errors
3	v2 → v3	UI review of the running v2 application (8 changes to screens and visual design; specification unchanged)	62 tests (4 new from a traceability check), 62 passing; headless Chrome 82 of 82 checks passed, 0 console errors

Iteration	Prompt	Source of changes	Result
4	v3 → v4	UX redesign review of the running v3 application: task-first Home per role, one primary action per screen, consequence shown before commitment, three-step request flow, sign-in as a guided demo tour, Korean as default language; specification gains NFR-13 and a new wording of FR-20	Application rebuilt as v4; 77 tests, 77 passing; headless check tools/check.mjs 115 of 115 (26 scenes × Korean/English × 1280/390 px: exactly one filled primary button, disabled buttons with a visible reason, touch targets ≥ 44 px, no horizontal scroll; walkthrough of done criterion 4 in Korean)
5	v4 → v5	Reproduction run 1 (Section 5.6.2): a clean rebuild passed its own tests but not the team's, because the prompt fixed function names but not signatures and return shapes; the acceptance test files become part of the input. An independent code review found seven defects, fixed in the same iteration. Specification: BR-02 (acceptor re-checked for overlap, ACCEPTOR_BUSY), BR-10 (exact three-month window), BR-12, FR-04, FR-05 and FR-13 clarified; Korean display names for the seed people	7 defects fixed with 12 new regression tests; 77 tests, 77 passing; test files become part of the input; reproduction run 2 passed all 77 supplied tests
6	v5 → v6	Product-owner review of the running v5 application: availability removed (UC-02, FR-03 and table Availability retired); every free co-worker is asked; candidates can answer can't; a request nobody can take ends FAILED with a message to the requester and an acknowledge-only card for the owner (BR-13, FR-22, walkthrough S8)	84 tests, 84 passing (77 updated, 7 new: TC-13A–TC-13F, TC-003); headless check tools/check.mjs 128 of 128 at 1280/360 px, no console errors
7	v6 → v7	Product-owner request: payroll data for past months from the contracts, and last month's payroll prepared automatically on the first day of each month. Seed with full work history and payroll confirmed to August 2026; UC-14 (System Clock), FR-23, BR-14; notification PAYROLL_DRAFT_READY; walkthrough S9	89 tests, 89 passing (84 of iteration 6, four updated, plus TC-141–TC-145); headless check tools/check.mjs 149 of 149 at 1280/360 px, no console errors

Iteration	Prompt	Source of changes	Result
8	v7 → v8	Revision of the proposal and the team's decision to implement it: a shared database on Supabase as the default data mode, the browser version kept as the local demonstration mode, the server time as the System Clock in the shared mode; the part-time overtime rule BR-08 (iv) and the approval warning (FR-24, BR-15); NFR-14 to NFR-16; walkthrough S10	125 tests, 125 passing (113 of the application: the 89 of iteration 7 unchanged plus 24 new; and 12 of the project hub's editing feature); tools/shared-rest.mjs 64 of 64 REST checks; tools/shared-check.mjs 12 of 12 checks with two headless browsers

5.4.1 Iteration 1: building from version 1

Prompt version 1 was written after the requirements and design models, as planned for Week 9 of the schedule. The first build produced a working system, and the assistant recorded 21 points at which it had to make a choice that the specification did not determine: six concerned the precision of business rules (for example, whether a week in progress can have perfect attendance, and the rounding of holiday pay), six the data and seed, three the workflow and notifications, three the technical set-up and three the user interface. The individual findings, their resolution and the tests that verify them are listed in Section 6.9.

In nearly every case the implementation had already made a sensible choice. The value of the log was that it made these choices visible, so that the team could confirm them and write them into the contract; otherwise the next build might have chosen differently, and the behaviour described in this report would not be reproducible.

5.4.2 Iteration 2: version 2 (build ambiguities and document review)

Version 2 folded in all 21 findings of iteration 1 and the findings of a document review, in which the proposal and the interim report were checked against the specification. The review found, among other things, that the minimum-wage acknowledgment had no place in the data model, that the recipients of a cancelled request were not defined for UC-12, and that no scenario ever changed a worker's eligibility. Twenty numbered changes were made (Section 6.9 lists them with their tests). Two changed the code: the new column `Payroll.minWageAck` with the service `acknowledgeMinWage`, and the replacement of a saved database that predates the new column. Six tests were added.

The table below shows how the wording changed for four representative items. The first two changed the specification, the other two the prompt.

Table 53. Examples of wording before and after iteration 2

Where	Before (v1)	After (v2)
BR-06 (spec)	"perfect attendance: every shift of the week whose original worker is W and that was not given away was WORKED (not ABSENT)."	"... was not ABSENT (a shift that is still SCHEDULED does not break attendance, so the weekly result is a projection until the week is over)."
FR-19 (spec)	"A requester shall cancel their own request while it is REQUESTED or ACCEPTED; all targets are closed and notified."	"... all PENDING targets become CLOSED, and every target that was PENDING or ACCEPTED receives a REQUEST_CANCELLED notification."
Prompt §1	"package.json with "type": "module" and script "test": "node --test tests/"	"... script "test": "node --test tests/*.test.js" (a bare directory argument fails on Node 24)."
Prompt §2, Payroll	"minimum-wage warning row that must be acknowledged before "Confirm", confirmed rows read-only."	"minimum-wage warning on the row with an Acknowledge button (<code>acknowledgeMinWage</code> sets <code>Payroll.minWageAck = 1</code> , persisted); "Confirm" is disabled until every warning row is acknowledged; confirmed rows read-only; rows with <code>estimated = 1</code> carry an "Estimated" chip."

The first example shows a requirements error that only an implementation could reveal. As written in v1, BR-06 made every worker ineligible until the week was over, because future shifts are not yet WORKED; the approval panel could therefore never show Choi Minhø as eligible, which contradicted the worked example of the specification. The fourth example shows the typical development of a prompt sentence: a requirement stated as an intention ("must be acknowledged") becomes a design with a stored field, a named function and a defined screen state.

5.4.3 Iteration 3: version 3 (UI review)

The team reviewed the running version 2 on desktop and mobile screens. The review found three problems: the text was too small, the content sat in a narrow centred column with large empty areas on wide screens, and the weekly board read as a list of cards rather than a schedule. All changes were made in sections 2 and 3 of the prompt, and the specification did not change. The table below lists the eight changes. A traceability check made in the same iteration found that FR-01 and FR-20 had no automated test; the role-to-menu table was therefore moved from the router into the pure service function `menuFor(role)`, and four tests were added (TC-01C, TC-01D, TC-134, TC-135), which raised the count to 62.

Table 54. Changes of iteration 3 (UI review)

#	Change in the prompt	Change in the code	Verification
1	Sign-in rows show name, role and a one-line summary of the person's seed data	<code>ui/login.js</code> builds the summary from the fixed schedule and contract hours	Headless: all five summaries as expected
2	Every page starts with a title and one sentence on its purpose	<code>pageHead()</code> helper in <code>ui/i18n.js</code> , used by all eleven screens; sentences in English and Korean	Headless: every route for both roles
3	Owner overview strip above the board: approvals waiting, open requests, this week's shifts and hours	<code>overviewStrip()</code> in <code>ui/schedule.js</code>	Headless: "9 shifts · 50 h" for the seed week
4	Weekly board as a time grid with an hour axis, 44 px per hour, blocks placed and sized by time	<code>ui/schedule.js</code> rewritten around <code>board()</code> and <code>layoutDay()</code> ; overlapping shifts share the width in lanes	Headless geometry: Sat 10:00–16:00 is 264 px high
5	Empty states say what will appear and offer the next action	<code>emptyState()</code> helper; used on seven screens	Manual review
6	Type scale: base 15 px; page title 28 px, weight 800; small text never below 12 px; inputs 16 px on mobile	<code>styles.css</code> heading and body rules	Headless: page title computes to 28 px / 800
7	Navigation 220 px; content left-aligned after it with 32 px padding; gaps between groups at least twice those inside a group	Grid layout; group spacing on the view container	Headless: navigation width 220 px
8	Kept: day-by-day list below 900 px, stacked-card tables below 720 px	Below 900 px the grid drops its axis and blocks become cards in day order	Headless: every screen, both roles, both languages, 1440 and 360 px, no horizontal scroll

The weekly board illustrates how an imprecise prompt sentence leads to an acceptable but wrong result. Version 2 asked for "seven columns Monday–Sunday, rows are shifts ordered by start time". The assistant implemented exactly that: each day was a column of cards in time order. The result met the words but not the purpose, because the owner could not see at a glance when the café was staffed. Version 3 describes the intended picture instead: "a time grid ... a vertical hour axis from the earliest shift start to the latest shift end of the displayed week, rounded to whole hours (seed: 10:00–23:00), 44 px per hour ... Each shift is a block positioned and sized by its start and end time". The concrete values, including the expected axis for the seed data, made the result checkable by measuring pixel positions in the headless browser.

The verification of iteration 3 also found one defect of its own: the 32-pixel gap between groups had first been set on the outer content element, but the screens render inside an inner view element, so the header, the overview strip and the week navigation touched each other. The gap was moved to the view element, and the check was repeated.

5.4.4 Iteration 4: version 4 (UX redesign)

The fourth iteration was started after the review of the version 3 application from the point of view of a consumer-app product team. The review accepted the functions and the visual quality of version 3 but found that the interface still followed the structure of the use cases rather than the tasks of its users: a candidate had to open the inbox to find a request, the owner had to open the approvals page to see the effect of a swap, and every screen offered several actions of equal weight. Version 4 therefore rewrote section 2 of the prompt around eight UX principles and a new navigation (Section 4.9), rewrote section 3 with new design tokens, and added done criterion 5. The specification gained the usability requirement NFR-13 and a new wording of FR-20 (sign-in through a guided demo tour or the account list); the business rules, the schema and the service commands did not change. Two read queries (`getWorkerHome`, `getOwnerHome`) and the pure function `menuFor(role)` were added to the service layer so that the new Home screens and the role menus contain no logic of their own.

The table below shows how two prompt sentences changed.

Table 55. Examples of wording before and after iteration 4

Where	Before (v3)	After (v4)
Prompt §2, inbox / Home	" Inbox (UC-05, FR-21) : notifications newest first; a REQUEST_RECEIVED item for a PENDING target has Accept and Decline buttons directly on the item (NFR-01: at most 2 clicks)."	" 대타 요청이 왔어요 — one card per PENDING target: who asks, the shift, reason, deadline countdown, the effect on me ("수락하면 이번 주 16시간 → 21시간 · 주휴수당 그대로"), and two buttons 수락하기 (primary) / 거절 (text). Accept is one tap (NFR-01)."
Prompt §2, request	" Request substitute (UC-04) : from one of the worker's own future shifts, open a form: reason (optional), deadline (default: the earlier of now + 24 h and shift start)."	"Requesting a substitute is a three-step flow (choose shift → reason and deadline → review and send) with a step indicator ("1/3"), a back button, and a review screen that names the people who will receive the request."

The build of version 4 changed sixteen items, recorded row by row in the prompt log. Most were confined to the presentation layer: every screen places its page-level action in a shared page header with one primary button, which is fixed above the tab bar on phones; a disabled button renders the reason beneath it; the sign-in screen became a numbered three-step tour; the request flow became three steps with a review screen that lists the recipients; and the style sheet, the Korean and English text tables and the date formats were rewritten. The service layer received only read queries — `getWorkerHome`, `getOwnerHome`, `previewCandidates`, `getShiftDetail` and `routesFor` — so the business rules, the schema and the existing service commands were not touched. Three tests were added for the new queries (TC-064 to TC-066), and TC-134 was updated for the new menus. The browser checks moved into the repository as `tools/check.mjs`, which checks done criteria 3, 4 and 5 on 26 named screen states.

The build also recorded eleven new ambiguities. Examples are the missing module for the Home screens and for shared components (the build added `ui/home.js` and `ui/components.js`), the conflict between §1, which placed "Reset demo data" in the header menu, and §2.1, which placed it in the demo tools, and the question of which button is the primary one when a Home screen lists several items that need action. A review of the screenshots in both languages at 1280 px and 390 px found a further eleven presentation defects, such as a toast of the previous person that stayed visible after switching accounts, weekday chips 39 px wide at phone width and English plural errors ("1 shifts"); all were fixed before the result below was taken.

Result of iteration 4: the application was rebuilt as version 4; `npm test` ran 77 tests, all passing (this count already includes the twelve regression tests of iteration 5, which were written in parallel); and `tools/check.mjs` passed 115 of 115 checks with no console errors. The checks cover 26 scenes in Korean and English at 1280 px and 390 px — exactly one filled primary button, a visible reason on every disabled button, touch targets of at least 44 px and no horizontal scroll — and the walkthrough of done criterion 4 in Korean. The phone width of the check was later changed to 360 px, the width that NFR-02 names; the final check of Section 6.7 uses it.

5.4.5 Iteration 5: version 5 (acceptance tests as input)

The fifth iteration followed the first reproduction run, described in Section 5.6.2. The rebuilt system worked and passed its own 54 tests, but only 13 of the team's 25 rule tests passed against it, and the team's service tests could not be loaded at all, because the rebuild had given the database factory a different name. The diagnosis was that the prompt fixed the names of modules and functions but not their signatures and return shapes, so two implementations that both satisfy the specification disagreed at the level of the interfaces. Version 5 therefore adds a section 0 to the prompt: the two test files are supplied with the prompt, must be read first and must not be changed, and they define the API. The same iteration folded two points reported by the clean session into the specification — the acceptor is re-checked for an overlapping shift at acceptance and at approval (BR-02, error ACCEPTOR_BUSY), and the three-month probation window is defined exactly (BR-10) — and fixed seven defects found by an independent code review of the reference implementation. The defects and their fixes are listed in the prompt log (Appendix B).

The seven defects — among them the missing re-check of the acceptor, attendance records that could be changed after they were final, and a 90 % probation floor computed in floating point — were fixed with twelve new regression tests: TC-01E, TC-01F, TC-01G, TC-054, TC-055, TC-072, TC-073, TC-08C, TC-106, TC-107, TC-124 and TC-125. The specification was clarified in BR-02, BR-10 and BR-12 and in FR-04, FR-05 and FR-13 (the last table of Section 6.9 lists each defect with its fix and its tests). After reproduction run 2 had started, the prompt was also given a Korean display-name map: in the Korean interface the seed people and the store are shown with their Korean names from spec §10 (for example 최민호님), while the stored names remain English.

Result of iteration 5: `npm test` runs 77 tests, all passing; prompt version 5 makes the two test files part of the input; and reproduction run 2 (Section 5.6.2) passed all 77 supplied tests unchanged.

5.4.6 Iteration 6: version 6 (requirements change: nobody can take the shift)

The sixth iteration was a change of the requirements, not a correction of the build. The product owner used the running version 5 and found a dead end: when no co-worker's availability matched a request, the owner received only a notice that there was no candidate and had nothing to do, and the availability slots were extra data that workers had to keep up to date. The change request, its impact analysis and the updated traceability are described in Section 3.11.4. In short, availability is removed (UC-02, FR-03 and the table Availability retired), every other active co-worker without an overlapping shift is asked (BR-01), candidates can answer *can't*, and a request that nobody can take ends as FAILED (BR-13, FR-22). The requester is then told to contact the owner, and the owner receives a notification whose only action is to acknowledge it.

The specification changed first, as the change procedure requires (spec §15), and prompt version 6 followed. In the prompt, the worker Home card offers **불가** ("can't") next to **수락하기**; the requester's tracker shows each co-worker's answer and, for a failed request, the card "모든 동료가 대타가 불가능하다고 해요. 사장님께 연락드려 보세요."; the owner Home gains the card **대타를 못 구했어요** with only **확인했어요**; the *Me* screen shows the worker's profile instead of an availability editor; and a new done criterion 5 requires walkthrough S8 to work in the browser, so the former criterion 5 (one primary action per screen) became criterion 6.

The prompt log lists nine changes, each with its code change and its tests. In `rules.js`, `isEligibleCandidate` lost its availability argument, and a new pure function `hasNoTaker` decides BR-13. In `db.js`, the table Availability was dropped, FAILED was added to the status check, and a database saved by version 5 is recognised as outdated and replaced by the seed. In `services.js`, `saveAvailability` and `getAvailability` were removed; `createSubRequest` and `respondToRequest` end a request FAILED through one shared function, and they return the new status; the two Home queries return unacknowledged REQUEST_FAILED notifications; and `markNotificationsRead` accepts a list of notification ids, so that acknowledging one card does not mark everything read. Because the seed request now has three candidates instead of two, several existing tests had to change their expected values.

Result of iteration 6: `npm test` runs 84 tests, all passing — the 77 tests of iteration 5, updated where they assumed availability, plus TC-13A to TC-13F and TC-003 (Section 6.9); and `tools/check.mjs` passes 128 of 128 checks at 1280 px and 360 px in Korean and English with no console errors: the sign-in screen (done criterion 3), the Korean tour walkthrough (criterion 4), five checks of walkthrough S8 (criterion 5), and the audit of 28 screen states for one filled primary button, visible reasons on disabled buttons, touch targets of at least 44 px and no horizontal scroll (criterion 6).

5.4.7 Iteration 7: version 7 (work history and automatic monthly payroll)

The seventh iteration, like the sixth, came from the product owner. Using version 6, the owner asked for payroll data for the past months, based on the employees' contracts, and for last month's payroll to be filled in automatically on the first day of every month. The change request and its impact analysis are described in Section 3.11.5. The specification gained the use case UC-14, whose actor is the System Clock, the requirement FR-23 and the business rule BR-14 (spec §16). Prompt version 7 adds the System Clock's second task to §1, the owner's Home card **9월 급여 초안이 준비됐어요** ("September payroll draft is ready") with the action **급여 확인하기** to §2, a month switcher for *Pay* → *Monthly* that reaches back to the first contract month, and a new done criterion 6 for walkthrough S9, so the headless count of primary buttons became criterion 7.

The prompt log lists eight changes. The seed now builds each worker's work history by calling the same services as the application: shifts from the contract start to 20 September are inserted day by day, their attendance is recorded and confirmed, and the payroll of every month from March to August 2026 is generated, acknowledged where a warning exists, and confirmed. The stored history is therefore exactly what the rules compute, which TC-142 checks by regenerating every month. The whole seed runs in one transaction. The new service `prepareMonthlyPayroll(nowIso)` determines the previous month and does nothing if that month lies before the first contract month or already has Payroll rows; otherwise it calls `generatePayroll` and sends one PAYROLL_DRAFT_READY notification to the owner. The router calls it after `expireOverdue()` before every screen, so the draft exists before the owner opens Home on the first of the month. Two read queries serve the screens: `getOwnerHome` returns the unread PAYROLL_DRAFT_READY notification as `payrollReady` and counts it as a task, and `listPayrollMonths` returns every month with its status for the month switcher. Because a browser may hold a database saved from the earlier, shorter seed, the storage key changed from `shiftswap.db.v1` to `shiftswap.db.v2`.

The history changed several figures that earlier chapters and tests had quoted. September 2026 now contains four weeks whose Sunday falls in the month, so Choi Minh'o's September holiday pay rose from 33,024 won to $4 \times 33,024 = 132,096$ won and his September total to 844,176 won; the tests that assert September figures (TC-088, TC-095, TC-114) and the seed test TC-001 were updated. The seed contains 142 shifts, 133 confirmed attendance records and 10 confirmed payroll rows.

Result of iteration 7: `npm test` runs 89 tests, all passing — the 84 tests of iteration 6, four of them updated for the history, plus TC-141 to TC-145 (Section 6.9); and `tools/check.mjs` passes 149 of 149 checks at 1280 px and 360 px in Korean and English with no console errors. Its five new checks cover walkthrough S9 (done criterion 6): the confirmed months March to August reached with <, the owner's Home card on 1 October, **급여 확인하기** opening the automatic September draft with Doyun's warning, the notification marked read, and still one draft and one notification on 15 October. The audit of done criterion 7 now covers 32 screen states.

5.4.8 Iteration 8: version 8 (shared database and part-time overtime)

The eighth iteration, unlike the sixth and the seventh, did not start from a review of the running application but from the revised proposal and the team's decision to implement it (Section 3.11.6, spec §17). The revision made two changes. First, the data of the demonstration are kept in a shared database on Supabase, so that several people who open the application see and change the same store; the browser-only version remains available as the local demonstration mode, which is also the fallback when the database cannot be reached (risk R6). In the shared mode the System Clock is the server's time, and the demo clock cannot be set. Password authentication stays out of scope: the people are still chosen from the account list. Second, the premium rule was extended for part-time workers in workplaces with five or more regular employees, whose hours beyond the contractual hours are limited to twelve a week and paid with a premium of at least 50 % [15]. The specification gained FR-24, NFR-14 to NFR-16 and BR-15, part (iv) of BR-08 with the system rules (a)–(e), the `app_` tables of the shared database in §8, the shared-mode behaviour and its trust boundary in §9, walkthrough S10 and the shared-mode seed in §10, and the test groups TC-09x, TC-15x and TC-16x in §11.

Prompt version 8 supplies a third test file, `tests/shared.test.js`, and the server side of the shared mode, `db/app-shared.sql` and `tools/seed-sql.mjs`, which a rebuild must read but not change or run (§0). Section 1 describes the two data modes, `app/config.js` with the database address, the publishable key and the store, the gateway `app/core/shared.js`, the replica functions of `db.js`, the server clock and the command wrapper `ctx.run()`; section 2 describes the demo tools and the sign-in notice of each mode, the shared-mode messages and the amber BR-15 notice on the owner's decision card; and the new done criterion 8 requires the shared mode to work with two browsers on a throwaway store. The specification was changed before the prompt, as in every iteration.

The prompt log lists six changes. The server side, `db/app-shared.sql`, holds the eleven tables of spec §8 as `app_` tables with a store column, row triggers for the state rules, the three functions `app_snapshot`, `app_commit` and `app_reset`, read-only grants and rate limits (Section 4.5.2); its seed template of 353 rows is generated from the seed code by `tools/seed-sql.mjs`, so the two cannot drift apart (TC-16A). On the client, `core/shared.js` exports `createGateway`, `restTransport`, `diffTables` and `classify`: every command runs the unchanged service function on an in-memory replica, and the gateway sends the changed rows with the base revision (Section 4.4). `main.js` chooses the mode, routes every command of the screen modules through `ctx.run()` with a busy state, polls the revision every 10 seconds and on focus, and saves the System Clock duties after drawing, pausing them for a minute after a refusal. In `rules.js`, `premiumHours` gained the part-time hours of BR-08 (iv), and the new functions `isPartTime` and `partTimeLimitCheck` serve the rule and the warning; `generatePayroll` passes each worker's contractual hours and the shifts of the same weeks outside the month. The services `seedPayrollHistory` and `completeSharedSeed` let the local seed and the shared seed complete the payroll history in the same way.

The iteration also revealed the limits of what the prompt can fix. The rules of a shared database — who may change which row, how many changes a store accepts and what happens when two people act at the same moment — cannot be enforced by the browser, which anyone can modify; they therefore live in the supplied SQL, and the prompt asks the assistant to build a client for a server that already exists rather than to invent one. The same reasoning led to the supplied test file: the change-set format, the re-run rules and the pause after a refusal are part of the contract between client and server, so they are fixed by TC-160 to TC-16B rather than described in prose.

Result of iteration 8: `npm test` runs 125 tests, all passing — the 89 tests of iteration 7 unchanged, twelve part-time tests (TC-096 to TC-09C and TC-151 to TC-155), twelve shared-mode tests (TC-160 to TC-16B) and the twelve tests of the project hub's editing feature in `tests/collab.test.js`; `tools/shared-rest.mjs`, which calls the database with the publishable key on a throwaway store, passes 64 of 64 checks; and `tools/shared-check.mjs`, which drives two headless browsers on a throwaway store, passes 12 of 12 checks: the shared notice on the sign-in screen, the server time in the demo tools, two simultaneous acceptances of which exactly one succeeds, the owner's approval seen in the other browser through polling, and the reset for everyone. `tools/check.mjs` runs in the local demonstration mode, where the done criteria 3 to 7 are checked (Section 6.7), and passes 154 of 154 checks at 1280 px and 360 px in Korean and English with no console errors; its new check S10 confirms that the owner's decision card shows Choi Minh'o's hours rising from 24 to 29 and the BR-15 warning of 13 hours beyond contract, with approval still possible.

5.5 From Prompt to Code

Section 1 of the prompt names every file and every exported function, so the structure of the code follows directly from the prompt. The tree below shows the files of the implementation with their line counts.

```
shiftswap/
├── app/
│   ├── index.html          22  page skeleton, loads vendor/sql-wasm.js and main.js
│   └── config.js           6   shared database address, publishable key, store
├── (iteration 8)
│   ├── main.js            450  bootstrap, mode choice, hash router, header, role menus,
│   │   demo tools, ctx.run(), System Clock, polling
│   ├── styles.css         674  design tokens, layout, board, responsive rules
│   └── core/
```

	clock.js	56	demo clock and server time (A3)
	db.js	280	schema (spec §8), seed (spec §10), tx(), persistence,
replica	load and dump		
	rules.js	320	BR-01 ... BR-11, BR-13 and BR-15 as pure functions
	services.js	1155	one function per use-case step; read queries
	shared.js	243	shared-mode gateway: replica, change sets, re-runs,
polling, reset			
	ui/		
15 warning	approvals.js	112	UC-06 decision card with before/after comparison and BR-
	attendance.js	102	UC-07
	components.js	185	icons, buttons, page header with its primary action,
tracker, sheets			
	home.js	163	worker and owner Home (NFR-13, FR-22, FR-23)
	i18n.js	1144	Korean and English text, formatting helpers
	inbox.js	39	notifications (FR-21)
	login.js	127	UC-13 sign-in as demo tour, notice of the data mode
	me.js	31	Me screen: own profile and attendance (UC-07)
	payroll.js	162	Pay route, UC-09 monthly payroll, month switcher
	requests.js	427	UC-04 three-step flow, UC-05, UC-12, FAILED card (Swaps)
	schedule.js	301	UC-03 time grid, day list, shift detail
	settings.js	80	UC-10
	slots.js	76	fixed-schedule slot editor used by workers.js
	weekly.js	35	UC-08 weekly summary cards
	workers.js	108	UC-01 (Staff)
	vendor/		sql-wasm.js, sql-wasm.wasm (sql.js 1.14.2)
db/			
	app-shared.sql	1116	shared database: app_ tables, triggers, RPCs, grants,
seed template			
	hub-collab.sql	216	project hub: document versions and their limits
tests/			
	rules.test.js	336	unit tests of rules.js
	services.test.js	1031	integration tests of services.js
	shared.test.js	261	tests of the shared-mode gateway (TC-16x)
	collab.test.js	160	tests of the project hub's editing feature
tools/			headless browser checks (check.mjs, shared-check.mjs),
REST check			(shared-rest.mjs), seed template (seed-sql.mjs),
screenshots, NFR-03			
	package.json	13	
	spec/spec.md	316	the contract (not reproduced; see the repository)
	prompts/		final-prompt.md (v8), prompt-v1.md ... prompt-v7.md, spec-
v5.md ... spec-v7.md, prompt-log.md			

Line counts from `wc -l` at iteration 8; application code 6,298 lines (core 2,054; presentation, router, style sheet and configuration 4,244) and test code 1,628 lines of the application tests (rules, services, shared), excluding the vendored sql.js files and the server SQL.

Because the prompt names the modules and functions, the mapping from prompt to code is exact: the contract paragraph and spec §8 and §10 produce `core/db.js` (checked by TC-001 and TC-002); the rule and service lists of §1 produce `core/rules.js` and `core/services.js` (checked by the rule and service test files); the shared-mode part of §1 produces `config.js` and `core/shared.js` against the supplied `db/app-shared.sql` (checked by `tests/shared.test.js`); §2 produces the screen modules and the router; §3 produces the style sheet (checked in the headless browser); and §4 defines the acceptance checks of Section 5.6. This exactness is the reason the traceability matrix of Section 6.8 could be derived from the code by searching for function names.

5.6 Reproducibility Check

The course requires a final prompt that, when re-run through vibe coding, reliably reproduces a working system. A prompt that was refined on one code base may, however, depend on that code base without anyone noticing: a sentence may be understandable only to an assistant that has seen the previous version. The reproducibility check therefore rebuilds the system from nothing and tests the result with the team's test suite. It was carried out five times, with prompt versions 3, 5, 6, 7 and 8. The final run, with version 8, passed all 113 acceptance tests unchanged and met done criteria 1–7; criterion 8, which needs the database owner's connection, was checked by the team at the data level (Section 5.6.2).

5.6.1 Method

1. **Clean start.** A new session of the AI coding assistant is started with no conversation history, in an empty folder that is outside the repository. The session has no access to the repository, the existing code, the prompt log or the reports.

2. **Input.** The session receives exactly what Appendix A prescribes: the text of the final prompt followed by the complete specification, the specification file `spec/spec.md` in the repository (<https://github.com/keemlogan/shiftswap/blob/main/spec/spec.md>). From version 5 on, the two team test files are placed in the folder before the session starts, as the prompt prescribes. No other instruction is given, apart from answering "continue" if the session stops before it reports completion.
3. **Build.** The assistant builds the system on its own, including its own tests, and reports when it considers the done criteria of the prompt met. The number of interventions and the elapsed time are recorded.
4. **Own tests.** The rebuilt system's own `npm test` is run and its result recorded.
5. **Team test suite.** The team's test files `tests/rules.test.js` and `tests/services.test.js` are run against the rebuilt code. In run 1 (prompt v3) they were copied into the rebuilt folder after the build, replacing the rebuilt tests; from run 2 (prompt v5) they are part of the input and must pass unchanged. Because the prompt fixes the module paths, the exported function names and the schema, the team's tests can call the rebuilt code directly. This is the decisive measurement: it checks the rebuilt system against the same test cases that the reference implementation passes (77 for prompt version 5, 84 for version 6, 89 for version 7, 113 for version 8), including the boundary values of Section 6.5, the concurrency test of Section 6.6 and the notification recipients of spec §8.
6. **Done criteria 2–4.** The rebuilt application is served with a static server and opened in headless Chrome: the sign-in screen must appear without console errors (criterion 3), and the walkthrough of criterion 4 is performed. Criterion 2 (seed state and walkthrough S7) is covered by TC-001 and TC-08B of the team suite. From run 4 on, the session checks criteria 4–7 with its own headless script; criterion 8 of version 8 (two devices on a throwaway store of the shared database) is run by the team, because the session has no database connection.
7. **Comparison.** Any failing team test is traced to its cause and classified as (a) a genuine deviation from the specification, (b) an ambiguity in the prompt or specification that the rebuild resolved differently, or (c) a difference that the tests over-specify (for example the wording of an error message) and that does not affect a requirement. Categories (a) and (b) are defects of the prompt or specification and are reported as such.

Two limits of the method should be stated. Each rebuild is a single run, and a generative assistant does not produce the same code twice, so one successful run shows that the prompt *can* reproduce the system, not that every run will. And the team's tests check the rules, services and data; the layout of the screens is checked only by the done criteria and the headless checks, not pixel by pixel against the reference.

5.6.2 Results

Run 1 (prompt version 3)

The first run was made on 2026-09-27 with the prompt as it stood after iteration 3. A new assistant session with no history and no access to the project received one file — the prompt of version 3 followed by `spec/spec.md` — in an empty folder, with the instruction to do what it says. The table below gives the results.

Table 56. Results of reproduction run 1 (prompt v3)

Measurement	Result
Size of the rebuilt application	4,540 lines of application code, including UI and CSS
Rebuilt system's own tests	54 tests, 54 passed, 0 failed
Done criteria 1–4 of the prompt	Met: headless Chrome walkthrough without console errors; "Already taken by Choi Minh"; 16 → 21 scheduled hours in the approval panel; handover marker; no horizontal scroll at 360 px
Team's tests/ <code>rules.test.js</code> against the rebuild	25 tests, 13 passed, 12 failed
Team's tests/ <code>services.test.js</code> against the rebuild	Did not load: <code>SyntaxError: The requested module '../app/core/db.js' does not provide an export named 'createDatabase'</code> (the rebuild exports <code>initDb</code>)

The twelve failing rule tests were TC-014, TC-021, TC-022, TC-023, TC-082, TC-084, TC-086, TC-101, TC-103, TC-111, TC-112 and TC-113. They fail on the form of the interface, not on behaviour: for example, whether `validateRequest` returns a list of error codes or a boolean, and how `computeWeeklySummary` reports the reason for ineligibility. At the level of the specification the rebuild reproduced the behaviour the reports rely on — the worked example of 3.2 holiday hours and 33,024 won, first acceptance wins, and walkthrough S7.

The diagnosis, recorded in `docs/reproducibility.md` in the repository, is that version 3 fixed the *names* of modules and functions but not their *signatures* and *return shapes*. Two correct implementations of the same specification can therefore disagree at the API level, and a test suite written against one does not run against the other. By the classification of Section 5.6.1, the failures are of category (b): an underspecification of the prompt, not a deviation from the requirements. The clean session also reported fourteen ambiguities of its own. Most were presentation choices or agreed with the reference imple-

mentation; two were folded into the specification (BR-02 and BR-10), and three API-level choices — whether the notification list runs in a transaction, re-entrant transactions, and the re-check of the acceptor — are now fixed by the supplied tests.

The change made in response is iteration 5 (Section 5.4.5): the test files became part of the input and define the API. Run 1 thus did what a reproducibility check is for. It showed that the prompt reproduced the behaviour but not the interfaces, and it turned that finding into a change of the prompt.

Run 2 (prompt version 5)

The second run was made with prompt version 5. As in run 1, a new assistant session with no conversation history worked only inside an empty folder. It received `PROMPT.md`, containing the final prompt of version 5 followed by the specification, and the two supplied test files `tests/rules.test.js` and `tests/services.test.js`. The table below gives the results.

Table 57. Results of reproduction run 2 (prompt v5)

Measurement	Result
npm test of the rebuilt system	85 tests, 85 passed: the 77 supplied tests and 8 tests the session added in <code>tests/extra.test.js</code>
Supplied tests on the first run of the core modules	All 77 passed
Supplied test files unchanged	Byte-identical before and after the build: shasum of <code>rules.test.js</code> 93d3c5c8168255b20a2d0013f513fb1d899d2204, of <code>services.test.js</code> b6846b7233733db054bbb71883bf863c01c544f5
Done criteria 1–2	Pass
Done criterion 3	Pass: the application opens without console errors
Done criterion 4	Pass: tour walkthrough at 360 px and 1280 px, including "16시간 → 21시간" on the candidate's card, acceptance in one tap, "먼저 수락했어요" for the second candidate, the owner's approval and the handover marker on the schedule
Done criterion 5	Pass: exactly one primary button on 17 screens at each width plus the steps of the request flow; every disabled button with its reason; touch targets of at least 44 px; no horizontal scroll at 360 px; no FR or UC identifiers in the interface

The session reported 19 ambiguities. The notable ones were the following. The module list of §1 names no module for the Home screens, so the session added `home.js`. The specification said that the English names are shown unchanged, while the example sentences of the prompt used Korean names; the session followed the specification, and the prompt was then clarified with a Korean display-name map (iteration 5, Section 5.4.5). The placement of the primary button on Home, on its card or in the sticky bottom bar, was left open. "Reset demo data" appeared both in the header menu and in the demo tools. A late accept or approval saves the request as EXPIRED and then throws `REQUEST_CLOSED`. And expiry closes only the targets that are still PENDING.

Compared with run 1, in which 13 of the 25 rule tests passed and the service tests did not load, run 2 passed all 77 supplied tests. Supplying the acceptance tests as part of the input therefore fixed the reproducibility of the system at the API level, which was the gap identified in run 1. Three limits apply to this conclusion. It rests on a single run. The interface was compared with the reference by the done criteria, not pixel by pixel. And the Korean display-name clarification was added to the prompt after run 2 had started, so run 2 was made without it.

Run 3 (prompt version 6)

The third run was made on 2026-09-27 with prompt version 6, after the requirements change of iteration 6 (Section 5.4.6). The method was that of run 2: a new assistant session with no conversation history worked only inside an empty folder. It received the prompt followed by the specification, and the 84 supplied tests in `tests/rules.test.js` and `tests/services.test.js`. The table below gives the results.

Table 58. Results of reproduction run 3 (prompt v6)

Measurement	Result
npm test of the rebuilt system	89 tests, 89 passed: the 84 supplied tests and 5 tests the session added
Supplied tests on the first run	All 84 passed; the team re-ran the supplied files in the rebuilt folder: 84 of 84

Measurement	Result
Supplied test files unchanged	Identical checksums at the start and at the end: shasum of rules.test.js 83182afc4cc26a0fea6b127fbb379b63b01f6668, of services.test.js 1032e7d08436da7010f94b88e1ea1fa61d3c7fb6
Done criteria 1–2	Pass: supplied tests, seed state and walkthrough S7
Done criterion 3	Pass: the sign-in screen "달빛카페 데모" opens without console errors
Done criterion 4	Pass: tour walkthrough including "16시간 → 21시간", one-tap acceptance, "최민호님이 먼저 수락했어요" for the second candidate, the owner's decision card with "(33,024원 유지)" and the handover marker
Done criterion 5	Pass: walkthrough S8 in the browser — three recipients, three answers 불가, Hana's FAILED card, and the owner's card "대타를 못 구했어요", which 확인했어요 removes
Done criterion 6	Pass: one filled primary button per screen, reasons on disabled buttons, touch targets of at least 44 px, no horizontal scroll at 360 px

The session reported several ambiguities; the main ones were the following. First, §10 of the specification still said that the Korean interface shows the English names, while §2.0 of the prompt required a Korean display map; the session followed the prompt. Second, the rule of exactly one primary button per screen conflicted with Home cards that each carry an action; the session kept the first enabled card action as the primary and drew the others grey. Third, the file layout of the prompt listed no module for Home, so the session placed Home in existing modules. The first and third points had already been corrected in the repository — the sentence of §10, and the module list of prompt §1 with home.js, me.js, components.js and slots.js — but only after run 3 had started, so the run was made without these corrections. Fourth, the session noted three behaviours that the documents leave implicit: saving the settings replaces the whole minimum-wage table, generating the payroll of a confirmed month returns the confirmed rows, and attendance can be recorded once the shift has started.

Run 4 (prompt version 7)

The fourth run was made on 2026-09-27 with the final prompt, version 7, after the requirements change of iteration 7 (Section 5.4.7). The method was that of runs 2 and 3: a new assistant session with no conversation history worked only inside an empty folder, and received the prompt followed by the specification and the 89 supplied tests. The table below gives the results.

Table 59. Results of reproduction run 4 (prompt v7)

Measurement	Result
npm test of the rebuilt system	94 tests, 94 passed: the 89 supplied tests and 5 tests the session added
Supplied tests on the first run	All 89 passed on the first run of the core modules; the team re-ran the supplied files in the rebuilt folder: 89 of 89
Supplied test files unchanged	Identical checksums at the start and at the end: shasum of rules.test.js 83182afc4cc26a0fea6b127fbb379b63b01f6668, of services.test.js 72f08f2d5c9f17c031a0ad0d1b0891a6dedd2482
Done criteria 1–3	Pass
Done criteria 4–7	Pass, checked by the session's own headless check: 68 screen states at 360 px and 1280 px, 249 assertions, 0 failures and no console errors. It covered the tour (criterion 4), walkthrough S8 (5), walkthrough S9 (6: March to August confirmed; on 2026-10-01 the Home card "9월 급여 초안이 준비됐어요" and the automatic September draft with Doyun's warning; no second draft on 10-03 and 10-20) and the one-primary audit (7)
Module layout	The same UI modules as the reference, among them home.js, me.js, components.js and slots.js

The session reported 21 ambiguities; the main ones were the following. First, the export lists of prompt §1 omit several functions that the supplied tests call, among them hasNoTaker, prepareMonthlyPayroll, listPayrollMonths, routesFor and previewCandidates; following the rule of §0, the session built them as the tests call them. Second, the rule of one filled primary button per screen again met Home cards that each carry an action; the session kept the first action primary and drew later ones grey, and gave screens without a natural action a navigation button as primary. Third, the seeded weekly summaries were stated only up to the week of 14 September, although the week of 21 September is also worked; the session seeded them to that week and let later weeks be computed on demand. Fourth, the session had to decide how an overdue re-

quest is expired when a candidate accepts or the owner approves it; it committed the expiry first and then reported REQUEST_CLOSED, which the supplied tests require. The remaining points were choices that the documents leave open, such as whether a payroll row is created for a worker without shifts in the month, and on which segment the *Pay* tab opens.

Run 5 (prompt version 8)

The fifth run was made on 2026-09-29 with the final prompt, version 8, after iteration 8 (Section 5.4.8). A new assistant session with no conversation history worked only inside an empty folder and received the prompt followed by the specification, the three supplied test files with 113 tests and the two supplied server files `db/app-shared.sql` and `tools/seed-sql.mjs`; it had no database connection and was told not to run the SQL. The first session built the core and interface modules and passed all 113 tests, but stopped before it had written the page skeleton and the style sheet; a second session with the same restrictions wrote the two files from prompt §3 and checked the done criteria. The table below gives the results.

Table 60. Results of reproduction run 5 (prompt v8)

Measurement	Result
npm test of the rebuilt system	113 tests, 113 passed; the session added no tests. The team re-ran the supplied files in the rebuilt folder: 113 of 113
Supplied files unchanged	Identical checksums at the start and at the end: <code>shasum</code> of <code>rules.test.js</code> <code>f8e84742b5b68e32cb8a8600afd727f33277a717</code> , of <code>services.test.js</code> <code>0249358faee378ab9372c432b91db1af52e2887e</code> , of <code>shared.test.js</code> <code>49ca71a6d05e15e18917912ee509ff26669c824d</code> ; the two server files likewise
Code size	4,319 lines under <code>app/</code> without the vendored <code>sql.js</code> (core 1,853; interface 1,720)
Done criteria 1–7	Pass in the local demonstration mode. The session's own headless check passed 25 of 25 checks, among them 50 screen audits at 360 px and 1280 px; it covered the tour (criterion 4), walkthrough S8 (5), walkthrough S9 (6) and the one-primary audit (7), and first found three small defects of the rebuild (buttons disabled without a reason, a missing icon file), which the session fixed
Done criterion 8	Not run by the session (no database connection). The team ran it at the data level: two separate Node processes, each with the rebuilt shared-data gateway and service layer and the real REST transport on a throwaway store, accepted the same request at the same moment; the database held one ACCEPTED request with one accepted target, and the other process received ALREADY_TAKEN after its stale change set had been re-run (two runs, both passed; the stores were deleted). The rebuilt screens were not driven in two browsers

The session reported 17 ambiguities. One was a defect of the supplied material: prompt §1 says that the publishable key of the database is given in the header comment of `db/app-shared.sql`, but the header did not contain it, so the rebuild started only in the local demonstration mode. The team added the address, the key and the demo store to the header comment; the SQL itself is unchanged. The other points repeat those of earlier runs or are choices that the documents leave open: the rule of one filled primary button per screen against Home cards that each carry an action, the notice that another candidate was faster (a message after a late tap in §2.3, a card without a tap in criterion 4), the month of the automatic payroll draft derived from its notification, and the expiry of an overdue request committed before REQUEST_CLOSED is reported.

Summary of the five runs

The table below summarises the five reproduction runs.

Table 61. Summary of the reproduction runs

Run	Prompt	Input	Team's acceptance tests against the rebuild	Done criteria
1	v3	prompt + specification	rules 13 of 25; services did not load	1–4 met
2	v5	prompt + specification + tests	77 of 77	1–5 met
3	v6	prompt + specification + tests	84 of 84	1–6 met
4	v7	prompt + specification + tests	89 of 89	1–7 met

Run	Prompt	Input	Team's acceptance tests against the rebuild	Done criteria
5	v8	prompt + specification + tests + server files	113 of 113	1–7 met; 8 at the data level by the team

Run 1 showed that the prompt reproduced the behaviour of the system but not its interfaces. From version 5 on, the acceptance tests are part of the input, and all four later runs passed every supplied test unchanged, including runs 3, 4 and 5 after changes of the requirements. The final prompt, version 8, therefore reproduces a system that passes all 113 acceptance tests of the reference implementation and meets done criteria 1–7; for criterion 8 the rebuilt data layer resolved simultaneous acceptances from two processes correctly against the shared database. The limits stated in Section 5.6.1 remain: there was one run per prompt version, the interface was compared with the reference by the done criteria and not pixel by pixel, and some clarifications that the runs reported were added to the prompt and the specification only after the runs had started; the incomplete export lists of prompt §1 reported in run 4 were covered by the supplied tests rather than by the prose, and the missing key reported in run 5 was added to the supplied server file after the run.

6. Testing and Validation

This chapter shows how the team checked that ShiftSwap does what Chapter 3 requires. The lecture distinguishes unit testing, integration testing, system testing, acceptance testing and regression testing [6], and it defines validation as checking that the software is what the customer wants. Section 6.1 describes the test strategy and how these levels were applied. Section 6.2 gives the test environment. Section 6.3 lists every automated test of the application with its expected result, and Section 6.4 reports the result of running them together with the checks that need the shared database or a browser. Sections 6.5 and 6.6 explain the two families of tests that need the most care: boundary values of the legal rules and concurrent acceptance, which since iteration 8 can be tested across two browsers. Section 6.7 covers the non-functional requirements, Section 6.8 the traceability matrix, and Section 6.9 the defects and specification gaps found during the build iterations and how each was resolved.

6.1 Test Strategy

The layered architecture of Chapter 4 decides where each kind of test belongs. The table below maps the lecture's test levels to what was done for ShiftSwap. From iteration 8 the application has two data modes, and the shared mode adds a client layer (the gateway) and a server layer (the PostgreSQL database), each with a test level of its own.

Table 62. Test levels and how they were applied

Level	Target	Method
Unit testing	Domain rules (<code>rules.js</code>)	36 tests in <code>tests/rules.test.js</code> call the pure rule functions with hand-built inputs, including the boundary values of Section 6.5 and the part-time rules of BR-08 (iv) and BR-15. No database is involved.
Unit testing	Shared-data gateway (<code>shared.js</code>)	12 tests in <code>tests/shared.test.js</code> run the gateway on a SQLite working copy against an in-process fake of the shared database that behaves like <code>app_snapshot</code> , <code>app_commit</code> and <code>app_reset</code> (revisions, STALE answers, refusals queued by the test). They check change sets, the re-run of a stale command, the mapping of server errors, offline behaviour, polling, the reset, the REST transport and that the seed template of the server is up to date.
Integration testing	Services with the real schema (<code>services.js</code> + <code>db.js</code> + <code>rules.js</code>)	65 tests in <code>tests/services.test.js</code> create a fresh in-memory SQLite database with the seed data before every test (<code>beforeEach</code>), reset the demo clock, call the service functions and assert on the resulting rows, notifications and error codes.
Integration testing	The shared database (PostgreSQL, <code>db/app-shared.sql</code>)	<code>tools/shared-rest.mjs</code> runs 64 checks over REST with the publishable key, that is, with the rights of the application, against the real database. It works on a throwaway store <code>e2e-...</code> that it creates and deletes: the read and write rights, the RPC functions, the row triggers, STALE, the limits and the reset (Section 6.4).
System testing	The running application in a browser	Scenario walkthroughs of Section 3.8 on the screens, for both roles, in Korean and English; for the final build the headless check <code>tools/check.mjs</code> opens 33 named screen states in both languages at 1280 px and 360 px in the local demonstration mode, which has a fixed clock (Section 6.7 and Chapter 7). In the shared mode, <code>tools/shared-check.mjs</code> drives two headless browsers on a throwaway store.
Acceptance testing	Done criteria of the final prompt	The eight done criteria of the prompt (Chapter 5) serve as acceptance tests. Criterion 4, the walkthrough from acceptance to the handover marker, is automated at the service level as TC-063 and in the browser by <code>tools/check.mjs</code> ; criterion 5 (walkthrough S8, nobody can take the shift) is automated at the service level as TC-13E, criterion 6 (walkthrough S9, the automatic payroll draft) as TC-145, and criterion 7 (one primary action per screen) is checked by the same script. Criterion 8, the shared mode with two browsers, is checked by <code>tools/shared-check.mjs</code> . Walkthrough S10 (part-time overtime) is tested at the service level as TC-153 to TC-155 and in the browser by one further check of <code>tools/check.mjs</code> .

Level	Target	Method
Regression testing	Every change of prompt or code	The whole suite is run after every generation or modification of the code; a change is accepted only if all tests pass (the practice of continuous integration in the lecture's description of extreme programming [6]). The three application test files are supplied to the coding assistant together with prompt version 8 (Section 5.4.8), so a rebuild has to satisfy them.

Two principles guided the choice of test cases. First, the tests were derived from the requirements, not from the code: each test group corresponds to a business rule (spec §11), and each test is named with its identifier and a sentence stating the expected behaviour, for example TC-082 no holiday allowance at 14.99 h. This is the test-case generation technique of requirements validation described in Section 3.10.3. Second, the seed data of spec §10 doubles as the test fixture. The integration tests start from the same café, the same five people and the same open request that appear in the scenarios and in the screenshots, so a number checked in a test — 3.2 holiday hours, 33,024 won, 16 → 21 scheduled hours — is the number the reader sees in this report and on the screen.

The identifiers extend the groups of spec §11 by groups that are not tied to a business rule: TC-00x for the seed data and storage, TC-07x for attendance (UC-07), TC-13x for privacy and notifications (NFR-11, FR-21), and TC-16x for the shared mode (NFR-04, NFR-14, NFR-15). Within a group, the third character is a sequence number (1–9, then A, B), so TC-01A follows TC-019. The tests TC-01C, TC-01D, TC-134 and TC-135 were added after a traceability check in iteration 3, TC-064 to TC-066 for the Home queries of iteration 4, twelve regression tests for the defects of the independent code review in iteration 5, and TC-13A to TC-13F and TC-003 for the change request of iteration 6 (Section 6.9). The iteration 6 tests extend the group TC-13x because BR-13 has the same number; TC-13F is a rule-level test and is therefore listed with the rule tests. Iteration 7 opened the group TC-14x (TC-141 to TC-145) for BR-14, FR-23 and the seeded work history. Iteration 8 added the part-time overtime tests TC-096 to TC-09C to the group TC-09x, because they belong to BR-08, and opened the groups TC-15x (TC-151 to TC-155) for BR-15 and FR-24 and TC-16x (TC-160 to TC-16B) for the shared mode.

One further test file is not part of the application. `tests/collab.test.js` holds 12 tests of the collaborative-editing module of the project hub (version labels and the highlighting of changes in the report text). `npm test` runs it together with the application's tests, but its tests are not listed in Section 6.3 and are counted separately in Section 6.4.

6.2 Test Environment

Table 63. Test environment

Item	Value
Test runner	Node.js built-in test runner <code>node:test</code> with <code>node:assert/strict</code> [12]; command <code>npm test</code> , which runs <code>node --test tests/*.test.js</code>
Node.js version	v25.9.0 (<code>node -v</code>) for the runs of iteration 8 on 2026-09-29; v24.14.0 for the earlier iterations
Database library	<code>sql.js</code> 1.14.2 (from <code>package-lock.json</code>), the same WebAssembly build as <code>app/vendor/</code>
Machine	Apple M4, 16 GB memory, macOS 26.2 for the runs of iteration 8; Apple M2, 24 GB memory, macOS 26.5 for the earlier iterations
Test data	Seed data of spec §10, recreated in memory before each integration test; demo clock reset to 2026-09-28 09:00. The gateway tests use a fake shared database whose server time is 2026-09-28 09:00, the seed time
Shared database	Supabase project <code>shiftswap</code> (free plan), PostgreSQL 17.6 (<code>select version()</code>), the tables and functions of <code>db/app-shared.sql</code> . The REST test and the two-browser check use the publishable key, like the application, and a throwaway store <code>e2e-<time></code> that the script creates and deletes through the owner connection of the database. The demo store <code>dalbit</code> is only read
Browser checks	Google Chrome 152 in headless mode, driven by <code>tools/check.mjs</code> (local demonstration mode, fixed clock) and <code>tools/shared-check.mjs</code> (shared mode, two browser instances); 1440 px and 360 px width for the builds of versions 2 and 3, 1280 px and 360 px for the final build

The service tests import the application modules unchanged. `createDatabase(SQL, {seed: true})` builds the database without browser storage, and the demo clock is set with `setNow()` where a test needs a later time, for example one minute before and at a deadline. The gateway tests replace the network by a fake transport, so they need neither a database server

nor the network. No unit test reads the real system time, which makes the results the same on every run and on every machine. The checks against the shared database are different: they depend on the network and on the server time, and each of them starts a store of its own, so that they neither depend on nor disturb the demonstration data.

6.3 Test Cases

The table below lists all 113 automated tests of the application in the order in which they appear in the three test files: 36 in `rules.test.js`, 65 in `services.test.js` and 12 in `shared.test.js`. The list was extracted from the files with a script, so it contains exactly the tests that run; the text of the columns "Covers" and "Expected result" is not part of the files and was written from the assertions of each test. The column "Covers" gives the business rules, requirements or specification sections each test checks; "Expected result" states what the assertions require. The twelve tests of the project hub in `tests/col-lab.test.js` are not application tests and are not listed.

Table 64. Automated test cases (tests/rules.test.js, tests/services.test.js and tests/shared.test.js)

ID	Name	File	Covers	Expected result
TC-011	overlap is half-open: touching ranges do not overlap	rules	BR-01	Touching ranges 10:00–16:00 and 16:00–22:00 do not overlap; one extra minute does.
TC-012	durationHours supports end <= start as next day	rules	BR-01, BR-11	18:00–23:00 = 5 h; 22:00–06:00 = 8 h (next day); overnight ranges overlap correctly.
TC-013	weekStartOf returns the ISO Monday	rules	BR-05	Wednesday 2026-09-30 and Sunday 2026-10-04 map to Monday 2026-09-28.
TC-014	eligible when an active worker, not the requester and free at that time (no availability, spec §15)	rules	BR-01	Choi Minh (active worker, free on Wednesday evening) is eligible for Wed 18:00–23:00, also with shifts that only touch the requested one or fall on another day; availability is not used.
TC-015	not eligible: requester, inactive, owner, or overlapping shift that day	rules	BR-01	The requester, an inactive worker, the owner and a worker with an overlapping shift that day are each not eligible.
TC-13F	BR-13: no taker when there is no target or every target declined	rules	BR-13	No taker for an empty list of responses and for three DECLINED; a PENDING or ACCEPTED response means there is still a taker.
TC-021	valid request passes	rules	BR-03, FR-06	Valid request returns no error, also with deadline equal to shift start.
TC-022	only the current worker of a SCHEDULED future shift can request	rules	BR-03	NOT_OWN_SHIFT, NOT_SCHEDULED and SHIFT_STARTED are reported.
TC-023	at most one open request, deadline after now and not after shift start	rules	BR-03	OPEN_REQUEST_EXISTS, DEADLINE_NOT_FUTURE (deadline = now) and DEADLINE_AFTER_START (one minute late).
TC-081	holiday allowance at exactly 15 h	rules	BR-06, BR-07	15.00 contractual hours: eligible, 3 holiday hours.
TC-082	no holiday allowance at 14.99 h	rules	BR-06	14.99 contractual hours: not eligible, 0 hours, reason CONTRACT_BELOW_15.
TC-083	holiday hours are capped at 40 contractual hours	rules	BR-07	holidayHours(40) = holidayHours(52) = 8; holidayHours(16) = 3.2; contract hours of Minh = 16.
TC-084	an ABSENT own shift breaks perfect attendance	rules	BR-06	An ABSENT own shift breaks perfect attendance (reason ABSENT on 09-29); actual hours 5.
TC-085	given-away shift under policy EXCUSED keeps attendance	rules	BR-06	Shift given away under EXCUSED keeps eligibility; scheduled hours drop to 5.
TC-086	given-away shift under policy ABSENT breaks attendance	rules	BR-06	Same shift under ABSENT policy: not eligible, reason GAVE_AWAY.

ID	Name	File	Covers	Expected result
TC-087	substitute shifts taken do not change contractual hours; absence on them does not count	rules	BR-06	A substitute shift taken does not change contractual hours (16); an absence on it does not remove eligibility.
TC-091	premium is zero with 4 regular employees	rules	BR-08	Premium hours are all zero with 4 regular employees.
TC-092	premium with 5 employees: daily overtime and night hours add up	rules	BR-08	10:00–23:00 with 5 employees: 5 daily overtime + 1 night hour = 6.
TC-093	overnight shift night hours 22:00–06:00	rules	BR-08	Overnight 21:00–07:00: 8 night hours and 2 daily overtime hours.
TC-094	weekly hours beyond 40 not already counted as daily overtime	rules	BR-08	Six days (8 h × 5 + 10 h): 2 daily and 8 weekly overtime hours, no double counting.
TC-096	part-time with C = 10 and 5 employees: 14 h in the week are 4 h beyond contract; none with 4 employees (BR-08 iv b)	rules	BR-08	With C = 10 and 5 employees a week of 14 h has 4 h beyond contract; with 4 employees none; the next Monday's 5 h start a new week and add none.
TC-097	no hours beyond contract without a fixed schedule (C = 0) or with C ≥ 40 (BR-08 iv a)	rules	BR-08	C = 40 and C = 0 give no hours beyond contract; isPartTime is false for 0 and 40 and true for 0.5 and 39.99.
TC-098	an hour is counted once: a 10 h day with C = 5 is 2 h beyond 8 plus 3 h more beyond contract (BR-08 iv d)	rules	BR-08	A 10 h day with C = 5: 2 daily + 3 beyond contract = 5 premium hours, the larger of 2 and 5 and not 7.
TC-099	night hours are added separately to hours beyond contract, also on an overnight shift (BR-08 iv e)	rules	BR-08	Friday's 4 h beyond C = 10 and 3 night hours add up to 7; an overnight Tuesday 22:00–06:00 gives 3 h beyond contract and 9 night hours (12).
TC-09A	S10 worked example across the month boundary: 13 h beyond contract in October, premium 72,240 won (BR-08 iv c)	rules	BR-08, BR-11	Worked example of S10 across the month boundary: October has 13 h beyond contract and 1 night hour, September only its night hours; days outside the month only move where the first 16 h end; premium pay 72,240 won (13 × 10,320 × 0.5 + 1 night hour × 5,160).
TC-09B	hours counted as weekly overtime (ii) are not counted again as beyond contract (BR-08 iv d)	rules	BR-08	The week of TC-094 (50 h) with C = 30: 2 daily + 8 weekly + 10 beyond contract = 20 premium hours, none counted twice.
TC-101	minimum wage by year	rules	BR-09	Minimum wage 2026 = 10,320; 2025 = 10,030; 2027 falls back to 10,320; 2024 = none.
TC-102	probation applies only when all BR-10 conditions hold	rules	BR-10	Probation holds for Jung Hana on 09-15 and 11-30; fails on each missing condition separately.
TC-103	minimum-wage warning uses 90 % only under probation	rules	BR-09, BR-10	Warning for 10,000 won without probation, none with probation (floor 9,288), warning for 9,000.
TC-106	BR-10 window: contractStart inclusive to the same date three months later exclusive; no probationEnd, no probation	rules	BR-10	Window from 2026-09-01 (inclusive) to 2026-12-01 (exclusive): 08-31 and 12-01 do not apply, 09-01 and 11-30 do; with no probation end (null, empty or missing) probation never applies.
TC-107	the 90 % probation floor is exact: 9,288 won at a 10,320 minimum gives no warning	rules	BR-09, BR-10	Under probation at a minimum of 10,320 won: 9,288 won gives no warning, 9,287 won gives a warning.

ID	Name	File	Covers	Expected result
TC-111	payroll totals: base + holiday + premium, rounded down	rules	BR-07, BR-11	$16\text{ h} \times 10,320 = 165,120 + 33,024\text{ holiday} = 198,144\text{ won.}$
TC-112	premium pay at 5 employees and estimated flag	rules	BR-08, BR-11	One 18:00–23:00 shift at 5 employees: premium 5,250 won; estimated flag set.
TC-113	fractional amounts are rounded down to the won	rules	BR-07, BR-11	Fractions rounded down: 0.33 h $\rightarrow$ 3,300 won; 1.33 h holiday $\rightarrow$ 13,301 won.
TC-151	12 h beyond contract gives no warning; 12.5 h and 13 h do (BR-15)	rules	BR-15, FR-24	With C = 16 and 5 employees: 28 h scheduled (12 h beyond contract) gives no warning; 28.5 h and 29 h (12.5 and 13 h) do; $28.1 - 16.1$ is exactly 12 h, no warning.
TC-152	no warning with 4 regular employees, with C ≥ 40 or without a fixed schedule (C = 0) (BR-15, BR-08 a)	rules	BR-15, FR-24	No warning with 4 regular employees (29 h, C = 16), with C = 40 (53 h) or with C = 0 (13 h), although 13 h lie beyond contract in each case.
TC-001	seed reproduces spec §10	services	spec §10	Seed: Dalbit Café, 4 employees, EXCUSED, two minimum-wage rows, five accounts; 18 shifts in the weeks of 09-21 and 09-28; every shift before 09-28 WORKED with a confirmed attendance; payroll confirmed up to 2026-08 and none for September; the open request with three PENDING targets and three REQUEST_RECEIVED; no Availability table.
TC-016	seed request candidates equal the BR-01 computation	services	BR-01, FR-07	Recreated seed request yields exactly Choi Minh, Jung Hana and Kang Doyun.
TC-017	fixed schedules of one worker must not overlap (FR-02)	services	FR-02	Overlapping fixed shifts on the same weekday are refused (FIXED_OVERLAP).
TC-018	generating the same week twice creates no duplicates (FR-04)	services	FR-04	Seed week creates 0 new shifts; a new week creates 9; a second call creates 0.
TC-019	owner edits a shift; overlap with the same worker is refused (FR-05)	services	FR-05	Owner adds, moves and deletes a shift; a move onto another shift of the same worker is refused.
TC-01A	deleting a shift with a request or attendance is refused (FR-05)	services	FR-05	Deleting a shift with a request or an attendance record is refused (SHIFT_IN_USE).
TC-01B	deleting a fixed-schedule row keeps the shifts it generated (FR-02)	services	FR-02	Removing a fixed-schedule row keeps its shifts with fixedScheduleId = NULL.
TC-01C	owner registers and edits a worker with all FR-01 fields	services	FR-01, FR-04	A new worker is stored with every FR-01 field (name trimmed, empty contract end stored as null); an edit changes wage, simple-labour and active flags; an inactive worker's fixed shift is not generated (9 new shifts).
TC-01D	worker registration refuses invalid input with a message that says how to fix it (FR-01)	services	FR-01	Empty name, zero, fractional or missing wage, missing contract start, and a contract or probation end before the start are each refused with their code and an explanatory message; no worker row is added.
TC-024	creating a request notifies each candidate (FR-07)	services	FR-07, BR-01	Kang Doyun's Saturday request notifies Lee Seoyeon and Jung Hana (two REQUEST_RECEIVED).

ID	Name	File	Covers	Expected result
TC-025	no eligible worker still creates the request, which ends FAILED at once (FR-08, BR-13)	services	FR-08, BR-13	A Saturday request of Doyun with every co-worker busy has no candidate, ends FAILED at once, and REQUEST_FAILED goes to the owner and Doyun.
TC-026	invalid requests are refused (BR-03)	services	BR-03, FR-06	Open request, closed shift, foreign shift and late deadline are refused with their codes.
TC-051	two acceptances of the same request: only the first succeeds	services	BR-02, NFR-04	Minho's acceptance succeeds; Doyun's later acceptance fails with ALREADY_TAKEN naming Choi Minho; targets Minho ACCEPTED, Hana and Doyun CLOSED.
TC-052	acceptance notifies requester and owner, closed target is told (FR-10)	services	FR-10	REQUEST_ACCEPTED to owner and requester; TARGET_CLOSED to Hana and Doyun.
TC-053	a decline keeps the request open for other candidates	services	FR-09	A can't (decline) leaves the request REQUESTED; another candidate can still accept.
TC-061	approval reassigns the shift and keeps the original worker; weekly summaries recomputed	services	BR-04, FR-12	Shift reassigned to Minho, original worker Seoyeon, status SCHEDULED; summaries 21 h / 5 h; two REQUEST_APPROVED.
TC-062	rejection leaves the shift unchanged; only ACCEPTED can be decided	services	BR-04, FR-12	Approval of a REQUESTED request refused; rejection leaves the shift unchanged; two REQUEST_REJECTED.
TC-063	done-criterion 4 walkthrough: accept, already taken, approval panel, approve, handover on board	services	FR-09, FR-11, FR-12, NFR-01	Done-criterion 4: accept, "already taken", preview 16 → 21 h with eligibility unchanged, approve, handover on the board.
TC-064	worker home: Minho sees the request with the effect of accepting (16 → 21 h, 33,024 won unchanged)	services	FR-09, FR-11, NFR-01, NFR-13	Minho's Home shows the seed request with 16 → 21 h and 33,024 won unchanged; after he accepts, Doyun's Home shows that Minho was faster and Seoyeon's shows her request as accepted.
TC-065	owner home: decision card with the approval effect, then nothing left after approval	services	FR-11, FR-12, NFR-13	Owner Home: one open request with three pending targets, 9 shifts · 50 h; after acceptance one decision card (Seoyeon 10 → 5 h, Minho 16 → 21 h, 33,024 won unchanged); none after approval; a pay warning and an attendance to confirm count as two tasks.
TC-066	review step previews recipients; shift detail explains why a shift cannot be deleted	services	BR-01, FR-05, FR-07	Recipients preview for Minho's Saturday: Lee Seoyeon and Jung Hana; for Hana's Sunday: Seoyeon, Minho and Doyun; none when every co-worker is busy; the Wednesday shift detail shows its open request and one history entry and cannot be deleted.
TC-088	worked example: Minho 3.2 h holiday allowance, 33,024 KRW per week in September payroll	services	BR-06, BR-07, BR-11	Week of 09-21: Minho 3.2 h; Seoyeon not eligible (contract below 15 h); September holiday pay $4 \times 33,024 = 132,096$ won (four weeks with their Sunday in September).
TC-089	marking a shift absent removes holiday eligibility for that week	services	BR-06, FR-13	Marking Minho's SCHEDULED Tuesday 09-29 absent after it started removes eligibility for that week (reason ABSENT on 09-29).
TC-08A	both substitution-attendance policies through approval	services	BR-06, FR-17	After an approved swap Minho stays eligible under EXCUSED and loses eligibility under ABSENT.

ID	Name	File	Covers	Expected result
TC-08B	demo walkthrough S7: ABSENT policy, Minho gives away Sat 10-03 and loses holiday allowance	services	BR-06, FR-11	Walkthrough S7: candidates Seoyeon and Hana, Doyun excluded; preview shows eligibility change; after approval not eligible.
TC-095	payroll premium is zero at 4 employees and non-zero at 5	services	BR-08	September premium 0 at 4 employees; at 5, Seoyeon's nine evening shifts give 9 night hours $\times 10,500 \times 0.5 = 47,250$ won.
TC-09C	S10 payroll at 5 employees: October pays Minho 13 h beyond contract and 1 night hour; September only night hours (BR-08 iv)	services	BR-08, BR-11	S10 at 5 employees with the week worked as scheduled (29 h): October pays 13 h beyond contract and 1 night hour ($14 \times 10,320 \times 0.5$), September only its 10 night hours.
TC-104	payroll warns for Kang Doyun and blocks confirmation until acknowledged	services	BR-09, BR-10, FR-16	Warning for Doyun, none for Hana (probation); confirmation blocked until acknowledged.
TC-105	minimum-wage acknowledgement is persisted and a regenerated draft needs a new one	services	BR-09, NFR-05	Acknowledgement survives a reload from storage; a regenerated draft resets it and blocks confirmation again.
TC-114	September payroll for Choi Minho: worked + estimated hours, four holiday weeks	services	BR-11	Minho September: 69 h (64 worked + 5 estimated on 09-29), $712,080 + 4 \times 33,024 = 844,176$ won, estimated.
TC-115	confirmed payroll is read-only; regenerating replaces drafts only	services	FR-15, BR-11	After confirmation, marking a shift absent and regenerating leaves the confirmed totals unchanged.
TC-121	request expires at its deadline; requester and owner notified	services	BR-12, FR-18	No expiry at 20:59; expiry at 21:00; targets CLOSED; REQUEST_EXPIRED to owner and requester.
TC-122	an ACCEPTED request expires when the shift starts without approval	services	BR-12	An ACCEPTED request expires at the shift start and can no longer be approved.
TC-123	cancel closes and notifies all targets (UC-12)	services	FR-19	Cancel sets CANCELLED, notifies all three targets, and a second cancel is refused.
TC-071	worker records attendance for a past shift; owner confirms it	services	FR-13	Recording before the shift starts or on a foreign shift is refused; record + confirm gives WORKED and 4.92 actual hours.
TC-131	a worker never receives other workers' phone or wage (NFR-11)	services	NFR-11	A worker sees null phone and wage for everyone else, own wage visible; the owner sees all phones.
TC-132	inbox lists newest first with unread count; mark read clears it (FR-21)	services	FR-21	Newest notification first, unread count 1, zero after marking read.
TC-133	notification recipients follow the spec §8 table	services	FR-19, FR-21, FR-22, spec §8	Every notification kind reaches exactly the recipients of the spec §8 table; a single can't notifies no one; REQUEST_FAILED goes to the owner and the requester when every target has answered can't.
TC-002	a saved database from before spec v2 (no Payroll.minWageAck) is replaced by the seed	services	NFR-05	A saved database without Payroll.minWageAck is replaced by the seed.

ID	Name	File	Covers	Expected result
TC-134	each role sees only its own menus; other routes are not offered (FR-20)	services	FR-20	Worker tabs Home, Schedule, Swaps, Me (plus notifications); owner tabs Home, Schedule, Staff, Pay (plus notifications, settings); no route of the other role; an unknown role gets none; the returned list is a copy.
TC-135	a worker sees only their own attendance rows and no one else's phone or wage (NFR-11)	services	NFR-11	Seoyeon's attendance list contains only her own rows (the owner sees 9); other workers' phone and wage are null, her own are shown.
TC-054	acceptance is refused when the candidate already works an overlapping shift that day (BR-02, ACCEPTOR_BUSY)	services	BR-02, FR-09	Minho, given an overlapping 19:00-21:00 shift, cannot accept (ACCEPTOR_BUSY); request, targets and notifications unchanged; Doyun can still accept.
TC-055	approval is refused when the acceptor got an overlapping shift after accepting (BR-02, ACCEPTOR_BUSY)	services	BR-02, FR-12	An acceptor who received an overlapping shift after accepting cannot be approved (ACCEPTOR_BUSY); shift and notifications unchanged; rejection is still possible.
TC-072	an ABSENT shift cannot be confirmed as worked (spec §7, FR-13, ATTENDANCE_CLOSED)	services	FR-13, spec §7	Once a shift is ABSENT, confirming it or marking it absent again is refused (ATTENDANCE_CLOSED); the status stays ABSENT.
TC-073	a WORKED shift cannot be marked absent or confirmed again (spec §7, FR-13, ATTENDANCE_CLOSED)	services	FR-13, spec §7	A WORKED shift cannot be marked absent or confirmed again (ATTENDANCE_CLOSED); the status stays WORKED.
TC-01E	regenerating a week does not re-create a shift the owner moved to another day (FR-04)	services	FR-04	After Seoyeon's Monday shift is moved to Tuesday, regenerating the week creates no shift; Monday stays empty.
TC-01F	regenerating a week after the fixed rows were deleted and re-added creates no overlapping duplicates (FR-04)	services	FR-04	After Seoyeon's fixed rows are deleted and re-added, regenerating the week creates no duplicate; a new week still creates 9 shifts.
TC-01G	changing worker, date or time of a shift with an open request is refused (FR-05, SHIFT_OPEN_REQUEST)	services	FR-05	Changing worker, date or time of the Wednesday shift is refused while its request is REQUESTED or ACCEPTED (SHIFT_OPEN_REQUEST); allowed after cancellation.
TC-08C	recomputing a week deletes the summary of a worker who no longer has a shift that week (spec §8)	services	BR-06, BR-07, FR-14, spec §8	After all of Minho's shifts of the week move to Seoyeon, his weekly summary row is deleted and the October payroll carries no stale 33,024 won holiday pay.
TC-124	accepting after the deadline expires the request and is refused (BR-12, REQUEST_CLOSED)	services	BR-12, FR-09	Accepting at 22:00 after the 21:00 deadline: REQUEST_CLOSED; request EXPIRED, targets CLOSED, owner and requester notified, no REQUEST_ACCEPTED.
TC-125	approving at the deadline or after the shift start expires the request and is refused (BR-12, REQUEST_CLOSED)	services	BR-12, FR-12	Approving at the deadline or at the shift start is refused (REQUEST_CLOSED); the request is EXPIRED, the shift keeps its worker and no REQUEST_APPROVED is sent.
TC-13A	all three targets answer can't: FAILED, requester and owner notified, later accept refused (FR-22, BR-13)	services	BR-13, FR-22	After three answers of can't the request is FAILED with all targets DECLINED; REQUEST_FAILED to owner and requester; accepting and cancelling are refused (REQUEST_CLOSED), approval too; expiry leaves it unchanged.

ID	Name	File	Covers	Expected result
TC-13B	no eligible candidate at creation: FAILED immediately, no targets (FR-08, BR-13)	services	BR-13, FR-08	A request without eligible candidate is FAILED at creation, has no targets, notifies owner and requester, and appears as a FAILED item on both Home screens.
TC-13C	one can't and one pending: still REQUESTED and no notification (FR-22)	services	FR-22	Two of three answers of can't leave the request REQUESTED without any notification and without FAILED items; the last candidate can still accept.
TC-13D	Home exposes the failed item; markNotificationsRead with its id removes it (BR-13, FR-21)	services	BR-13, FR-21, FR-22	The requester's and the owner's Home show the FAILED item (targets see none); the owner's task count includes it; marking its notification read removes only that item; another user's id is ignored.
TC-13E	demo walkthrough S8: Hana's Sun 10-04 request, all three can't, FAILED card for Hana and the owner (spec §10)	services	BR-13, FR-22, spec §10 S8	Walkthrough S8: Hana's Sunday request reaches Seoyeon, Minho and Doyun; after three answers of can't it is FAILED, Hana and the owner see the item, the owner acknowledges it, and the shift stays Hana's.
TC-003	a saved database from before iteration 6 (Availability table, no FAILED status) is replaced by the seed	services	NFR-05	A saved database from before iteration 6 (with the Availability table and without FAILED) is replaced by the seed, in which a request can end FAILED.
TC-141	seed history: shifts from contractStart to 09-20 follow the fixed schedule, all WORKED and confirmed (spec §10)	services	spec §10	Each worker's shifts from the contract start to 10-04 follow the fixed schedule without gap or duplicate around 09-20; the history up to 09-20 (58, 48, 6 and 12 shifts) is WORKED with confirmed attendance at the scheduled times.
TC-142	seed payroll: confirmed months per worker equal the generatePayroll recomputation (BR-06...BR-11)	services	BR-06-BR-11, spec §10	Ten CONFIRMED payroll rows (Seoyeon 03-08, Minho 06-08, Doyun 08); Minho's holiday pay 4, 4 and 5 × 33,024; Doyun 36 h, 360,000 won, warning acknowledged; regenerating every month gives the stored rows.
TC-143	prepareMonthlyPayroll: September draft on 10-01 with one notification, nothing again in October, nothing before (BR-14, FR-23)	services	BR-14, FR-23	Nothing is prepared in September or before the first contract month; on 10-01 the September DRAFT (4 rows, equal to generatePayroll) and one PAYROLL_DRAFT_READY; nothing more in October, the acknowledgement is kept; the October draft follows on 11-01.
TC-144	owner Home payrollReady, listPayrollMonths and the notification yearMonth (UC-14)	services	FR-23, UC-14, NFR-13	listPayrollMonths lists March–August as CONFIRMED and September as NONE, then DRAFT (prepared automatically); the owner Home shows payrollReady and counts it as a task; marking the notification read clears it.
TC-145	demo walkthrough S9: confirmed Mar–Aug, clock to 10-01, September draft with Doyun's warning, no second draft (spec §10)	services	BR-14, FR-23, spec §10 S9	Walkthrough S9: March–August confirmed; at 10-01 the September draft with Doyun's warning; confirmation blocked until acknowledged, then 4 rows confirmed; no second draft on 10-20.
TC-153	demo walkthrough S10: with 5 employees the preview shows Minho 24 → 29 h and warns of 13 h beyond contract (BR-15, FR-24)	services	BR-15, FR-24, FR-11	S10: the owner sets 5 regular employees and adds Fri 10:00–18:00; the preview shows Minho 24 → 29 h (contract 16) and 13 h beyond contract with a warning; only the acceptor is checked; the Home decision card carries the same warning; approval is not blocked.

ID	Name	File	Covers	Expected result
TC-154	S10 with Friday 10:00-17:00: Minho 23 → 28 h is exactly 12 h beyond contract, no warning (BR-15)	services	BR-15, FR-24	S10 with Friday 10:00-17:00: Minho 23 → 28 h is exactly 12 h beyond contract, no warning.
TC-155	S10 with 4 regular employees: Minho 24 → 29 h, 13 h beyond contract, no warning (BR-15)	services	BR-15, FR-24	S10 with 4 regular employees: Minho 24 → 29 h, 13 h beyond contract, no warning.
TC-160	every column name survives the snake_case mapping of the shared database	shared	spec §8, §9	camel(snake(c)) returns c for every column of the 11 tables.
TC-161	diffTables finds inserts, updates and deletes with every column in snake_case	shared	spec §9	diffTables returns update, insert and delete rows with snake_case columns; identical tables give no change.
TC-162	a command saves exactly its row changes; replica, server and server time agree	shared	spec §9	Minho's acceptance sends one change set with only Notification inserts and SubRequest and SubRequestTarget updates; revision 1 → 2; the server's tables equal the replica; the server time 09:00 is used and the clock cannot be set.
TC-163	first acceptance across browsers: a stale change set is recomputed on the current data (NFR-04)	shared	BR-02, NFR-04	Minho's change reaches the server first; Doyun's stale change set is refused, the gateway reloads and re-runs the command: ALREADY_TAKEN, nothing committed, acceptor Minho, revision 2.
TC-164	still stale after three re-runs: SHARED_BUSY, and the replica shows the server data	shared	spec §9, NFR-04	After four STALE answers in a row (the first attempt and three re-runs) the command fails with SHARED_BUSY; one reload; the replica shows the server data.
TC-165	a change the server refuses is re-run once on reloaded data, then reported as SHARED_REFUSED	shared	spec §9, NFR-14	A refusal of the first attempt is re-run once on reloaded data and then succeeds; two refusals in a row give SHARED_REFUSED with the server's detail; the request stays ACCEPTED and the replica equals the server.
TC-166	server answers map to the error codes shown in both languages	shared	spec §9, NFR-14	Twelve server and transport answers map to their codes (STALE, rate and daily limits, full store, refused, unavailable, offline); the nine user-facing codes have texts in Korean and English.
TC-167	offline: nothing is saved and the replica goes back to the last saved state	shared	spec §9	A change sent while offline fails with SHARED_OFFLINE and the replica returns to the saved state (request REQUESTED, revision 1); a rule error without changes passes through unchanged and sends nothing.
TC-168	System Clock duties are saved after drawing; a refusal pauses them for a minute	shared	BR-14, spec §9, §10	At server time 10-01 the payroll history is completed once and the September draft is prepared; the change sets are saved (server equals replica; March–August CONFIRMED, September DRAFT); a second call does nothing; a refused duty reloads the replica and pauses System Clock duties.
TC-169	polling reloads another person's change; reset for everyone reloads the seed	shared	NFR-05, NFR-15, spec §9	A poll without a change returns false; another user's change is loaded by the next poll (one reload); the reset for everyone reloads the seed.
TC-16A	the seed template of the shared database is up to date with the seed code (spec §10)	shared	spec §10	The seed template in db/app-shared.sql equals the template generated from the seed code.

ID	Name	File	Covers	Expected result
TC-16B	the REST transport sends only the publishable key and reports server errors	shared	NFR-14, spec §9	The transport calls the REST address of app_store with the apikey header only (no Authorization header), reports a server answer as STALE and a network failure as SHARED_OFFLINE.

6.4 Test Results

The table below gives the result of running `npm test` on the environment of Section 6.2 on 2026-09-29, against the final build (prompt v8, Chapter 5). Its groups are the 113 tests of the application listed in Section 6.3.

Table 65. Test results of the application by group (npm test, 2026-09-29)

Group	Subject	Tests	Passed	Failed
TC-00x	Seed data and storage (spec §10, NFR-05)	3	3	0
TC-01x	Overlap, eligibility, workers and schedule editing (BR-01, FR-01, FR-02, FR-04, FR-05)	16	16	0
TC-02x	Request validity and creation (BR-03, FR-06–FR-08)	6	6	0
TC-05x	First acceptance wins; acceptor re-checked for overlap (BR-02, NFR-04)	5	5	0
TC-06x	Approval effect and Home queries (BR-04, FR-11, NFR-13)	6	6	0
TC-07x	Attendance (UC-07, FR-13)	3	3	0
TC-08x	Holiday allowance (BR-06, BR-07)	12	12	0
TC-09x	Premium pay, including part-time hours beyond contract (BR-08)	12	12	0
TC-10x	Minimum wage and probation (BR-09, BR-10)	7	7	0
TC-11x	Monthly payroll (BR-11)	5	5	0
TC-12x	Expiry and cancellation (BR-12, FR-18, FR-19)	5	5	0
TC-13x	Privacy, notifications, role menus and no taker (NFR-11, FR-20–FR-22, BR-13)	11	11	0
TC-14x	Work history and automatic monthly payroll (spec §10, BR-14, FR-23)	5	5	0
TC-15x	Part-time weekly limit warning (BR-15, FR-24)	5	5	0
TC-16x	Shared mode: change sets, stale re-run, errors, seed template (NFR-04, NFR-14, NFR-15)	12	12	0
Total (rules: 36, services: 65, shared: 12)		113	113	0

Together with the 12 tests of the project hub in `tests/collab.test.js`, which passed as well, `npm test` ran 125 tests, and all 125 passed. The summary lines of the runner were:

```
i tests 125
i suites 0
i pass 125
i fail 0
i cancelled 0
i skipped 0
i todo 0
i duration_ms 1241.741834
```

The suite contains no skipped, pending or "todo" tests. The whole run takes between one and two seconds, which kept it practical to run the suite after every change, as the regression strategy of Section 6.1 requires. The count of application tests grew with the iterations, as recorded in the prompt log: 52 tests in the build of version 1; 58 after iteration 2 (TC-002, TC-01A, TC-01B, TC-105, TC-08B and TC-133); 62 after the traceability check of iteration 3 (TC-01C, TC-01D, TC-134, TC-135); 65 with the Home queries of iteration 4 (TC-064, TC-065, TC-066); 77 with the twelve regression tests of iteration 5; 84 after the change request of iteration 6, which added TC-13A to TC-13F and TC-003 and rewrote the tests that assumed availability (TC-014, TC-015, TC-025); 89 after iteration 7, which added TC-141 to TC-145 and updated TC-001, TC-088, TC-095 and TC-114 for the seeded history; and 113 after iteration 8, which added twelve part-time tests (TC-096 to TC-09C).

and TC-151 to TC-155) and twelve shared-mode tests (TC-160 to TC-16B) and left the 89 earlier tests unchanged. Each test was introduced together with the specification change or defect fix it verifies (Section 6.9).

The unit tests use a fake of the shared database, so they cannot show that the real database behaves in the same way. Two further scripts therefore run against the real database and are reported separately. `tools/shared-rest.mjs` passed 64 of 64 checks; the table below groups them by topic. `tools/shared-check.mjs` passed 12 of 12 checks with two headless browsers: the sign-in screen states that the demonstration is shared and offers the local mode; the demo tools show the server time and no clock controls; Minho and Doyun both see the open request and tap 수락하기/ at the same moment, exactly one of them gets the shift and the other is told that someone accepted first, and the server holds one ACCEPTED request with one accepting target; the owner sees the decision card, approves, and the server holds the APPROVED request; the second browser shows the new state within 20 seconds; the reset for everyone brings the open request back; there are no console errors in either browser; and the throwaway store is deleted. Finally, `tools/check.mjs` was run again on 2026-09-29 in the local demonstration mode and passed 154 of 154 checks (Section 6.7, NFR-13 row).

Table 66. Checks of `tools/shared-rest.mjs` against the shared database, by topic (64 of 64 passed)

Topic	Checks	What is checked
Read access and reset seed	8	snapshot of a new store (rev 1, 142 shifts, five workers), server time, the seed request still open after the move, unknown store, tables readable, counters not readable
No direct writes	5	direct insert, update and delete denied; internal reset and template functions not callable
Change sets and their validation	7	STALE, empty or unknown-table change set, unknown store, missing row (CONFLICT), more than 2,000 rows, more than 256 kB
Commits and state rules	34	commits and revisions, transitions of \$7, first acceptance on one revision, one open request per shift, server time (deadline, creation time, expiry), read-only attendance and payroll, no deletes
Limits	5	61st change set in a minute, 3,001st in a day, 20,000 rows, counters restart
Reset	3	seed restored, second reset within a minute, 51st reset in a day
Demo store and clean-up	2	the demo store <code>dalbit</code> exists and is readable; the throwaway store is deleted
Total	64	

The results of the two scripts that use the shared database are those recorded at the end of iteration 8; the number of checks was confirmed by reading the scripts (64 and 12 calls of the check function). Both scripts need network access and the owner connection of the database, so unlike `npm test` they are not part of the regression run after every change.

6.5 Boundary-Value Analysis

The legal rules of ShiftSwap are threshold rules: a worker is either above or below 15 hours, a workplace either has five employees or not, a part-time worker is either beyond the twelfth hour over the contract or not, a date is either inside or outside the probation period. Errors in such rules typically occur exactly at the threshold — a $>$ written for $\geq$, or a date compared inclusively instead of exclusively. The tests therefore place inputs on both sides of every threshold, as the table below shows.

Table 67. Boundary values tested for the legal and workflow rules

Rule	Threshold	Values below / at / above	Tests
BR-06 contractual hours	≥ 15 h	14.99 h $\rightarrow$ not eligible; 15.00 h $\rightarrow$ eligible (3 holiday hours); 16 h $\rightarrow$ eligible (3.2 h)	TC-082, TC-081, TC-083
BR-07 holiday-hours cap	$\min(h, 40)$	16 h $\rightarrow$ 3.2; 40 h $\rightarrow$ 8; 52 h $\rightarrow$ 8 (capped)	TC-083
BR-08 employees	≥ 5 regular employees	4 $\rightarrow$ premium 0; 5 $\rightarrow$ premium paid; part-time hours beyond contract: 4 employees $\rightarrow$ 0, 5 employees $\rightarrow$ 4 h	TC-091, TC-092, TC-095, TC-096
BR-08 daily hours	> 8 h per day	8 h days $\rightarrow$ no daily overtime; 10 h and 13 h days $\rightarrow$ 2 h and 5 h	TC-092, TC-094
BR-08 weekly hours	> 40 regular h per week	48 regular hours in a week $\rightarrow$ 8 weekly overtime hours, excluding the 2 daily ones	TC-094

Rule	Threshold	Values below / at / above	Tests
BR-08 night window	22:00–06:00	18:00–23:00 → 1 night hour; 21:00–07:00 → 8 night hours	TC-092, TC-093, TC-112
BR-08 (iv) part-time	$0 < C < 40$ contractual hours	$C = 0$ → no hours beyond contract; $C = 0.5$ and 39.99 → part-time; $C = 40$ → not part-time, no hours beyond contract	TC-097
BR-08 (iv) beyond contract	hours after the first C hours of the ISO week	$C = 10$, week of 14 h → 4 h; the next Monday starts a new week → still 4 h; a 10 h day with $C = 5$ → 2 daily + 3 beyond contract, counted once (5, not 7)	TC-096, TC-098, TC-09B
BR-08 (iv) month boundary	only hours on dates inside the month are paid	$C = 16$, Tue 5 h + Wed 5 h + Thu 5 h: the 16th hour is one hour into Friday, so Fri 7 h + Sat 6 h = 13 h are beyond contract, all in October; September pays night hours only	TC-09A, TC-09C
BR-10 contract length	≥ 365 days	2026-09-01 to 2027-08-30 (363 days) → no reduction; to 2027-09-01 (365 days) → reduction; open-ended → reduction	TC-102
BR-10 three months	$\text{date} < \text{contractStart} + 3 \text{ months}$	2026-08-31 → does not apply; 2026-09-01 and 2026-11-30 → apply; 2026-12-01 → does not, even with a later probation end	TC-102, TC-106
BR-10 probation end	$\text{date} \leq \text{probationEnd}$	probation end 10-15, date 10-16 → does not apply; no probation end → does not apply	TC-102
BR-09 minimum wage	$\text{wage} < \text{floor}$	10,320 at minimum 10,320 → no warning; 10,000 → warning; 10,000 under probation (floor 9,288) → no warning; 9,000 → warning; under probation 9,288 → no warning, 9,287 → warning	TC-103, TC-107
BR-01 overlap	half-open [s, e)	10:00–16:00 vs 16:00–22:00 → no overlap; 10:00–16:01 → overlap	TC-011
BR-15 hours beyond contract	$S - C > 12 \text{ h}$	$C = 16$: 28 h → 12 h beyond, no warning; 28.5 h → 12.5 h and 29 h → 13 h, warning; 28.1 – 16.1 → exactly 12 h, no warning. In the walkthrough: 23 → 28 h (12 h) no warning, 24 → 29 h (13 h) warning	TC-151, TC-154, TC-153
BR-15 employees and contract	≥ 5 regular employees and $0 < C < 40$	4 employees with 13 h beyond contract → no warning, 5 employees → warning; $C = 40$ (53 h) and $C = 0$ (13 h) → no warning	TC-152, TC-155, TC-153
BR-03 deadline	$\text{now} < \text{deadline} \leq \text{shift start}$	$\text{deadline} = \text{now}$ → refused; $= \text{shift start}$ → accepted; $\text{shift start} + 1 \text{ min}$ → refused	TC-021, TC-023
BR-12 expiry	$\text{deadline} \leq \text{now}$ or $\text{shift start} \leq \text{now}$	now 20:59 with deadline 21:00 → open; now 21:00 → expired; approval at the deadline or at the shift start → refused and expired	TC-121, TC-125

The probation dates deserve a comment. For Jung Hana, whose contract starts on 2026-09-01, the three-month limit is 2026-12-01, computed by adding calendar months rather than 90 days. The test checks 2026-11-30, the last day on which the reduction may apply, and 2026-12-01, the first day on which it may not; it also checks 2026-12-01 with a probation end deliberately set later (2027-03-01), to prove that the three-month limit is applied independently of the probation end written in the contract. The contract-length tests use two end dates two days apart around the 365-day threshold.

The part-time limit of BR-15 is exclusive: exactly 12 hours beyond contract gives no warning, and 12.5 hours do, as the rule states. Comparing $S - C$ in floating point would put a week of 28.1 hours with a contract of 16.1 hours over the limit ($28.1 - 16.1$ is 12.000000000000002), so the rule compares the two values on two decimals, and TC-151 fixes this case. The hours of a part-time worker beyond the contract depend on the order of the hours: TC-09A places the crossing of the contractual 16 hours one hour into a Friday and the month boundary between the Wednesday and the Thursday of that week, so that the days of September only move the crossing point and are not paid in October.

6.6 Concurrency Test

NFR-04 requires that two acceptances of the same request can never both succeed. TC-051 tests this directly with the seed request: Choi Minhø accepts, and then Kang Doyun accepts the same request. The expected result has four parts: the first call returns ACCEPTED; the second throws a `ServiceError` with the code `ALREADY_TAKEN` and the parameter `name = 'Choi Minhø'`, which is what the inbox displays; the request's acceptor is Minhø; and the targets are ACCEPTED for Minhø and CLOSED for Doyun. TC-052 checks the notifications of the same situation, and TC-063 checks the state the second candidate sees in the inbox (TAKEN, with the acceptor's name).

TC-051 calls the two acceptances one after the other on one working copy. That is faithful to SD-2 for one JavaScript thread, which executes one transaction at a time, and it verifies the design property that makes the outcome independent of timing: the service re-reads the request inside the transaction, and the update is conditional on `status = 'REQUESTED'`, with the number of changed rows checked. Up to iteration 7 this was all that could be tested, because the client-only architecture had no shared database and therefore no truly parallel transactions. In the shared mode of iteration 8 two people on two phones can act on the same request at the same moment, and the guarantee rests on the server: it applies a change set only to the revision it was computed on, under a row lock on the store. The table below lists the cases at each level.

Table 68. Concurrency cases and their tests

Case	Expected result	Test
Two acceptances one after the other on one working copy	First ACCEPTED; second ALREADY_TAKEN naming Choi Minhø; targets ACCEPTED and CLOSED; notifications as in spec §8.	TC-051, TC-052, TC-063
The second acceptor's replica is one revision behind (STALE re-run)	The server refuses the stale change set; the gateway reloads and runs the command again on the current data; the normal answer ALREADY_TAKEN follows, nothing is committed, the acceptor is Minhø, the revision is 2.	TC-163
The replica stays stale	After the first attempt and three re-runs the command fails with SHARED_BUSY; the replica shows the server data.	TC-164
Two change sets on the same base revision sent to the real database at once	Exactly one succeeds and the other is refused with STALE; the request is ACCEPTED once with one acceptor (Minhø or Doyun, whichever is committed first).	REST checks "two acceptances on the same revision" and "ACCEPTED once"
Two open requests for one shift in one change set	Refused by the server (unique index on open requests per shift, BR-03).	REST check "two open requests for one shift are refused"
Acceptance or approval after the deadline by server time	Refused by the server; an overdue request expires.	REST checks "acceptance after the deadline" and "an overdue request expires"
Two browsers tap 수락하기 at the same moment	Exactly one browser gets the shift and the other is told that someone accepted first; the server holds one ACCEPTED request with one accepting target.	shared-check.mjs SC-2 (three checks)

The last three rows run against the real database and the real user interface, so they check what the unit tests can only assume: that the row lock, the revision check and the triggers of `db/app-shared.sql` give the outcome that the fake server of `tests/shared.test.js` models. The REST test does not decide which of the two acceptors wins, because that depends on which request reaches the database first; it checks that the outcome is one of the two and never both. The check with two browsers has the same property. What none of the tests covers is the load of many users at once; the limits of NFR-14 (60 change sets a minute per store) bound it, and Section 8.3 discusses the limitation of the whole-store replica.

6.7 Verification of Non-Functional Requirements

The non-functional requirements were verified with the methods named in their specification. The table below gives the evidence and the status for each. Where a check has not been carried out, the table says so.

Table 69. Verification of the non-functional requirements

ID	Method	Evidence	Status
----	--------	----------	--------

ID	Method	Evidence	Status
NFR-01	UI walkthrough	In the final build the pending request appears as a card on the candidate's Home screen, with the effect of accepting and the Accept button on the card, so acceptance takes one tap after sign-in. Checked in the Korean walkthrough of <code>tools/check.mjs</code> (done criterion 4); the data behind the card is covered by TC-064 and the state logic by TC-063.	Met
NFR-02	Browser check at 360 px and 1440 px	The prompt log records headless Chrome checks of every screen for both roles in English and Korean at 1440 px and 360 px with no horizontal scroll for the builds of version 2 (70 of 70 checks) and version 3 (82 of 82 checks). For the final build, <code>tools/check.mjs</code> checks 33 screen states in both languages at 1280 px and 360 px for horizontal scroll; all pass (Section 6.7, NFR-13 row). The check found one overflow in iteration 8 (Section 6.9).	Met
NFR-03	Timed load with 20 workers and 12 weeks	Browser measurement on the final (iteration 6) interface with <code>tools/nfr03.mjs</code> (Apple M2, headless Chrome 153, viewport 1280 × 800), owner signed in, 20 workers, 12 generated weeks (588 shifts). Route change to rendered DOM, median / maximum of 10 runs: Schedule 3.1 / 5.7 ms, Pay → Weekly 7.7 / 15.7 ms, Pay → Monthly 13.4 / 15.2 ms; including paint: 27.5 / 28.4 ms, 10.6 / 28.7 ms and 23.3 / 35.1 ms. The slowest screen took 15.7 ms to the DOM and 35.1 ms to paint, against the limit of 1,000 ms. Cold start (reload to rendered schedule): median 39 ms, range 27–63 ms. The procedure and data set are recorded in <code>docs/nfr03-measurement.md</code> in the repository.	Met
NFR-04	Unit tests TC-05x, TC-16x; REST test of the shared database	TC-051, TC-052, TC-053 and TC-063 pass; in the shared mode TC-163 and TC-164 (stale re-run), the two REST checks on two change sets with the same base revision (exactly one succeeds, the other is STALE) and SC-2 of the two-browser check (Section 6.6).	Met
NFR-05	Manual test; REST test	Local mode: TC-105 reloads the database from a storage object and finds the acknowledgement kept; TC-002 and TC-003 check the replacement of an outdated copy (from before iteration 2 and before iteration 6); the prompt log records a headless reload and reset check. Shared mode: the REST test commits a change and reads it back in the next snapshot, and its reset check restores the seed; TC-169 and SC-4 of the two-browser check show the reset for everyone.	Met
NFR-06	Manual test in four browsers	Chrome checked (headless and desktop), in both data modes. The code uses only standard ES modules, <code>fetch</code> , <code>localStorage</code> , <code>sessionStorage</code> and <code>WebAssembly</code> ; the shared mode needs no server of its own besides the hosted database. Checks in Edge, Safari and Firefox are not recorded at the time of writing.	Chrome met; others pending
NFR-07	Deployed URL	<code>https://keemlogan.github.io/shiftswap/app/</code> returned HTTP 200 with the application's <code>index.html</code> when checked on 2026-09-29; <code>app/config.js</code> and <code>app/core/shared.js</code> of iteration 8 are served as well (HTTP 200).	Met
NFR-08	Repository	Static HTML, CSS and ES modules; no build step; <code>sql.js</code> vendored in <code>app/vendor/</code> ; hosted on GitHub Pages; the shared data are in the Supabase PostgreSQL database, reached with the publishable key from <code>app/config.js</code> and set up by <code>db/app-shared.sql</code> .	Met
NFR-09	Review	UML models drawn with PlantUML [14] in UML 2.5 notation [8]; all code identifiers, table and column names in English.	Met

ID	Method	Evidence	Status
NFR-10	Unit tests TC-08x, TC-09x, TC-15x	All TC-08x, TC-09x, TC-10x and TC-15x tests pass, including the boundary values of Section 6.5. For the part-time rule the tests check the system rules (a)–(e) of BR-08 and BR-15, which are design decisions where the law leaves the method open, not the law itself.	Met
NFR-11	Role test	TC-131 and TC-135: a worker receives null for other people's phone and wage and sees only their own attendance rows; TC-134: the worker routes contain no owner screen. In the shared mode this is a rule of the screens and the services, not of the database: every row can be read with the publishable key (Section 8.3).	Met
NFR-12	Design review	A search of the application code finds no use of the geolocation interface; attendance is entered by the worker and confirmed by the owner (TC-071).	Met
NFR-13	Headless check that counts the filled primary buttons on every screen of both roles (exactly one expected), checks that every disabled button has a visible reason and that touch targets are at least 44 × 44 px (done criterion 7 of the final prompt); walkthrough of the sign-in tour (done criterion 4)	tools/check.mjs, run in the local demonstration mode on 2026-09-29: 154 of 154 checks pass with no console errors on the final build (prompt v8): 33 screen states × Korean/English × 1280/360 px for done criterion 7 (132 checks: one filled primary button, visible reasons on disabled buttons, touch targets ≥ 44 px, no horizontal scroll), the sign-in screen (criterion 3, 2 checks), the Korean tour walkthrough (criterion 4, 8 checks), five checks of walkthrough S8 (criterion 5), five checks of walkthrough S9 (criterion 6), one check of the console and one check S10 of the part-time warning (below). The screen state added in iteration 8 is owner-overtime-warning, the owner's decision card with the BR-15 warning. In S10 the owner's decision card shows Minhø 24 → 29 h and "13 h beyond contract" and the button 승인하기/ stays enabled. The data shown on the Home screens and decision cards is covered by TC-064, TC-065 and TC-153.	Met
NFR-14	REST test of the shared database (iteration 8)	tools/shared-rest.mjs, 64 of 64 checks, with the publishable key on a throwaway store (Section 6.4): direct insert, update and delete are refused; the internal reset and template functions cannot be called; the rate-limit counters cannot be read; change sets are refused when stale, empty, for an unknown table or store, for a missing row, above 2,000 rows or 256 kB; the transitions of \$7, acceptance and approval before the deadline only, expiry only when overdue, one open request per shift, read-only confirmed payroll and creation times within 10 minutes of server time are enforced by the triggers; the 61st change set in a minute, the 3,001st in a day, a store above 20,000 rows, a second reset within a minute and the 51st reset in a day are refused. TC-16B checks that the client sends the publishable key only. Identity is not verified, as the requirement states (Section 8.3).	Met
NFR-15	Two-browser check	SC-3 of tools/shared-check.mjs: after the owner's approval in the first browser, the second browser shows the new state within 20 seconds (the check waits up to 20 s in steps of one second and passed). The polling interval of 10 seconds is set in main.js, and TC-169 checks that a poll loads another person's change. The delay itself was not timed.	Met by design; delay not timed
NFR-16	Workflow log; manual test	The workflow .github/workflows/supabase-keepalive.yml is in the repository and runs every six hours and on demand. Its run list holds one run, started by hand on 2026-09-29, which succeeded. No scheduled run and no test of the offer of the local mode when the database cannot be reached are recorded at the time of writing; TC-167 checks only the gateway's behaviour when a request fails offline.	Partly met

6.8 Traceability Matrix

The specification requires that every functional requirement maps to at least one use case, every use case to at least one screen and service function, and every business rule to at least one test case (spec §12). The matrix below was derived from the code, not from the design documents: the screen column lists the UI modules that call the service functions, found by searching each module in `app/ui/` for its calls to `services.js`, and the test column lists the tests that call those functions or the rule functions behind them and assert the requirement's behaviour.

Table 70. Traceability matrix: functional requirement → use case → screen → service → business rule → tests

FR	UC	Screen (app/ui)	Service function(s)	BR	Tests
FR-01	UC-01	workers.js	registerWorker	BR-09, BR-10 (data)	TC-01C, TC-01D
FR-02	UC-01	workers.js	saveFixedSchedule	BR-06	TC-017, TC-01B, TC-083
FR-03	UC-02	Retired in iteration 6 together with the table Availability (Section 3.11.4); no screen, service or test			
FR-04	UC-03	schedule.js	generateWeek	BR-05	TC-018, TC-013, TC-01C, TC-01E, TC-01F
FR-05	UC-03	schedule.js	editShift	BR-01 (overlap)	TC-019, TC-01A, TC-01G, TC-066
FR-06	UC-04	requests.js	createSubRequest, listRequestableShifts, defaultDeadline	BR-03	TC-021, TC-022, TC-023, TC-026
FR-07	UC-04	requests.js	createSubRequest	BR-01	TC-014, TC-015, TC-016, TC-024, TC-133
FR-08	UC-04	requests.js	createSubRequest, previewCandidates	BR-01, BR-13	TC-025, TC-066, TC-13B
FR-09	UC-05	home.js, requests.js	getWorkerHome, respondToRequest	BR-02, BR-12, BR-13	TC-051, TC-053, TC-054, TC-063, TC-064, TC-124, TC-13A, TC-163
FR-10	UC-05	requests.js	respondToRequest	BR-02	TC-052, TC-133
FR-11	UC-06	home.js, approvals.js	getOwnerHome, getApprovalPreview (shared effect computation)	BR-06, BR-07, BR-15	TC-063, TC-064, TC-065, TC-08B, TC-153
FR-12	UC-06	approvals.js	decideRequest, recomputeWeek	BR-02, BR-04, BR-12	TC-061, TC-062, TC-055, TC-065, TC-125, TC-133
FR-13	UC-07	attendance.js	recordAttendance, confirmAttendance, markAbsent	BR-06	TC-071, TC-072, TC-073, TC-089
FR-14	UC-08	weekly.js	recomputeWeek	BR-05, BR-06, BR-07	TC-081-TC-089, TC-08A, TC-08C
FR-15	UC-09	payroll.js	generatePayroll, confirmPayroll, getPayroll	BR-07, BR-08, BR-11	TC-111-TC-115, TC-095, TC-09A, TC-09C
FR-16	UC-09	payroll.js	generatePayroll, acknowledgeMinWage, confirmPayroll	BR-09, BR-10	TC-101-TC-107
FR-17	UC-10	settings.js	updateSettings, getSettings	BR-06, BR-08, BR-09	TC-08A, TC-08B, TC-095
FR-18	UC-11	(router main.js)	expireOverdue	BR-12	TC-121, TC-122, TC-124, TC-125, TC-133
FR-19	UC-12	requests.js	cancelRequest	—	TC-123, TC-133

FR	UC	Screen (app/ui)	Service function(s)	BR	Tests
FR-20	UC-13	login.js, main.js	menuFor, routesFor, listWorkers, getWorker	NFR-11	TC-134 (menus and routes), TC-131, TC-135 (privacy), TC-169 (reset for everyone); shared-check.mjs SC-1 (shared notice on the sign-in screen)
FR-21	UC-04-06, 11, 12	inbox.js, main.js	listNotifications, markNotificationsRead, unreadCount	—	TC-132, TC-133, TC-13D
FR-22	UC-05	home.js, requests.js	respondToRequest, getWorkerHome, getOwnerHome, markNotificationsRead	BR-13	TC-13A, TC-13C, TC-13D, TC-13E, TC-133
FR-23	UC-14	home.js, payroll.js (main.js runs the System Clock)	prepareMonthlyPayroll, getOwnerHome, listPayrollMonths, getPayroll	BR-14	TC-143, TC-144, TC-145, TC-168
FR-24	UC-06	home.js, approvals.js	getApprovalPreview, getOwnerHome (with swapEffect and partTimeLimitCheck)	BR-15, BR-08 (a)	TC-151, TC-152, TC-153, TC-154, TC-155

An earlier version of the matrix showed two gaps: FR-01 (registering a worker) had no automated test, and FR-20 (role menus) was enforced only by the router in main.js, which the Node tests do not load. Both were closed in iteration 3. TC-01C and TC-01D test registerWorker with every field and every validation error, and the role-to-menu table was moved from the router into the pure service function menuFor(role) (with routesFor(role) added in iteration 4), which TC-134 tests. Every active functional requirement now has at least one automated test, including FR-22, which was added in iteration 6, and FR-24, which was added in iteration 8; the retired FR-03 is kept in the matrix as a struck-through row so that its number is not reused.

The table below summarises coverage from the other direction. Every business rule, including BR-13 and BR-15, has at least one test, as spec §12 requires; most have several, at the rule level and through the services. The shared mode adds a server that enforces BR-02, BR-03 and BR-12 a second time; its checks are listed in Sections 6.4 and 6.6 and are not counted in the table, which counts the test cases of Section 6.3 only.

Table 71. Business-rule coverage by test cases

BR	Rule	Implementing functions	Tests	Count
BR-01	Eligible candidates	isEligibleCandidate, overlaps	TC-011, TC-012, TC-014, TC-015, TC-016, TC-024, TC-066, TC-08B	8
BR-02	First acceptance wins	respondToRequest	TC-051, TC-052, TC-053, TC-054, TC-055, TC-063, TC-163	7
BR-03	Request validity	validateRequest	TC-021, TC-022, TC-023, TC-026	4
BR-04	Approval effect	decideRequest	TC-061, TC-062	2
BR-05	Week	weekStartOf	TC-013, TC-018	2
BR-06	Holiday-allowance eligibility	computeWeeklySummary, contractHours	TC-081, TC-082, TC-084–TC-089, TC-08A, TC-08B, TC-08C	11
BR-07	Holiday-allowance hours and pay	holidayHours, computePayrollRow	TC-081, TC-083, TC-088, TC-111, TC-113	5
BR-08	Premium pay	premiumHours, isPartTime	TC-091–TC-099, TC-09A–TC-09C, TC-112	13

BR	Rule	Implementing functions	Tests	Count
BR-09	Minimum-wage check	minimumWageFor, computePayrollRow, acknowledgeMinWage, confirmPayroll	TC-101, TC-103, TC-104, TC-105, TC-107	5
BR-10	Probation reduction	probationApplies	TC-102, TC-103, TC-104, TC-106, TC-107	5
BR-11	Monthly payroll	computePayrollRow, generatePayroll	TC-111-TC-115, TC-088, TC-09A, TC-09C	8
BR-12	Expiry	expireOverdue, isOverdue (and the server triggers, REST test)	TC-121, TC-122, TC-124, TC-125, TC-133	5
BR-13	No taker	hasNoTaker, respondToRequest, createSubRequest	TC-13A-TC-13F, TC-025, TC-003	8
BR-14	Automatic monthly payroll draft	prepareMonthlyPayroll, generatePayroll	TC-141-TC-145, TC-168	6
BR-15	Part-time weekly limit warning	partTimeLimitCheck, getApprovalPreview	TC-151-TC-155	5

6.9 Defects and Specification Gaps Resolved During the Iterations

The build iterations are described in Chapter 5; this section lists what they found and how each finding was verified. In iteration 1, the AI coding assistant built the system from prompt version 1 and recorded every point at which the specification was ambiguous or silent, together with the choice it made. A document review of the proposal and interim report against the specification found further gaps. In iteration 2 each finding was resolved by changing the specification or the prompt, and where necessary the code, and each resolution was tied to a test. The table below lists the findings. Most of them were gaps in the specification rather than errors in the code: the implementation had already made a reasonable choice, and the change made that choice part of the contract, so that a regenerated system must make the same choice.

Table 72. Findings of iterations 1 and 2, their resolution and verification

#	Finding	Resolution (specification / prompt / code)	Verified by
1	node --test tests/ fails on Node 24 (directory argument read as a module path)	Prompt: test script node --test tests/*.test.js	npm test runs
2	Read-only queries for the screens and an error type were not in the service list	Prompt: read-only query functions and ServiceError(code, message, params)	All service tests
3	BR-06 required every shift to be WORKED, so a week in progress could never be eligible, contradicting the worked example	BR-06: a SCHEDULED shift does not break attendance; the weekly result is a projection	TC-085, TC-063
4	Unclear which workers have a weekly summary, and whether the seed contains derived rows	Spec §8: a summary exists only for workers with a shift in the week; the seed has no WeeklySummary or Payroll rows	TC-001, TC-088
5	Meaning of "estimated" and of shifts without attendance in the payroll	BR-11: WORKED → recorded hours; SCHEDULED → scheduled hours and estimated = 1; ABSENT → 0	TC-114, TC-112
6	Premium pay at month level mixed daily, weekly and night rules without saying how a week split by the month end is treated	BR-08: weekly overtime uses only the days of the week inside the month; premium pay rounded down	TC-092-TC-095
7	Date on which BR-10 is evaluated for a monthly payroll	BR-10: first day of the month, or contract start if later	TC-102, TC-104

#	Finding	Resolution (specification / prompt / code)	Verified by
8	"Effective hourly wage" in BR-07 undefined; rounding of holiday pay unclear	BR-07: <code>Worker.hourlyWage</code> ; rounded down per week, then summed	TC-111, TC-113
9	Format of attendance times	Spec §8: HH:MM on the shift date; end $\leq$ start means next day	TC-071
10	Recipients of cancel and expiry notifications not specified (also found in the review of UC-12)	Spec §8 recipients table; FR-19; a decline notifies no one	TC-133 (new), TC-052, TC-121, TC-123
11	Seed request creation time, phone numbers and stored name language not given	Spec §10: <code>createdAt 2026-09-28T08:30</code> , fictional phone numbers, English names	TC-001, TC-131
12	Deleting a shift that already has a request or attendance	FR-05: refused with an explanatory message	TC-01A (new)
13	Deleting a fixed-schedule row that already generated shifts	FR-02: shifts kept with <code>fixedScheduleId = NULL</code>	TC-01B (new)
14	Storage of the demo clock and language; effect of reset on the clock	Prompt: <code>keys shiftswap.clock.v1, shiftswap.lang</code> ; reset also resets the clock	Headless reset and reload check
15	No module for shared UI helpers	Prompt: escaping, formatting and the form error box in <code>i18n.js</code>	Review
16	Stripe pattern conflicted with "no gradients"; tables did not fit 360 px	Prompt: the stripe may use <code>repeating-linear-gradient</code> ; tables become stacked cards below 720 px	Headless check at 360 px
17	The minimum-wage acknowledgement was not stored, so it was lost on reload (document review)	Schema: <code>Payroll.minWageAck</code> ; BR-09; new service <code>acknowledgeMinWage</code> ; Confirm disabled until every warning is acknowledged	TC-104, TC-105 (new), TC-115
18	Saved databases from before the schema change lacked the new column	<code>openDatabase</code> checks the schema and replaces an outdated copy with the seed	TC-002 (new)
19	No scenario in which a swap changes eligibility, so FR-11's highlight was never exercised	Spec §10: demonstration walkthrough S7 with the ABSENT policy	TC-08B (new)
20	Date of Week 14 in NFR-07 inconsistent with the course calendar	NFR-07 corrected to 2026-12-04; spec §14 calendar	—

Two findings changed the code; the others changed only the specification and the prompt. Finding 17 is the most instructive. The original requirement said that a minimum-wage warning "blocks confirmation until acknowledged", but the data model had no place to store an acknowledgement, and the version 1 build handled it as a parameter of the confirm call. The review of the interim report noticed that an owner who acknowledged a warning and then reloaded the page would find the acknowledgement gone. The fix touched every level at once — a new column in the schema, a new clause in BR-09, a new service function, a new button, and two tests — which shows why the change procedure of Section 3.11.3 starts with the specification.

Findings from the UI review of the running application in iteration 3 (prompt version 3) concern layout and presentation rather than rules; they are listed with the prompt changes in Chapter 5, and the screens that result from them are shown in Chapter 7.

Iteration 5 added a second kind of finding: defects in the code of the reference implementation, found by an independent code review after reproduction run 1 (Section 5.6.2). A reviewer script reproduced each defect against the version 3 code before it was fixed. The table below lists the seven defects and the regression tests that now guard against them.

Table 73. Defects found by the independent code review (iteration 5) and their regression tests

#	Defect	Fix (specification / code)	Regression tests
1	A candidate who already works an overlapping shift on the date could accept, and could be approved	BR-02: the acceptor is re-checked at acceptance and at approval; ACCEPTOR_BUSY is thrown inside the transaction, so nothing changes	TC-054, TC-055
2	A WORKED or ABSENT shift could be confirmed or marked absent again	Spec §7, FR-13: confirm and mark-absent require a SCHEDULED shift (ATTENDANCE_CLOSED); TC-089 now uses a SCHEDULED shift	TC-072, TC-073, TC-089 (changed)
3	Regenerating a week re-created a shift the owner had moved, or duplicated shifts after the fixed rows were re-entered	FR-04: a fixed slot is skipped when its fixed schedule already produced a shift in the week or the worker already has an overlapping shift that day	TC-01E, TC-01F
4	Recomputing a week left the summary row of a worker who no longer had a shift that week, so payroll could count stale holiday pay	Spec §8: such rows are deleted by recomputeWeek	TC-08C
5	Worker, date or time of a shift with an open request could be changed	FR-05: refused with SHIFT_OPEN_REQUEST; the error has a message in both languages	TC-01G
6	An overdue request could still be accepted or approved until the next expiry check	BR-12: accept and approve expire an overdue request themselves and refuse with REQUEST_CLOSED; the expiry is committed before the error is thrown	TC-124, TC-125
7	The 90 % probation floor was computed in floating point, and $10320 * 0.9$ is not exactly 9,288	BR-10 window stated exactly (contract start inclusive to three months later exclusive, only with a probation end); floor computed as $\text{minimumHourly} * 9 / 10$	TC-106, TC-107

Iteration 6 was a requirements change rather than a correction (Section 3.11.4). Its effect on the tests is listed in the table below, taken from the prompt log. Tests that relied on availability were rewritten, and the new rule BR-13 and requirement FR-22 received tests of their own.

Table 74. Test changes for the change request of iteration 6

#	Specification change	Tests
1	BR-01 without availability: eligible means an active worker, not the requester, with no overlapping shift that day	TC-014, TC-015 rewritten; TC-016 and TC-066 updated (three recipients for the seed request and for Hana's Sunday shift)
2	UC-02 and FR-03 retired; table Availability removed	TC-001 checks that the table no longer exists; the set-up of TC-08A, TC-122 and TC-125 no longer registers availability
3	Seed request with three targets (Minho, Hana, Doyun)	TC-001, TC-051, TC-052, TC-054, TC-065, TC-123 updated
4	SubRequest.status gains FAILED; an older saved database is replaced	TC-003 (new)
5	BR-13, FR-08: no candidate at creation ends the request FAILED	TC-025 rewritten; TC-13B and TC-13F (new)
6	BR-13, FR-22: the last can't ends the request FAILED; a single can't notifies no one	TC-13A, TC-13C (new); TC-133 updated (REQUEST_FAILED instead of NO_CANDIDATE)
7	A FAILED request cannot be accepted, cancelled or approved and does not expire	TC-13A

#	Specification change	Tests
8	FR-22: the failed request stays on Home until acknowledged and counts as an owner task	TC-13D (new)
9	Demonstration walkthrough S8	TC-13E (new)

Iteration 7 added a requirement (Section 3.11.5). Its tests are listed in the table below, taken from the prompt log.

Table 75. Test changes for the change request of iteration 7

#	Specification change	Tests
1	Spec §10: work history from each contract start to 2026-09-20, worked with confirmed attendance	TC-141 (new); TC-001 updated
2	Spec §10: payroll confirmed for every month up to 2026-08, computed as <code>generatePayroll</code>	TC-142 (new)
3	UC-14, FR-23, BR-14: the previous month's draft on or after the first day, at most once, never confirmed	TC-143 (new)
4	PAYROLL_DRAFT_READY on the owner's Home; month switcher of Pay → Monthly	TC-144 (new)
5	Demonstration walkthrough S9	TC-145 (new)
6	A database saved from an earlier seed is not kept (storage key <code>shiftswap.db.v2</code>)	TC-002, TC-003, TC-105 use the new key
7	September figures that change with the history	TC-088 (4 × 33,024 holiday pay), TC-095 (9 night hours for Seoyeon), TC-114 (Minho 69 h, four holiday weeks) updated

Iteration 8 changed the architecture as well as the rules (Section 3.11.6). Its tests are listed in the table below, taken from the prompt log. The 89 earlier tests were not changed; the new tests were added to the groups of the rules they verify and to the two new groups TC-15x and TC-16x.

Table 76. Test changes for the change request of iteration 8

#	Specification change	Tests and checks
1	BR-08 (iv) and the system rules (a)–(e): part-time hours beyond contract, weekly and chronological, counted once, night hours separate	TC-096 to TC-09C (7 tests)
2	BR-15, FR-24: warning at more than 12 h beyond contract with five or more regular employees, not blocking	TC-151 to TC-155 (5 tests); check S10 of <code>tools/check.mjs</code>
3	§8, §9: the tables as <code>app_ tables</code> , the functions <code>app_snapshot</code> , <code>app_commit</code> , <code>app_reset</code> , row triggers, grants, limits (NFR-14)	<code>tools/shared-rest.mjs</code> (64 checks); TC-16A (seed template up to date)
4	§9 steps 1–6: replica, change sets with the base revision, stale re-run, one re-run after a refusal, System Clock work saved after drawing, polling	TC-160 to TC-169, TC-16B (11 tests)
5	§10 shared-mode seed: the template moved by whole weeks, payroll history completed by the application	TC-162, TC-168, TC-16A; REST check "app_reset restores the seed"
6	FR-20, UC-13, §9 step 8: shared notice on the sign-in screen, demo tools with the server time, reset for everyone	<code>tools/shared-check.mjs</code> (12 checks); TC-169

The tests of the iteration found no defect in the rules or the services, and the shared-mode checks found none in the database. The one defect of iteration 8 was found in the layout of the sign-in screen at 360 px. The button in the notice on the shared mode (`.mode-notice .btn`) did not wrap its text, which made the page wider than the screen and violated NFR-02. The style rule was changed to `white-space: normal` in `app/styles.css`, and the layout check of done criterion 7 has passed for every screen state since. No test of the rules could have found this defect, since it concerns a new interface element, which confirms the value of running the layout check over every screen state after each iteration.

7. Deployment and User Guide

This chapter explains how ShiftSwap is deployed and how it is used. Section 7.1 describes the deployment on GitHub Pages and the shared database, what happens when the application is opened in each of its two data modes, how to reset or run it locally, which browsers have been checked, and the project hub on which the team edits the project documents together. Section 7.2 is a user guide organised by role and task, which follows the demonstration data of spec §10 through one complete substitution and then through attendance, weekly summary and payroll. Section 7.3 gives a script for a live demonstration, and Section 7.4 lists common problems and their solutions.

7.1 Deployment

7.1.1 Hosting on GitHub Pages

The application itself is a static web application (NFR-08): deploying it means publishing files, and there is no application server of its own to install or start. Since iteration 8 the default data mode also uses a hosted database, which is set up once, independently of the published files (see below). The repository is hosted on GitHub and published with GitHub Pages [13] from the root of the `main` branch, with HTTPS enforced. An empty `.nojekyll` file in the root tells GitHub Pages to serve the files as they are, without processing them. The table below gives the addresses.

Table 77. Deployment addresses

Item	Address
Application	https://keemlogan.github.io/shiftswap/app/
Project page (deliverables hub)	https://keemlogan.github.io/shiftswap/
Source repository	https://github.com/keemlogan/shiftswap
Pages source	Branch <code>main</code> , folder / (repository root); HTTPS enforced
Shared database	Supabase project <code>shiftswap</code> , https://dbahxuegxnqsthntpuqy.supabase.co ; demonstration store <code>dalbit</code> ; address, publishable key and store in <code>app/config.js</code>

Publishing a new version consists of committing the changed files and pushing them to `main`; GitHub Pages rebuilds the site within a few minutes. Because all URLs inside the application are relative, the files work unchanged under the sub-path `/shiftswap/app/`. The deployment was checked on 2026-09-27: the application URL returned HTTP 200 with the application's `index.html`, and the Pages configuration reported the status "built".

The shared database is deployed separately. The script `db/app-shared.sql` creates the tables, row triggers, functions, grants and seed template of the shared mode (Section 4.5.2); it is idempotent and is applied by the team through the owner connection of the database, which is never stored in the repository. The application receives only the publishable key, which allows it to read the tables and call the three functions `app_snapshot`, `app_commit` and `app_reset`. On the free plan of Supabase, a project that shows low activity over seven days is paused until it is resumed manually from the dashboard [16]; the scheduled workflow `.github/workflows/supabase-keepalive.yml` therefore sends a read query every six hours, and because GitHub disables scheduled workflows in a public repository after 60 days without repository activity [17], the team also checks the public URL every week (Section 4.2, NFR-16).

7.1.2 What happens when the application is opened

The application has two data modes (spec §9). The **shared mode** is the default: everyone who opens the application works on the same demonstration store in the shared database, and the time is the server's time in Korea. The **local demonstration mode** keeps the data in the browser, as all versions up to iteration 7 did, and has a demo clock that can be set. The address parameter `?mode=local` or `?mode=shared` selects a mode, and the choice is remembered in the browser under `shiftswap.-mode`; the parameter `?store=` selects another existing store and is used by the automated checks with throwaway stores.

1. The browser loads `app/index.html`, the style sheet, `vendor/sql-wasm.js` and the module `main.js`, which imports `config.js`. While the data are being prepared, the page shows a loading message.
2. `main.js` initialises `sql.js`, which loads the WebAssembly file `vendor/sql-wasm.wasm` (about 650 KB) from the same site. In both modes the rules and services run on a SQLite database in the browser.
3. **Shared mode.** The gateway calls `app_snapshot` for the store `dalbit`. The answer contains the store's revision number, the server time and every row of the store; the application builds an in-memory database from it (the replica) and sets its clock to the server time. The router then runs the System Clock duties on the replica — completing the payroll history after a reset, `expireOverdue()` and `prepareMonthlyPayroll()` — draws the sign-in screen and saves what the duties changed with `app_commit`. From now on, every command is saved in the shared database before its result is shown, the revision is polled every 10 seconds and whenever the window regains focus, and a change made by someone else is drawn as soon as the user is idle (no sheet open, nothing typed, nothing being saved). Only the mode, the language, the

tour progress and the signed-in person are kept in the browser. If the database cannot be reached, the page states that the shared database could not be reached and offers the button *Open the local demonstration mode* (NFR-16).

4. **Local demonstration mode.** If the browser's `localStorage` holds a saved database under `shiftswap.db.v2` with the current schema, it is opened. Otherwise the schema of spec §8 is created, the demonstration data of spec §10 is inserted — including each worker's work history since the contract start and the confirmed payroll up to August 2026 — and the database is saved. A database saved by a version before iteration 7, under `shiftswap.db.v1`, is not used. The demo clock is read from `shiftswap.clock.v1` (default Monday 2026-09-28 09:00). The router runs the System Clock — `expireOverdue()`, then `prepareMonthlyPayroll()` — and renders the sign-in screen; from now on, every change is saved to `localStorage` after its transaction commits.
5. In both modes the language is read from `shiftswap.lang`; without a saved choice the interface is in Korean. The progress of the demo tour is kept under `shiftswap.tour.v1`, and an unfinished substitute request is kept in `sessionStorage`, so that a reload does not lose the flow. The sign-in screen doubles as a guided demo tour (Section 4.9) and states which mode is in use.

The fonts (Pretendard and JetBrains Mono) are loaded from public font services. If they cannot be reached, the browser uses its own fonts; the application works the same, only its appearance differs. The first load therefore needs a network connection for the application files. In the local demonstration mode later work happens entirely in the browser; in the shared mode every change is saved in the shared database, so the connection is needed throughout.

7.1.3 Resetting and running locally

Reset for everyone (shared mode). The *Demo tools* panel, opened from the clock chip in the header, contains *Reset the demo data for everyone*. Its confirmation states that the data return to their starting state for everyone and that all requests, approvals and attendance records made so far are erased. The function `app_reset` then replaces the data of the store with the seed template, whose dates are moved forward by whole weeks to the current date, so that the weekdays of the scenarios are preserved; every open client then reloads the data, and the application completes the payroll history on its first load after the reset. The server allows one reset a minute and 50 a day per store (NFR-14). Because the reset affects every visitor, it should be used before a demonstration rather than during someone else's use.

Local reset (local demonstration mode). In the local mode the panel contains *Reset demo data*. After a confirmation in the panel, it deletes the saved database, recreates the demonstration data, sets the demo clock back to 2026-09-28 09:00 and returns to the sign-in screen. This restores the exact starting state of the scenarios in Section 3.8 and of the user guide. Clearing the site data in the browser has the same effect in this mode; it does not change the shared data.

Local run. To run the application from a copy of the repository, serve the folder with any static web server and open the `app/` path, for example:

```
git clone https://github.com/keemlogan/shiftswap.git
cd shiftswap
python3 -m http.server 8000          # then open http://localhost:8000/app/
npm install && npm test              # optional: run the automated tests (Chapter 6)
```

A local copy uses `app/config.js` and therefore opens the same shared store as the deployed application; add `?mode=local` to the address (for example `http://localhost:8000/app/?mode=local`) to work only in the browser. If the address and key in `config.js` are empty, the application offers only the local mode.

Opening `app/index.html` directly from the file system does not work, because browsers do not load WebAssembly and ES modules from `file://` addresses; the application then shows an error message that asks the user to serve the folder over HTTP and reload the page.

7.1.4 Browser support

The application uses only standard features: ES modules, `fetch`, `localStorage`, `sessionStorage` and WebAssembly. It was tested in Google Chrome on desktop and in headless Chrome at 1280 px and 360 px width with `tools/check.mjs` in the local demonstration mode (Section 6.7), and in the shared mode with two headless Chrome instances on a throwaway store with `tools/shared-check.mjs`. NFR-06 also names Edge, Safari and Firefox; checks in these browsers had not been recorded when this chapter was written (Section 6.7). If a browser blocks storage, for example in some private-browsing modes, the application still runs: in the local mode the data are then kept only until the tab is closed, and in the shared mode the data are unaffected, but the mode, language and tour progress are not remembered.

7.1.5 The project hub and collaborative editing of the documents

The project page at the root of the site is the deliverables hub: it collects the deliverables of the project and shows each document in a preview. Since iteration 8, three documents — the proposal, the interim report and the final report — can be edited on the hub by the team, which used this function to review the documents together. The function consists of `hub/collab.js` with its settings in `hub/collab-config.js` and the database script `db/hub-collab.sql`; the versions are stored in the same Supabase project as the shared data of the application, in separate tables.

- **Editing and versions.** 편집 (Edit) makes the English text of the preview editable. 저장 (Save) asks for the editor's name (작업자명) and stores the text as a new version; no version is ever deleted. A version saved on the current version becomes the next current version (v2, v3, ...); a version saved on an older version, or after another member has saved in the meantime, is kept as a branch (v1.1, v1.1.1, ...), so that no work is overwritten. The change history (변경 이력) lists all versions with their labels, authors and times.
- **Viewing changes and restoring.** Viewing a version highlights what changed against its parent version, with the number of changes and a button to move to the next change. The highlights are not part of the document: editing, saving and printing use the text without them. Restoring an earlier version creates a new current version with the old content; if another member saved in the meantime, the restore is refused and the history is reloaded.
- **Korean view.** A switch EN | 한국어 shows a stored Korean translation of the displayed version, read-only. If a version has no stored translation yet, a browser that provides a built-in translation function can create one, which is then stored for everyone.
- **PDF 저장.** The button prints the document without highlights and with its own page margins, so that the browser adds no header or footer with date and address; tools/print-check.mjs checks this.
- **Safeguards.** Anyone may read the versions, and writing is possible only through three database functions, which keep the history append-only and assign the labels under a row lock. Because the hub has no accounts, saves and restores are limited to ten a minute and 100 a day per document, and the size of a version is capped per document: 300 kB for the proposal, 400 kB for the interim report and 1.5 MB for the final report, with 400 kB, 600 kB and 2 MB for a Korean translation. All stored and displayed content passes through an HTML sanitiser (DOMPurify). The hub checks the current version every 20 seconds and tells the editor when another member has saved. If the store cannot be reached, the hub shows the static copy of the document and disables editing.

The keep-alive workflow of Section 7.1.1 reads the current versions of these documents, which also keeps the shared data of the application active. The editing function has its own tests: twelve unit tests in tests/collab.test.js, which npm test runs together with the application's tests, and the browser check tools/collab-check.mjs.

7.2 User Guide

The guide follows the demonstration data through one complete substitution and then through the owner's other tasks. Dalbit Café has one owner, Park Jiyoung, and four part-time workers: Lee Seoyeon, Choi Minho, Jung Hana and Kang Doyun. The demo clock starts on Monday 2026-09-28 at 09:00, and Lee Seoyeon has already asked for a substitute for her Wednesday evening shift because of a midterm exam. The interface opens in Korean. Most screenshots are in English, which is selected in the header menu, and three show the Korean default. Desktop screenshots were taken at 1280 px width and phone screenshots at 390 px. The screenshots were taken in the local demonstration mode, whose data start from the same seed as the shared store after a reset. In the shared mode other visitors may have changed the data, and steps that move the clock (Sections 7.2.8 and 7.2.10) are available only in the local mode.

7.2.1 Signing in with the demo tour (all users)

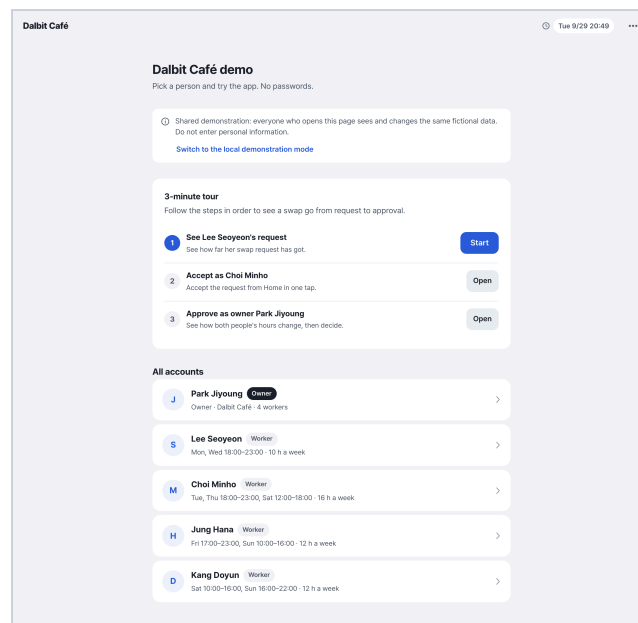


Figure 19. Sign-in as a demo guide (UC-13, FR-20) in the default shared mode: the notice "Shared demonstration: everyone who opens this page sees and changes the same fictional data. Do not enter personal information." with the text button *Switch to the local demonstration mode*; the header shows the server time; the three-step tour — see Lee Seoyeon's request, accept as Choi Minho, approve as Park Jiyoung — with *Start* as the only primary button, and all five accounts with their fixed shifts and weekly hours

The first screen is a guide to the demo. The *3-minute tour* card lists three numbered steps; each step's button signs in as the right person and opens the screen where the step happens, and completed steps are marked. Below it, *All accounts* lists every person with a one-line summary, for example "Tue, Thu 18:00–23:00, Sat 12:00–18:00 · 16 h a week" for Choi Minho. There are no passwords, so anyone can act as any person; the demonstration therefore contains only fictional data. Above the tour, a notice states the data mode: in the shared mode "Shared demonstration: everyone who opens this page sees and changes the same fictional data. Do not enter personal information.", in the local mode that the data are kept in this browser only; a text button below it switches to the other mode. Once signed in, the header shows the store name, the clock chip (the demo clock, or the server time in the shared mode), the bell with the number of unread notifications, the current person (select it to *View as someone else*) and the menu with the language switch and *Sign out*.

7.2.2 Worker Home: answering a request (candidate)

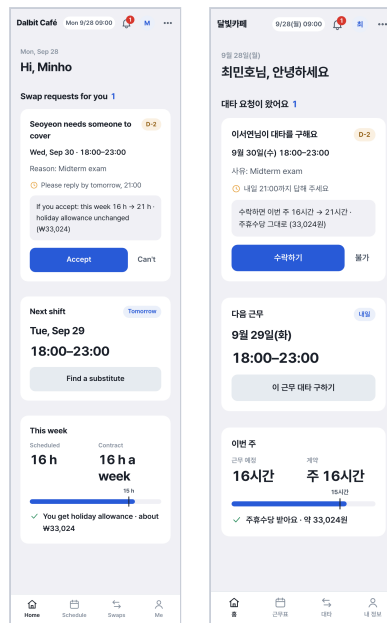


Figure 20. Worker Home (UC-05, NFR-13). (a) Choi Minho, English: the request card for Wed, Sep 30 · 18:00–23:00 with the reason, the deadline "Please reply by tomorrow, 21:00", the effect "If you accept: this week 16 h → 21 h · holiday allowance unchanged (₩33,024)", *Accept* and *Can't*; below, the next shift and this week's hours against the 15-hour line. (b) The same screen in the Korean default interface, which shows the seed people with their Korean names ("최민호님, 안녕하세요"; the request from 이서연님) and the buttons 수락하기 and 불가

A worker's Home lists what needs the worker now. For a candidate, the first card is the request: who asks, which shift, why, by when, and — before any button — what accepting would do to the candidate's own week. *Accept* takes one tap (NFR-01). A message then confirms that the shift is taken and that the owner will decide. If another candidate was faster, the card says so and closes. *Can't* is also one tap: it closes the card with a short confirmation and tells no one, unless it was the last answer missing and everyone has said they can't, in which case the request ends (Section 7.2.5). Further down, Home shows the next shift with *Find a substitute*, the week's scheduled and contract hours with the holiday allowance in words and money, and any past shifts still to record.

7.2.3 Requesting a substitute and following it (requester)

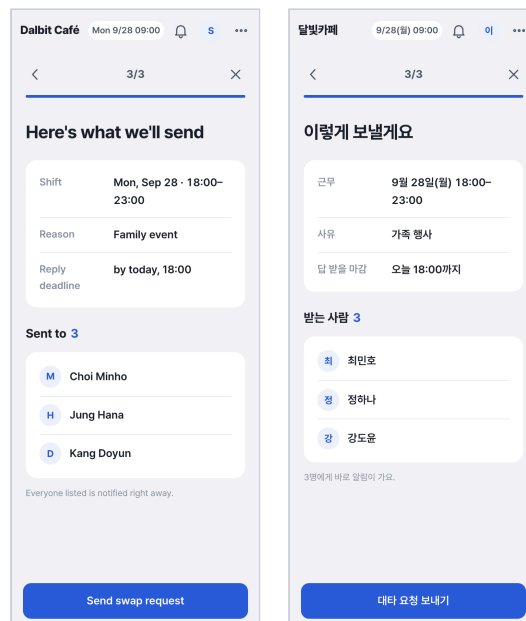


Figure 21. Request flow, step 3 of 3 (UC-04). (a) English: "Here's what we'll send" — shift Mon, Sep 28 · 18:00–23:00, reason "Family event", reply deadline "by today, 18:00", "Sent to 3": Choi Minho, Jung Hana and Kang Doyun, none of whom works on Monday evening, with the note "Everyone listed is notified right away", and the single button *Send swap request*. (b) The same step in Korean, with the recipients 최민호, 정하나 and 강도윤

Find a substitute — on Home, on the *Swaps* tab or in the detail of one's own shift — starts a flow of three steps with a step indicator, a back arrow and a close button. Step 1 asks which shift; a shift that already has an open request is shown as "Already looking for cover" and cannot be chosen (BR-03). Step 2 asks for a reason (chips such as *Exam* or *Family event*, or free text) and a reply deadline (*In 24 hours*, *2 hours before* or a chosen time), and repeats the deadline as a sentence; the default is the earlier of 24 hours from now and the shift start. Step 3 shows the request and the people who will receive it: every co-worker who is not already working at that time, computed at that moment (BR-01). If no co-worker is free, the step says so and explains that the owner will be told at once (FR-08).

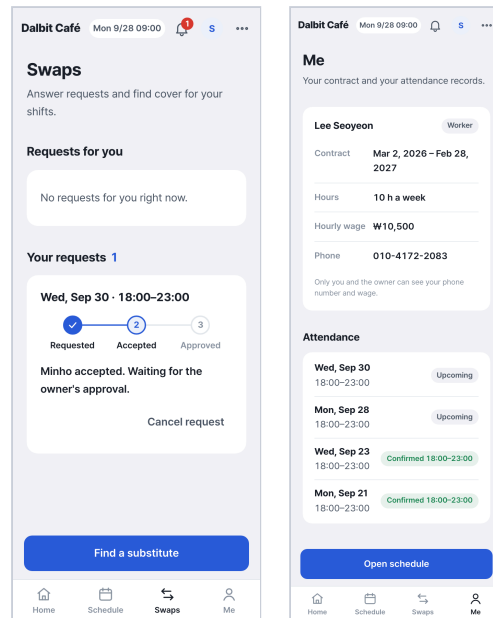


Figure 22. (a) *Swaps* tab of Lee Seoyeon (UC-12): the Wednesday request as a tracker Requested ✓ → Accepted → Approved, "Minho accepted. Waiting for the owner's approval.", *Cancel request*, and *Find a substitute* as the primary action. (b) *Me* tab (UC-07): her contract (Mar 2, 2026 – Feb 28, 2027), 10 h a week, hourly wage ₩10,500 and phone ("Only you and the owner can see your phone number and wage", NFR-11), and her attendance records — two upcoming shifts and two confirmed ones of the week of 21 September; *Open schedule* is the primary action. Since iteration 6 the tab has no availability editor

The *Swaps* tab lists requests received, the worker's own requests and past requests. Each open request is a tracker with three steps — Requested, Accepted, Approved — with each co-worker's answer so far (waiting, can't or accepted) and a sentence saying what happens next; rejected, expired, cancelled and failed requests end with a labelled state and the reason. *Cancel request* withdraws an open request, and everyone it was sent to is told. The *Me* tab holds the worker's own profile — contract, weekly contract hours, hourly wage and phone, which only the worker and the owner can see (NFR-11) — and the attendance list, in which a past shift is recorded with *Record*. Since iteration 6 it has no availability editor: workers are no longer asked to maintain the times they could take extra shifts.

7.2.4 Owner Home: approving a swap (owner)

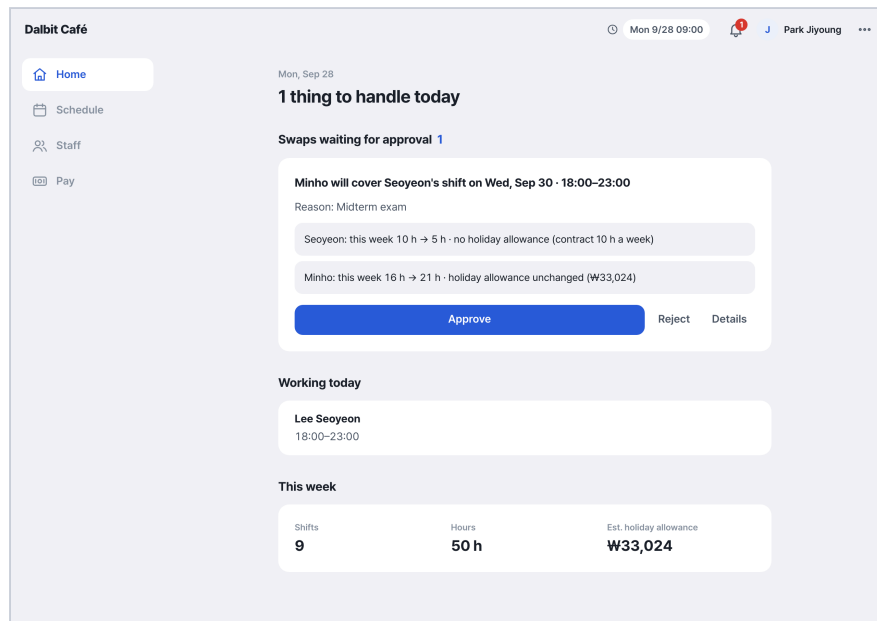


Figure 23. Owner Home (UC-06, FR-11): "1 thing to handle today"; the decision card "Minho will cover Seoyeon's shift on Wed, Sep 30 · 18:00-23:00" with the impact lines "Seoyeon: this week 10 h → 5 h · no holiday allowance (contract 10 h a week)" and "Minho: this week 16 h → 21 h · holiday allowance unchanged (₩33,024)", and *Approve*, *Reject*, *Details*; below, who works today and the week's shifts, hours and estimated holiday allowance

The owner's Home starts with the number of things to handle today and a decision card for each accepted swap. The card states the swap in one sentence and gives one line per person with scheduled hours before and after and the holiday allowance; if a worker's eligibility would change, the line is highlighted in amber. *Details* opens the full before/after comparison. *Approve* reassigns the shift, recomputes both workers' weekly summaries and informs both; *Reject* leaves the shift with the requester. In a store with five or more regular employees, when the acceptor is a part-time worker who would work more than 12 hours beyond the contract that week, the card also shows an amber notice, for example, in walkthrough S10 with five regular employees and an extra Friday shift, "Minho would work 13 h beyond contract this week. The limit for part-time workers is 12 h a week."; the notice informs the owner but does not disable *Approve* (FR-24, BR-15). Home also lists requests still waiting for replies, attendance records to confirm (*Confirm* or *Mark absent*), unacknowledged pay warnings, who works today, and the week's totals.

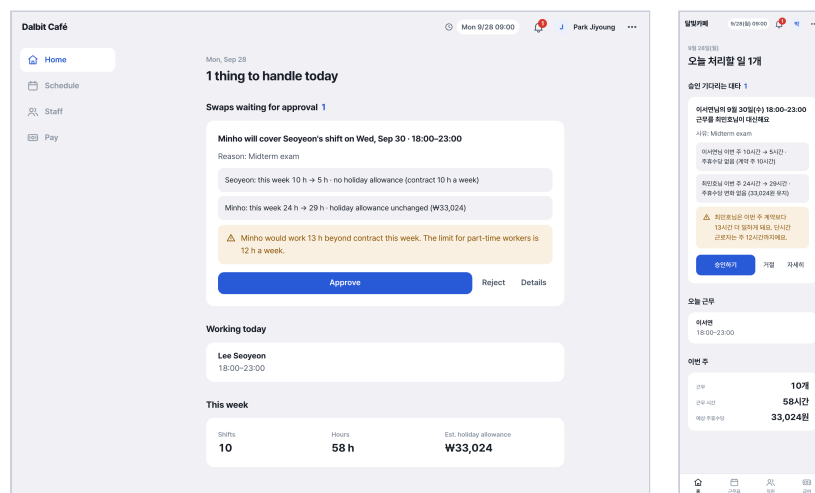


Figure 24. Walkthrough S10, the part-time overtime warning (FR-24, BR-15), local demonstration mode. (a) Owner Home, English, after the owner set five regular employees and added a shift for Minho on Fri, Oct 2, 10:00-18:00: "Minho: this week 24 h → 29 h · holiday allowance unchanged (₩33,024)" and the amber notice "Minho would work 13 h beyond contract this week. The limit for part-time workers is 12 h a week."; *Approve* remains the enabled primary button. (b) The same card in Korean at 360 px: "최민호님은 이번 주 계약보다 13시간 더 일하게 돼요. 단시간 근로자는 주 12시간까지만 일해요."

7.2.5 When nobody can take the shift (requester and owner)

Demonstration walkthrough S8 of the specification shows what happens when nobody can take a shift (BR-13, FR-22). Jung Hana requests a substitute for her Sunday shift, 4 October, 10:00–16:00. The request goes to Lee Seoyeon, Choi Minho and Kang Doyun; Doyun is asked because his own Sunday shift begins at 16:00, when Hana's ends. Each of the three signs in and taps *Can't* on the request card. The first two answers change nothing for anyone else, and Hana's tracker shows them as they arrive. With the third answer the request ends at once, without waiting for its deadline. Hana's *Swaps* tab then shows the card "Every co-worker said they can't cover this shift. Please contact the owner." The owner's Home shows a card "No one can cover" for Hana's Sunday shift, whose only action is *Got it*: there is nothing left for the system to try, and the owner settles the shift with Hana directly. Acknowledging removes the card. The same ending occurs immediately when a request is sent while no co-worker is free at that time.

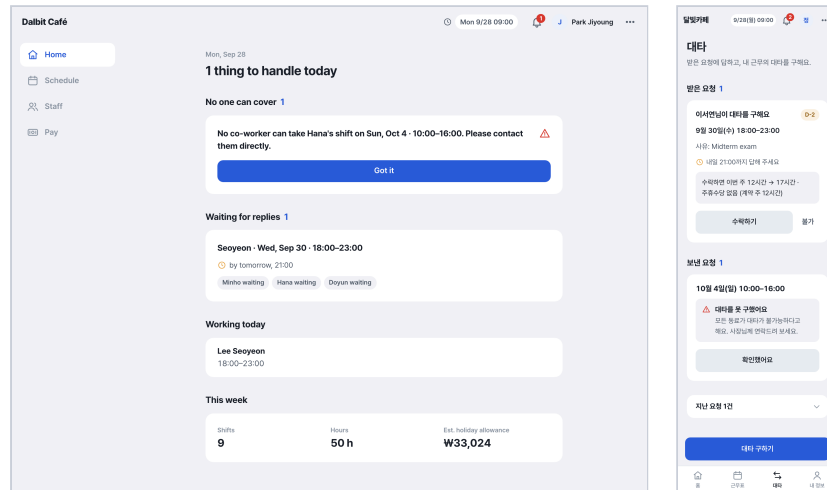


Figure 25. Walkthrough S8, nobody can take the shift (BR-13, FR-22). (a) Owner Home, English: the section "No one can cover" with the card "No co-worker can take Hana's shift on Sun, Oct 4 · 10:00–16:00. Please contact them directly." and *Got it* as its only action; below, Seoyeon's request still waiting for replies (Minho, Hana and Doyun waiting) and the week's totals. (b) Jung Hana's *Swaps* tab in Korean at 360 px: under 보낸 요청, the Sunday request 10월 4일(일) 10:00–16:00 with "대타를 못 구했어요 — 모든 동료가 대타가 불가능하다고 해요. 사장님께 연락드려 보세요." and 확인했어요; above it, Seoyeon's request that Hana herself can still answer

7.2.6 The schedule (all users)

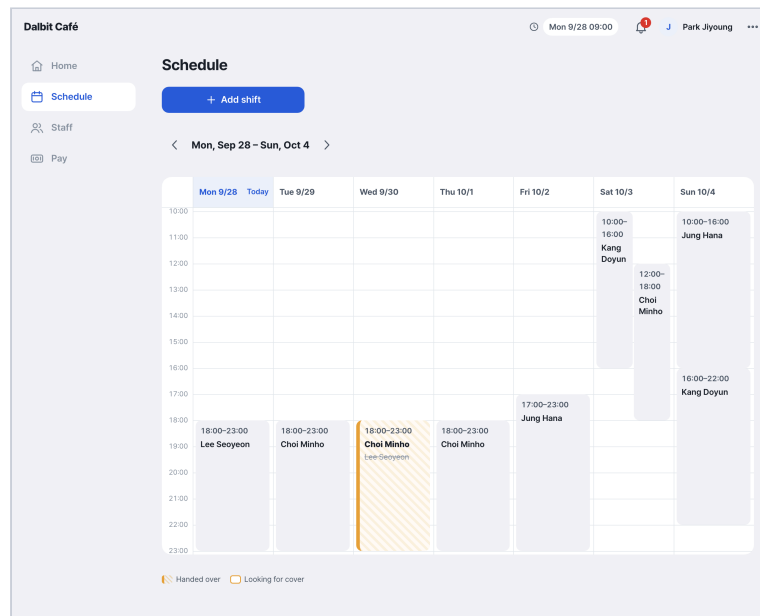


Figure 26. Schedule after the approval (UC-03, FR-12), owner view: the Wednesday 9/30 shift shows Choi Minho in bold and Lee Seoyeon struck through, with the amber left edge and stripes of a handed-over shift; *Add shift* is the primary action

On wide screens the schedule shows the week as a time grid: seven day columns, an hour axis from 10:00 to 23:00, today marked, and each shift a block placed by its time. A shift with an open request has a dashed amber outline and the chip

"Looking for cover"; a handed-over shift has an amber edge and stripes. A worker's own shifts are highlighted in blue, and the worker's primary action is *Find a substitute*. Selecting a shift opens its detail with its swap history and only the actions the viewer may take. For the owner these are *Change time*, *Change worker* and *Delete*; deleting is refused with its reason for a shift that has a request or an attendance record (FR-05). The owner's primary action is *Add shift*, or *Create this week's schedule* for a week without shifts (FR-04).

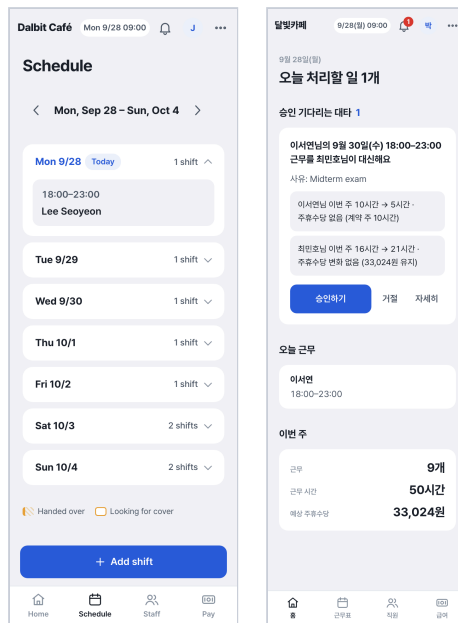


Figure 27. On a phone. (a) The owner's schedule at 360 px (NFR-02): days in order, today expanded and the other days collapsed with their shift count, *Add shift* fixed above the tab bar Home · Schedule · Staff · Pay. (b) The owner's Home in the Korean default interface, with the decision card, its approve button and the week's totals

7.2.7 Pay: weekly summary and monthly payroll (owner)

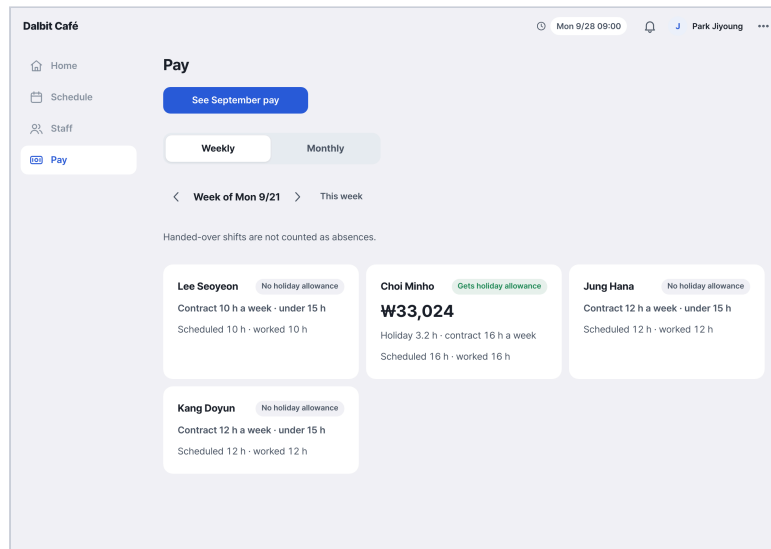


Figure 28. Pay → Weekly (UC-08), week of Mon 9/21: one card per worker; only Choi Minho "Gets holiday allowance", ₩33,024 (holiday 3.2 h · contract 16 h a week); the others "No holiday allowance" with the reason "Contract ... a week · under 15 h"; the line above the cards states that handed-over shifts are not counted as absences

The *Pay* tab has two segments. *Weekly* shows, for the chosen week, each worker's contract, scheduled and worked hours, whether the week earns the weekly holiday allowance, and the amount or the reason. For a week in progress the result is a projection, because scheduled shifts do not break attendance until they are marked absent.

Dalbit Café

Mon 9/28 09:00

J

Park Jiyoung

...

Home

Schedule

Staff

Pay

Pay

Weekly

Monthly

<

September 2026

>

Draft

Worker	Hours	Base pay	Holiday pay	Premium	Total	Status
Lee Seoyeon	45 h	₩472,500	₩0	₩0	₩472,500	Draft Estimated
Choi Minho	69 h	₩712,080	₩132,096	₩0	₩844,176	Draft Estimated
Jung Hana	48 h	₩495,360	₩0	₩0	₩495,360	Draft Probation minimum 90 %
Kang Doyun	48 h	₩480,000	₩0	₩0	₩480,000	Draft Below minimum wage
Doyun's hourly wage of ₩10,000 is below the 2026 minimum wage of ₩10,320						Acknowledge
All workers					₩2,292,036	

Estimated: shifts without confirmed attendance are counted with their scheduled hours.

Premium pay applies only with 5 or more regular employees (now 4).

Rebuild draft

Figure 29. Pay → Monthly (UC-09), September 2026 draft created on 28 September, with the chip "Draft" next to the month: Lee Seoyeon 45 h, ₩472,500 and Choi Minho 69 h, ₩712,080 + ₩132,096 holiday pay = ₩844,176 (both "Estimated"); Jung Hana 48 h, ₩495,360 with "Probation minimum 90 %"; Kang Doyun 48 h, ₩480,000, "Below minimum wage" with the sentence "Doyun's hourly wage of ₩10,000 is below the 2026 minimum wage of ₩10,320" and *Acknowledge*; all workers ₩2,292,036; *Confirm September pay* disabled, with its reason under it

Monthly shows the payroll of a month. The arrows < > move between months, back to the first contract month, March 2026, and a chip next to the month shows its state: "Confirmed" (확정됨) or "Draft" (초안). Without a draft, the primary action is *Create September pay draft*, with the month named in the button. Each row has hours, base pay, holiday pay, premium and total, with flags: "Estimated" when some shifts have no confirmed attendance and are counted with their scheduled hours (BR-11), "Probation minimum 90 %" when all probation conditions hold (BR-10), and "Below minimum wage" with an explanation (BR-09). *Confirm September pay* stays disabled, with the reason written under it, until every warning has been acknowledged with *Acknowledge*; the acknowledgement is saved. A confirmed month is read-only. *Rebuild draft* replaces only draft rows, and warnings must then be acknowledged again. The note under the table explains that premium pay applies only with five or more regular employees.

Dalbit Café

Mon 9/28 09:00

J

Park Jiyoung

...

Home

Schedule

Staff

Pay

Pay

Weekly

Monthly

<

August 2026

>

Confirmed

August pay is confirmed and can no longer be changed.

Worker	Hours	Base pay	Holiday pay	Premium	Total	Status
Lee Seoyeon	45 h	₩472,500	₩0	₩0	₩472,500	Confirmed
Choi Minho	70 h	₩722,400	₩165,120	₩0	₩887,520	Confirmed
Kang Doyun	36 h	₩360,000	₩0	₩0	₩360,000	Confirmed Below minimum wage
Doyun's hourly wage of ₩10,000 is below the 2026 minimum wage of ₩10,320						Acknowledged
All workers					₩1,720,020	

Premium pay applies only with 5 or more regular employees (now 4).

Figure 30. Pay → Monthly for a month of the seeded history (UC-09, iteration 7): August 2026 with the chip "Confirmed" and the sentence "August pay is confirmed and can no longer be changed"; Lee Seoyeon 45 h, ₩472,500; Choi Minho 70 h, ₩722,400 + ₩165,120 holiday pay (five weeks with their Sunday in August) = ₩887,520; Kang Doyun 36 h, ₩360,000, "Below minimum wage", marked "Acknowledged"; all workers ₩1,720,020; the primary button reads "August pay confirmed" and is disabled

The months before September come from the seeded work history and are confirmed, so they are read-only: the button at the top says that the month is confirmed, and the rows cannot be changed. Jung Hana has no row in August, because her contract starts on 1 September.

7.2.8 Last month's payroll prepared automatically (owner)

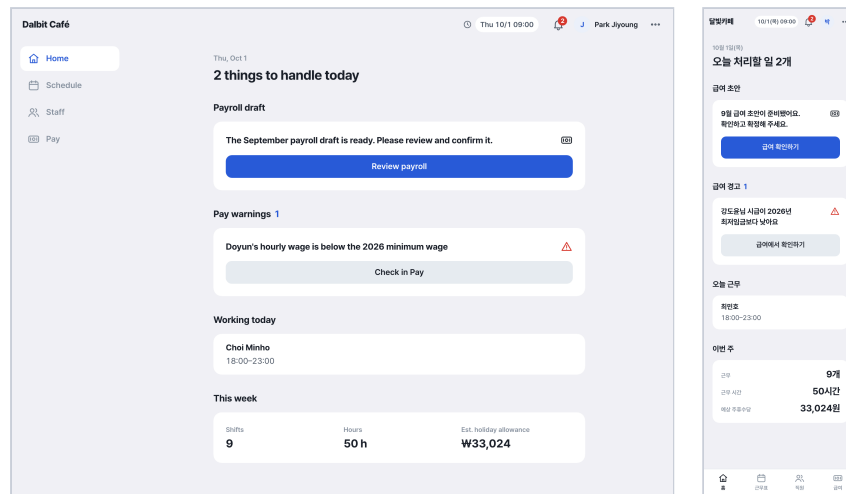


Figure 31. Walkthrough S9, the automatic payroll draft (UC-14, FR-23, BR-14). (a) Owner Home, English, with the demo clock at Thu 10/1 09:00: "2 things to handle today"; the section "Payroll draft" with the card "The September payroll draft is ready. Please review and confirm it." and the primary button *Review payroll*; below, "Pay warnings" with Doyun's below-minimum wage and *Check in Pay*. (b) The same screen in Korean at 360 px: "9월 급여 초안이 준비됐어요. 확인하고 확정해 주세요." with *급여 확인하기*, and the warning card for 강도윤님

On or after the first day of a month, the System Clock prepares the previous month's payroll draft by itself (walkthrough S9 of the specification). To see it in the local demonstration mode, open *Demo tools* and set the clock to 1 October 2026; in the shared mode the draft is prepared when the server time reaches the first day of a month. The owner's Home then shows the card that the September payroll draft is ready; *Review payroll* opens *Pay* → *Monthly* for September and marks the notification as read. The draft is the same as one created by hand, and it carries the note "This draft was made automatically on the 1st of the month." (매월 1일에 자동으로 만든 초안이에요). The system never confirms a payroll: Doyun's below-minimum warning must still be acknowledged, and the owner confirms the month as described in Section 7.2.7. Moving the clock further within October creates no second draft and no second notification. If the owner has already created or confirmed the September payroll before 1 October, nothing is prepared.

7.2.9 Staff and settings (owner)

Dalbit Café Mon 9/28 09:00 Park Jiyong

Home Schedule **Staff** Pay

< All staff

Choi Minho

Save

Details

Name: Choi Minho Phone: 010-5290-3316

Hourly wage (won): 10320 Contract start: 06/01/2026

Contract end: 12/31/2026 Probation end: mm/dd/yyyy

Simple labour: ☐ Currently employed: ☒

Fixed shifts

Pick the days and set the times. The weekly schedule is built from these.

Mon	Tue	Wed	Thu	Fri	Sat	Sun
	18:00 - 23:00					
	18:00 - 23:00					
	12:00 - 18:00					

Contract of 16 h a week.

Figure 32. Staff → worker detail (UC-01): Choi Minho's name, phone, hourly wage, contract start and end, optional probation end, the *Simple labour* and *Currently employed* switches, and the fixed-shift editor with the weekday chips Tue, Thu and Sat and the result "Contract of 16 h a week"; *Save* is the primary action

Staff lists the workers with contract hours, wage and contract period; *Add worker* is the primary action. A worker's detail holds the personal and contract data and the fixed weekly shifts, edited by choosing weekdays and times. The editor shows the resulting weekly contract hours, which decide the holiday allowance. Leave the contract end empty for an open-ended contract; the probation end is optional and is used only for the 90 % minimum-wage rule; *Simple labour* marks work that the law excludes from that reduction. Overlapping fixed shifts are refused with a message naming the conflict.

Dalbit Café Mon 9/28 09:00 Park Jiyong

Home Schedule Staff Pay

Settings

Store details and the rules pay is calculated with.

Save

Store name: Dalbit Café

Regular employees: 4

With 5 or more, overtime and night premiums apply. Now 4.

Handed-over shifts

☒ Do not count as absence
The day was changed by agreement, so perfect attendance is kept. (Default)

☐ Count as absence
A week with a handed-over shift loses perfect attendance and its holiday allowance.

Minimum wage by year

Year	Hourly (won)
2025	10030
2026	10320

+ Add row

Figure 33. Settings (UC-10), from the header menu: store name, 4 regular employees ("With 5 or more, overtime and night premiums apply"), the handed-over shift policy as two cards — "Do not count as absence" (default) and "Count as absence" — each with its effect, and the minimum wage by year (2025: 10,030 won; 2026: 10,320 won)

Settings holds the values that the pay rules depend on: the number of regular employees, which switches premium pay on from five; the treatment of shifts handed over by an approved swap; and the minimum wage by year. When a new minimum wage is announced, add a row for the new year; the program does not need to change.

7.2.10 Demo tools

The clock chip in the header ("Mon 9/28 09:00") opens *Demo tools*, a pop-over on desktops and a sheet on phones. Its content depends on the mode. In the local demonstration mode it contains the current demo time with *Set this time*, *+1 hour* and *+1 day*, *Reset demo data*, which asks for confirmation inside the panel, and *Switch to the shared demonstration mode*. Moving the clock past a request's deadline or its shift start makes the request expire the next time any screen is shown (UC-11), and the requester and the owner are told. In the shared mode the panel states that the demonstration is shared and that the time is the server time in Korea, which cannot be changed; it shows the server time, *Reset the demo data for everyone* with its confirmation (Section 7.1.3), and *Switch to the local demonstration mode*. Requests then expire on time by the server clock, checked every second while the application is open.

While a change is being saved in the shared mode, the screen is marked as busy and further commands wait. If the server refuses a change or another person changed the same data at the same moment, the application reloads the shared data and repeats the command, so the user normally sees the ordinary result, such as "Minho was faster"; the messages that remain possible are listed in Section 7.4.

7.3 Demonstration Script

The script below runs the core workflow in about five minutes, following the tour on the sign-in screen (done criterion 4 of the final prompt). Start from the seed data: in the shared mode with *Reset the demo data for everyone* shortly before the demonstration, or, for a guided tour that must not be disturbed by other visitors, in the local demonstration mode (?mode=local) with *Reset demo data*. Steps 7 and 9 move the demo clock and therefore need the local mode; step 10 needs the shared mode.

Table 78. Live demonstration script

#	Person	Action	Expected result
1	Lee Seoyeon	Tour step 1 (<i>Start</i>)	<i>Swaps</i> shows the Wednesday request as a tracker at "Requested"
2	Choi Minho	Tour step 2; <i>Accept</i> on the Home card	The card showed 16 h → 21 h, holiday allowance unchanged; a message confirms the shift is taken
3	Kang Doyun	Open Home (<i>View as someone else</i>)	The card says that Minho was faster
4	Park Jiyoung	Tour step 3; <i>Approve</i> on the decision card	The card showed Seoyeon 10 → 5 h and Minho 16 → 21 h, ₩33,024 unchanged; approval confirmed
5	Park Jiyoung	Open <i>Schedule</i>	Wednesday 9/30 shows the handover: Choi Minho in bold, Lee Seoyeon struck through
6	Park Jiyoung	<i>Pay</i> → <i>Monthly</i> , September 2026, create the draft	Minho's holiday pay ₩33,024; warning on Kang Doyun's row; confirmation disabled until acknowledged
7 (optional, local)	Any	Create a request, then <i>+1 day</i> in Demo tools	The request expires; requester and owner are notified (UC-11)
8 (optional)	Jung Hana, then Lee Seoyeon, Choi Minho, Kang Doyun, then Park Jiyoung	Hana requests a substitute for Sun 10/4 10:00–16:00; each of the three taps <i>Can't</i> ; the owner opens Home (walkthrough S8, done criterion 5)	After the third answer the request has failed: Hana sees "Please contact the owner"; the owner's card "No one can cover" disappears after <i>Got it</i>
9 (optional, local)	Park Jiyoung	<i>Pay</i> → <i>Monthly</i> , page back with <; then <i>Demo tools</i> , set the clock to 2026-10-01 and open Home (walkthrough S9, done criterion 6)	March to August are "Confirmed" and read-only; on 1 October Home shows that the September payroll draft is ready, and <i>Review payroll</i> opens the draft with Doyun's warning to acknowledge
10 (optional, shared)	Choi Minho and Kang Doyun on two devices, then Park Jiyoung	After a reset for everyone, both tap <i>Accept</i> on the Wednesday request at the same moment; the owner approves on one device (done criterion 8)	Exactly one acceptance succeeds and the other device says who was faster; the approval appears on the other devices within about 10 seconds of polling

7.4 Troubleshooting

Table 79. Common problems and solutions

Problem	Cause and solution
An error message asks to serve the folder over HTTP	The page was opened from the file system. Use the deployed URL or a local web server (Section 7.1.3).
The data differs from this guide	In the shared mode other visitors have changed the shared store; in the local mode earlier changes are saved in the browser. Open Demo tools and select <i>Reset the demo data for everyone</i> or <i>Reset demo data</i> , or switch to the local mode for an undisturbed tour.
A notice says that the shared database could not be reached	The device is offline, or the database is unavailable, for example because the free project has been paused [16]. Select <i>Open the local demonstration mode</i> to continue with the seed data in the browser, or reload when the connection is back; a paused project is resumed by the team from the dashboard.
"The shared database cannot be reached, so the change was not saved"	The connection was lost while saving. The screen returns to the last saved state; repeat the action when the device is online.
"Other people are changing the same data right now"	The change collided with changes of other people three times in a row. Wait a moment and repeat the action.
"The shared database did not accept this change"	A rule of the server refused the change, usually because the data had changed in the meantime. The latest data have been loaded; check the screen and repeat the action if it is still possible.
A message about 60 changes a minute, 3,000 a day, 20,000 rows, or the reset limits	The limits of the shared store (NFR-14). Wait a minute, continue in the local demonstration mode, or, for a full store, reset the demo data for everyone. Only saved changes count towards the limits.
The clock cannot be set	The shared mode uses the server time. Switch to the local demonstration mode in Demo tools.
The interface is in Korean	Korean is the default. Choose English in the header menu; the choice is remembered.
A request card no longer has <i>Accept</i>	Another candidate accepted first, or the request expired because the demo clock passed its deadline or the shift start. The card or the notification states which.
A shift cannot be chosen in step 1 of the request flow	It already has an open request, has started, or is not the worker's own (BR-03).
A shift cannot be recorded in <i>Me</i>	It has not started yet according to the clock. In the local mode, move the clock forward in Demo tools.
<i>Confirm September pay</i> is disabled	The reason is written under the button: a minimum-wage warning must be acknowledged first, or the wage corrected in <i>Staff</i> and the draft rebuilt.
A document on the project hub cannot be edited	The version store cannot be reached, so the hub shows the static copy (Section 7.1.5). Reload later; saving is also refused above ten saves a minute or 100 a day per document.

8. Conclusion

This report has described ShiftSwap, a web application that replaces the telephone-and-spreadsheet handling of substitute shifts in a small part-time workplace. This chapter assesses the result against the objectives of Chapter 1, summarises the size of the system, states its limitations, proposes future work, and records what the team learned about requirements engineering and vibe coding.

8.1 Achievement of the Objectives

The table below assesses each objective G1–G6 against its measure. Only what can be verified from the tests of Chapter 6, the code or the deployed site is claimed. The objectives concern the behaviour of the system; whether that behaviour reduces the owner's work in a real store has not been measured, because no field trial took place (Section 8.3).

Table 80. Achievement of objectives G1–G6

ID	Objective	Evidence	Assessment
G1	Remove the owner's one-to-one contacts	<code>createSubRequest</code> notifies every eligible worker in the transaction that creates the request (TC-024, TC-016); when no co-worker can be asked or every one of them answers that they can't, the request ends at once and the requester and the owner are told (TC-025, TC-13A). The owner's only action in the workflow is one approve or reject decision (TC-061, TC-062).	Achieved in the system; effect in a store not measured
G2	Let a candidate answer quickly	A pending request appears as a card on the candidate's Home screen with <i>Accept</i> on the card, so accepting takes one tap after opening the application (NFR-01, NFR-13). Only the first acceptance succeeds (TC-051), also between two browsers on the shared database, where <code>tools/shared-check.mjs</code> lets two candidates accept at the same moment (Section 5.4.8).	Achieved
G3	Keep schedule and weekly hours of both workers correct after a swap	Approval reassigns the shift, keeps the original worker and recomputes both weekly summaries in one transaction (TC-061). The approval panel shows before and after values for both workers — Choi Minh 16 → 21 scheduled hours, eligibility unchanged (TC-063) — and highlights a change of eligibility (TC-08B).	Achieved
G4	Apply the legal pay rules and flag violations before payday	BR-06 to BR-11 are pure functions tested at their boundary values (Section 6.5), and every business rule BR-01 to BR-15 has at least one test (Section 6.8); the part-time premium of BR-08 (iv) and the approval warning of BR-15 are tested with TC-096 to TC-09C and TC-151 to TC-155. The payroll warns for Kang Doyun's wage and blocks confirmation until the owner acknowledges it (TC-104, TC-105).	Achieved
G5	Deliver a system of the size the course requires	13 use cases (8 core) against a minimum of 5; 11 database tables against a minimum of 4, held in both data modes (in the shared database as 11 <code>app_</code> tables plus two service tables).	Achieved
G6	Deliver on time and reproducibly	The application is deployed at https://keemlogan.github.io/shiftswap/app/ (NFR-07). The final prompt and the specification are in the repository (Chapter 5). In reproduction run 4, a new assistant session given only the final prompt (version 7), the specification and the two test files in an empty folder built a system that passes all 89 acceptance tests unchanged and meets done criteria 1–7 (Section 5.6.2); the run with the final prompt, version 8, passed all 113 acceptance tests unchanged and met criteria 1–7, and the team checked criterion 8 at the data level. Limits: one run per prompt version, and the interface was compared by the done criteria, not pixel by pixel.	Achieved

Beyond the six objectives, the revised proposal asked for a shared database, and iteration 8 delivered it: in the default shared mode the owner and the workers use one demonstration store from their own devices, the server re-checks the state rules and the time of every change, and a change made on one device appears on the others within about ten seconds (Sections 4.5.2 and 5.4.8). The browser-only version remains available as the local demonstration mode. The shared mode is verified by 64 of 64 REST checks against a throwaway store and 12 of 12 checks with two headless browsers.

8.2 Size and Complexity of the System

The table below summarises the size of the specification, the models and the implementation. The counts of identifiers come from `spec/spec.md`; the line counts come from `wc -l` and exclude the vendored `sql.js` files.

Table 81. Size of the ShiftSwap specification, models and implementation

Item	Count	Remark
Actors	3	Worker, Owner, System Clock
Use cases	13	8 core (UC-01, UC-03–UC-09), 5 supporting; 9 Must, 4 Should; UC-02 retired in iteration 6, UC-14 added in iteration 7
Functional requirements	23	FR-01, FR-02, FR-04–FR-24; FR-03 retired in iteration 6, FR-24 added in iteration 8
Non-functional requirements	16	10 product, 3 organisational, 3 external; NFR-14 to NFR-16 added in iteration 8
Business rules	15	6 of them derived from labour law (BR-06–BR-10, BR-15)
Database tables	11	SQLite schema of spec §8; Availability removed in iteration 6. The shared database holds them as 11 <code>app_ tables</code> plus <code>app_store</code> and <code>app_store_gate</code> , with 7 row triggers and 3 functions
State machines / states	5 / 18	SubRequest (7), SubRequestTarget (4), Shift (3), Attendance (2), Payroll (2)
Notification kinds	9	With a fixed recipients table (spec §8)
UML diagrams of the system	16	2 use-case, 1 class, 1 ER, 1 component, 1 deployment, 5 sequence (SD-1, SD-2, SD-3a, SD-3b, SD-4), 3 state, 2 activity
Screens	11	Sign-in and ten working screens (Section 4.9)
Automated tests	125	36 rule tests, 65 service tests, 12 shared-mode tests and 12 tests of the project hub's editing feature; all pass. In addition, 64 REST checks of the shared database and 12 two-browser checks of the shared mode
Lines of application code	6,298	Core modules 2,054 (services 1155, rules 320, db 280, shared-data gateway 243, clock 56); presentation 4,244 (router, 15 UI modules including translations and shared components, page skeleton, style sheet and configuration)
Lines of test code	1,628	<code>tests/rules.test.js</code> 336, <code>tests/services.test.js</code> 1031, <code>tests/shared.test.js</code> 261 (without the hub's <code>tests/collab.test.js</code> , 160)
Lines of server SQL	1,116	<code>db/app-shared.sql</code> : tables, triggers, functions, grants and the seed template of the shared database

Line counts were taken with `wc -l` on the final build (prompt version 8).

By the measures the course names — use cases and tables — the system is well above the minimum. Its complexity lies less in the number of screens than in the interaction of rules: a single approval changes a shift, two weekly summaries, possibly two holiday allowances and later a payroll, and each of these depends on settings that the owner can change. The 18 states and the 15 business rules are where most of the analysis and test effort went. Iteration 8 added a second kind of complexity: the same rules must hold when several devices change one store at the same time, which the design meets with revisions, re-runs and row triggers on the server rather than with more screens.

8.3 Limitations

The system has the following limitations. Most follow from the decision to build a demonstration system on static hosting and a free hosted database without accounts (design decisions D1 and D13); they are listed so that the reader can judge what the results above do and do not show.

- **No authentication; identity is not enforced.** Sign-in is the choice of a demo account without a password (FR-20). In the shared mode anyone who opens the application can act as any person, including the owner, and change the shared store. The row triggers enforce the state rules, the server time and the limits, but not who makes a change, and every row of the store, including phone numbers and wages, can be read with the publishable key. The role restriction and the privacy rule NFR-11 therefore protect against accidental, not deliberate, disclosure, and the demonstration contains only fictional data.
- **Whole-store replica.** Each client loads a snapshot of the entire store when it starts and again after every change made elsewhere, and every commit is checked against one revision number per store. Two unrelated changes made at the same moment therefore also conflict, and the second is re-run; after three conflicts the user is asked to try again. This suits a store with a few people and a demonstration of a few hundred rows, but the cost of a reload grows with the history, and the store is capped at 20,000 rows. Changes by others are seen by polling every ten seconds, not pushed.
- **Rate limits count only successful commits.** The limits of 60 changes a minute and 3,000 a day per store are counted in the same transaction as the change, so a refused or failed attempt is not counted. The limits bound the data that can be written, not the number of requests that reach the server.
- **Free plan and keep-alive.** On the free plan, a project that shows low activity over seven days is paused until it is resumed manually [16]. The six-hourly keep-alive workflow prevents this, but GitHub disables scheduled workflows in a public repository after 60 days without repository activity [17]. If the database is paused or unreachable, the application offers only the local demonstration mode until the team resumes the project.
- **Browser storage in the local mode.** In the local demonstration mode the data are kept in `localStorage`, which the user or the browser can clear and which is limited to a few megabytes per site. Neither mode provides backups or export.
- **Clock-driven work needs an open client.** In the shared mode requests expire and the monthly payroll draft is prepared by the server time, but the work is performed and saved by whichever client has the application open; if nobody opens it, nothing expires until someone does. In the local mode expiry follows the demo clock and is triggered by navigation.
- **Legal scope.** The pay rules cover the minimum wage, the weekly holiday allowance, overtime and night premiums, the premium for part-time hours beyond the contract and the probation reduction. The part-time worker's consent to extra hours is not recorded, and the 12-hour limit of BR-15 is a warning to the owner, not a block. Holiday-work premium, annual leave, severance pay, tax and social insurance are outside the scope, and the four-week average of the 15-hour rule is simplified to the weekly contractual value (BR-06). The treatment of a handed-over shift is a setting because the law does not decide it. The system applies these rules consistently but is not legal advice.
- **Evaluation.** The requirements came from one reference store, and the interviews and prototype sessions were used to confirm requirements, not to measure them. No store has used the system in operation, so the effect on the number of contacts per request (problem P1) has not been measured.
- **Test and reproduction gaps.** The browser checks of NFR-06 are recorded only for Chrome, and the shared mode was checked with two headless browsers on one computer rather than with several phones on mobile networks. The reproducibility of each prompt version rests on one run (Section 5.6.2), and a generative assistant does not produce the same code twice.

8.4 Future Work

Earlier versions of this section named a shared database as the most important extension; iteration 8 implemented it (Section 5.4.8). The layered design made this a change of the data-access layer: the rules and the services run unchanged on a replica, and a gateway saves their changes in the shared database. The following would be the next increments:

1. authentication with real accounts and row ownership on the server, so that the owner and each worker can change only their own data, and NFR-11 is enforced by the database rather than by the interface;
2. updates pushed by the database to the open clients instead of polling every ten seconds, and loading only the changed rows instead of the whole store;
3. push or messenger notifications for new requests, in addition to the in-app inbox;
4. expiry and the monthly payroll draft driven by a scheduled job on the server instead of an open client;
5. a field trial in the reference store, measuring the number of contacts per request and the time to fill a shift before and after, which is the measurement G1 ultimately stands for;
6. recorded checks in Edge, Safari and Firefox and on real phones, and repeated reproduction runs of the final prompt;
7. recording the part-time worker's consent to extra hours, export of confirmed payroll as a payslip, and several stores per owner, for which the shared database already separates the data by store.

8.5 Lessons Learned

The specification is the program's real source. In vibe coding, the AI coding assistant writes the code, and the quality of the code is bounded by the precision of what it is told. The team's most useful decision was to keep one specification file with every identifier, rule, state and seed value, and to change it before changing anything else. When a question arose — who is notified when a request is cancelled, whether a week in progress can be "perfect" — the answer was written into the specification, and the next generated version had to follow it. This is requirements management as the lecture describes it [7]: individual requirements are tracked, their dependencies are recorded, and changes go through a defined procedure. Without it, three documents, a set of UML diagrams and the code would have drifted apart within a week.

Generation exposes incompleteness. The lecture states that in practice a complete and consistent requirements document is impossible [7]. The build iterations confirmed this concretely: in iteration 1 the AI coding assistant met more than twenty points where the specification was silent or ambiguous and had to choose. Recording each choice and turning it into specification text (Section 6.9) worked as a requirements review performed by the implementer. The lesson for vibe coding is that the log of the generator's decisions is as valuable as the code, because it shows where the requirements were weaker than the team believed.

Testable rules make generated code checkable. Code produced by an AI coding assistant cannot be trusted because it looks right. Placing every business rule in a pure function and deriving tests from the rules at their boundary values (14.99 and 15 hours, four and five employees, the last day of probation) gave the team an objective check that did not depend on reading every line. When the specification changed, the tests changed first. This is the test-first practice of extreme programming [6] applied to a prompt instead of to hand-written code.

Models find what text hides. The state diagram of the substitute request made it easy to see that every open state needs an exit by time and by the requester, and the traceability matrix showed which requirements had no test. The demonstration walkthrough S7 was added because the team noticed that no scenario ever changed a worker's eligibility, so the highlight required by FR-11 had never been exercised. Chapter 2 of the lecture presents models as a means of discussion, documentation and implementation [6]; in this project they served mostly as a means of checking.

The incremental model suited the course. Placing all Must use cases in the first increment meant that the substitute workflow, the core of the project, was working and testable before the payroll rules were built, as the lecture's description of the incremental model promises [6]. The weakness the lecture warns of, degradation of structure as increments are added, was contained by fixing the layered architecture before the first increment; the second increment added functions to the existing layers and did not change their boundaries.

Names are not interfaces. Reproduction run 1 rebuilt a system that behaved as specified but passed only 13 of the team's 25 rule tests, because the prompt fixed the names of functions but not their signatures and return shapes. Supplying the acceptance tests as part of the input closed this gap: run 2 passed all 77 supplied tests unchanged, and runs 3 and 4, after the requirements changes of iterations 6 and 7, all 84 and all 89. For vibe coding, the lesson is that the executable tests belong to the contract, not only to the verification of one build.

Retire requirements; do not reuse their numbers. The change request of iteration 6 removed a use case, a functional requirement and a table after the product owner had used the running system. Because the traceability links were part of the specification, the impact analysis was a search for identifiers, and the change reached the code, the tests and this report within one iteration. Keeping the retired numbers FR-03 and UC-02 unused preserved the meaning of every earlier record that mentions them.

A shared server is part of the contract. Iteration 8 showed that the rules that matter most once several devices share one store — who may change which row, and which of two simultaneous acceptances wins — cannot be left to generated client code, because any browser can be modified. The team therefore wrote the server side once and supplied it with the prompt, together with a test file for the protocol between client and server, and let the unchanged services run on a replica; as a result, the 89 tests of iteration 7 passed unchanged in the new architecture.

Documents need the same discipline as code. Because code can be regenerated in hours, the documents became the slower part of the project, and they drifted more easily than the code. Several corrections in this report — the recipients of a cancelled request, the storage of the minimum-wage acknowledgement, the course calendar — were found by checking documents against the specification and the tests. A team using vibe coding should plan as much time for keeping the requirements, models and report consistent as for generating the software.

ShiftSwap shows that a small team can take a real problem from a part-time job through problem definition, requirements, design, implementation by vibe coding and testing within one semester, and arrive at a system whose behaviour is specified, traceable and tested. What remains is to enforce identity on the shared server and to measure the system's effect in a real store.

References

1. Ministry of Employment and Labor, "2026 minimum wage KRW 10,320 per hour," MOEL notice, Aug. 2025. [Online]. Available: https://www.moel.go.kr/news/enews/report/enewsView.do?news_seq=18144
2. Korea Policy Briefing, "2026년 시간당 최저임금 10,320원" (2026 hourly minimum wage KRW 10,320). [Online]. Available: <https://www.korea.kr/news/policyNewsView.do?newsId=148956812>
3. Labor Standards Act (근로기준법), Art. 11, 18, 55, 56; Enforcement Decree of the Labor Standards Act, Art. 30. National Law Information Center. [Online]. Available: <https://www.law.go.kr/법령/근로기준법>
4. Minimum Wage Act (최저임금법), Art. 5 ②; Enforcement Decree of the Minimum Wage Act, Art. 3. National Law Information Center. [Online]. Available: <https://www.law.go.kr/법령/최저임금법>
5. Ministry of Government Legislation, Easy-Law (찾기 쉬운 생활법령정보), "기간제 및 단시간근로자: 「근로기준법」 적용의 특례" (fixed-term and part-time workers: special provisions on the application of the Labor Standards Act; paid holidays of part-time workers and the 15-hour threshold). [Online]. Available: <https://www.easylaw.go.kr/CSP/CnpClsMain.laf?csmSeq=1347&ccfNo=3&cciNo=2&cnpClsNo=1>
6. D. A. Gustafson, *Schaum's Outline of Software Engineering*. New York: McGraw-Hill, 2002 (course textbook; Ch. 1 software life cycle, Ch. 2 software process and other models, Ch. 8 requirements).
7. I. Sommerville, *Software Engineering*, 10th ed. Boston: Pearson, 2016 (requirements engineering and the classification of non-functional requirements used in lecture Chapter 8).
8. Object Management Group, *OMG Unified Modeling Language (OMG UML)*, Version 2.5.1, Dec. 2017. [Online]. Available: <https://www.omg.org/spec/UML/2.5.1>
9. sql.js contributors, "sql.js: SQLite compiled to JavaScript/WebAssembly," version 1.14.2. [Online]. Available: <https://sql.js.org>
10. Deputy, "Allow team members to swap or offer shifts," Deputy Help Center. [Online]. Available: <https://help.deputy.com/hc/en-au/articles/4688726501135-Allow-team-members-to-swap-or-offer-shifts>
11. International Organization for Standardization, *ISO 8601-1:2019 Date and time — Representations for information interchange — Part 1: Basic rules*. Geneva: ISO, 2019.
12. OpenJS Foundation, "Node.js documentation: Test runner (node:test)," Node.js v24. [Online]. Available: <https://nodejs.org/api/test.html>
13. GitHub, "About GitHub Pages," GitHub Docs. [Online]. Available: <https://docs.github.com/en/pages/getting-started-with-github-pages/about-github-pages>
14. PlantUML contributors, "PlantUML: open-source tool that uses simple textual descriptions to draw UML diagrams." [Online]. Available: <https://plantuml.com>
15. Act on the Protection, etc. of Fixed-term and Part-time Employees (기간제 및 단시간근로자 보호 등에 관한 법률), Art. 3 ②, Art. 6; Enforcement Decree of the same Act, Art. 2 and Annex 1. National Law Information Center. [Online]. Available: <https://www.law.go.kr/법령/기간제및단시간근로자보호등에관한법률>
16. Supabase, "Free project pausing," Supabase Docs. [Online]. Available: <https://supabase.com/docs/guides/platform/free-project-pausing>
17. GitHub, "Events that trigger workflows," GitHub Docs, section "schedule." [Online]. Available: <https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows>
18. When I Work, "Getting your shifts covered," When I Work Help Center. [Online]. Available: <https://help.wheniwork.com/articles/getting-your-shifts-covered/>
19. When I Work, "Giving your team schedule flexibility," When I Work Help Center. [Online]. Available: <https://help.wheniwork.com/articles/giving-your-team-schedule-flexibility/>
20. Homebase, help article on trading and covering shifts, Homebase Support. [Online]. Available: <https://support.joinhomebase.com/s/article/Trading-and-covering-shifts>
21. Deputy, "Manage different languages in the workplace with Deputy," Deputy Help Center. [Online]. Available: <https://help.deputy.com/hc/en-au/articles/4621854798223-Manage-different-languages-in-the-workplace-with-Deputy>
22. Shiftee, "시프티 200% 활용하기: 근무일정 수정 요청과 전자결재" (making full use of Shiftee: schedule change requests and electronic approval), Shiftee blog, 2019. [Online]. Available: <https://shiftee.io/ko/blog/article/tipScheduleRequest>
23. Shiftee, frequently asked question on the conditions under which the weekly holiday allowance is recognised, Shiftee Help Center. [Online]. Available: <https://shiftee.io/ko/help>
24. Shopl, "아르바이트 직원마다 다른 근무 스케줄을 관리하는 방법" (how to manage a different work schedule for each part-time employee), Shopl blog. [Online]. Available: <https://www.shoplworks.com/blog-guides/managing-different-work-schedules-part-time-employees>
25. Albami (알바미), service website. [Online]. Available: <https://albame.co.kr/>
26. Gig for owners (기그 사장님), app listings on Google Play and the App Store. [Online]. Available: <https://play.google.com/store/apps/details?id=connectfit.logout.ceo>; <https://apps.apple.com/kr/app/id1523613072>

Appendix A. Final Vibe-Coding Prompt (v8)

The complete prompt, verbatim. To reproduce the system it is given to the coding assistant together with the three test files and the two supplied server files (db/app-shared.sql, tools/seed-sql.mjs) and followed by the specification file spec/spec.md (all in the repository). Source file: prompts/final-prompt.md.

ShiftSwap — Final Vibe-Coding Prompt (v8)

Version history: v1 was built in iteration 1; v2 (iteration 2) resolved the build ambiguities and document-review gaps; v3 (iteration 3) applied the UI review of the running app; v4 (iteration 4) applied the UX redesign review from the point of view of a consumer-app product team (task-first home screens, one primary action per screen, consequences shown before committing); v5 (iteration 5) makes the acceptance test files part of the input after reproduction run 1 showed that two correct rebuilds disagree on function signatures (see docs/reproducibility.md); v6 (iteration 6) applies a requirements change from the product-owner review: availability is no longer collected, every co-worker is asked, a "can't" answer lets a request end as FAILED with a message to the requester and an acknowledge-only card for the owner (spec §15); v7 (iteration 7) adds the work and payroll history from the contracts and the automatic monthly payroll draft (spec §16); v8 (iteration 8) propagates the revised proposal: a shared database on Supabase as the default data mode with the browser-only version kept as the local demonstration mode, the server time as the System Clock in the shared mode, and the part-time overtime rule with the approval warning (spec §17). Earlier versions are kept as prompts/prompt-v1.md ... prompts/prompt-v7.md; the reasons for every change are in prompts/prompt-log.md.

How to use: in an empty folder, place tests/rules.test.js, tests/services.test.js and tests/shared.test.js and the supplied server files db/app-shared.sql and tools/seed-sql.mjs from the repository, then paste this whole prompt, followed by the complete contents of spec/spec.md, into a coding agent that can create files and run shell commands. The agent must produce the same system that is deployed at <https://keemlogan.github.io/shiftswap/app/>.

You are building **ShiftSwap**, a shift schedule and substitute management web application for a small café with part-time workers. The complete specification (actors, use cases UC-01...UC-14 (UC-02 retired), functional requirements FR-01...FR-24 (FR-03 retired), non-functional requirements NFR-01...NFR-16, business rules BR-01...BR-15, states, SQLite schema, seed data, test IDs) follows this prompt. Treat the specification as the contract: use its identifiers, table and column names, state names and seed data exactly. Do not add features that are not in it.

0. Acceptance tests (supplied — read them first)

The folder already contains tests/rules.test.js, tests/services.test.js and tests/shared.test.js. They are the executable form of the specification and they fix the API: module paths, exported names, argument order, return shapes, error codes and the Node-side database factory. Read all three files completely before writing code. The server side of the shared mode is supplied and already deployed: db/app-shared.sql (tables, row triggers, the RPC functions app_snapshot, app_commit, app_reset, grants and policies, and the seed template) and tools/seed-sql.mjs (regenerates the seed template in that file from the seed code and is used by tests/shared.test.js). **Do not modify these two files and do not run the SQL**; read the header of db/app-shared.sql to learn the RPC arguments, the answers and the error messages. **Do not modify, rename, skip or delete any test**. Write the implementation so that npm test passes all of them unchanged. Where a test and the prose below seem to disagree about an API detail, the test wins; where they disagree about behaviour, stop and report the conflict instead of guessing. You may add further tests in a new file tests/extra.test.js.

1. Technology constraints

- Static site only: index.html, CSS, and JavaScript ES modules. No build step, no framework, no application server of its own. It must work when the folder is served by GitHub Pages under the sub-path /shiftswap/app/ (use only relative URLs).
- Database: SQLite in the browser with **sql.js** (WebAssembly). Vendor sql-wasm.js and sql-wasm.wasm into app/vendor/ (copy them from the sql.js npm package) so the app does not depend on a CDN at runtime.
- Two data modes (spec §9). **Shared mode** is the default whenever app/config.js contains the database URL and the publishable key: export const SUPABASE_URL = 'https://dbahxuegnqsthntpuqy.supabase.co', SUPABASE_KEY = the publishable key given in the header comment of db/app-shared.sql (public by design), STORE = 'dalbit'. It talks to the database with fetch only (no client library): POST {URL}/rest/v1/rpc/<name> with named JSON arguments and the header apikey (add Authorization: Bearer only for a key starting with eyJ), and GET {URL}/rest/v1/app_store?slog=eq.<store>&select=rev to poll the revision. **Local demonstration mode** is the behaviour of v7 (sql.js in localStorage, adjustable demo clock); ?mode=local / ?mode=shared switch and the choice is remembered under shiftswap.mode; ?store=<slog> selects another existing store (used by the checks with throwaway stores).
- Shared mode works on a replica: on start app_snapshot(p_store) returns {rev, now, tables} (rows in snake_case); load them into an in-memory sql.js database with the schema of §8 and let now() follow the server time. Every user command runs the unchanged service function on the replica; the gateway compares all tables before and after, sends the changed rows ([{t, op: 'insert'|'update'|'delete', row}], snake_case) with the base revision to app_commit(p_store, p_base_rev, p_changes), and on STALE reloads and re-runs the command (at most three times); on a refusal by a server rule it reloads and re-runs once so that the user gets the service's own message (e.g. ALREADY_TAKEN). Commands, system work, polling and reset never overlap (one queue). Poll the revision every 10 s and on focus; redraw after a remote change only when the user is idle (no sheet open, nothing typed, nothing being saved). The System Clock duties and the weekly recomputation that a screen triggers are saved after drawing; when the server refuses them, pause them for a minute.

- Persistence (local mode): after every committed transaction, export the database and save it to `localStorage` under the key `shiftswap.db.v2` (base64). The demo clock is stored under `shiftswap.clock.v1` and the language under `shiftswap.lang`. On start, load them if present, otherwise create the schema and insert the seed data. Provide "Reset demo data" in the header menu; it also resets the clock to `2026-09-28T09:00`. In the shared mode the reset calls `app_reset(p_store)`, which restores the seed for everyone (rate-limited by the server).
- Folder layout:
 - `app/index.html`, `app/styles.css`, `app/main.js` (router + bootstrap; chooses the mode)
 - `app/config.js` — the shared-mode settings above.
 - `app/core/shared.js` — the shared-mode gateway: `diffTables(before, after)`, `classify(err)`, `restTransport({url, key, fetchFn})`, `createGateway({SQL, transport, store, onReload, retries, pause})` with `load`, `command(fn)`, `settle`, `poll`, `reset`, `busy`, `systemAllowed`, `rev`, and the key mappers `snake / camel` (the tests fix the details).
 - `app/core/db.js` — `init`, `schema` (exactly the SQL of spec §8), `seed` (spec §10), `tx(fn)` helper that runs `BEGIN...COMMIT / ROLLBACK` and persists after commit, `all(sql, params)`, `get(sql, params)`, `run(sql, params)`; for the shared mode `TABLES` (the 11 tables in dependency order), `keyOf(table)`, `loadTables(SQL, tables)` (replace the database by an in-memory replica) and `dumpTables()`.
 - `app/core/clock.js` — System Clock: `now()` returns the stored demo datetime (default `2026-09-28T09:00`) in the local mode, `setNow(iso)`, `advance(minutes)`; in the shared mode `useServerTime(serverMs, localMs)` makes `now()` follow the server time in Korea (`usesServerTime()`), and `setNow` refuses.
 - `app/core/rules.js` — pure functions only (no DB, no DOM): `overlaps`, `durationHours` (supports `end ≤ start` as next day), `weekStartOf`, `isEligibleCandidate`, `validateRequest`, `contractHours`, `computeWeeklySummary`, `holidayHours`, `premiumHours` (with the part-time item (iv) of BR-08 when `options.contractHours` is given), `isPartTime`, `partTimeLimitCheck` (BR-15), `minimumWageFor`, `probationApplies`, `computePayrollRow`. Each function has a JSDoc comment citing the BR it implements.
 - `app/core/services.js` — one exported function per use-case step, each running inside `tx()`: `registerWorker`, `saveFixedSchedule`, `generateWeek`, `editShift`, `createSubRequest`, `respondToRequest`, `decideRequest`, `cancelRequest`, `expireOverdue`, `recordAttendance`, `confirmAttendance`, `markAbsent`, `recomputeWeek`, `generatePayroll`, `confirmPayroll`, `updateSettings`, `listNotifications`, `markNotificationsRead`, `acknowledgeMinWage`. Services call rules for every decision and write Notification rows exactly as the recipients table in spec §8 says. `services.js` also exports read-only query functions for the screens (not wrapped in `tx`), e.g. `getWeekShifts`, `listMyRequests`, `listOwnerRequests`, `getApprovalPreview`, `getPayroll`, `unreadCount`, `defaultDeadline`, and a `ServiceError(code, message, params)` class that the UI translates by code. It also exports the pure function `menuFor(role)` returning the navigation items a role may open (the router uses it to hide menus and to redirect routes the role may not open, FR-20), and home-screen queries `getWorkerHome(workerId)` and `getOwnerHome()` that return, in one call, every item §2.3 / §2.5 needs (including the before/after effect of accepting or approving and the BR-15 warning of the acceptor). For the shared seed it exports `seedPayrollHistory(lastMonth)` and `completeSharedSeed(today)` (spec §10, shared-mode seed).
 - `app/ui/` — one module per screen: `login.js`, `home.js` (worker and owner Home, §2.3/§2.5), `me.js` (worker Me: profile + attendance), `schedule.js` (weekly board, both roles), `requests.js` (worker: my requests + new request form), `inbox.js`, `approvals.js` (owner), `attendance.js`, `weekly.js` (owner weekly summary), `payroll.js`, `workers.js` (owner: workers, fixed schedules), `settings.js` (owner: workplace + minimum wage), `components.js` (shared UI pieces: icons, buttons with disabled reasons, page header, tracker, sheets, toasts), `slots.js` (weekday + time slot editor used for fixed schedules), `i18n.js` (translations plus shared helpers: HTML escaping, number/date/money formatting, the form error box).
 - `tests/rules.test.js` and `tests/services.test.js` run with `node --test` (use the `sql.js` npm package in Node for service tests). Name each test with its spec test ID (e.g. `TC-081 holiday allowance at exactly 15 h`).
 - `package.json` with `"type": "module"` and script `"test": "node --test tests/*.test.js"` (a bare directory argument fails on Node 24).
- Every command of a screen goes through `ctx.run(fn)` and is awaited: in the local mode it just calls `fn`; in the shared mode it is the gateway's `command(fn)`, and the screen shows a busy state (`aria-busy`, dimmed, no clicks) until the change is saved.
- The System Clock actor runs `expireOverdue()` (UC-11) and then `prepareMonthlyPayroll()` (UC-14, BR-14: draft for the previous month if it has no rows yet, plus one `PAYROLL_DRAFT_READY` notification for the owner) on app start, on every route change, and after the demo clock changes; in the shared mode it also runs `completeSharedSeed()` first and uses the server time, and the server refuses an expiry before the deadline. The seed builds the work history of spec §10 by calling the same services (generate shifts, record and confirm attendance, generate and confirm payroll with acknowledgements) so that the stored history equals what the rules compute.

2. Screens and behaviour

2.0 UX principles (apply to every screen; they override any screen description that seems to conflict)

1. **One screen, one job.** Every screen has exactly one primary action, drawn as the only filled blue button on that screen. On screens narrower than 900 px the primary action is a full-width button fixed to the bottom (16 px side inset, above the tab bar, `env(safe-area-inset-bottom)` respected). Other actions are grey filled (secondary) or text buttons.
2. **What needs me comes first.** Each role opens on a **Home** screen that lists the things waiting for this person right now, most urgent first, and each item carries its own action button, so the user never has to find the right menu.

3. **Show the consequence before the commitment.** Whenever an action changes someone's hours or pay (accepting, approving, confirming payroll), the screen states the effect in one plain sentence plus numbers before the button — e.g. "최민호님 이번 주 16시간 → 21시간 · 주휴수당 변화 없음 (33,024원 유지)".
4. **Ask one thing at a time.** Requesting a substitute is a three-step flow (choose shift → reason and deadline → review and send) with a step indicator ("1/3"), a back button, and a review screen that names the people who will receive the request.
5. **Say what happened and what happens next.** After every action show a result screen or a toast that states the outcome and the next step — e.g. "최민호님, 강도윤님에게 요청을 보냈어요. 누군가 수락하면 알려 드릴게요." — never a silent state change.
6. **Status as progress.** A substitute request is shown everywhere as a three-step tracker 요청 → 수락 → 승인 (Requested → Accepted → Approved); REJECTED, EXPIRED and CANCELLED are shown as a clearly labelled end state with the reason.
7. **The user's words.** The default UI language is Korean in polite 해요체 ("대타를 구하고 있어요"), with a complete English translation selectable in the menu. IDs such as FR/UC/BR, status codes and table names never appear in the UI. In the Korean UI the seed people and store are shown with their Korean names from spec §10 (박지영, 이서연, 최민호, 정하나, 강도윤, 달빛카페) and addressed as full name + 님 ("최민호님") — a display map in `i18n.js`, the stored names stay English; names not in the map are shown as stored. Dates read like "9월 30일(수) 18:00–23:00" / "Wed, Sep 30 · 18:00–23:00"; relative times where they help ("내일 21:00까지", "D-2").
8. **Nothing blocks without a reason.** A disabled button always has a one-line explanation beneath it (e.g. "최저임금 경고 1건을 확인해야 확정할 수 있어요").

2.1 Navigation

- Worker tabs: **홈** (Home), **근무표** (Schedule), **대타** (Swaps), **내 정보** (Me: profile and attendance). Owner tabs: **홈**, **근무표**, **직원** (Staff: workers and fixed schedules), **급여** (Pay: weekly summary and monthly payroll). **설정** (Settings), language and sign-out are in the header menu.
- ≥ 900 px: left navigation (220 px) with the same items and inline SVG line icons; < 900 px: bottom tab bar (64 px, icon + label), hidden while the request flow is open.
- Header: store name; a bell icon with the unread count opening the notification list (FR-21); the compact demo clock chip ("9/28(월) 09:00") that opens **Demo tools**; the current person's name with **다른 사람으로 보기** (switch account) that opens the account list without resetting data.
- **Demo tools** panel (sheet on mobile, popover on desktop): demo clock date-time input, **+1시간**, **+1일**, **데모 데이터 초기화** (Reset demo data, asks for confirmation in the panel itself) and **공유 데모 모드로 바꾸기**. The clock no longer sits permanently in the header. In the shared mode the chip shows the server time and the panel shows instead: one sentence that the demo is shared and the time is the server time, **서버 시각** (read-only), **모두의 데모 데이터 초기화** with a confirmation that says every visitor's data go back to the start, and **로컬 데모 모드로 바꾸기**.
- Shared-mode messages (toasts, both languages, spec §9): another person changed the same data at the same moment several times (SHARED_BUSY), refused by the server (SHARED_REFUSED, data reloaded), offline (SHARED_OFFLINE, nothing saved), unavailable, the per-minute / per-day limits, a full store, and the reset limits. If the database cannot be reached on start, show a notice with the button **로컬 데모 모드로 열기**.

2.2 Sign-in = demo guide (UC-13)

- Title "달빛카페 데모" and one sentence: "비밀번호 없이 사람을 골라 체험해요."
- A **3분 체험** (3-minute tour) card listing three numbered steps — the order matters, so numbers are used: ① 이서연으로 대타 요청 보기 ② 최민호로 수락하기 ③ 박지영 사장님으로 승인하기. Each step has a button that signs in as that person and opens the screen where the step happens (Swaps / Home / Home). The card marks steps already done in this browser.
- Below, **모든 계정**: each account with name, role and its one-line summary (owner: "사장님 · 달빛카페 · 직원 4명"; worker: fixed shifts and weekly contract hours, e.g. "월·수 18:00–23:00 · 주 10시간").
- A notice above the tour states the mode with a text button to switch: shared — "공유 데모예요. 이 페이지를 연 모든 사람이 같은 가상 데이터를 보고 바꿀 수 있어요. 개인 정보는 입력하지 마세요."; local — the data stay in this browser and the clock can be set in the demo tools. In the shared mode the tour's progress follows the shared data and restarting the tour is the reset for everyone (with its confirmation).
- The chosen account is kept in `sessionStorage`.

2.3 Worker Home

In this order, each section omitted when empty:

1. Greeting "이서연님, 안녕하세요" and today's date.
2. **대타 요청이 왔어요** — one card per PENDING target: who asks, the shift ("9월 30일(수) 18:00–23:00"), reason, deadline countdown ("내일 21:00까지 답해 주세요"), the effect on me ("수락하면 이번 주 16시간 → 21시간 · 주휴수당 그대로"), and two buttons **수락하기** (primary) / **불가** (text, "can't" = DECLINED). Accept is one tap (NFR-01); 불가 is one tap and the card closes with the toast "불가로 답했어요". After accepting: result toast "수요일 근무를 맡았어요. 사장님 승인을 기다려요." If someone else was faster: "최민호님이 먼저 수락했어요" and the card closes.
3. **내가 보낸 요청** — tracker cards for the worker's open requests with each co-worker's answer so far (대기 중 / 불가 / 수락) and a text button **요청 취소**. A request that ended FAILED shows, until the worker taps **확인했어요**, the card "모든 동료가 대타가 불가능하다고 해요. 사장님께 연락드려 보세요." (BR-13).
4. **다음 근무** — a large card with date, time, D-day and the button **이 근무 대타 구하기** (starts the flow with this shift preselected).

5. 이번 주 — scheduled hours, contract hours, and holiday allowance in words and money: eligible "주휴수당 받아요 · 약 33,024원" or not "계약이 주 15시간 미만이라 주휴수당이 없어요 (주 10시간)", with a bar showing contract hours against the 15-hour line.
6. 기록할 근무 — past own shifts without attendance, each with 기록하기.

2.4 Request substitute flow (UC-04, UC-12)

- Step 1 "어떤 근무를 바꿀까요?": own future SCHEDULED shifts as selectable cards; a shift that already has an open request is shown disabled with "이미 대타를 구하고 있어요".
- Step 2 "사유와 마감을 정해 주세요": reason chips 시험 / 병원 / 가족 행사 / 개인 사정 / 직접 입력 (optional); deadline chips "24시간 뒤" / "근무 2시간 전" / "직접 선택", default = the earlier of now + 24 h and shift start (BR-03); the chosen deadline is shown as a sentence.
- Step 3 "이렇게 보낼게요": summary of shift, reason and deadline, and 받는 사람 — the co-workers who will be asked (BR-01: every active co-worker except those already working at that time) with names, or — when nobody can be asked — the warning "지금 근무가 비어 있는 동료가 없어요. 보내면 바로 사장님께 알려 드려요." (FR-08). Primary 대타 요청 보내기.
- Result screen: the tracker at step 1, the recipients, and 홈으로.
- 대타 tab: sections 받은 요청 (same cards as Home), 보낸 요청 (tracker cards with cancel), 지난 요청 (collapsed history with end states). Primary action of the tab: 대타 구하기.

2.5 Owner Home

1. Heading "오늘 처리할 일 N개" (or "지금 처리할 일이 없어요" with the week's summary).
2. 승인 기다리는 대타 — a decision card per ACCEPTED request: the sentence "이서연님의 9월 30일(수) 18:00–23:00 근무를 최민호님이 대신해요", then one impact line per person (scheduled hours before → after this week, holiday allowance status and amount before → after; a changed eligibility is highlighted in amber with the words "주휴수당이 바뀌어요"), then, when BR-15 applies (≥ 5 regular employees, part-time acceptor, more than 12 h beyond contract that week), the amber notice "최민호님은 이번 주 계약보다 13시간 더 일하게 돼요. 단시간 근로자는 주 12시간까지예요." (it does not block the decision, FR-24), then 승인하기 (primary) / 거절 (text). A 자세히 toggle shows the full FR-11 before/after table.
3. 답을 기다리는 요청 — open REQUESTED requests with deadline countdown and each co-worker's answer (대기 중 / 불가). 대타를 못 구했어요 — one card per unacknowledged REQUEST_FAILED notification: "정하남님의 10월 4일(일) 10:00–16:00 근무는 동료 모두 불가예요. 직접 연락해 주세요." with only 확인했어요 (marks the notification read; no other action).
4. 확인할 출근 기록 — recorded but unconfirmed attendance with 확인 / 결근 처리.
5. 급여 초안이 준비됐어요 — shown while the owner's PAYROLL_DRAFT_READY notification is unread: "9월 급여 초안이 준비됐어요. 확인하고 확정해 주세요." with the primary-style action 급여 확인하기 (opens Pay → Monthly for that month and marks the notification read). 급여 경고 — unacknowledged minimum-wage warnings of the current draft, linking to Pay.
6. 오늘 근무 — who works today and when; 이번 주 — shifts, hours and the estimated holiday-allowance total.

2.6 Schedule (UC-03)

- ≥ 900 px: the time grid of v3 (day columns Mon–Sun, hour axis from earliest start to latest end rounded to whole hours, 44 px per hour, hourly lines, today marked, blocks positioned by time, overlapping shifts side by side). < 900 px: day-by-day list with today first expanded.
- Tapping a shift opens a detail sheet (mobile) or side panel (desktop) with its details, its history ("이서연 → 최민호 · 9/28 승인") and only the actions relevant to the viewer: the worker's own future shift → 대타 구하기; owner → 시간 바꾸기, 담당자 바꾸기, 삭제 (refused with the reason when it has a request or attendance, FR-05).
- Owner: when the displayed week has no shifts, the empty state's primary button is 이번 주 근무표 만들기 (FR-04); otherwise the primary button is 근무 추가. Week navigation: previous / next / 이번 주.
- The handover and pending markers of §3 are used here and on every shift card elsewhere.

2.7 Pay (owner; UC-08, UC-09)

- Two segments: 주간 집계 and 월 급여; period chosen with < > buttons ("2026년 9월").
- Weekly summary: one card per worker: contract hours, scheduled/actual hours, eligibility as words with the reason ("계약 주 10시간 · 15시간 미만"; "9/29(화) 결근") and holiday hours/amount.
- Monthly payroll: the < > month switcher reaches every month back to the first contract month; confirmed months show "확정됨" and their rows read-only; a month that the System Clock prepared shows the note "매월 1일에 자동으로 만든 초안이에요". Primary 9월 급여 초안 만들기 when no draft exists; then one card (mobile) / row (desktop) per worker with base hours, base pay, holiday pay, premium, total and flags (예상치 = estimated, 수습 최저임금 90 %, 최저임금 미만). A below-minimum row shows the sentence "강도윤님 시급 10,000원이 2026년 최저임금 10,320원보다 낮아요" and 확인했어요 (acknowledgeMinWage). The sticky primary 9월 급여 확정하기 is disabled with its reason until all warnings are acknowledged; confirmed months are read-only and say so.

2.8 Staff, Me, Settings

- 직원 (UC-01): worker cards (name, contract hours per week, hourly wage, contract period); tapping opens the worker's detail with the fixed-schedule editor: weekday chips + start/end time selects, one row per weekday, add/remove; save shows the resulting weekly contract hours. Primary 직원 추가.

- **내 정보** (UC-07): the worker's profile (contract, weekly contract hours, hourly wage, phone — visible only here and to the owner) and the list of attendance records with 기록하기 for ended shifts. There is no availability editor (removed in iteration 6).
- **설정** (UC-10): store name, number of regular employees with the sentence of what it changes ("5명 이상이면 연장·야간·가산수당이 붙어요"), the substitution-attendance policy as two radio cards each explaining the effect in one sentence, and the minimum-wage table.
- Notifications list: each notification as a sentence with time; actionable ones link to the card where the action is.
- Role guard: a worker never sees other workers' phone numbers or wages (NFR-11).

3. Visual design (do not deviate)

- Font: Pretendard Variable for everything (<https://cdn.jsdelivr.net/gh/orioncactus/pretendard@v1.3.9/dist/web/variable/pretendardvariable-dynamic-subset.min.css>) with font-variant-numeric: tabular-nums on times, hours and money. No second font. Korean: word-break: keep-all.
- Type scale: body 16 px / 1.6; home heading 26 px / 700 / -0.02em; card title 17 px / 700; key number 28 px / 700; caption 13 px (never below 12 px).
- Colors: --ink #191F28, --ink-2 #4E5968, --ink-3 #8B95A1, --surface #F2F4F6 (page background), --paper #FFFFFF (cards), --line #E5E8EB, --shift #2B5BD7 (primary and own shifts), --shift-soft #EBF1FD, --handover #E9A23B, --handover-soft #FDF3E1, --alert #D93A30, --alert-soft #FDECEB, --ok #1F8A55.
- Cards: white on the grey page, 16 px radius, 20 px padding (24 px ≥ 900 px), no border, no shadow; 12 px between cards, 32 px between sections.
- Buttons: height 52 px on mobile / 44 px on desktop, 12 px radius, 16 px / 600 label; primary filled --shift; secondary filled #E8EBF0 with --ink; text buttons --ink-2; pressed state scale(0.96); minimum touch target 44 × 44 px.
- Icons: inline SVG line icons (24 px, 1.75 px stroke, currentColor) for tabs, bell, clock, chevrons; no icon fonts, no emoji.
- Signature element: a handed-over shift has a 6 px amber left edge and a subtle 45° amber stripe (repeating-linear-gradient, the only gradient), the new worker's name in bold and the original worker struck through; a shift with an open request has a dashed amber outline and the chip "대타 구하는 중".
- Motion: sheets and step changes slide/fade in 180 ms cubic-bezier(0.2, 0, 0, 1); no motion under prefers-reduced-motion.
- Layout: content max-width 720 px for Home, flows and lists (centred in the content area), full width up to 1200 px for the schedule grid and payroll table; at 360 px no horizontal scroll.
- Accessibility: visible focus rings, real <button>s, labelled inputs, status never by colour alone, the step indicator announced ("3단계 중 2단계").

4. Done criteria

1. npm test passes and covers every test ID group listed in spec §11 with at least one test each, including: two acceptances of the same request where only the first succeeds; holiday eligibility at 14.99 h vs 15 h; the 40 h cap; both substitution-attendance policies; premium zero at 4 employees and non-zero at 5; probation applies only when all BR-10 conditions hold; the worked example of spec §10 (Choi Minho 3.2 h, 33,024 KRW).
2. The seed state reproduces spec §10 exactly, including the open request with three candidates (Choi Minho, Jung Hana, Kang Doyun), and the S7 walkthrough of §10 works (a test proves it).
3. Serving the repository root with any static server and opening /app/ shows the sign-in screen with no console errors.
4. Walking through the sign-in tour in the browser: as Lee Seoyeon the Swaps tab shows her open request as a tracker; as Choi Minho, Home shows the request card with the effect "16시간 → 21시간" and 수락하기 accepts it in one tap; as Kang Doyun the card says Minho was faster; as the owner, Home shows the decision card for Minho (16 → 21 h, holiday allowance unchanged) and 승인하기 approves it; the schedule then shows the handover marker.
5. The S8 walkthrough of spec §10 works in the browser: as Jung Hana request Sun 10-04 10:00–16:00; as Lee Seoyeon, Choi Minho and Kang Doyun tap 불가; Hana's Swaps tab shows the FAILED card with the contact-the-owner message; the owner's Home shows the "대타를 못 구했어요" card and 확인했어요 removes it.
6. The S9 walkthrough of spec §10 works in the browser: Pay → Monthly shows March–August 2026 as confirmed; after setting the demo clock to 2026-10-01 the owner's Home shows "9월 급여 초안이 준비됐어요" and Pay → Monthly shows the September 2026 draft with Doyun's warning to acknowledge; moving the clock further within October does not create a second draft.
7. Every screen has exactly one filled primary button (a headless check counts them), every disabled button has a visible reason, and all touch targets are at least 44 × 44 px. Criteria 3–7 are checked in the local demonstration mode (?mode=local).
8. The shared mode works with two browsers on a throwaway store (?mode=shared&store=e2e-..., created and deleted with the owner connection): the sign-in screen says that the demo is shared; the demo tools show the server time without clock controls; when Choi Minho and Kang Doyun tap 수락하기 at the same moment exactly one succeeds and the other is told who was faster, and the server holds one ACCEPTED request; the owner's approval appears in the other browser within 20 s; the reset for everyone brings the open request back.

Appendix B. Prompt Log (index)

The prompt log records every ambiguity, review finding and change made between prompt versions. To keep the report within 150 pages it is not reproduced here; Section 5.4 summarises each iteration, and the complete log is the file `prompts/prompt-log.md` in the repository (<https://github.com/keemlogan/shiftswap/blob/main/prompts/prompt-log.md>). The table lists its sections and the number of entries (ambiguities, findings or changes) in each. Source file: `prompts/prompt-log.md`.

Table 82. Sections of the prompt log

Section	Entries
Iteration 1 — ambiguities found while building	21
Iteration 2 — changes from v1 to v2 and result	22
Iteration 3 — changes from v2 to v3 and result	9
Iteration 5 — independent code review and reproduction run 1	9
Iteration 4 — changes from v3 to v4 and result	27
Iteration 6 — requirements change: availability removed, can't answer and FAILED	9
Iteration 7 — work history and automatic monthly payroll	8
Iteration 8 — shared database and part-time overtime (proposal revision)	7